

CODING SUPER HUB

Complete Source Codebase

Complete, Unabridged Technical Specification & Source Implementation

150 DEVELOPER TOOLS • 45+ LANGUAGES • FULL-STACK GEMINI AI RUNNER

SYSTEM & ARCHITECTURE SPECIFICATION

Application:

Coding Super Hub (Online Multi-Language Hub)

Generated Date:

9/11/2026, 6:37:53 PM

Architecture:

Full-Stack (React 19 + Vite 6 + Express 4 + Gemini AI)

Total Files:

25 Files

Included Tools:

150 Fully Interactive Utilities (Tools #1 - #150)

AI Runtime:

Google GenAI (gemini-3.8-flash)

Styling Engine:

Tailwind CSS v4 with Light/Dark Theme Support

Export Type:

Complete Unabridged PDF

TABLE OF CONTENTS / MANIFEST

#	File Path	Module Description	Lines
1	metadata.json	AI Studio app metadata & capabilities	7
2	package.json	Project manifest & dependencies	41
3	tsconfig.json	TypeScript compiler configuration	27
4	vite.config.ts	Vite build & server plugins setup	23
5	.env.example	Environment variables documentation	10
6	index.html	HTML5 application root entry point	19
7	server.ts	Express + Vite backend & Gemini AI endpoints	312
8	src/main.tsx	React DOM client mount point	11
9	src/types.ts	Global TypeScript interfaces & enums	44
10	src/index.css	Tailwind CSS imports & custom styles	2
11	src/App.tsx	Master Application layout & state orchestrator	574
12	src/components/Navbar.tsx	Navigation, search, category filter bar	213
13	src/components/CodeEditor.tsx	Multi-language editor, runner & sandbox	969
14	src/components/AIAssistant.tsx	AI Copilot chat with Markdown & Code Apply	531
15	src/components/LanguageHub.tsx	45+ Programming languages explorer & starter cards	216
16	src/components/ToolCard.tsx	Universal reusable tool container with copy/clear	138
17	src/components/ShortcutsModal.tsx	Keyboard shortcuts cheat sheet modal	82
18	src/data/languages.ts	Full catalog & starters for 45+ languages	341
19	src/data/toolsMetadata.ts	Registry & metadata of all 150 dev tools	1411
20	src/utils/helpers.ts	Conversion, formatting, math & string utilities	260
21	src/utils/crypto.ts	Hash, cipher & encoding cryptographic helpers	282
22	src/components/tools/ToolsTextFormat.tsx	Tools #1 - #20 (Text, Formatting, Conversion, Hashes)	986
23	src/components/tools/ToolsMathCalculators.tsx	Tools #21 - #30 (Math, Calculations, Financial, Conversion)	1383
24	src/components/tools/ToolsDataWeb.tsx	Tools #31 - #100 (Data, Web, CSS, JSON, SEO, Dev Cheat Sheets)	1967

25	src/components/tools/ToolsAdvancedDev.tsx - #150 (Diff, Ciphers, Crypto, CSS & JS Snippets)	2010
----	---	------

FILE [1/25]

metadata.json

7 lines • 0.3 KB • AI Studio app metadata & capabilities

```
1 {  
2   "name": "Coding Super Hub",  
3   "description": "All-in-one developer toolbox featuring 150+ coding tools, multi-language code editor, programming  
4   language support, and AI code completion",  
5   "requestedPermissions": [],  
6   "majorCapabilities": ["MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"]  
7 }
```

FILE [2/25]

package.json

41 lines • 1.1 KB • Project manifest & dependencies

```
1 {
2   "name": "react-example",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "scripts": {
7     "dev": "tsx server.ts",
8     "build": "vite build && esbuild server.ts --bundle --platform=node --format=cjs --packages=external --sourcemap",
9     "start": "node dist/server.cjs",
10    "preview": "vite preview",
11    "clean": "rm -rf dist server.js",
12    "generate:pdf": "node scripts/generate_code_pdf.cjs",
13    "lint": "tsc --noEmit"
14  },
15  "dependencies": {
16    "@google/genai": "^2.4.0",
17    "@tailwindcss/vite": "^4.1.14",
18    "@vitejs/plugin-react": "^5.0.4",
19    "dotenv": "^17.2.3",
20    "express": "^4.21.2",
21    "lucide-react": "^0.546.0",
22    "motion": "^12.23.24",
23    "pdfkit": "^0.20.2",
24    "react": "^19.0.1",
25    "react-dom": "^19.0.1",
26    "react-markdown": "^10.1.0",
27    "vite": "^6.2.3"
28  },
29  "devDependencies": {
30    "@types/express": "^4.17.21",
31    "@types/node": "^22.14.0",
32    "@types/pdfkit": "^0.17.6",
33    "autoprefixer": "^10.4.21",
34    "esbuild": "^0.25.0",
35    "tailwindcss": "^4.1.14",
36    "tsx": "^4.21.0",
37    "typescript": "^5.8.2",
38    "vite": "^6.2.3"
39  }
40 }
41
```

FILE [3/25]

tsconfig.json

27 lines • 0.5 KB • TypeScript compiler configuration

```
1 {
2   "compilerOptions": {
3     "target": "ES2022",
4     "experimentalDecorators": true,
5     "useDefineForClassFields": false,
6     "module": "ESNext",
7     "lib": [
8       "ES2022",
9       "DOM",
10      "DOM.Iterable"
11    ],
12    "skipLibCheck": true,
13    "moduleResolution": "bundler",
14    "isolatedModules": true,
15    "moduleDetection": "force",
16    "allowJs": true,
17    "jsx": "react-jsx",
18    "paths": {
19      "@/*": [
20        "./*"
21      ]
22    },
23    "allowImportingTsExtensions": true,
24    "noEmit": true
25  }
26 }
27
```

FILE [4/25]

vite.config.ts

23 lines • 0.7 KB • Vite build & server plugins setup

```
1 import tailwindcss from '@tailwindcss/vite';
2 import react from '@vitejs/plugin-react';
3 import path from 'path';
4 import {defineConfig} from 'vite';
5
6 export default defineConfig(() => {
7   return {
8     plugins: [react(), tailwindcss()],
9     resolve: {
10       alias: {
11         '@': path.resolve(__dirname, '.'),
12       },
13     },
14     server: {
15       // HMR is disabled in AI Studio via DISABLE_HMR env var.
16       // Do not modify "file watching is disabled to prevent flickering during agent edits.
17       hmr: process.env.DISABLE_HMR !== 'true',
18       // Disable file watching when DISABLE_HMR is true to save CPU during agent edits.
19       watch: process.env.DISABLE_HMR === 'true' ? null : {},
20     },
21   };
22 });
23
```

FILE [5/25]

.env.example

10 lines • 0.4 KB • Environment variables documentation

```
1 # GEMINI_API_KEY: Required for Gemini AI API calls.
2 # AI Studio automatically injects this at runtime from user secrets.
3 # Users configure this via the Secrets panel in the AI Studio UI.
4 GEMINI_API_KEY="MY_GEMINI_API_KEY"
5
6 # APP_URL: The URL where this applet is hosted.
7 # AI Studio automatically injects this at runtime with the Cloud Run service URL.
8 # Used for self-referential links, OAuth callbacks, and API endpoints.
9 APP_URL="MY_APP_URL"
10
```

FILE [6/25]

index.html

19 lines • 0.8 KB • HTML5 application root entry point

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>Coding Super Hub</title>
7     <meta name="description" content="All-in-one developer toolbox featuring 150+ coding tools, multi-language code
8 editor, programming lang
9 <meta property="og:title" content="Coding Super Hub" />
10 <meta property="og:description" content="All-in-one developer toolbox featuring 150+ coding tools, multi-
11 language code editor, programming
12 <meta property="og:type" content="website" />
13     <meta name="twitter:card" content="summary_large_image" />
14   </head>
15   <body>
16     <div id="root"></div>
17     <script type="module" src="/src/main.tsx"></script>
18   </body>
19 </html>
```


FILE [7/25]

server.ts

312 lines • 9.7 KB • Express + Vite backend & Gemini AI endpoints

```
1 import express from "express";
2 import path from "path";
3 import fs from "fs";
4 import { createServer as createViteServer } from "vite";
5 import { GoogleGenAI } from "@google/genai";
6 import dotenv from "dotenv";
7
8 dotenv.config();
9
10 const PORT = 3000;
11
12 // Valid models from @google/genai guidelines with high-throughput fallbacks
13 const CANDIDATE_MODELS = [
14   "gemini-3.8-flash",
15   "gemini-flash-latest",
16   "gemini-3.1-flash-lite",
17 ];
18
19 function isTransientError(error: any): boolean {
20   const status =
21     error?.status ||
22     error?.code ||
23     error?.error?.code ||
24     error?.error?.status ||
25     (error?.response ? error.response.status : undefined);
26   const msg = String(
27     error?.message || error?.error?.message || error || ""
28   ).toLowerCase();
29
30   return (
31     status === 503 ||
32     status === 429 ||
33     status === "UNAVAILABLE" ||
34     status === "RESOURCE_EXHAUSTED" ||
35     msg.includes("503") ||
36     msg.includes("429") ||
37     msg.includes("high demand") ||
38     msg.includes("unavailable") ||
39     msg.includes("overloaded") ||
40     msg.includes("resource has been exhausted") ||
41     msg.includes("spikes in demand are usually temporary") ||
42     msg.includes("try again later")
43   );
44 }
45
46 async function generateWithFallback(
47   ai: GoogleGenAI,
48   params: {
49     contents: any;
50     config?: any;
51     preferredModel?: string;
52   }
53 ) {
54   const preferred = params.preferredModel || "gemini-3.8-flash";
55   const modelsToTry = [
56     preferred,
57     ...CANDIDATE_MODELS.filter((m) => m !== preferred),
58   ];
59
60   let lastError: any = null;
61
62   for (const model of modelsToTry) {
63     for (let attempt = 1; attempt <= 2; attempt++) {
64       try {
65         const response = await ai.models.generateContent({
66           model,
```

server.ts (continued)

```

67         contents: params.contents,
68         config: params.config,
69     });
70     return { response, modelUsed: model };
71 } catch (err: any) {
72     lastError = err;
73     console.warn(
74         `[Gemini API] Attempt ${attempt} on model "${model}" failed: ${err?.message || err}`
75     );
76
77     if (isTransientError(err)) {
78         // Exponential backoff with jitter before retry or next model
79         const delayMs = attempt === 1 ? 600 + Math.random() * 400 : 1200;
80         await new Promise((resolve) => setTimeout(resolve, delayMs));
81     } else {
82         // If fatal error (e.g. invalid key format), don't keep retrying
83         throw err;
84     }
85 }
86 }
87 }
88
89 throw lastError;
90 }
91
92 async function startServer() {
93     const app = express();
94     app.use(express.json({ limit: "10mb" }));
95
96     // Health check endpoint
97     app.get("/api/health", (_req, res) => {
98         res.json({ status: "ok", service: "Coding Super Hub API" });
99     });
100
101     // Server-side Gemini AI generation endpoint
102     app.post("/api/ai/ask", async (req, res) => {
103         try {
104             const { prompt, history = [], language = "javascript", customApiKey } = req.body;
105
106             if (!prompt || typeof prompt !== "string" || !prompt.trim()) {
107                 return res.status(400).json({ error: "Prompt is required" });
108             }
109
110             const apiKey =
111                 (typeof customApiKey === "string" && customApiKey.trim()) ||
112                 process.env.GEMINI_API_KEY;
113
114             if (!apiKey) {
115                 return res.status(400).json({
116                     error:
117                         "No Gemini API key available. Please configure GEMINI_API_KEY in the environment or provide a key in
118 the AI Assistant panel.",
119                 });
120
121                 const ai = new GoogleGenAI({
122                     apiKey,
123                     httpOptions: {
124                         headers: {
125                             "User-Agent": "aistudio-build",
126                         },
127                     },
128                 });
129
130                 const systemInstruction = `You are Coding Super AI inside Coding Super Hub – an all-in-one developer
131 workspace with 150+ developer tags.
132 Selected language context: ${language}.
133 Your goal:
134 - Act as a senior, highly competent full-stack coding copilot.
135 - Give accurate, modern, runnable code when asked.
136 - When debugging, pinpoint the exact root cause, explain clearly in brief points, and provide the complete
corrected code.`

```

server.ts (continued)

```

136 - Keep explanations structured, concise, and easy to scan.
137 - Return Markdown with language-tagged code blocks.`;
138
139     const contents: Array<{ role: string; parts: Array<{ text: string }> }> = [];
140
141     // Add conversation history if present
142     if (Array.isArray(history)) {
143         for (const item of history.slice(-8)) {
144             if (item && item.text) {
145                 contents.push({
146                     role: item.role === "bot" || item.role === "model" ? "model" : "user",
147                     parts: [{ text: String(item.text) }],
148                 });
149             }
150         }
151     }
152
153     // Add current user prompt
154     contents.push({
155         role: "user",
156         parts: [{ text: prompt.trim() }],
157     });
158
159     const { response, modelUsed } = await generateWithFallback(ai, {
160         preferredModel: "gemini-3.8-flash",
161         contents,
162         config: {
163             systemInstruction,
164             temperature: 0.7,
165         },
166     });
167
168     const text = response.text || "No response generated.";
169     return res.json({ answer: text, modelUsed });
170 } catch (error: any) {
171     console.error("Gemini API error:", error);
172     const isHighDemand = isTransientError(error);
173     const userMessage = isHighDemand
174         ? "This model is currently experiencing temporary high demand on Google servers. Automatic retries were attempted across fallbacks."
175         : "Failed to process AI request";
176     return res.status(isHighDemand ? 503 : 500).json({
177         error: userMessage,
178         isHighDemand,
179     });
180 }
181 });
182
183 // Multi-Language Code Runner Endpoint (Python, C++, Rust, Go, Java, Bash, PHP, etc.)
184 app.post("/api/code/run", async (req, res) => {
185     try {
186         const { language = "text", code, customApiKey } = req.body;
187         if (!code || typeof code !== "string" || !code.trim()) {
188             return res.status(400).json({ error: "Code is required to run." });
189         }
190
191         const apiKey =
192             (typeof customApiKey === "string" && customApiKey.trim()) ||
193             process.env.GEMINI_API_KEY;
194
195         if (!apiKey) {
196             return res.status(400).json({
197                 stdout: "",
198                 stderr: "Notice: Please configure GEMINI_API_KEY or enter your custom key in the AI Copilot panel to run non-browser languages.",
199                 executionTime: "0.00s",
200             });
201         }
202     }
203
204     const ai = new GoogleGenAI({

```

server.ts (continued)

```

205     apiKey,
206     httpOptions: {
207       headers: {
208         "User-Agent": "aistudio-build",
209       },
210     },
211   });
212
213   const prompt = `You are a high-precision multi-language virtual compiler and execution runtime engine.
214 Simulate executing or compiling the following ${language} code.
215 Accurately compute standard output (stdout), runtime warnings, standard error (stderr), and process return code.
216
217 Return ONLY a single valid JSON object with this exact schema:
218 {
219   "stdout": "standard output string",
220   "stderr": "error or warning string if any, otherwise empty string",
221   "exitCode": 0,
222   "executionTime": "0.05s",
223   "notes": "brief compiler/interpreter note (e.g. Python 3.12 or gcc 14.1)"
224 }
225
226 Do not include any other markdown or text outside the JSON object.
227
228 Code:
229 \`\`\`${language.toLowerCase()}
230 ${code.slice(0, 10000)}
231 \`\`\`;
232
233   let parsed;
234   try {
235     const { response, modelUsed } = await generateWithFallback(ai, {
236       preferredModel: "gemini-3.8-flash",
237       contents: [{ role: "user", parts: [{ text: prompt }] }],
238       config: {
239         temperature: 0.1,
240         responseType: "application/json",
241       },
242     });
243
244     const raw = response.text || "{}";
245     parsed = JSON.parse(raw);
246     if (!parsed.notes) {
247       parsed.notes = `${language} virtual runtime (${modelUsed})`;
248     }
249   } catch (geminiErr: any) {
250     if (isTransientError(geminiErr)) {
251       return res.status(200).json({
252         stdout: "",
253         stderr: "& b Notice: The virtual execution engine is temporarily under high demand on Google servers.
Please wait a few seconds a",
254         exitCode: 1,
255         executionTime: "0.00s",
256         notes: "High demand spike - Retry available",
257       });
258     }
259     throw geminiErr;
260   }
261
262   return res.json(parsed);
263 } catch (error: any) {
264   console.error("Code runner API error:", error);
265   return res.status(500).json({
266     stdout: "",
267     stderr: `Runner Error: ${error?.message || "Execution failed"}`,
268     exitCode: 1,
269     executionTime: "0.00s",
270     notes: "Execution halted",
271   });
272 }
273 });

```

server.ts (continued)

```
274
275 // Download Full Codebase PDF Endpoint
276 app.get("/api/download/codebase-pdf", (_req, res) => {
277   const pdfPath = path.join(process.cwd(), "public", "CodingSuperHub_Complete_SourceCode.pdf");
278   if (fs.existsSync(pdfPath)) {
279     res.setHeader("Content-Type", "application/pdf");
280     res.setHeader(
281       "Content-Disposition",
282       'attachment; filename="CodingSuperHub_Complete_SourceCode.pdf"'
283     );
284     const stream = fs.createReadStream(pdfPath);
285     stream.pipe(res);
286   } else {
287     res.status(404).json({ error: "Codebase PDF not found. Please generate it first." });
288   }
289 });
290
291 // Vite middleware setup
292 if (process.env.NODE_ENV !== "production") {
293   const vite = await createViteServer({
294     server: { middlewareMode: true },
295     appType: "spa",
296   });
297   app.use(vite.middlewares);
298 } else {
299   const distPath = path.join(process.cwd(), "dist");
300   app.use(express.static(distPath));
301   app.get("/*", (_req, res) => {
302     res.sendFile(path.join(distPath, "index.html"));
303   });
304 }
305
306 app.listen(PORT, "0.0.0.0", () => {
307   console.log(`Coding Super Hub server running on port ${PORT}`);
308 });
309 }
310
311 startServer();
312
```

FILE [8/25]

src/main.tsx

11 lines • 0.2 KB • React DOM client mount point

```
1 import {StrictMode} from 'react';
2 import {createRoot} from 'react-dom/client';
3 import App from './App.tsx';
4 import './index.css';
5
6 createRoot(document.getElementById('root')!).render(
7   <StrictMode>
8     <App />
9   </StrictMode>,
10 );
11
```

FILE [9/25]

src/types.ts

44 lines • 0.7 KB • Global TypeScript interfaces & enums

```
1  export type ToolCategory =
2    | "All"
3    | "Web"
4    | "Text"
5    | "Formatting"
6    | "Converters"
7    | "Security"
8    | "Encryption"
9    | "Hashes"
10   | "Math"
11   | "Time"
12   | "Color"
13   | "Generators"
14   | "HTML"
15   | "CSS"
16   | "JavaScript"
17   | "API"
18   | "Data"
19   | "DevOps"
20   | "Utilities"
21   | "Miscellaneous";
22
23  export interface ToolItem {
24    id: number;
25    title: string;
26    category: ToolCategory;
27    description?: string;
28    keywords?: string[];
29  }
30
31  export interface LanguageInfo {
32    name: string;
33    category: string;
34    starterCode?: string;
35    description?: string;
36    extension?: string;
37  }
38
39  export interface AIChatMessage {
40    role: "user" | "bot";
41    text: string;
42    timestamp?: number;
43  }
44
```

FILE [10/25]

src/index.css

2 lines • 0.0 KB • Tailwind CSS imports & custom styles

```
1 @import "tailwindcss";  
2
```


FILE [11/25]

src/App.tsx

574 lines • 23.6 KB • Master Application layout & state orchestrator

```
1 import React, { useState, useEffect, useRef, useMemo } from "react";
2 import { Navbar } from "../components/Navbar";
3 import { CodeEditor } from "../components/CodeEditor";
4 import { AIAssistant } from "../components/AIAssistant";
5 import { LanguageHub } from "../components/LanguageHub";
6 import { ShortcutsModal } from "../components/ShortcutsModal";
7 import { ToolsTextFormat } from "../components/tools/ToolsTextFormat";
8 import { ToolsMathCalculators } from "../components/tools/ToolsMathCalculators";
9 import { ToolsDataWeb } from "../components/tools/ToolsDataWeb";
10 import { ToolsAdvancedDev } from "../components/tools/ToolsAdvancedDev";
11 import { ToolCategory } from "../types";
12 import { safeStorageGet, safeStorageSet } from "../utils/helpers";
13 import { ALL_TOOLS_METADATA } from "../data/toolsMetadata";
14 import {
15   Wrench,
16   Sparkles,
17   Zap,
18   Globe2,
19   Lock,
20   ChevronDown,
21   Layers,
22   ArrowUp,
23   SearchX,
24   RotateCcw,
25 } from "lucide-react";
26
27 const TOOL_CATEGORY_COUNTS: Record<string, number> = {
28   All: 150,
29   Web: 12,
30   Text: 20,
31   Formatting: 9,
32   Converters: 11,
33   Security: 10,
34   Encryption: 0,
35   Hashes: 7,
36   Math: 21,
37   Time: 6,
38   Color: 4,
39   Generators: 6,
40   HTML: 4,
41   CSS: 17,
42   JavaScript: 12,
43   API: 3,
44   Data: 1,
45   DevOps: 2,
46   Utilities: 5,
47   Miscellaneous: 0,
48 };
49
50 export default function App() {
51   const [darkMode, setDarkMode] = useState<boolean>(() => {
52     try {
53       if (typeof window !== "undefined") {
54         const saved = safeStorageGet("csh_theme");
55         if (saved) return saved === "dark";
56         if (typeof window.matchMedia === "function") {
57           return window.matchMedia("(prefers-color-scheme: dark)").matches;
58         }
59       }
60     } catch {
61       // Fallback cleanly
62     }
63     return false;
64   });
65
66   const [searchQuery, setSearchQuery] = useState("");
```

src/App.tsx (continued)

```

67   const [selectedCategory, setSelectedCategory] = useState<ToolCategory>("All");
68   const [activeTab, setActiveTab] = useState<"tools" | "languages" | "editor" | "ai">("tools");
69   const [showShortcuts, setShowShortcuts] = useState(false);
70   const [showBackToTop, setShowBackToTop] = useState(false);
71
72   // Cross-component communication state
73   const [activeCode, setActiveCode] = useState("");
74   const [activeLanguage, setActiveLanguage] = useState("HTML/CSS/JS");
75   const [incomingAiQuery, setIncomingAiQuery] = useState("");
76   const activeCodeGetterRef = useRef<() => { tab: string; code: string } | null>(null);
77   const applyCodeToEditorRef = useRef<((lang: string, code: string) => void) | null>(null);
78
79   // Dynamically compute category counts from tools registry
80   const dynamicCategoryCounts = useMemo(() => {
81     const counts: Record<string, number> = {
82       All: ALL_TOOLS_METADATA.length + 1, // +1 for Code Editor
83     };
84     for (const t of ALL_TOOLS_METADATA) {
85       counts[t.category] = (counts[t.category] || 0) + 1;
86     }
87     return counts;
88   }, []);
89
90   // Compute number of matching tools for active search & category
91   const matchingToolsCount = useMemo(() => {
92     const q = searchQuery.toLowerCase().trim();
93     return ALL_TOOLS_METADATA.filter((t) => {
94       const matchesCategory =
95         selectedCategory === "All" || selectedCategory === t.category;
96       if (!matchesCategory) return false;
97       if (!q) return true;
98       return (
99         t.title.toLowerCase().includes(q) ||
100         t.category.toLowerCase().includes(q) ||
101         `${t.id}`.includes(q) ||
102         t.keywords.some((k) => k.toLowerCase().includes(q))
103       );
104     }).length;
105   }, [searchQuery, selectedCategory]);
106
107   // Sync dark mode class
108   useEffect(() => {
109     try {
110       if (darkMode) {
111         document.documentElement.classList.add("dark");
112         safeStorageSet("csh_theme", "dark");
113       } else {
114         document.documentElement.classList.remove("dark");
115         safeStorageSet("csh_theme", "light");
116       }
117     } catch {
118       // Fallback cleanly
119     }
120   }, [darkMode]);
121
122   // Global Keyboard Shortcuts
123   useEffect(() => {
124     const handleKeyDown = (e: KeyboardEvent) => {
125       // Toggle Shortcuts modal on "?" (when not typing in an input or textarea)
126       if (
127         e.key === "?" &&
128         !["INPUT", "TEXTAREA"].includes((e.target as HTMLInputElement).tagName)
129       ) {
130         e.preventDefault();
131         setShowShortcuts((prev) => !prev);
132       }
133       // Toggle dark mode (Ctrl/Cmd + Shift + L)
134       if ((e.ctrlKey || e.metaKey) && e.shiftKey && e.key.toLowerCase() === "l") {
135         e.preventDefault();

```

src/App.tsx (continued)

```

136         setDarkMode((prev) => !prev);
137     }
138     // Focus Search (Ctrl/Cmd + Shift + F)
139     if ((e.ctrlKey || e.metaKey) && e.shiftKey && e.key.toLowerCase() === "f") {
140         e.preventDefault();
141         const searchInput = document.getElementById("search-all-tools-input");
142         searchInput?.focus();
143     }
144 };
145 window.addEventListener("keydown", handleKeyDown);
146 return () => window.removeEventListener("keydown", handleKeyDown);
147 }, []);
148
149 // Back to top scroll listener
150 useEffect(() => {
151     const handleScroll = () => {
152         if (window.scrollY > 400) {
153             setShowBackToTop(true);
154         } else {
155             setShowBackToTop(false);
156         }
157     };
158     window.addEventListener("scroll", handleScroll);
159     return () => window.removeEventListener("scroll", handleScroll);
160 }, []);
161
162 const handleSendCodeToAI = (code: string, language: string, promptText?: string) => {
163     const query =
164         promptText ||
165         `Please review, explain, and optimize the following ${language} code:
166         \n\n\`\`\`${language.toLowerCase()}${code}\`\`\`
167         \n\n`;
168     // Scroll to AI assistant
169     document
170         .getElementById("ai-assistant-section")
171         ?.scrollIntoView({ behavior: "smooth" });
172 };
173
174 const handleSelectLanguageFromHub = (lang: string) => {
175     setActiveLanguage(lang);
176     setActiveTab("editor");
177 };
178
179 const handleAskAIFromHub = (prompt: string) => {
180     setIncomingAiQuery(prompt);
181 };
182
183 const scrollToSection = (sectionId: string) => {
184     document.getElementById(sectionId)?.scrollIntoView({ behavior: "smooth" });
185 };
186
187 return (
188     <div className="min-h-screen bg-slate-100 dark:bg-slate-950 text-slate-900 dark:text-slate-100 transition-colors duration-200">
189         <TopNavigation */>
190         <Navbar
191             darkMode={darkMode}
192             onToggleDarkMode={() => setDarkMode(!darkMode)}
193             searchQuery={searchQuery}
194             onSearchChange={setSearchQuery}
195             selectedCategory={selectedCategory}
196             onSelectCategory={setSelectedCategory}
197             onOpenShortcuts={() => setShowShortcuts(true)}
198             categoryCounts={dynamicCategoryCounts}
199             totalTools={ALL_TOOLS_METADATA.length + 1}
200             onJumpToSection={scrollToSection}
201         />
202
203         <main className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6 space-y-8">
204             </* Hero Section Banner */>
205             <section

```

src/App.tsx (continued)

```
205         id="hero-section"
206         className="relative overflow-hidden rounded-3xl bg-gradient-to-br from-blue-600 via-indigo-600 to-
207 slate-900 text-white p-6 sm:p-10"
208         <div className="relative z-10 max-w-3xl space-y-4">
209             <div className="inline-flex items-center space-x-2 px-3 py-1 rounded-full bg-white/10 backdrop-blur-md
210 border border-white/20">
211                 <Sparkles className="w-3.5 h-3.5 text-yellow-300" />
212                 <span>Full-Stack Developer Super Hub 2026</span>
213             </div>
214             <h1 className="text-3xl sm:text-4xl lg:text-5xl font-extrabold tracking-tight text-white leading-tight">
215                 150+ Coding Tools, Multi-Language Sandbox & AI Assistant
216             </h1>
217             <p className="text-sm sm:text-base text-blue-100/90 leading-relaxed">
218                 Instantly format, convert, test, encrypt, and debug code. Includes a live
219                 in-browser execution sandbox for 65+ programming languages, plus Gemini AI coding copilot.
220             </p>
221             <div className="flex flex-wrap items-center gap-3 pt-2">
222                 <button
223                     type="button"
224                     onClick={() => scrollToSection("code-editor-section")}
225                     className="px-4 py-2.5 bg-white text-blue-700 font-bold rounded-xl text-xs sm:text-sm hover:bg-
226 blue-50 transition shadow-sm"
227                 >
228                     Open Code Sandbox
229                 </button>
230                 <button
231                     type="button"
232                     onClick={() => scrollToSection("all-tools-grid")}
233                     className="px-4 py-2.5 bg-blue-700/60 hover:bg-blue-700 text-white font-semibold rounded-xl text-xs
234 sm:text-sm border border-
235                 >
236                     Browse 150+ Tools
237                 </button>
238                 <button
239                     type="button"
240                     onClick={() => scrollToSection("ai-assistant-section")}
241                     className="px-4 py-2.5 bg-indigo-500/40 hover:bg-indigo-500/60 text-white font-semibold rounded-xl
242 text-xs sm:text-sm border-
243                 >
244                     <Sparkles className="w-4 h-4 text-yellow-300" />
245                     <span>Ask AI Copilot</span>
246                 </button>
247             </div>
248             </div>
249             {/* Quick Metrics Bar in Hero */}
250             <div className="mt-8 pt-6 border-t border-white/10 grid grid-cols-2 sm:grid-cols-4 gap-4 text-center">
251                 <div className="p-3 rounded-2xl bg-white/5 backdrop-blur-xs border border-white/10">
252                     <div className="text-xl sm:text-2xl font-black text-white">150+</div>
253                     <div className="text-xs text-blue-200">Developer Tools</div>
254                 </div>
255                 <div className="p-3 rounded-2xl bg-white/5 backdrop-blur-xs border border-white/10">
256                     <div className="text-xl sm:text-2xl font-black text-white">65+</div>
257                     <div className="text-xs text-blue-200">Supported Languages</div>
258                 </div>
259                 <div className="p-3 rounded-2xl bg-white/5 backdrop-blur-xs border border-white/10">
260                     <div className="text-xl sm:text-2xl font-black text-white">Gemini 2.5</div>
261                     <div className="text-xs text-blue-200">Server AI Engine</div>
262                 </div>
263                 <div className="p-3 rounded-2xl bg-white/5 backdrop-blur-xs border border-white/10">
264                     <div className="text-xl sm:text-2xl font-black text-white">100%</div>
265                     <div className="text-xs text-blue-200">Private & Secure</div>
266                 </div>
267             </div>
268         </section>
269
270         {/* Section Tabs Quick Switcher */}
271         <div className="flex items-center justify-between border-b border-slate-200 dark:border-slate-800 pb-3">
272             <div className="flex items-center space-x-2 overflow-x-auto py-1">
273                 <button
```

src/App.tsx (continued)

```

274         type="button"
275         onClick={() => scrollToSection("all-tools-grid")}
276         className="px-3.5 py-1.5 rounded-xl text-xs font-semibold bg-white dark:bg-slate-800 text-slate-800
277 dark:text-slate-200 border
278         <Wrench className="w-3.5 h-3.5 text-blue-500" />
279         <span>150 Tools Grid</span>
280     </button>
281     <button
282         type="button"
283         onClick={() => scrollToSection("code-editor-section")}
284         className="px-3.5 py-1.5 rounded-xl text-xs font-semibold bg-white dark:bg-slate-800 text-slate-800
285 dark:text-slate-200 border
286         <Zap className="w-3.5 h-3.5 text-amber-500" />
287         <span>Multi-Language Sandbox</span>
288     </button>
289     <button
290         type="button"
291         onClick={() => scrollToSection("ai-assistant-section")}
292         className="px-3.5 py-1.5 rounded-xl text-xs font-semibold bg-white dark:bg-slate-800 text-slate-800
293 dark:text-slate-200 border
294         <Sparkles className="w-3.5 h-3.5 text-indigo-500" />
295         <span>AI Coding Assistant</span>
296     </button>
297     <button
298         type="button"
299         onClick={() => scrollToSection("languages-hub-section")}
300         className="px-3.5 py-1.5 rounded-xl text-xs font-semibold bg-white dark:bg-slate-800 text-slate-800
301 dark:text-slate-200 border
302         <Globe2 className="w-3.5 h-3.5 text-emerald-500" />
303         <span>65+ Languages</span>
304     </button>
305 </div>
306 </div>
307
308 /* Core Workspace: Code Sandbox & AI Copilot in Responsive Grid */
309 <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 items-start">
310     /* Left Column: Multi-Language Code Editor (7 cols on lg) */
311     <div className="lg:col-span-7">
312         <CodeEditor
313             onSendToAI={handleSendCodeToAI}
314             initialLanguage={activeLanguage}
315             onLanguageChange={setActiveLanguage}
316             registerActiveCodeGetter={({fn} => {
317                 activeCodeGetterRef.current = fn;
318             })}
319             registerApplyCodeHandler={({fn} => {
320                 applyCodeToEditorRef.current = fn;
321             })}
322         />
323     </div>
324
325     /* Right Column: AI Coding Assistant (5 cols on lg) */
326     <div className="lg:col-span-5">
327         <AIAssistant
328             incomingPrompt={incomingAiQuery}
329             currentCode={activeCode}
330             currentLanguage={activeLanguage}
331             onApplyCodeToEditor={({lang, code} => {
332                 if (applyCodeToEditorRef.current) {
333                     applyCodeToEditorRef.current(lang, code);
334                 }
335                 scrollToSection("code-editor-section");
336             })}
337             getActiveCode={() =>
338                 activeCodeGetterRef.current?.() || {
339                     tab: activeLanguage,
340                     code: activeCode,
341                 }
342         >

```

src/App.tsx (continued)

```

343         />
344     </div>
345 </div>
346
347     {/* 65+ Programming Languages Catalog Section */}
348     <LanguageHub
349       onSelectLanguage={handleSelectLanguageFromHub}
350       onAskAI={handleAskAIFromHub}
351     />
352
353     {/* 150+ Tools Section */}
354     <section id="all-tools-grid" className="space-y-4 pt-4">
355       <div className="flex flex-wrap items-center justify-between gap-3 border-b border-slate-200 dark:border-
356         slate-800 pb-4">
357         <div className="flex items-center space-x-2">
358           <Wrench className="w-5 h-5 text-blue-600 dark:text-blue-400" />
359           <h2 className="text-xl font-bold text-slate-900 dark:text-white">
360             Developer Toolbox (Tools 2 - 150)
361           </h2>
362           <span className="px-2.5 py-0.5 text-xs font-bold rounded-full bg-blue-100 text-blue-800 dark:bg-
363             blue-900 dark:text-blue-200">
364             {selectedCategory === "All" ? "All Categories" : selectedCategory}
365           </span>
366           <span className="px-2 py-0.5 text-xs font-medium rounded-full bg-slate-100 dark:bg-slate-800 text-
367             slate-600 dark:text-slate-400">
368             {matchingToolsCount} {matchingToolsCount === 1 ? "tool" : "tools"}
369           </span>
370         </div>
371         <p className="text-xs text-slate-500 dark:text-slate-400 mt-1">
372           Fast, responsive, zero-latency in-browser utilities for everyday engineering tasks
373         </p>
374       </div>
375
376       {searchQuery && (
377         <div className="text-xs text-slate-500 dark:text-slate-400 flex items-center space-x-2">
378           <span>
379             Filtered by:{" "}
380             <span className="font-semibold text-blue-600 dark:text-blue-400">
381               "{searchQuery}"
382             </span>
383           </span>
384           <button
385             type="button"
386             onClick={() => setSearchQuery("")}
387             className="text-xs text-slate-400 hover:text-rose-500 underline ml-1"
388           >
389             Clear
390           </button>
391         </div>
392       )}
393     </div>
394
395     {/* Tools Grid or Empty State */}
396     {matchingToolsCount === 0 ? (
397       <div className="my-8 p-10 rounded-2xl border border-slate-200 dark:border-slate-800 bg-white dark:bg-
398         slate-900/60 text-center flex flex-col">
399         <div className="w-14 h-14 rounded-2xl bg-blue-50 dark:bg-blue-950/60 flex items-center justify-center
400           text-blue-600 dark:text-slate-400">
401           <SearchX className="w-7 h-7" />
402         </div>
403         <div className="max-w-md">
404           <h3 className="text-base font-bold text-slate-900 dark:text-white">
405             No developer tools found
406           </h3>
407           <p className="text-xs text-slate-500 dark:text-slate-400 mt-1 leading-relaxed">
408             {searchQuery
409               ? `No developer tools matched "${searchQuery}" in category "${selectedCategory}".`
410               : `No developer tools currently registered in category "${selectedCategory}".`}
411           </p>
412         </div>
413       </div>
414     ) : (
415       <div className="flex flex-wrap items-center justify-center gap-2 pt-1">
416         {searchQuery && (

```

src/App.tsx (continued)

```

412         <button
413             type="button"
414             onClick={() => setSearchQuery("")}
415             className="px-3.5 py-1.5 rounded-lg bg-blue-600 hover:bg-blue-500 text-white text-xs font-
semibold shadow-xs>transition
416
417             <RotateCcw className="w-3.5 h-3.5" />
418             <span>Clear Search</span>
419         </button>
420     )}
421     {selectedCategory !== "All" && (
422         <button
423             type="button"
424             onClick={() => setSelectedCategory("All")}
425             className="px-3.5 py-1.5 rounded-lg bg-slate-100 dark:bg-slate-800 hover:bg-slate-200
dark:hover:bg-slate-700 text-slate
426
427             View All Categories (150 tools)
428         </button>
429     )}
430 </div>
431
432 <div className="pt-4 border-t border-slate-100 dark:border-slate-800/80 w-full max-w-lg">
433     <span className="text-[11px] uppercase tracking-wider font-semibold text-slate-400 block mb-2">
434         Popular Searches
435     </span>
436     <div className="flex flex-wrap items-center justify-center gap-1.5">
437         {[
438             "JSON",
439             "Base64",
440             "Regex",
441             "Hash",
442             "UUID",
443             "Color",
444             "SQL",
445             "JWT",
446             "Markdown",
447             "Epoch",
448             "Flexbox",
449         ].map((tag) => (
450             <button
451                 key={tag}
452                 type="button"
453                 onClick={() => {
454                     setSearchQuery(tag);
455                     setSelectedCategory("All");
456                 }}
457                 className="px-2.5 py-1 rounded-md bg-slate-100 dark:bg-slate-800 hover:bg-blue-50
dark:hover:bg-blue-950/60 hover:text
458
459                 {tag}
460             </button>
461         ))}
462     </div>
463 </div>
464 </div>
465 ) : (
466 <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-5">
467     </* Tools 2 - 20 (Text, Formatting, Conversion, Hashes) */>
468     <ToolsTextFormat
469         searchQuery={searchQuery}
470         selectedCategory={selectedCategory}
471     />
472
473     </* Tools 21 - 50 (Math, Calculations, Financial, Conversion) */>
474     <ToolsMathCalculators
475         searchQuery={searchQuery}
476         selectedCategory={selectedCategory}
477     />
478
479     </* Tools 51 - 100 (Data, Web, CSS, JSON, SEO, Dev Cheat Sheets) */>
480     <ToolsDataWeb

```

src/App.tsx (continued)

```
481         searchQuery={searchQuery}
482         selectedCategory={selectedCategory}
483       />
484
485       {/* Tools 101 - 150 (Diff, Ciphers, Advanced Crypto, Modern CSS & JS Snippets) */}
486       <ToolsAdvancedDev
487         searchQuery={searchQuery}
488         selectedCategory={selectedCategory}
489       />
490     </div>
491   )}
492 </section>
493
494   {/* Footer */}
495   <footer className="mt-16 pt-8 pb-12 border-t border-slate-200 dark:border-slate-800 text-center text-xs
496   text-slate-500 dark:text-slate-400 flex flex-wrap items-center justify-center gap-6 font-medium text-slate-700 dark:text-
497   slate-300"> <button
498     type="button"
499     onClick={() => scrollToSection("hero-section")}
500     className="hover:text-blue-600 transition"
501   >
502     Top
503 </button>
504   <button
505     type="button"
506     onClick={() => scrollToSection("code-editor-section")}
507     className="hover:text-blue-600 transition"
508   >
509     Editor Sandbox
510 </button>
511   <button
512     type="button"
513     onClick={() => scrollToSection("ai-assistant-section")}
514     className="hover:text-blue-600 transition"
515   >
516     AI Coding Assistant
517 </button>
518   <button
519     type="button"
520     onClick={() => scrollToSection("languages-hub-section")}
521     className="hover:text-blue-600 transition"
522   >
523     Languages Hub
524 </button>
525   <button
526     type="button"
527     onClick={() => scrollToSection("all-tools-grid")}
528     className="hover:text-blue-600 transition"
529   >
530     150 Tools
531 </button>
532   <button
533     type="button"
534     onClick={() => setShowShortcuts(true)}
535     className="hover:text-blue-600 transition"
536   >
537     Shortcuts (?)
538 </button>
539   <a
540     href="/api/download/codebase-pdf"
541     download="CodingSuperHub_Complete_SourceCode.pdf"
542     className="inline-flex items-center space-x-1 px-2.5 py-0.5 rounded-full bg-emerald-100 dark:bg-
543     emerald-950 text-emerald-700"
544   >
545     <span>📄 Download Codebase PDF (188 Pages)</span>
546   </a>
547 </div>
548 <p>
549   Coding Super Hub &bull; 150+ developer tools, real-time code sandbox & Gemini AI integration.
```


src/App.tsx (continued)

```
550         </p>
551     </footer>
552 </main>
553
554     {/* Floating Back-to-Top Button */}
555     {showBackToTop && (
556         <button
557             type="button"
558             onClick={() => window.scrollTo({ top: 0, behavior: "smooth" })}
559             className="fixed bottom-6 right-6 p-3 rounded-full bg-blue-600 hover:bg-blue-500 text-white shadow-lg
z-40 transition-hover:scale-105"
560             title="Back to Top"
561             >
562             <ArrowUp className="w-5 h-5" />
563         </button>
564     )}
565
566     {/* Keyboard Shortcuts Modal */}
567     <ShortcutsModal
568         isOpen={showShortcuts}
569         onClose={() => setShowShortcuts(false)}
570     />
571 </div>
572 );
573 }
574
```

FILE [12/25]

src/components/Navbar.tsx

213 lines • 7.7 KB • Navigation, search, category filter bar

```
1 import React from "react";
2 import { ToolCategory } from "../types";
3 import {
4   Code2,
5   Search,
6   Moon,
7   Sun,
8   Bot,
9   Keyboard,
10  Globe2,
11  Sparkles,
12  Terminal,
13  FileDown,
14 } from "lucide-react";
15
16 interface NavbarProps {
17   searchQuery: string;
18   onSearchChange: (query: string) => void;
19   selectedCategory: ToolCategory;
20   onSelectCategory: (category: ToolCategory) => void;
21   categoryCounts?: Record<string, number>;
22   darkMode: boolean;
23   onToggleDarkMode: () => void;
24   onOpenShortcuts: () => void;
25   onJumpToSection?: (sectionId: string) => void;
26   totalTools?: number;
27 }
28
29 const CATEGORIES: ToolCategory[] = [
30   "All",
31   "Web",
32   "Text",
33   "Formatting",
34   "Converters",
35   "Security",
36   "Encryption",
37   "Hashes",
38   "Math",
39   "Time",
40   "Color",
41   "Generators",
42   "HTML",
43   "CSS",
44   "JavaScript",
45   "API",
46   "Data",
47   "DevOps",
48   "Utilities",
49   "Miscellaneous",
50 ];
51
52 export const Navbar: React.FC<NavbarProps> = ({
53   searchQuery,
54   onSearchChange,
55   selectedCategory,
56   onSelectCategory,
57   categoryCounts = {},
58   darkMode,
59   onToggleDarkMode,
60   onOpenShortcuts,
61   onJumpToSection,
62   totalTools = 150,
63 }) => {
64   return (
65     <header className="sticky top-0 z-40 bg-slate-900/95 backdrop-blur-md border-b border-slate-800 text-slate-100 shadow-lg">
66       <div className="max-w-xl mx-auto px-4 sm:px-6 py-3">
```

src/components/Navbar.tsx (continued)

```

67      {/* Top brand & actions bar */}
68      <div className="flex flex-wrap items-center justify-between gap-3">
69        <div className="flex items-center space-x-3">
70          <div className="w-10 h-10 rounded-xl bg-blue-600 flex items-center justify-center text-white shadow-md
71            shadow-blue-500/20 terminal className="w-6 h-6" />
72        </div>
73        <div>
74          <div className="flex items-center space-x-2">
75            <h1 className="text-xl font-bold tracking-tight text-white">
76              Coding Super Hub
77            </h1>
78            <span className="px-2 py-0.5 text-xs font-semibold rounded-full bg-blue-500/20 text-blue-300 border
79              border-blue-500/30">{TotalTools} Tools
80            </span>
81            <span className="hidden sm:inline-flex px-2 py-0.5 text-xs font-semibold rounded-full bg-
82              emerald-500/20 text-emerald-300 border
83            </span>
84          </div>
85          <p className="text-xs text-slate-400">
86            All-in-one developer workspace, editor, and 65+ language hub
87          </p>
88        </div>
89      </div>
90
91      <div className="flex items-center space-x-2">
92        <button
93          type="button"
94          onClick={() => onJumpToSection?.("ai-assistant-section")}
95          className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-indigo-600/30 hover:bg-
96            indigo-600/50 text-white border
97          >
98          <Bot className="w-3.5 h-3.5" />
99          <span>Super AI</span>
100        </button>
101
102        <button
103          type="button"
104          onClick={() => onJumpToSection?.("code-editor-section")}
105          className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-slate-800 hover:bg-slate-700 text-
106            slate-200 border border-s
107          >
108          <Code2 className="w-3.5 h-3.5" />
109          <span>Editor</span>
110        </button>
111
112        <button
113          type="button"
114          onClick={() => onJumpToSection?.("languages-hub-section")}
115          className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-slate-800 hover:bg-slate-700 text-
116            slate-200 border border-s
117          >
118          <Globe2 className="w-3.5 h-3.5" />
119          <span>Languages</span>
120        </button>
121
122        <a
123          href="/api/download/codebase-pdf"
124          download="CodingSuperHub_Complete_SourceCode.pdf"
125          className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-emerald-600/30 hover:bg-
126            emerald-600/50 text-white border
127          id="download-codebase-pdf-btn"
128          >
129          <FileDown className="w-3.5 h-3.5" />
130          <span>Code PDF</span>
131        </a>
132
133        <button
134          type="button"
135          onClick={onOpenShortcuts}

```

src/components/Navbar.tsx (continued)

```

136         className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700 text-slate-300 border border-slate-700
137     transition"
138         title="Keyboard shortcuts (?)"
139         aria-label="Keyboard shortcuts"
140     >
141         <Keyboard className="w-4 h-4" />
142     </button>
143
144     <button
145         type="button"
146         onClick={onToggleDarkMode}
147         className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700 text-slate-300 border border-slate-700
148     transition"
149         title={darkMode ? "Switch to Light Mode" : "Switch to Dark Mode"}
150         aria-label="Toggle Theme"
151     >
152         {darkMode ? (
153             <Sun className="w-4 h-4 text-amber-400" />
154         ) : (
155             <Moon className="w-4 h-4 text-blue-400" />
156         )}
157     </button>
158 </div>
159 </div>
160
161 {/* Global Search Bar */}
162 <div className="mt-3 relative">
163     <div className="absolute inset-y-0 left-0 pl-3 flex items-center pointer-events-none text-slate-400">
164         <Search className="w-4 h-4" />
165     </div>
166     <input
167         id="toolSearchInput"
168         type="search"
169         value={searchQuery}
170         onChange={(e) => onSearchChange(e.target.value)}
171         placeholder="Search 150 tools by name, utility (e.g. jwt, hash, base64, bmi, regex, uuid, flexbox)..."
172         className="w-full pl-9 pr-24 py-2 bg-slate-800/90 text-sm text-slate-100 placeholder-slate-400 rounded-
173     lg border border-slate-700
174     <div className="absolute inset-y-0 right-0 pr-3 flex items-center pointer-events-none">
175         <kbd className="hidden sm:inline-block px-2 py-0.5 text-[10px] font-mono text-slate-400 bg-slate-900
176     border border-slate-700 rounded-sm" />
177     </div>
178 </div>
179
180 {/* Category Pills Scroller */}
181 <div className="mt-2.5 flex items-center gap-1.5 overflow-x-auto pb-1 scrollbar-none text-xs">
182     {CATEGORIES.map((cat) => {
183         const isSelected = selectedCategory === cat;
184         const count = cat === "All" ? totalTools : (categoryCounts?.[cat] ?? 0);
185         return (
186             <button
187                 key={cat}
188                 type="button"
189                 onClick={() => onSelectCategory(cat)}
190                 className={`whitespace-nowrap px-3 py-1 rounded-full font-medium transition flex items-center
191             gap-1.5 ${
192                 isSelected
193                     ? "bg-blue-600 text-white shadow-sm"
194                     : "bg-slate-800 text-slate-300 hover:bg-slate-700 border border-slate-700/60"
195                 }`}
196             >
197                 <span>{cat}</span>
198                 <span
199                     className={`text-[10px] px-1.5 py-0.2 rounded-full ${
200                         isSelected
201                             ? "bg-blue-800/80 text-blue-100"
202                             : "bg-slate-700 text-slate-400"
203                         }`}
204                 >
205                     {count}
206                 </span>

```

src/components/Navbar.tsx (continued)

```
205         </button>
206     );
207     }}
208 </div>
209 </div>
210 </header>
211 );
212 };
213
```

FILE [13/25]

src/components/CodeEditor.tsx 969 lines • 35.5 KB • Multi-language editor, runner & sandbox

```
1 import React, { useState, useRef, useEffect } from "react";
2 import {
3   Play,
4   Copy,
5   Trash2,
6   Check,
7   Sparkles,
8   Download,
9   Upload,
10  Terminal,
11  Eye,
12  RefreshCw,
13  FileCode,
14  AlertCircle,
15  CheckCircle2,
16 } from "lucide-react";
17 import { LANGUAGE_CATALOG } from "../data/languages";
18
19 interface CodeEditorProps {
20   onSendToAI?: (code: string, language: string, promptText?: string) => void;
21   selectedLanguage?: string;
22   initialLanguage?: string;
23   onLanguageChange?: (lang: string) => void;
24   registerActiveCodeGetter?: (fn: () => { tab: string; code: string }) => void;
25   registerApplyCodeHandler?: (fn: (lang: string, code: string) => void) => void;
26 }
27
28 interface ConsoleLogItem {
29   id: string;
30   type: "info" | "warn" | "error" | "stdout" | "system";
31   text: string;
32   time: string;
33 }
34
35 export const CodeEditor: React.FC<CodeEditorProps> = ({
36   onSendToAI,
37   selectedLanguage,
38   initialLanguage = "HTML",
39   onLanguageChange,
40   registerActiveCodeGetter,
41   registerApplyCodeHandler,
42 }) => {
43   const [internalLang, setInternalLang] = useState(initialLanguage);
44   const currentLang = selectedLanguage || internalLang;
45
46   // Web tabs
47   const [activeTab, setActiveTab] = useState<string>("html");
48   const [htmlCode, setHtmlCode] = useState(
49     `<div class="container">\n  <h1>ðŸ‘Œ Hello Coding Super Hub!</h1>\n  <p>Live, multi-language sandbox with instant
50 execution and AI copilot
51 `);
52   const [cssCode, setCssCode] = useState(
53     `body {\n  font-family: system-ui, sans-serif;\n  padding: 24px;\n  background: #f8fafc;\n  color: #0f172a;
54 \n}\n.container {\n  max-widt
55 `);
56   const [jsCode, setJsCode] = useState(
57     `const btn = document.getElementById("btn");\nbtn.addEventListener("click", () => {\n  console.log("Button
58 clicked!");\n  alert("ðŸ’Š Java
59 `);
60
61   // Non-web language codes storage
62   const [customCodes, setCustomCodes] = useState<Record<string, string>>({});
63
64   // UI state
65   const [copied, setCopied] = useState(false);
66   const [exported, setExported] = useState(false);
67   const [lastRunTime, setLastRunTime] = useState<string>("");
68   const [srcDoc, setSrcDoc] = useState<string>("");
69   const [outputTab, setOutputTab] = useState<"preview" | "terminal">("preview");
```

src/components/CodeEditor.tsx (continued)

```
67   const [terminalLogs, setTerminalLogs] = useState<ConsoleLogItem[]>([]);
68   const [isRunning, setIsRunning] = useState(false);
69   const [toastMsg, setToastMsg] = useState<string | null>(null);
70   const [isDragging, setIsDragging] = useState(false);
71   const [cursorPos, setCursorPos] = useState({ line: 1, col: 1 });
72
73   // Refs
74   const fileInputRef = useRef<HTMLInputElement>(null);
75   const textareaRef = useRef<HTMLTextAreaElement>(null);
76   const lineNumbersRef = useRef<HTMLDivElement>(null);
77
78   const isWebLanguage = (lang: string) => {
79     const l = lang.toLowerCase().trim();
80     return l === "html" || l === "css" || l === "javascript";
81   };
82
83   const showToast = (msg: string) => {
84     setToastMsg(msg);
85     setTimeout(() => setToastMsg(null), 3500);
86   };
87
88   // Get active code string
89   const getCurrentCode = (): string => {
90     if (isWebLanguage(currentLang)) {
91       if (activeTab === "html") return htmlCode;
92       if (activeTab === "css") return cssCode;
93       return jsCode;
94     }
95     return customCodes[currentLang] || "";
96   };
97
98   // Set active code string
99   const setCurrentCode = (newCode: string) => {
100     if (isWebLanguage(currentLang)) {
101       if (activeTab === "html") setHtmlCode(newCode);
102       else if (activeTab === "css") setCssCode(newCode);
103       else setJsCode(newCode);
104     } else {
105       setCustomCodes((prev) => ({ ...prev, [currentLang]: newCode }));
106     }
107   };
108
109   // Expose active code getter to parent for AI assistant
110   useEffect(() => {
111     registerActiveCodeGetter?.(() => {
112       const code = getCurrentCode();
113       const tab = isWebLanguage(currentLang)
114         ? activeTab.toUpperCase()
115         : currentLang;
116       return { tab, code };
117     });
118   }, [currentLang, activeTab, htmlCode, cssCode, jsCode, customCodes, registerActiveCodeGetter]);
119
120   // Expose code applier so AI can paste snippets
121   useEffect(() => {
122     registerApplyCodeHandler?.((lang: string, code: string) => {
123       const l = lang.toLowerCase().trim();
124       if (l === "html") {
125         setActiveTab("html");
126         setHtmlCode(code);
127         handleLanguageSelect("HTML");
128       } else if (l === "css") {
129         setActiveTab("css");
130         setCssCode(code);
131         handleLanguageSelect("CSS");
132       } else if (l === "javascript" || l === "js") {
133         setActiveTab("js");
134         setJsCode(code);
135         handleLanguageSelect("JavaScript");
```

src/components/CodeEditor.tsx (continued)

```

136     } else {
137       // Find matching language
138       const match = LANGUAGE_CATALOG.find(
139         (item) => item.name.toLowerCase() === l
140       );
141       const targetLang = match ? match.name : currentLang;
142       setInternalLang(targetLang);
143       onLanguageChange?.(targetLang);
144       setCustomCodes((prev) => ({ ...prev, [targetLang]: code }));
145     }
146     showToast(`Loaded ${lang} snippet into Code Editor`);
147   });
148 }, [currentLang, onLanguageChange, registerApplyCodeHandler]);
149
150 // Listen to sandbox postMessages for console output
151 useEffect(() => {
152   const handleMessage = (e: MessageEvent) => {
153     if (e.data && e.data.type === "csh_sandbox_console") {
154       const item: ConsoleLogItem = {
155         id: Math.random().toString(36).substring(2, 9),
156         type: e.data.level === "error" ? "error" : e.data.level === "warn" ? "warn" : "info",
157         text: e.data.text || "",
158         time: e.data.time || new Date().toLocaleTimeString(),
159       };
160       setTerminalLogs((prev) => [...prev.slice(-99), item]);
161     }
162   };
163   window.addEventListener("message", handleMessage);
164   return () => window.removeEventListener("message", handleMessage);
165 }, []);
166
167 // Sync line numbers scroll with textarea scroll
168 const handleScroll = () => {
169   if (textareaRef.current && lineNumbersRef.current) {
170     lineNumbersRef.current.scrollTop = textareaRef.current.scrollTop;
171   }
172 };
173
174 // Cursor tracking
175 const updateCursorPosition = () => {
176   if (!textareaRef.current) return;
177   const text = textareaRef.current.value.substring(
178     0,
179     textareaRef.current.selectionStart
180   );
181   const lines = text.split("\n");
182   setCursorPos({
183     line: lines.length,
184     col: lines[lines.length - 1].length + 1,
185   });
186 };
187
188 // Handle Tab key indentation
189 const handleKeyDown = (e: React.KeyboardEvent<HTMLTextAreaElement>) => {
190   if (e.key === "Tab") {
191     e.preventDefault();
192     const textarea = e.currentTarget;
193     const start = textarea.selectionStart;
194     const end = textarea.selectionEnd;
195     const val = textarea.value;
196
197     const updated = val.substring(0, start) + "  " + val.substring(end);
198     setCurrentCode(updated);
199
200     setTimeout(() => {
201       if (textareaRef.current) {
202         textareaRef.current.selectionStart = textareaRef.current.selectionEnd = start + 2;
203         updateCursorPosition();
204       }

```


src/components/CodeEditor.tsx (continued)

```

205     }, 0);
206   }
207 };
208
209 // Execution: Native web vs Multi-language virtual runner
210 const runCode = async () => {
211   const nowStr = new Date().toLocaleTimeString();
212   setLastRunTime(nowStr);
213
214   if (isWebLanguage(currentLang)) {
215     // In-browser Web Sandbox
216     try {
217       const consoleTrap = `
218         <script>
219           (function() {
220             function formatArg(a) {
221               if (typeof a === 'object') {
222                 try { return JSON.stringify(a, null, 2); } catch(e) { return String(a); }
223               }
224               return String(a);
225             }
226             var _log = console.log, _warn = console.warn, _error = console.error;
227             function send(level, args) {
228               try {
229                 var str = Array.from(args).map(formatArg).join(' ');
230                 window.parent.postMessage({ type: 'csh_sandbox_console', level: level, text: str, time: new
231 Date().toLocaleTimeString() }, '*');
232               } catch(e) {}
233             }
234             console.log = function() { send('info', arguments); _log.apply(console, arguments); };
235             console.warn = function() { send('warn', arguments); _warn.apply(console, arguments); };
236             console.error = function() { send('error', arguments); _error.apply(console, arguments); };
237             window.onerror = function(msg, url, line) {
238               send('error', ['Uncaught Error (' + line + '): ' + msg]);
239             };
240             })();
241           <\/script>`;
242
243       const generated = `<!DOCTYPE html>
244 <html>
245   <head>
246     <meta charset="utf-8">
247     <style>${cssCode}<\/style>
248   <\/head>
249   <body>
250     ${htmlCode}
251     ${consoleTrap}
252     <script>
253       try {
254         ${jsCode.replace(/<\/script>/gi, "<\/script")}
255       } catch(err) {
256         console.error(err);
257         var errDiv = document.createElement("div");
258         errDiv.style.color = "#ef4444";
259         errDiv.style.padding = "10px";
260         errDiv.style.background = "#fee2e2";
261         errDiv.style.borderRadius = "6px";
262         errDiv.style.marginTop = "12px";
263         errDiv.textContent = "Runtime Error: " + (err && err.message ? err.message : String(err));
264         document.body.appendChild(errDiv);
265       }
266     <\/script>
267   <\/body>
268 <\/html>`;
269
270     setSrcDoc(generated);
271   } catch (err) {
272     console.error(err);
273   }
274 } else {
275   // Non-Web Language (Python, C++, Rust, Go, Java, Bash, etc.)

```

src/components/CodeEditor.tsx (continued)

```
274     const codeToRun = getCurrentCode();
275     if (!codeToRun.trim()) {
276       showToast("Please write code before running.");
277       return;
278     }
279
280     setIsRunning(true);
281     setOutputTab("terminal");
282     setTerminalLogs((prev) => [
283       ...prev,
284       {
285         id: Math.random().toString(36).substring(2, 9),
286         type: "system",
287         text: `🔧 Compiling & Executing ${currentLang} script...`,
288         time: nowStr,
289       },
290     ]);
291
292     try {
293       const res = await fetch("/api/code/run", {
294         method: "POST",
295         headers: { "Content-Type": "application/json" },
296         body: JSON.stringify({
297           language: currentLang,
298           code: codeToRun,
299         }),
300       });
301       const data = await res.json();
302
303       const newLogs: ConsoleLogItem[] = [];
304       if (data.notes) {
305         newLogs.push({
306           id: Math.random().toString(36).substring(2, 9),
307           type: "system",
308           text: `[Environment: ${data.notes}]`,
309           time: nowStr,
310         });
311       }
312       if (data.stdout && data.stdout.trim()) {
313         newLogs.push({
314           id: Math.random().toString(36).substring(2, 9),
315           type: "stdout",
316           text: data.stdout.trim(),
317           time: nowStr,
318         });
319       }
320       if (data.stderr && data.stderr.trim()) {
321         newLogs.push({
322           id: Math.random().toString(36).substring(2, 9),
323           type: "error",
324           text: data.stderr.trim(),
325           time: nowStr,
326         });
327       }
328       newLogs.push({
329         id: Math.random().toString(36).substring(2, 9),
330         type: "system",
331         text: `🏁 Process finished with exit code ${data.exitCode ?? 0} (${data.executionTime || "0.04s"})`,
332         time: nowStr,
333       });
334
335       setTerminalLogs((prev) => [...prev, ...newLogs]);
336     } catch (err: any) {
337       setTerminalLogs((prev) => [
338         ...prev,
339         {
340           id: Math.random().toString(36).substring(2, 9),
341           type: "error",
342           text: `Execution failed: ${err?.message || String(err)}`,
```

src/components/CodeEditor.tsx (continued)

```
343         time: nowStr,
344     },
345     });
346     } finally {
347         setIsRunning(false);
348     }
349 }
350 };
351
352 useEffect(() => {
353     runCode();
354     // eslint-disable-next-line react-hooks/exhaustive-deps
355 }, []);
356
357 const handleCopy = () => {
358     let fullBundle: string;
359     if (isWebLanguage(currentLang)) {
360         fullBundle = `<!-- HTML -->\n${htmlCode}\n\n/* CSS */\n${cssCode}\n\n// JavaScript\n${jsCode}`;
361     } else {
362         fullBundle = getCurrentCode();
363     }
364     navigator.clipboard.writeText(fullBundle);
365     setCopied(true);
366     setTimeout(() => setCopied(false), 1500);
367 };
368
369 const isTextLanguage = (langName: string): boolean => {
370     if (!langName) return false;
371     const lower = langName.trim().toLowerCase();
372     if (
373         lower === "text" ||
374         lower === "plain text" ||
375         lower === "plaintext" ||
376         lower === "txt" ||
377         lower === "markdown" ||
378         lower === "md" ||
379         lower === "documentation" ||
380         lower === "notes"
381     ) {
382         return true;
383     }
384
385     const catalogItem = LANGUAGE_CATALOG.find(
386         (l) => l.name.toLowerCase() === lower
387     );
388     if (catalogItem) {
389         const cat = catalogItem.category.toLowerCase();
390         if (cat.includes("documentation") || cat.includes("text")) {
391             return true;
392         }
393     }
394     return false;
395 };
396
397 const handleExportFile = () => {
398     const currentCode = getCurrentCode();
399     const isText = isTextLanguage(currentLang);
400     const ext = isText ? "txt" : "code";
401     const fileName = `code.${ext}`;
402
403     const blob = new Blob([currentCode], { type: "text/plain;charset=utf-8" });
404     const url = URL.createObjectURL(blob);
405     const link = document.createElement("a");
406     link.href = url;
407     link.download = fileName;
408     document.body.appendChild(link);
409     link.click();
410     document.body.removeChild(link);
411     URL.revokeObjectURL(url);

```

src/components/CodeEditor.tsx (continued)

```
412
413     setExported(true);
414     setTimeout(() => setExported(false), 1500);
415 };
416
417 // Import / Open file
418 const processUploadedFile = (file: File) => {
419     const reader = new FileReader();
420     reader.onload = (e) => {
421         const content = e.target?.result as string;
422         if (typeof content !== "string") return;
423
424         const fileName = file.name.toLowerCase();
425         let detectedLang = currentLang;
426
427         // Extension to language mapper
428         if (fileName.endsWith(".html") || fileName.endsWith(".htm")) {
429             detectedLang = "HTML";
430             setActiveTab("html");
431             setHtmlCode(content);
432         } else if (fileName.endsWith(".css")) {
433             detectedLang = "CSS";
434             setActiveTab("css");
435             setCssCode(content);
436         } else if (
437             fileName.endsWith(".js") ||
438             fileName.endsWith(".mjs") ||
439             fileName.endsWith(".cjs")
440         ) {
441             detectedLang = "JavaScript";
442             setActiveTab("js");
443             setJsCode(content);
444         } else {
445             const extMap: Record<string, string> = {
446                 ".py": "Python",
447                 ".ts": "TypeScript",
448                 ".tsx": "TypeScript",
449                 ".rs": "Rust",
450                 ".go": "Go",
451                 ".cpp": "C++",
452                 ".cc": "C++",
453                 ".c": "C",
454                 ".cs": "C#",
455                 ".java": "Java",
456                 ".kt": "Kotlin",
457                 ".swift": "Swift",
458                 ".php": "PHP",
459                 ".rb": "Ruby",
460                 ".sh": "Bash",
461                 ".sql": "SQL",
462                 ".json": "JSON",
463                 ".md": "Markdown",
464                 ".yaml": "YAML",
465                 ".yml": "YAML",
466                 ".txt": "Plain Text",
467             };
468
469             for (const [ext, lName] of Object.entries(extMap)) {
470                 if (fileName.endsWith(ext)) {
471                     detectedLang = lName;
472                     break;
473                 }
474             }
475             setCustomCodes((prev) => ({ ...prev, [detectedLang]: content }));
476         }
477
478         setInternalLang(detectedLang);
479         onLanguageChange?.(detectedLang);
480         showToast(`Imported "${file.name}" (${content.split("\n").length} lines)`);
```

src/components/CodeEditor.tsx (continued)

```
481     };
482     reader.readAsText(file);
483   };
484
485   const handleFileInputChange = (e: React.ChangeEvent<HTMLInputElement>) => {
486     const files = e.target.files;
487     if (files && files[0]) {
488       processUploadedFile(files[0]);
489     }
490     // reset input
491     if (fileInputRef.current) fileInputRef.current.value = "";
492   };
493
494   // Drag and drop handlers
495   const handleDragOver = (e: React.DragEvent) => {
496     e.preventDefault();
497     setIsDragging(true);
498   };
499
500   const handleDragLeave = () => {
501     setIsDragging(false);
502   };
503
504   const handleDrop = (e: React.DragEvent) => {
505     e.preventDefault();
506     setIsDragging(false);
507     if (e.dataTransfer.files && e.dataTransfer.files[0]) {
508       processUploadedFile(e.dataTransfer.files[0]);
509     }
510   };
511
512   useEffect(() => {
513     if (initialLanguage && initialLanguage !== internalLang) {
514       setInternalLang(initialLanguage);
515       loadLanguageStarter(initialLanguage);
516     }
517     // eslint-disable-next-line react-hooks/exhaustive-deps
518   }, [initialLanguage]);
519
520   const loadLanguageStarter = (langName: string) => {
521     const found = LANGUAGE_CATALOG.find(
522       (l) => l.name.toLowerCase() === langName.toLowerCase()
523     );
524     if (!found || !found.starterCode) return;
525
526     if (langName.toLowerCase() === "html") {
527       setActiveTab("html");
528       setHtmlCode(found.starterCode);
529     } else if (langName.toLowerCase() === "css") {
530       setActiveTab("css");
531       setCssCode(found.starterCode);
532     } else if (langName.toLowerCase() === "javascript") {
533       setActiveTab("js");
534       setJsCode(found.starterCode);
535     } else {
536       setCustomCodes((prev) => ({
537         ...prev,
538         [langName]: prev[langName] || found.starterCode || "",
539       }));
540     }
541   };
542
543   const handleClear = () => {
544     if (isWebLanguage(currentLang)) {
545       if (activeTab === "html") setHtmlCode("");
546       else if (activeTab === "css") setCssCode("");
547       else setJsCode("");
548     } else {
549       setCustomCodes((prev) => ({ ...prev, [currentLang]: "" }));
```

src/components/CodeEditor.tsx (continued)

```

550     }
551   };
552
553   const handleLanguageSelect = (langName: string) => {
554     setInternalLang(langName);
555     onLanguageChange?.(langName);
556     loadLanguageStarter(langName);
557   };
558
559   const activeCode = getCurrentCode();
560   const lineCount = Math.max(1, activeCode.split("\n").length);
561   const charCount = activeCode.length;
562
563   return (
564     <div
565       id="code-editor-section"
566       className={`relative bg-white dark:bg-slate-800/90 rounded-2xl border ${
567         isDragging
568           ? "border-dashed border-blue-500 ring-2 ring-blue-500/20"
569           : "border-slate-200 dark:border-slate-700"
570       } p-5 shadow-sm my-6 transition`}
571       onDragOver={handleDragOver}
572       onDragLeave={handleDragLeave}
573       onDrop={handleDrop}
574     >
575       {/* Hidden File Input for Open File */}
576       <input
577         type="file"
578         ref={fileInputRef}
579         onChange={handleFileInputChange}
580         className="hidden"
581         accept=".txt,.html,.htm,.css,.js,.mjs,.cjs,.ts,.tsx,.py,.cpp,.c,.cs,.go,.rs,.java,.kt,.swift,.php,.rb,.sh,.json,.md,.sql,.yaml,.yml"
582       />
583
584       {/* Drag overlay notice */}
585       {isDragging && (
586         <div className="absolute inset-0 z-50 rounded-2xl bg-blue-600/10 backdrop-blur-xs flex items-center justify-center pointer-events-none" style={{width: 100%, height: 100%}}>
587           <div className="px-5 py-3 rounded-xl bg-blue-600 text-white font-medium text-sm shadow-xl flex items-center space-x-2">
588             <div className="w-5 h-5 animate-bounce" />
589             <span>Drop file here to import into Code Editor</span>
590           </div>
591         </div>
592       )}
593
594       {/* Toast message */}
595       {toastMsg && (
596         <div className="absolute top-3 right-5 z-40 px-3.5 py-2 rounded-xl bg-slate-900 text-white text-xs font-medium shadow-lg backdrop-blur" style={{width: 300px, height: 40px}}>
597           <span>{toastMsg}</span>
598         </div>
599       )}
600
601       {/* Top Header */}
602       <div className="flex flex-wrap items-center justify-between gap-3 border-b border-slate-200 dark:border-slate-700/80 pb-4">
603         <div className="flex items-center space-x-2">
604           <span className="px-2 py-0.5 rounded text-xs font-mono font-bold bg-blue-100 text-blue-800 dark:bg-blue-900/60 dark:text-blue-30" />
605           <span>1</span>
606         </span>
607         <h2 className="text-lg font-bold text-slate-900 dark:text-white">
608           Online Code Editor & Runner
609         </h2>
610         <span
611           className={`px-2 py-0.5 text-xs font-semibold rounded-full border ${
612             isWebLanguage(currentLang)
613               ? "bg-emerald-50 text-emerald-700 dark:bg-emerald-950/60 dark:text-emerald-300 border-emerald-200 dark:border-emerald-800"
614               : "bg-indigo-50 text-indigo-700 dark:bg-indigo-950/60 dark:text-indigo-300 border-indigo-200 dark:border-indigo-800"
615             }`}
616         >

```

src/components/CodeEditor.tsx (continued)

```

{isWebLanguage(currentLang) ? "Web Sandbox" : "AI Multi-Language Runner"}
</span>
</div>
<p className="text-xs text-slate-500 dark:text-slate-400 mt-1">
  {isWebLanguage(currentLang)
    ? "Live browser execution for HTML, CSS & JavaScript with console interceptor"
    : `Write & run ${currentLang} with virtual compiler output & stdout/stderr console`}
</p>
</div>

<div className="flex items-center gap-2">
  <select
    value={currentLang}
    onChange={(e) => handleLanguageSelect(e.target.value)}
    className="text-xs px-3 py-1.5 rounded-lg bg-slate-100 dark:bg-slate-900 text-slate-800 dark:text-slate-200 border border-slate-200"
    >
    {LANGUAGE_CATALOG.map((lang) => (
      <option key={lang.name} value={lang.name}>
        {lang.name} ({lang.category})
      </option>
    ))}
  </select>
</div>
</div>

/* Editor Tabs & Controls */
<div className="flex flex-wrap items-center justify-between gap-3 mt-4">
  /* Language Tabs */
  {isWebLanguage(currentLang) ? (
    <div className="flex items-center space-x-1 p-1 bg-slate-100 dark:bg-slate-900/90 rounded-xl border border-slate-200 dark:border-slate-800">
      <button
        type="button"
        onClick={() => setActiveTab("html")}
        className={`px-3 py-1.5 rounded-lg transition ${
          activeTab === "html"
            ? "bg-blue-600 text-white shadow-sm"
            : "text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white"
        }`}
      >
        HTML
      </button>
      <button
        type="button"
        onClick={() => setActiveTab("css")}
        className={`px-3 py-1.5 rounded-lg transition ${
          activeTab === "css"
            ? "bg-blue-600 text-white shadow-sm"
            : "text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white"
        }`}
      >
        CSS
      </button>
      <button
        type="button"
        onClick={() => setActiveTab("js")}
        className={`px-3 py-1.5 rounded-lg transition ${
          activeTab === "js"
            ? "bg-blue-600 text-white shadow-sm"
            : "text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white"
        }`}
      >
        JavaScript
      </button>
    </div>
  ) : (
    <div className="flex items-center space-x-2">
      <div className="px-3 py-1.5 rounded-lg bg-blue-600 text-white text-xs font-semibold flex items-center space-x-1.5 shadow-sm">
        <code className="w-3.5 h-3.5" />
        <span>{currentLang} Code</span>
      </div>
    </div>
  )}
</div>

```

src/components/CodeEditor.tsx (continued)

```

688         </div>
689         <button
690             type="button"
691             onClick={() => handleLanguageSelect("HTML")}
692             className="text-xs text-slate-500 dark:text-slate-400 hover:text-blue-600 underline"
693         >
694             Switch to Web (HTML/CSS/JS)
695         </button>
696     </div>
697 )}
698
699     {/* Action Buttons Toolbar */}
700     <div className="flex flex-wrap items-center gap-2">
701         {/* Run Code Button */}
702         <button
703             type="button"
704             onClick={runCode}
705             disabled={isRunning}
706             className="flex items-center space-x-1.5 px-3.5 py-1.5 rounded-lg bg-blue-600 hover:bg-blue-500
disabled:opacity-50 text-white"
707             title="Execute code (Ctrl+Enter)"
708         >
709             {isRunning ? (
710                 <RefreshCw className="w-3.5 h-3.5 animate-spin" />
711             ) : (
712                 <Play className="w-3.5 h-3.5 fill-current" />
713             )}
714             <span>{isRunning ? "Running..." : "Run Code"}</span>
715         </button>
716
717         {/* Copy Code */}
718         <button
719             type="button"
720             onClick={handleCopy}
721             className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-slate-100 dark:bg-slate-700 hover:bg-
slate-200 dark:hover:bg-
722             title="Copy code"
723         >
724             {copied ? (
725                 <Check className="w-3.5 h-3.5 text-emerald-500" />
726             ) : (
727                 <Copy className="w-3.5 h-3.5" />
728             )}
729             <span>{copied ? "Copied!" : "Copy Code"}</span>
730         </button>
731
732         {/* Open / Import File */}
733         <button
734             type="button"
735             id="openFileBtn"
736             onClick={() => fileInputRef.current?.click()}
737             className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-slate-100 dark:bg-slate-700 hover:bg-
slate-200 dark:hover:bg-
738             title="Import or drag & drop a code file"
739         >
740             <Upload className="w-3.5 h-3.5" />
741             <span>Open File</span>
742         </button>
743
744         {/* Export as File */}
745         <button
746             type="button"
747             id="exportAsFileBtn"
748             aria-label="Export as File"
749             onClick={handleExportFile}
750             className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-slate-100 dark:bg-slate-700 hover:bg-
slate-200 dark:hover:bg-
751             title="Export as .${isTextLanguage(currentLang) ? "txt" : "code"} file`"
752         >
753             {exported ? (
754                 <Check className="w-3.5 h-3.5 text-emerald-500" />
755             ) : (
756                 <Download className="w-3.5 h-3.5" />

```


src/components/CodeEditor.tsx (continued)

```

757         })
758         <span>{exported ? "Exported!" : "Export as File"}</span>
759     </button>
760
761     {/* Clear */}
762     <button
763         type="button"
764         onClick={handleClear}
765         className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-slate-100 dark:bg-slate-700 hover:bg-
766 slate-200 dark:hover:bg-slate-600 active:tab"
767     >
768         <Trash2 className="w-3.5 h-3.5" />
769         <span>Clear</span>
770     </button>
771
772     {/* Ask AI */}
773     <button
774         type="button"
775         onClick={() => {
776             const active = getCurrentCode();
777             const lang = isWebLanguage(currentLang)
778                 ? activeTab.toUpperCase()
779                 : currentLang;
780             onSendToAI?.(
781                 active,
782                 lang,
783                 `Explain, debug, and review this ${lang} code:\n\n\`\`\`${lang.toLowerCase()} ${active}\n\`\`\``
784             );
785         }}
786         className="hidden sm:flex items-center space-x-1 px-3 py-1.5 rounded-lg bg-indigo-50 dark:bg-
787 indigo-950 hover:bg-indigo-100 d
788         <Sparkles className="w-3.5 h-3.5" />
789         <span>Ask AI</span>
790     </button>
791 </div>
792 </div>
793
794     {/* Code Area with Line Numbers & Tab Indentation */}
795     <div className="mt-3 rounded-xl bg-slate-900 border border-slate-700 overflow-hidden shadow-inner">
796         <div className="flex relative">
797             {/* Line Numbers Gutter */}
798             <div
799                 ref={lineNumbersRef}
800                 className="py-3.5 pl-3 pr-2 select-none text-right font-mono text-xs text-slate-500 bg-slate-950/70
801 border-r border-slate-800 ov
802                 style={{ minWidth: "42px", maxHeight: "240px" }}
803             >
804                 {Array.from({ length: lineCount }, (_, i) => (
805                     <div key={i + 1} className="leading-6">
806                         {i + 1}
807                     </div>
808                 ))}
809             </div>
810
811             {/* Code Textarea */}
812             <textarea
813                 ref={textareaRef}
814                 id="mainCodeTextarea"
815                 value={activeCode}
816                 onChange={(e) => {
817                     setCurrentCode(e.target.value);
818                     updateCursorPosition();
819                 }}
820                 onKeyDown={handleKeyDown}
821                 onKeyUp={updateCursorPosition}
822                 onClick={updateCursorPosition}
823                 onScroll={handleScroll}
824                 rows={9}
825                 className="w-full font-mono text-sm py-3.5 px-3 bg-transparent text-slate-100 focus:outline-none resize-
y leading-6 selection:bg-
            <span>Write ${isWebLanguage(currentLang) ? activeTab.toUpperCase() : currentLang} code here
            (press Tab to indent)...</span>

```

src/components/CodeEditor.tsx (continued)

```

826         spellCheck={false}
827     />
828 </div>
829
830     {/* Editor Status Bar */}
831     <div className="flex items-center justify-between px-3 py-1.5 bg-slate-950/90 border-t border-slate-800
832 text-[11px] font-mono text-slate-400" style={{flex: 1}}>
833         <span>
834             Ln {cursorPos.line}, Col {cursorPos.col}
835         </span>
836         <span>•</span>
837         <span>
838             {lineCount} {lineCount === 1 ? "line" : "lines"}
839         </span>
840         <span>•</span>
841         <span>{charCount} chars</span>
842     </div>
843     <div className="flex items-center space-x-2">
844         <span className="text-slate-400">Tab: 2 spaces</span>
845         <span>•</span>
846         <span className="text-blue-400 font-semibold">{currentLang}</span>
847     </div>
848 </div>
849 </div>
850
851     {/* Output Panel: Live Preview & Terminal / Console */}
852     <div className="mt-4">
853         <div className="flex items-center justify-between mb-2 text-xs">
854             {/* Output Mode Switcher */}
855             <div className="flex items-center space-x-1 p-0.5 bg-slate-100 dark:bg-slate-900 rounded-lg border border-
856 slate-200 dark:border-slate-800">
857                 <button
858                     type="button"
859                     onClick={() => setOutputTab("preview")}
860                     className={`px-3 py-1 rounded-md transition flex items-center space-x-1.5 ${
861                         outputTab === "preview"
862                             ? "bg-white dark:bg-slate-800 text-blue-600 dark:text-blue-400 shadow-xs font-semibold"
863                             : "text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white"
864                     }`>
865                 >
866                     <Eye className="w-3 h-3" />
867                     <span>Live Preview</span>
868                 </button>
869             </div>
870
871             <button
872                 type="button"
873                 onClick={() => setOutputTab("terminal")}
874                 className={`px-3 py-1 rounded-md transition flex items-center space-x-1.5 ${
875                     outputTab === "terminal" || !isWebLanguage(currentLang)
876                         ? "bg-white dark:bg-slate-800 text-blue-600 dark:text-blue-400 shadow-xs font-semibold"
877                         : "text-slate-600 dark:text-slate-400 hover:text-slate-900 dark:hover:text-white"
878                 }`>
879                 >
880                     <Terminal className="w-3 h-3" />
881                     <span>Terminal & Console</span>
882                 {terminalLogs.length > 0 && (
883                     <span className="px-1.5 py-0.2 rounded-full bg-slate-200 dark:bg-slate-700 text-[10px] font-mono">
884                         {terminalLogs.length}
885                     </span>
886                 )}
887             </button>
888         </div>
889
890         <div className="flex items-center space-x-3 text-slate-500 dark:text-slate-400">
891             {outputTab === "terminal" && terminalLogs.length > 0 && (
892                 <button
893                     type="button"
894                     onClick={() => setTerminalLogs([])}

```

src/components/CodeEditor.tsx (continued)

```
895         className="hover:text-rose-500 text-[11px] underline"
896       >
897         Clear Terminal
898       </button>
899     )}
900     {lastRunTime && <span>Executed: {lastRunTime}</span>}
901   </div>
902 </div>
903
904   {/* View 1: Live Web Preview iframe */}
905   {isWebLanguage(currentLang) && outputTab === "preview" && (
906     inner">
907     <iframe
908       id="liveOutput"
909       title="Live Output Preview"
910       srcDoc={srcDoc}
911       sandbox="allow-scripts allow-modals"
912       className="w-full h-64 border-none bg-white"
913     />
914   </div>
915 )}
916
917   {/* View 2: Terminal / Console Output */}
918   {(outputTab === "terminal" || !isWebLanguage(currentLang)) && (
919     inner">
920     <div className="px-3 py-1.5 bg-slate-900 border-b border-slate-800 text-slate-400 flex items-center
921       justify-between text-[11px]">
922       <div className="flex items-center space-x-2">
923         <span className="w-2 h-2 rounded-full bg-emerald-400 animate-pulse"></span>
924         <span className="font-semibold text-slate-300">
925           {isWebLanguage(currentLang)
926             ? "Sandbox Console Log"
927             : `${currentLang} Standard Output (Stdout/Stderr)`}
928         </span>
929       </div>
930       <span className="text-slate-500">Interactive Stream</span>
931     </div>
932
933     <div className="p-3 h-64 overflow-y-auto space-y-1 text-slate-200 scrollbar-thin">
934       {terminalLogs.length === 0 ? (
935         text-xs">
936         <div className="h-full flex flex-col items-center justify-center text-slate-500 space-y-1 italic">
937           <Terminal className="w-6 h-6 stroke-1 text-slate-600" />
938           <span>No output yet. Click "Run Code" to execute.</span>
939         </div>
940       ) : (
941         terminalLogs.map((log) => (
942           <div key={log.id} className="flex items-start space-x-2 leading-relaxed">
943             <span className="text-slate-600 select-none text-[10px] shrink-0 pt-0.5">
944               {log.time}
945             </span>
946             {log.type === "system" && (
947               <span className="text-indigo-400 font-semibold">{log.text}</span>
948             )}
949             {log.type === "stdout" && (
950               <span className="text-emerald-300 whitespace-pre-wrap">{log.text}</span>
951             )}
952             {log.type === "info" && (
953               <span className="text-blue-300 whitespace-pre-wrap">{log.text}</span>
954             )}
955             {log.type === "warn" && (
956               <span className="text-amber-300 whitespace-pre-wrap">{log.text}</span>
957             )}
958             {log.type === "error" && (
959               <span className="text-rose-400 whitespace-pre-wrap">{log.text}</span>
960             )}
961           </div>
962         ))
963       )}
964     </div>
965   </div>
```

src/components/CodeEditor.tsx (continued)

```
964         })
965       </div>
966     </div>
967   );
968 };
969
```

FILE [14/25]

src/components/AIAssistant.tsx 31 lines • 21.2 KB • AI Copilot chat with Markdown & Code Apply

```
1 import React, { useState, useEffect, useRef } from "react";
2 import Markdown from "react-markdown";
3 import { AIChatMessage } from "../types";
4 import { safeStorageGet, safeStorageSet, safeStorageRemove } from "../utils/helpers";
5 import {
6   Bot,
7   Sparkles,
8   Send,
9   Trash2,
10  Key,
11  Eye,
12  EyeOff,
13  Code2,
14  Wrench,
15  BookOpen,
16  Check,
17  Copy,
18  ArrowRight,
19 } from "lucide-react";
20
21 interface AIAssistantProps {
22   currentLanguage?: string;
23   incomingPrompt?: string;
24   currentCode?: string;
25   getActiveCode?: () => { tab: string; code: string };
26   onApplyCodeToEditor?: (tab: string, code: string) => void;
27 }
28
29 export const AIAssistant: React.FC<AIAssistantProps> = ({
30   currentLanguage = "JavaScript",
31   incomingPrompt,
32   currentCode = "",
33   getActiveCode,
34   onApplyCodeToEditor,
35 }) => {
36   const [messages, setMessages] = useState<AIChatMessage[]>([]);
37   const [prompt, setPrompt] = useState("");
38   const [loading, setLoading] = useState(false);
39   const [customKey, setCustomKey] = useState("");
40   const [showKeyInput, setShowKeyInput] = useState(false);
41   const [showKeyText, setShowKeyText] = useState(false);
42   const [statusMsg, setStatusMsg] = useState<string>(
43     "Ready: Powered by Gemini AI via full-stack server integration."
44   );
45   const [copiedIndex, setCopiedIndex] = useState<number | null>(null);
46
47   const chatBottomRef = useRef<HTMLDivElement>(null);
48
49   const getActiveCodeSafe = () => {
50     if (getActiveCode) {
51       try {
52         return getActiveCode();
53       } catch {
54         // fallback
55       }
56     }
57     if (currentCode) {
58       return { tab: currentLanguage || "Code", code: currentCode };
59     }
60     return { tab: "Code", code: "" };
61   };
62
63   useEffect(() => {
64     const saved = safeStorageGet("csh_custom_key");
65     if (saved) setCustomKey(saved);
66   });
```

src/components/AIAssistant.tsx (continued)

```

67     const savedHistory = safeStorageGet("csh_ai_messages");
68     if (savedHistory) {
69         try {
70             setMessages(JSON.parse(savedHistory));
71         } catch {
72             // ignore
73         }
74     } else {
75         setMessages([
76             {
77                 role: "bot",
78                 text: `👋 Hello! I am **Coding Super AI**, your developer copilot for **Coding Super Hub**.\n\nI can:\n- 📄 Pa
79 **Debug and fix code**\n- 📅 Date.now(),
80             },
81         ]);
82     }
83 }, []);
84
85 useEffect(() => {
86     chatBottomRef.current?.scrollIntoView({ behavior: "smooth" });
87 }, [messages, loading]);
88
89 useEffect(() => {
90     if (incomingPrompt && incomingPrompt.trim()) {
91         handleSend(incomingPrompt);
92     }
93     // eslint-disable-next-line react-hooks/exhaustive-deps
94 }, [incomingPrompt]);
95
96 const saveKey = (val: string) => {
97     setCustomKey(val);
98     if (val.trim()) {
99         safeStorageSet("csh_custom_key", val.trim());
100         setStatusMsg("Custom Gemini key saved locally in this browser.");
101     } else {
102         safeStorageRemove("csh_custom_key");
103         setStatusMsg("Using default server-side Gemini configuration.");
104     }
105 };
106
107 const handleSend = async (customPrompt?: string) => {
108     const textToSend = (customPrompt || prompt).trim();
109     if (!textToSend) return;
110
111     const userMessage: AIChatMessage = {
112         role: "user",
113         text: textToSend,
114         timestamp: Date.now(),
115     };
116
117     const updated = [...messages, userMessage];
118     setMessages(updated);
119     setPrompt("");
120     setLoading(true);
121     setStatusMsg("#0 Coding Super AI is generating response...");
122
123     try {
124         const res = await fetch("/api/ai/ask", {
125             method: "POST",
126             headers: { "Content-Type": "application/json" },
127             body: JSON.stringify({
128                 prompt: textToSend,
129                 history: updated.slice(-6),
130                 language: currentLanguage,
131                 customApiKey: customKey.trim() || undefined,
132             }),
133         });
134
135         const data = await res.json();

```

src/components/AIAssistant.tsx (continued)

```

136     if (!res.ok) {
137         throw new Error(data.error || "Failed to reach AI service");
138     }
139
140     const botMessage: AIChatMessage = {
141         role: "bot",
142         text: data.answer || "No response received.",
143         timestamp: Date.now(),
144     };
145
146     const finalMessages = [...updated, botMessage];
147     setMessages(finalMessages);
148     safeStorageSet(
149         "csh_ai_messages",
150         JSON.stringify(finalMessages.slice(-16))
151     );
152     setStatusMsg("' Response generated successfully.");
153 } catch (err: any) {
154     const errMsg = err?.message || "An unexpected error occurred.";
155     const isHighDemand =
156         errMsg.includes("high demand") ||
157         errMsg.includes("503") ||
158         errMsg.includes("temporarily") ||
159         errMsg.includes("try again later");
160
161     const tipText = isHighDemand
162         ? "Google Cloud Gemini servers are currently experiencing a temporary peak load spike. Please wait a few
163 seconds and verify network connection or check if your API key or quota is active.";
164
165     const errorBotMessage: AIChatMessage = {
166         role: "bot",
167         text: `& p **Service Notice:** ${errMsg}\n\n*Action:* ${tipText}`,
168         timestamp: Date.now(),
169     };
170     setMessages([...updated, errorBotMessage]);
171     setStatusMsg(`& p Service Notice: ${errMsg.slice(0, 100)}`);
172 } finally {
173     setLoading(false);
174 }
175 };
176
177 const handleExplainCode = () => {
178     const active = getActiveCodeSafe();
179     if (!active.code.trim()) {
180         setStatusMsg("'L The active editor tab is empty. Write or paste code first.");
181         return;
182     }
183     const query = `Analyze and explain this ${active.tab} code in ${currentLanguage}:
184 \n\n${active.code.toLowerCase()}\n\n${active.code}\n
185     `;
186     handleSend(query);
187
188     const handleFixCode = () => {
189         const active = getActiveCodeSafe();
190         if (!active.code.trim()) {
191             setStatusMsg("'L The active editor tab is empty. Write or paste code first.");
192             return;
193         }
194         const query = `Please debug and fix this ${active.tab} code in ${currentLanguage}:
195 \n\n${active.code.toLowerCase()}\n\n${active.code}\n
196     `;
197     handleSend(query);
198
199     const handleGenerateCode = () => {
200         const p = prompt.trim();
201         if (!p) {
202             setPrompt(`Write a clean, responsive, production-ready ${currentLanguage} snippet for: `);
203             setStatusMsg("'9p Please finish describing what you want to generate, then press Ask AI.");
204             return;
205         }
206         handleSend(`Generate production-ready ${currentLanguage} code for: ${p}. Include brief explanation and usage
207 instructions.`);

```

src/components/AIAssistant.tsx (continued)

```

205     };
206
207     const clearChat = () => {
208         setMessages([]);
209         safeStorageRemove("csh_ai_messages");
210         setStatusMsg("Chat history cleared.");
211     };
212
213     const copyMessage = (text: string, index: number) => {
214         navigator.clipboard.writeText(text);
215         setCopiedIndex(index);
216         setTimeout(() => setCopiedIndex(null), 1500);
217     };
218
219     return (
220         <div
221             id="ai-assistant-section"
222             className="bg-gradient-to-br from-slate-900 via-indigo-950 to-slate-900 rounded-2xl border border-
223 indigo-900/60 p-5 shadow-xl text-whi
224
225         <div className="flex flex-wrap items-center justify-between gap-3 border-b border-indigo-900/50 pb-4">
226             <div className="flex items-center space-x-3">
227                 <div className="w-10 h-10 rounded-xl bg-indigo-600 flex items-center justify-center text-white shadow-lg
228 shadow-indigo-500/40">
229                     <Bot className="w-6 h-6" />
230                 </div>
231                 <div>
232                     <h2 className="text-lg font-bold text-white flex items-center gap-2">
233                         <span>ðŸ’Œ Coding Super AI</span>
234                         <span className="text-[11px] px-2 py-0.5 rounded-full bg-indigo-500/20 text-indigo-300 border border-
235 indigo-500/30 font-mono">gemini-1.5-flash</span>
236                     </h2>
237                     <p className="text-xs text-indigo-200/80">
238                         Explain, debug, generate, and optimize code in real-time
239                     </p>
240                 </div>
241             </div>
242
243             <div className="flex items-center space-x-2 text-xs">
244                 <button
245                     type="button"
246                     onClick={() => setShowKeyInput(!showKeyInput)}
247                     className="flex items-center space-x-1 px-2.5 py-1.5 rounded-lg bg-indigo-900/50 hover:bg-indigo-900
248 border border-indigo-700/50
249                     <Key className="w-3.5 h-3.5" />
250                     <span>{customKey ? "Custom Key Active" : "API Key Settings"}</span>
251                 </button>
252                 <button
253                     type="button"
254                     onClick={clearChat}
255                     className="flex items-center space-x-1 px-2.5 py-1.5 rounded-lg bg-slate-800/80 hover:bg-slate-800
256 border border-slate-700 text-
257                     <Trash2 className="w-3.5 h-3.5" />
258                     <span className="hidden sm:inline">Clear</span>
259                 </button>
260             </div>
261         </div>
262
263         {/* Key input drawer if requested */}
264         {showKeyInput && (
265             <div className="mt-3 p-3 bg-indigo-950/70 rounded-xl border border-indigo-800/60 text-xs flex flex-wrap
266 items-center gap-2">
267                 <span className="text-indigo-200">Custom Gemini Key (Optional):</span>
268                 <div className="relative flex-1 min-w-[220px]">
269                     <input
270                         type={showKeyText ? "text" : "password"}
271                         value={customKey}
272                         onChange={(e) => saveKey(e.target.value)}
273                         placeholder="Leave empty to use automatic server key..."
274                         className="w-full pl-3 pr-8 py-1.5 bg-slate-900 border border-indigo-700/60 rounded-lg text-slate-100
275 placeholder-slate-500 fo

```


src/components/AIAssistant.tsx (continued)

```

    />
    <button
      type="button"
      onClick={() => setShowKeyText(!showKeyText)}
      className="absolute right-2 top-2 text-slate-400 hover:text-slate-200"
    >
      {showKeyText ? <EyeOff className="w-3.5 h-3.5" /> : <Eye className="w-3.5 h-3.5" />}
    </button>
  </div>
  {customKey && (
    <button
      type="button"
      onClick={() => saveKey("")}
      className="px-2 py-1 bg-rose-900/60 hover:bg-rose-900 text-rose-200 rounded border border-rose-700"
    >
      Reset to Server Key
    </button>
  )}
</div>
)}

{/* Chat Messages Log */}
<div className="mt-4 h-72 overflow-y-auto rounded-xl bg-slate-950/90 border border-indigo-950/80 p-4 space-y-3 font-sans text-sm font-mono">
  {messages.map((msg, idx) => (
    <div
      key={idx}
      className={`flex flex-col ${
        msg.role === "user" ? "items-end" : "items-start"
      }`}
    >
      <div
        className={`max-w-[90%] sm:max-w-[80%] rounded-xl px-4 py-2.5 relative group ${
          msg.role === "user"
            ? "bg-blue-600 text-white rounded-br-none shadow-md"
            : "bg-slate-900 text-slate-100 border border-slate-800 rounded-bl-none shadow-md"
        }`}
      >
        {msg.role === "user" ? (
          <div className="whitespace-pre-wrap font-sans leading-relaxed break-words text-sm">
            {msg.text}
          </div>
        ) : (
          <div className="markdown-body font-sans leading-relaxed break-words text-sm text-slate-100">
            <Markdown
              components={{
                code({ inline, className, children, ...props }: any) {
                  const match = /language-(\w+)/.exec(className || "");
                  const lang = match ? match[1] : "";
                  const codeString = String(children).replace(/\n$/, "");

                  if (!inline && (match || codeString.includes("\n"))) {
                    return (
                      <div className="my-2.5 rounded-lg overflow-hidden border border-slate-700/80 bg-slate-950 text-left">
                        <div className="flex items-center justify-between px-3 py-1.5 bg-slate-800/80 text-xs">
                          <span className="uppercase font-bold text-blue-400">
                            {lang || "code"}
                          </span>
                          <div className="flex items-center space-x-1.5">
                            <button
                              type="button"
                              onClick={() => {
                                navigator.clipboard.writeText(codeString);
                                setStatusMsg("Code block copied to clipboard!");
                                setTimeout(
                                  () =>
                                    setStatusMsg(
                                      "Ready: Powered by Gemini AI via full-stack server integration."
                                    ),
                                2000
                              )
                            }
                          </div>
                        </div>
                      </div>
                    );
                  }
                }
              }}
            />
          </div>
        )}
      </div>
    >
  )}
</div>

```

src/components/AIAssistant.tsx (continued)

```

343         );
344     }}
345     className="flex items-center space-x-1 px-2 py-0.5 rounded bg-slate-700
346     hover:bg-slate-600 text-slate-20 title="Copy code snippet"
347     >
348         <Copy className="w-3 h-3" />
349         <span>Copy</span>
350     </button>
351     {onApplyCodeToEditor && (
352         <button
353             type="button"
354             onClick={() => {
355                 onApplyCodeToEditor(
356                     lang || currentLanguage || "JavaScript",
357                     codeString
358                 );
359                 setStatusMsg(
360                     `Loaded ${lang || "code"} snippet into active editor.`
361                 );
362             }}
363             className="flex items-center space-x-1 px-2 py-0.5 rounded bg-blue-600
364             hover:bg-blue-500 text-white te title="Load snippet into active editor"
365             >
366                 <ArrowRight className="w-3 h-3" />
367                 <span>Apply to Editor</span>
368             </button>
369         )}
370     </div>
371 </div>
372     <pre className="p-3 text-xs font-mono overflow-x-auto text-slate-100 leading-relaxed
373     bg-slate-950/90">
374         <code>{children}</code>
375     </pre>
376 </div>
377 );
378 }
379 return (
380     <code
381         className="px-1.5 py-0.5 rounded bg-slate-800 text-indigo-300 font-mono text-xs"
382         {...props}
383     >
384         {children}
385     </code>
386 );
387 },
388 p({ children }) {
389     return <p className="mb-2 last:mb-0 leading-relaxed">{children}</p>;
390 },
391 ul({ children }) {
392     return (
393         <ul className="list-disc list-inside mb-2 space-y-1">{children}</ul>
394     );
395 },
396 ol({ children }) {
397     return (
398         <ol className="list-decimal list-inside mb-2 space-y-1">{children}</ol>
399     );
400 },
401 h1({ children }) {
402     return (
403         <h1 className="text-base font-bold text-white mb-2 mt-3">{children}</h1>
404     );
405 },
406 h2({ children }) {
407     return (
408         <h2 className="text-sm font-bold text-white mb-1.5 mt-2.5">
409             {children}
410         </h2>
411     );
412 },

```

src/components/AIAssistant.tsx (continued)

```

412             h3({ children }) {
413                 return (
414                     <h3 className="text-xs font-bold text-blue-300 mb-1 mt-2">
415                         {children}
416                     </h3>
417                 );
418             },
419         }}
420     >
421         {msg.text}
422     </Markdown>
423 </div>
424 }}
425
426 <button
427     type="button"
428     onClick={() => copyMessage(msg.text, idx)}
429     className="absolute top-2 right-2 p-1 rounded bg-black/40 text-slate-300 opacity-0 group-
430 hover:opacity-100 transition:opacity"
431 >
432     {copiedIndex === idx ? (
433         <Check className="w-3.5 h-3.5 text-emerald-400" />
434     ) : (
435         <Copy className="w-3.5 h-3.5" />
436     )}
437 </button>
438 </div>
439 </div>
440 )))
441 {loading && (
442     <div className="flex items-center space-x-2 text-indigo-300 text-xs italic py-2">
443         <div className="w-2 h-2 rounded-full bg-indigo-400 animate-pulse" />
444         <div className="w-2 h-2 rounded-full bg-indigo-400 animate-pulse delay-100" />
445         <div className="w-2 h-2 rounded-full bg-indigo-400 animate-pulse delay-200" />
446         <span>Coding Super AI is thinking...</span>
447     </div>
448 )}
449 <div ref={chatBottomRef} />
450 </div>
451
452 {/* Action shortcuts */}
453 <div className="mt-3 flex flex-wrap items-center gap-2">
454     <button
455         type="button"
456         onClick={() => handleSend()}
457         disabled={loading || !prompt.trim()}
458         className="flex items-center space-x-1.5 px-3.5 py-1.5 rounded-lg bg-blue-600 hover:bg-blue-500
459 disabled:opacity-50 text-white font
460         <Sparkles className="w-3.5 h-3.5" />
461         <span>Ask AI</span>
462     </button>
463
464     <button
465         type="button"
466         onClick={handleExplainCode}
467         disabled={loading}
468         className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-indigo-800/60 hover:bg-indigo-800 text-
469 indigo-200 border border
470         <BookOpen className="w-3.5 h-3.5" />
471         <span>Explain Active Tab</span>
472     </button>
473
474     <button
475         type="button"
476         onClick={handleFixCode}
477         disabled={loading}
478         className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-indigo-800/60 hover:bg-indigo-800 text-
479 indigo-200 border border
480         <Wrench className="w-3.5 h-3.5" />

```

src/components/AIAssistant.tsx (continued)

```

481         <span>Fix Active Tab</span>
482     </button>
483
484     <button
485         type="button"
486         onClick={handleGenerateCode}
487         disabled={loading}
488         className="flex items-center space-x-1.5 px-3 py-1.5 rounded-lg bg-indigo-800/60 hover:bg-indigo-800 text-
489 indigo-200 border border
490         <Code2 className="w-3.5 h-3.5" />
491         <span>Generate Code</span>
492     </button>
493 </div>
494
495 {/* Input textarea */}
496 <div className="mt-3 relative">
497     <textarea
498         value={prompt}
499         onChange={(e) => setPrompt(e.target.value)}
500         onKeyDown={(e) => {
501             if ((e.ctrlKey || e.metaKey) && e.key === "Enter") {
502                 e.preventDefault();
503                 handleSend();
504             }
505         }}
506         placeholder="Ask anything or request refactoring (Ctrl+Enter to send)..."
507         rows={3}
508         className="w-full px-4 py-2.5 bg-slate-950 text-slate-100 placeholder-slate-500 rounded-xl border border-
509 indigo-900/80 focus:outli
510     <button
511         type="button"
512         onClick={() => handleSend()}
513         disabled={loading || !prompt.trim()}
514         className="absolute right-3 bottom-3 p-2 bg-blue-600 hover:bg-blue-500 disabled:opacity-40 text-white
515 rounded-lg transition"
516         title="Send query"
517     >
518         <Send className="w-4 h-4" />
519     </button>
520 </div>
521
522 {/* Status Bar */}
523 <div className="mt-2 text-xs text-indigo-300/80 flex items-center justify-between">
524     <span>{statusMsg}</span>
525     <span className="hidden sm:inline text-indigo-400/60 font-mono text-[11px]">
526         Shortcuts: Ctrl+Enter to send
527     </span>
528 </div>
529 </div>
530 );
531 };

```

FILE [15/25]

src/components/LanguageHub.tsx 16 lines • 8.9 KB • 45+ Programming languages explorer & starter cards

```
1 import React, { useState } from "react";
2 import { LANGUAGE_CATALOG } from "../data/languages";
3 import { LanguageInfo } from "../types";
4 import {
5   Globe2,
6   Search,
7   Code2,
8   Sparkles,
9   X,
10  Copy,
11  Check,
12  ChevronRight,
13 } from "lucide-react";
14
15 interface LanguageHubProps {
16   onSelectLanguage: (lang: string) => void;
17   onAskAI: (query: string) => void;
18 }
19
20 export const LanguageHub: React.FC<LanguageHubProps> = ({
21   onSelectLanguage,
22   onAskAI,
23 }) => {
24   const [search, setSearch] = useState("");
25   const [selectedLanguage, setSelectedLanguage] = useState<LanguageInfo | null>(
26     null
27   );
28   const [copied, setCopied] = useState(false);
29
30   const filtered = LANGUAGE_CATALOG.filter((item) => {
31     const q = search.toLowerCase().trim();
32     return (
33       item.name.toLowerCase().includes(q) ||
34       item.category.toLowerCase().includes(q) ||
35       (item.description && item.description.toLowerCase().includes(q))
36     );
37   });
38
39   const handleCopyCode = (code?: string) => {
40     if (!code) return;
41     navigator.clipboard.writeText(code);
42     setCopied(true);
43     setTimeout(() => setCopied(false), 1500);
44   };
45
46   const handleUseInEditor = (lang: LanguageInfo) => {
47     onSelectLanguage(lang.name);
48     setSelectedLanguage(null);
49     document
50       .getElementById("code-editor-section")
51       ?.scrollIntoView({ behavior: "smooth" });
52   };
53
54   const handleAskAI = (lang: LanguageInfo) => {
55     onAskAI(
56       `Teach me ${lang.name} (${lang.category}). Give me a structured guide covering:\n1. Core language paradigms &
57       stand out features\n2. Pr
58     );
59     setSelectedLanguage(null);
60     document
61       .getElementById("ai-assistant-section")
62       ?.scrollIntoView({ behavior: "smooth" });
63   };
64
65   return (
66     <div
67       id="languages-hub-section"
```

src/components/LanguageHub.tsx (continued)

```

67         className="bg-white dark:bg-slate-800/90 rounded-2xl border border-slate-200 dark:border-slate-700 p-5 shadow-
68         sm my-6 transition"
69         <div className="flex flex-wrap items-center justify-between gap-3 border-b border-slate-200 dark:border-
70         slate-700 pb-4">
71             <div className="flex items-center space-x-2">
72                 <Globe2 className="w-5 h-5 text-blue-600 dark:text-blue-400" />
73                 <h2 className="text-lg font-bold text-slate-900 dark:text-white">
74                     Programming Languages Hub
75                 </h2>
76                 <span className="px-2 py-0.5 text-xs font-semibold rounded-full bg-emerald-50 text-emerald-700 dark:bg-
77                 emerald-950 dark:text-white">{LANGUAGE_CATALOG.length} Languages
78             </span>
79         </div>
80         <p className="text-xs text-slate-500 dark:text-slate-400 mt-1">
81             Browse major programming, scripting, mobile, systems, hardware, and web formats with starter code
82         </p>
83     </div>
84
85     <div className="relative w-full sm:w-72">
86         <Search className="w-4 h-4 text-slate-400 absolute left-3 top-2.5" />
87         <input
88             type="text"
89             value={search}
90             onChange={(e) => setSearch(e.target.value)}
91             placeholder="Search language (e.g. Rust, Go, SQL)..."
92             className="w-full pl-9 pr-3 py-1.5 text-xs rounded-lg bg-slate-100 dark:bg-slate-900 text-slate-900
93             dark:text-slate-100 border b
94         </input>
95     </div>
96
97     {/ * Languages Grid */}
98     <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 lg:grid-cols-6 gap-2.5 mt-4">
99         {filtered.map((lang) => (
100             <button
101                 key={lang.name}
102                 type="button"
103                 onClick={() => setSelectedLanguage(lang)}
104                 className="text-left p-3 rounded-xl bg-slate-50 dark:bg-slate-900/60 hover:bg-blue-50 dark:hover:bg-
105                 slate-800/80 border border-s
106             <div>
107                 <div className="flex items-center justify-between">
108                     <span className="font-bold text-sm text-slate-900 dark:text-slate-100 group-hover:text-blue-600
109                     dark:group-hover:
110                     {lang.name}
111                 </span>
112                 <ChevronRight className="w-3.5 h-3.5 text-slate-400 group-hover:translate-x-0.5 transition" />
113             </div>
114             <span className="text-[11px] text-slate-500 dark:text-slate-400 line-clamp-1 mt-0.5">
115                 {lang.category}
116             </span>
117             </div>
118         </button>
119     )))
120     {filtered.length === 0 && (
121         <div className="col-span-full py-8 text-center text-sm text-slate-500 dark:text-slate-400">
122             No languages matched "{search}". Try searching for another name or category.
123         </div>
124     )}
125 </div>
126
127 {/ * Language Modal */}
128 {selectedLanguage && (
129     <div
130         className="fixed inset-0 z-50 bg-slate-950/60 backdrop-blur-sm flex items-center justify-center p-4"
131         onClick={() => setSelectedLanguage(null)}
132     >
133         <div
134             className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-700 w-full max-w-xl
135             rounded-2xl p-5 shadow-2xl"
136         >

```

src/components/LanguageHub.tsx (continued)

```

136         <div className="flex items-start justify-between">
137             <div>
138                 <div className="flex items-center space-x-2">
139                     <h3 className="text-xl font-bold text-slate-900 dark:text-white">
140                         {selectedLanguage.name}
141                     </h3>
142                     {selectedLanguage.extension && (
143                         <span className="px-2 py-0.5 text-xs font-mono bg-slate-100 dark:bg-slate-800 text-slate-700
144                         dark:text-slate-300 rounded">{selectedLanguage.extension}</span>
145                     )}
146                 </div>
147                 <span className="text-xs font-medium text-blue-600 dark:text-blue-400">
148                     {selectedLanguage.category}
149                 </span>
150             </div>
151             <button
152                 type="button"
153                 onClick={() => setSelectedLanguage(null)}
154                 className="p-1.5 rounded-lg text-slate-400 hover:text-slate-600 dark: hover:text-slate-200 bg-
155                 slate-100 dark:bg-slate-800"
156             >
157                 <X className="w-4 h-4" />
158             </button>
159         </div>
160
161         <p className="text-sm text-slate-600 dark:text-slate-300 leading-relaxed">
162             {selectedLanguage.description ||
163             "Versatile language supported in Coding Super Hub for development and learning."}
164         </p>
165
166         {selectedLanguage.starterCode && (
167             <div>
168                 <div className="flex items-center justify-between text-xs text-slate-500 dark:text-slate-400
169                 mb-1.5">
170                     <span className="font-semibold text-slate-700 dark:text-slate-300">
171                         Starter Code:
172                     </span>
173                     <button
174                         type="button"
175                         onClick={() => handleCopyCode(selectedLanguage.starterCode)}
176                         className="flex items-center space-x-1 text-blue-600 dark:text-blue-400 hover:underline"
177                     >
178                         {copied ? (
179                             <Check className="w-3.5 h-3.5 text-emerald-500" />
180                         ) : (
181                             <Copy className="w-3.5 h-3.5" />
182                         )}
183                         <span>{copied ? "Copied!" : "Copy Snippet"}</span>
184                     </button>
185                 </div>
186                 <pre className="p-3 bg-slate-950 text-slate-100 rounded-xl text-xs font-mono overflow-x-auto border
187                 border-slate-800 leading-relaxed">
188                     {selectedLanguage.starterCode}
189                 </pre>
190             </div>
191         )}
192
193         <div className="flex flex-wrap items-center gap-2 pt-2 border-t border-slate-200 dark:border-slate-800">
194             <button
195                 type="button"
196                 onClick={() => handleUseInEditor(selectedLanguage)}
197                 className="flex items-center space-x-1.5 px-4 py-2 rounded-xl bg-blue-600 hover:bg-blue-500 text-
198                 white font-medium text-xs"
199             >
200                 <Code2 className="w-4 h-4" />
201                 <span>Load in Editor</span>
202             </button>
203
204             <button
205                 type="button"
206                 onClick={() => handleAskAI(selectedLanguage)}
207                 className="flex items-center space-x-1.5 px-4 py-2 rounded-xl bg-indigo-600 hover:bg-indigo-500
208                 text-white font-medium text-

```

src/components/LanguageHub.tsx (continued)

```
205         >
206         <Sparkles className="w-4 h-4" />
207         <span>Learn with AI Copilot</span>
208       </button>
209     </div>
210   </div>
211 </div>
212   )}
213 </div>
214 );
215 };
216
```


FILE [16/25]

src/components/ToolCard.tsx 138 lines • 5.4 KB • Universal reusable tool container with copy/clear

```
1 import React, { useState } from "react";
2 import { ToolCategory } from "../types";
3 import { Copy, Check } from "lucide-react";
4
5 interface ToolCardProps {
6   id: number;
7   title: string;
8   category: ToolCategory;
9   description?: string;
10  children: React.ReactNode;
11  output?: string | React.ReactNode;
12  copyText?: string;
13  isError?: boolean;
14 }
15
16 const CATEGORY_COLORS: Record<string, string> = {
17   Web: "bg-cyan-500/10 text-cyan-700 dark:text-cyan-300 border-cyan-500/20",
18   Text: "bg-blue-500/10 text-blue-700 dark:text-blue-300 border-blue-500/20",
19   Formatting: "bg-indigo-500/10 text-indigo-700 dark:text-indigo-300 border-indigo-500/20",
20   Converters: "bg-violet-500/10 text-violet-700 dark:text-violet-300 border-violet-500/20",
21   Security: "bg-amber-500/10 text-amber-700 dark:text-amber-300 border-amber-500/20",
22   Encryption: "bg-red-500/10 text-red-700 dark:text-red-300 border-red-500/20",
23   Hashes: "bg-orange-500/10 text-orange-700 dark:text-orange-300 border-orange-500/20",
24   Math: "bg-emerald-500/10 text-emerald-700 dark:text-emerald-300 border-emerald-500/20",
25   Time: "bg-teal-500/10 text-teal-700 dark:text-teal-300 border-teal-500/20",
26   Color: "bg-pink-500/10 text-pink-700 dark:text-pink-300 border-pink-500/20",
27   Generators: "bg-purple-500/10 text-purple-700 dark:text-purple-300 border-purple-500/20",
28   HTML: "bg-orange-500/10 text-orange-700 dark:text-orange-300 border-orange-500/20",
29   CSS: "bg-sky-500/10 text-sky-700 dark:text-sky-300 border-sky-500/20",
30   JavaScript: "bg-yellow-500/10 text-yellow-700 dark:text-yellow-300 border-yellow-500/20",
31   API: "bg-fuchsia-500/10 text-fuchsia-700 dark:text-fuchsia-300 border-fuchsia-500/20",
32   Data: "bg-lime-500/10 text-lime-700 dark:text-lime-300 border-lime-500/20",
33   DevOps: "bg-slate-500/10 text-slate-700 dark:text-slate-300 border-slate-500/20",
34   Utilities: "bg-emerald-500/10 text-emerald-700 dark:text-emerald-300 border-emerald-500/20",
35   Miscellaneous: "bg-zinc-500/10 text-zinc-700 dark:text-zinc-300 border-zinc-500/20",
36 };
37
38 export const ToolCard: React.FC<ToolCardProps> = ({
39   id,
40   title,
41   category,
42   description,
43   children,
44   output,
45   copyText,
46   isError,
47 }) => {
48   const [copied, setCopied] = useState(false);
49
50   const handleCopy = () => {
51     const text = copyText || (typeof output === "string" ? output : "");
52     if (!text) return;
53     navigator.clipboard.writeText(text);
54     setCopied(true);
55     setTimeout(() => setCopied(false), 1500);
56   };
57
58   const badgeColor =
59     CATEGORY_COLORS[category] ||
60     "bg-slate-500/10 text-slate-700 dark:text-slate-300 border-slate-500/20";
61
62   return (
63     <div
64       id={`tool-${id}`}
65       data-tool-id={id}
66       data-category={category}
```

src/components/ToolCard.tsx (continued)

```

67         className="bg-white dark:bg-slate-800/90 rounded-2xl border border-slate-200 dark:border-slate-700 p-5 shadow-
68         sm transition hover:shadow"
69         <div>
70             { /* Header: Number, Title, Category Badge */ }
71             <div className="flex items-start justify-between gap-2 mb-3">
72                 <div>
73                     <div className="flex items-center space-x-2">
74                         <span className="px-2 py-0.5 rounded text-xs font-mono font-bold bg-slate-100 dark:bg-slate-900 text-
75                         slate-700 dark:text-slate-100">{id}</span>
76                     </div>
77                     <h3 className="font-bold text-slate-900 dark:text-slate-100 text-base leading-snug">
78                         {title}
79                     </h3>
80                 </div>
81                 {description && (
82                     <p className="text-xs text-slate-500 dark:text-slate-400 mt-1">
83                         {description}
84                     </p>
85                 )}
86             </div>
87             <span
88                 className={`px-2.5 py-0.5 text-[11px] font-semibold rounded-full border whitespace-nowrap ${badgeColor}`}
89             >
90                 {category}
91             </span>
92         </div>
93
94         { /* Tool Interactive Form Body */ }
95         <div className="space-y-3 mt-3">{children}</div>
96     </div>
97
98     { /* Result Output Container if present */ }
99     {output !== undefined && output !== null && output !== "" && (
100         <div className="mt-4 relative group">
101             <div
102                 className={`p-3.5 rounded-xl text-xs font-mono whitespace-pre-wrap break-words leading-relaxed border
103                 transition ${isError
104                     ? "bg-rose-50 dark:bg-rose-950/40 text-rose-800 dark:text-rose-200 border-rose-200 dark:border-
105                 rose-900"
106                 : "bg-slate-50 dark:bg-slate-900/90 text-slate-800 dark:text-slate-200 border-slate-200 dark:border-
107                 slate-700/80"}`}
108             >
109                 {output}
110             </div>
111
112             {(copyText || typeof output === "string") && (
113                 <button
114                     type="button"
115                     onClick={handleCopy}
116                     className="absolute top-2 right-2 flex items-center space-x-1 px-2 py-1 rounded-md bg-white dark:bg-
117                 slate-800 text-slate-600 dark:text-slate-300"
118                 >
119                     {copied ? (
120                         <>
121                             <Check className="w-3 h-3 text-emerald-500" />
122                             <span className="text-emerald-600 dark:text-emerald-400 font-medium">
123                                 Copied!
124                             </span>
125                         </>
126                     ) : (
127                         <>
128                             <Copy className="w-3 h-3" />
129                             <span>Copy</span>
130                         </>
131                     )}
132                 </button>
133             )}
134         </div>
135     )}
136 </div>

```

src/components/ToolCard.tsx (continued)

```
136     );  
137   };  
138
```

FILE [17/25]

src/components/ShortcutsModal.tsx • 82 Lines • 2.9 KB • Keyboard shortcuts cheat sheet modal

```

1  import React from "react";
2  import { X, Keyboard } from "lucide-react";
3
4  interface ShortcutsModalProps {
5    isOpen: boolean;
6    onClose: () => void;
7  }
8
9  export const ShortcutsModal: React.FC<ShortcutsModalProps> = ({ isOpen, onClose }) => {
10    if (!isOpen) return null;
11
12    const shortcuts = [
13      { desc: "Ask AI Assistant", keys: ["Ctrl/Cmd", "Enter"] },
14      { desc: "Run Active Editor Code", keys: ["Ctrl", "Shift", "R"] },
15      { desc: "Copy Active Editor Code", keys: ["Ctrl", "Shift", "C"] },
16      { desc: "Toggle Dark / Light Mode", keys: ["Ctrl", "Shift", "L"] },
17      { desc: "Focus Tool Search Bar", keys: ["Ctrl", "Shift", "F"] },
18      { desc: "Open / Close Shortcuts Help", keys: ["?"] },
19    ];
20
21    return (
22      <div
23        className="fixed inset-0 z-50 bg-slate-950/60 backdrop-blur-sm flex items-center justify-center p-4"
24        onClick={onClose}
25      >
26        <div
27          className="bg-white dark:bg-slate-900 border border-slate-200 dark:border-slate-700 w-full max-w-md
28            rounded-2xl p-6 shadow-2xl"
29          onClick={(e) => e.stopPropagation()}
30        >
31          <div className="flex items-center justify-between pb-3 border-b border-slate-200 dark:border-slate-800">
32            <div className="flex items-center space-x-2">
33              <Keyboard className="w-5 h-5 text-blue-600 dark:text-blue-400" />
34              <h3 className="font-bold text-slate-900 dark:text-white text-base">
35                Keyboard Shortcuts
36              </h3>
37            </div>
38            <button
39              type="button"
40              onClick={onClose}
41              className="p-1.5 rounded-lg text-slate-400 hover:text-slate-600 dark: hover:text-slate-200 bg-slate-100
42                dark:bg-slate-800"
43            >
44              <X className="w-4 h-4" />
45            </button>
46          </div>
47          <div className="space-y-2.5 text-xs">
48            {shortcuts.map((item, i) => (
49              <div
50                key={i}
51                className="flex items-center justify-between py-2 border-b border-slate-100 dark:border-slate-800/60
52                last:border-none"
53              >
54                <span className="text-slate-700 dark:text-slate-300 font-medium">
55                  {item.desc}
56                </span>
57                <div className="flex items-center space-x-1">
58                  {item.keys.map((k, idx) => (
59                    <kbd
60                      key={idx}
61                      className="px-2 py-1 bg-slate-100 dark:bg-slate-800 text-slate-800 dark:text-slate-200 border
62                      border-slate-300 dark:border-slate-700"
63                    >
64                      {k}
65                    </kbd>
66                  ))}
67                </div>
68              </div>
69            ))}
70          </div>
71        </div>
72      </div>
73    );

```

src/components/ShortcutsModal.tsx (continued)

```
67         </div>
68
69         <div className="pt-2 flex justify-end">
70             <button
71                 type="button"
72                 onClick={onClose}
73                 className="px-4 py-2 bg-blue-600 hover:bg-blue-500 text-white rounded-xl text-xs font-semibold shadow-
74 sm transition"
75                 >Got it
76             </button>
77         </div>
78     </div>
79 </div>
80 );
81 };
82
```

FILE [18/25]

src/data/languages.ts

341 lines • 17.2 KB • Full catalog & starters for 45+ languages

```

1 import { LanguageInfo } from "../types";
2
3 export const LANGUAGE_CATALOG: LanguageInfo[] = [
4   {
5     name: "JavaScript",
6     category: "Web / Full-stack",
7     extension: ".js",
8     description: "High-level, dynamically typed language powering web frontends, Node.js backends, and full-stack
9 apps",
10    starterCode: `// JavaScript Starter\nfunction greet(name) {\n  return `Hello, ${name}! Welcome to Coding
11 Super Hub.`;\n}\n\nconsole.l
12 {
13   name: "TypeScript",
14   category: "Web / Full-stack",
15   extension: ".ts",
16   description: "Typed superset of JavaScript that compiles to plain JavaScript.",
17   starterCode: `// TypeScript Starter\ninterface User {\n  id: number;\n  name: string;\n  role: "admin" |
18 "developer";\n}\n\nconst dev: U
19 {
20   name: "Python",
21   category: "AI / Data / General",
22   extension: ".py",
23   description: "Versatile, readable programming language widely used in AI, machine learning, data science, and
24 web",
25   starterCode: `# Python Starter\ndef fibonacci(n):\n    a, b = 0, 1\n    result = []\n    for _ in range(n):
26     \n        result.append(a)\n
27 {
28   name: "HTML",
29   category: "Markup / Web",
30   extension: ".html",
31   description: "Standard markup language for web document structure and semantics.",
32   starterCode: `<!DOCTYPE html>\n<html lang="en">\n<head>\n  <meta charset="UTF-8">\n  <title>Coding Super Hub</
33 title>\n</head>\n<body>\n
34 {
35   name: "CSS",
36   category: "Style / Web",
37   extension: ".css",
38   description: "Style sheet language used for describing the presentation of a document written in HTML.",
39   starterCode: `/* CSS Starter */\nbody {\n  margin: 0;\n  font-family: system-ui, -apple-system, sans-serif;\n
40 background: #0f172a;\n  c
41 {
42   name: "Rust",
43   category: "Systems / Web",
44   extension: ".rs",
45   description: "Blazingly fast and memory-efficient systems programming language with zero-cost abstractions and
46 memory safety guarantees",
47   starterCode: `Rust Starter\nfn main() {\n    let items = vec!["Speed", "Safety", "Concurrency"];\n    for
48 item, in items {\n        pr
49 {
50   name: "Go",
51   category: "Cloud / Systems",
52   extension: ".go",
53   description: "Statically typed, compiled language designed at Google for concurrent, scalable cloud services
54 and microservices",
55   starterCode: `package main\n\nimport "fmt"\n\nfunc main() {\n    fmt.Println("ðŸ‘€ Hello from Go in Coding Super
56 Hub!")\n}\n`,
57 {
58   name: "C++",
59   category: "Systems / General",
60   extension: ".cpp",
61   description: "High-performance systems programming language with object-oriented and generic features.",
62   starterCode: `#include <iostream>\n#include <vector>\n\nint main() {\n    std::vector<std::string> tools =
63 {"Editor", "Converters", "AI
64 {
65   name: "C",
66   category: "Systems / General",
67   extension: ".c",
68   description: "Foundational procedural programming language for operating systems, hardware drivers, and
69 embedded systems",
70   starterCode: `#include <stdio.h>\n\nint main() {\n    printf("Hello, World! Coding Super Hub in C\\n");\n
71     \n    return 0;\n}\n`,

```

src/data/languages.ts (continued)

```

67     {
68         name: "C#",
69         category: "Application / Game",
70         extension: ".cs",
71         description: "Modern, object-oriented language developed by Microsoft for .NET, Unity games, and enterprise
72 apps",
73 starterCode: `using System;\n\nnamespace Program {\n    static void Main() {\n        Console.WriteLine("ðŸ‘Œ Welcome
to C# in Coding Super Hu
74     },
75     {
76         name: "Java",
77         category: "Enterprise / Android",
78         extension: ".java",
79         description: "Class-based, object-oriented language designed to have as few implementation dependencies as
80 possible",
81 starterCode: `public class Main {\n    public static void main(String[] args) {\n
System.out.println("Hello from Java! Build once
82     },
83     {
84         name: "Kotlin",
85         category: "Android / JVM",
86         extension: ".kt",
87         description: "Modern, expressive programming language and the preferred choice for native Android development.",
88 starterCode: `fun main() {\n    val languages = listOf("Kotlin", "Java", "Swift", "Dart")\n
println("Languages: \${languages.joinToSt
89     },
90     {
91         name: "Swift",
92         category: "Apple / Mobile",
93         extension: ".swift",
94         description: "Fast, safe, and modern language created by Apple for iOS, macOS, watchOS, and tvOS apps.",
95 starterCode: `import Foundation\n\nlet message = "Hello from Swift on iOS / macOS!"\nprint(message)\`,
96     },
97     {
98         name: "Dart",
99         category: "Flutter / Mobile",
100        extension: ".dart",
101        description: "Client-optimized language for fast apps on any platform, powering Flutter multi-platform apps.",
102 starterCode: `void main() {\n    var tech = ['Flutter', 'Dart', 'Web', 'Mobile'];\n    print('Building with: \
\${tech.join(" + ")}');\n}\`,
103     },
104     {
105         name: "PHP",
106         category: "Web",
107         extension: ".php",
108         description: "Widely-used open source general-purpose scripting language especially suited for web backend
109 development",
110 starterCode: `<?php\n$greeting = "Hello from PHP 8.x!";\necho $greeting;\n?>\`,
111     },
112     {
113         name: "Ruby",
114         category: "Web / General",
115         extension: ".rb",
116         description: "Dynamic, open source programming language with a focus on developer happiness, simplicity, and
117 productivity",
118 starterCode: `5.times do |i|\n    puts "Ruby iteration #{i + 1}: Developer happiness first!"\nend\`,
119     },
120     {
121         name: "R",
122         category: "Statistics / Data",
123         extension: ".r",
124         description: "Language and environment for statistical computing, graphics, bioinformatics, and data analysis.",
125 starterCode: `data <- c(10, 20, 35, 40, 55)\nprint(summary(data))\nprint(paste("Mean:", mean(data)))\`,
126     },
127     {
128         name: "SQL",
129         category: "Database / Query",
130         extension: ".sql",
131         description: "Domain-specific language used in programming and designed for managing data held in relational
132 DBMS",
133 starterCode: `-- SQL Starter\nSELECT \n    id, name, category, created_at \nFROM \n    developer_tools \nWHERE \n
is_active = TRUE \nORDER
134     },
135     {
136         name: "Bash",
137         category: "Shell / DevOps",
138         extension: ".sh",
139         description: "Unix shell and command language for automation, scripting, deployment, and server orchestration.",
140 starterCode: `#!/usr/bin/env bash\nnecho "ðŸ‘Œ Starting Coding Super Hub build pipeline..."&date\nnecho "Done!"\`,

```

src/data/languages.ts (continued)

```

136     },
137     {
138         name: "PowerShell",
139         category: "Shell / Windows",
140         extension: ".ps1",
141         description: "Task automation and configuration management framework consisting of a command-line shell and
142 scripting language",
143         starterCode: "Write-Output \"PowerShell script running...\"\\nGet-Date\\nWrite-Host \"Success!\" -ForegroundColor
144 Green",
145     },
146     {
147         name: "Lua",
148         category: "Scripting / Games",
149         extension: ".lua",
150         description: "Powerful, efficient, lightweight, embeddable scripting language widely used in games like Roblox
151 and Neovim",
152         starterCode: "-- Lua Starter\\nlocal player = { name = \"Hero\", level = 10 }\\nprint(string.format(\"Player %s is
153 level %d\", player.name, pl
154 level,
155     },
156     {
157         name: "Julia",
158         category: "Scientific / Data",
159         extension: ".jl",
160         description: "High-level, high-performance, dynamic programming language for technical and numerical
161 computing",
162         starterCode: "function mandelbrot(z)\\n    c = z\\n    for n = 1:80\\n        if abs(z) > 2 return n - 1
163 end\\n    z = z^2 + c\\n    end\\n
164     },
165     {
166         name: "Solidity",
167         category: "Blockchain",
168         extension: ".sol",
169         description: "Statically-typed curly-braces programming language designed for developing smart contracts on
170 Ethereum",
171         starterCode: "// SPDX-License-Identifier: MIT\\npragma solidity ^0.8.20;\\n\\ncontract CodingHubToken {\\n
172 string public name = \"SuperHub
173     },
174     {
175         name: "Zig",
176         category: "Systems",
177         extension: ".zig",
178         description: "General-purpose programming language and toolchain for maintaining robust, optimal and reusable
179 software",
180         starterCode: "const std = @import(\"std\");\\n\\npub fn main() !void {\\n    const stdout =
181 std.io.getStdOut().writer();\\n    try stdout.prin
182     },
183     {
184         name: "GraphQL",
185         category: "API Query",
186         extension: ".graphql",
187         description: "Query language for APIs and runtime for fulfilling queries with existing data.",
188         starterCode: "query GetDeveloperTools($category: String) {\\n    tools(category: $category) {\\n        id\\n
189 title\\n        category\\n        keyword
190     }\\n    }\\n
191     },
192     {
193         name: "JSON",
194         category: "Data Format",
195         extension: ".json",
196         description: "Lightweight data-interchange format easy for humans to read and write, and easy for machines to
197 parse",
198         starterCode: "{\\n    \"app\": \"Coding Super Hub\",\\n    \"toolsCount\": 150,\\n    \"features\": [\"AI Assistant\", \"Code
199 Editor\", \"Language Hub\"]\\n}",
200     },
201     {
202         name: "YAML",
203         category: "Config / Data",
204         extension: ".yaml",
205         description: "Human-readable data-serialization language commonly used for configuration files in Kubernetes
206 and CT/CD",
207         starterCode: "version: \"3.9\"\\nservices:\\n    superhub:\\n        image: node:20-alpine\\n        ports:\\n            -
208 \"3000:3000\"\\n    environment:\\n
209     },
210     {
211         name: "Markdown",
212         category: "Documentation",
213         extension: ".md",
214         description: "Lightweight markup language with plain text formatting syntax for documentation and README
215 files",
216         starterCode: "# ðŸš€ Coding Super Hub\\n\\nAll-in-one developer toolbox featuring:\\n- **150+ Interactive Coding
217 Tools**\\n- **Multi-language
218     },
219     {
220         name: "Dockerfile",
221         category: "DevOps",
222         extension: "Dockerfile",
223         description: "Text document that contains all commands a user could call on the command line to assemble a
224 Docker container image.",

```


src/data/languages.ts (continued)

```

205     starterCode: `FROM node:20-alpine AS builder\nWORKDIR /app\nCOPY package*.json ./\nRUN npm ci\nCOPY . .\nRUN
206     npm run build\n\nEXPOSE 300
207     {
208         name: "Elixir",
209         category: "BEAM / Distributed",
210         extension: ".ex",
211         description: "Dynamic, functional language for building scalable, fault-tolerant and maintainable applications
212         on the Erlang VM.",
213         starterCode: `defmodule CodingHub do\n  def greet(name) do\n    "Hello, #{name}! Distributed power with
214         Elixir."\n  end\nend\n\nIO.puts
215     {
216         name: "Haskell",
217         category: "Functional",
218         extension: ".hs",
219         description: "Advanced purely functional programming language with strong static typing and lazy evaluation.",
220         starterCode: `quicksort :: (Ord a) => [a] -> [a]\nquicksort [] = []\nquicksort (x:xs) =\n  let smaller =
221         quicksort [a | a <- xs, a <=
222     {
223         name: "Scala",
224         category: "JVM / Data",
225         extension: ".scala",
226         description: "Concise, elegant language integrating both object-oriented and functional programming on the
227         JVM.",
228         starterCode: `object Main extends App {\n  val languages = List("Scala", "Java", "Kotlin")\n
229         languages.map(_.toUpperCase).foreach(print
230     {
231         name: "Perl",
232         category: "Scripting",
233         extension: ".pl",
234         description: "Highly capable, feature-rich programming language with over 30 years of development.",
235         starterCode: `#!/usr/bin/perl\nuse strict;\nuse warnings;\nmy $msg = "Hello from Perl!";\nprint "$msg\n";`,
236     },
237     {
238         name: "Assembly",
239         category: "Low-level",
240         extension: ".asm",
241         description: "Low-level programming language that has a very strong correspondence with architecture's machine
242         code.",
243         starterCode: `section .data\n  msg db 'Hello, World!', 0xa\n  len equ $ - msg\nsection .text\n  global
244         _start\n_start:\n  mov
245     {
246         name: "WebAssembly",
247         category: "Web / Low-level",
248         extension: ".wat",
249         description: "Binary instruction format for a stack-based virtual machine, designed for high-performance
250         execution on the web.",
251         starterCode: `(module\n  (func $add (param $a i32) (param $b i32) (result i32)\n    local.get $a\n    local.get
252         $b\n    i32.add\n  )\n
253     {
254         name: "Vyper",
255         category: "Blockchain",
256         extension: ".vy",
257         description: "Pythonic, contract-oriented programming language targeting the Ethereum Virtual Machine (EVM)
258         designed for security and
259         self-stored_data = x`,
260     {
261         name: "GDScript",
262         category: "Game Development",
263         extension: ".gd",
264         description: "High-level, dynamically typed programming language used to create content inside the Godot Game
265         Engine.",
266         starterCode: `extends Node2D\n\nfunc _ready():\n  print("Godot 4 node initialized!")`,
267     },
268     {
269         name: "Verilog",
270         category: "Hardware / HDL",
271         extension: ".v",
272         description: "Hardware description language (HDL) used to model electronic systems such as FPGAs and ASICs.",
273         starterCode: `module adder(\n  input [3:0] a,\n  input [3:0] b,\n  output [4:0] sum\n);\n  assign sum = a + b;
274     \nendmodule`,
275     {
276         name: "VHDL",
277         category: "Hardware / HDL",
278         extension: ".vhd",

```

src/data/languages.ts (continued)

```

274     description: "Standard hardware description language used in electronic design automation to describe digital
275 and mixed-signal systems",
276 y }, out STD_LOGIC );\nend N
277 {
278     name: "Nim",
279     category: "Systems / General",
280     extension: ".nim",
281     description: "Statically typed, compiled systems programming language combining the speed of C with the
282 expressive power of Python",
283 },
284 {
285     name: "Fortran",
286     category: "Scientific / HPC",
287     extension: ".f90",
288     description: "General-purpose, compiled imperative programming language especially suited to numeric
289 computer code",
290 },
291 {
292     name: "COBOL",
293     category: "Legacy / Enterprise",
294     extension: ".cbl",
295     description: "Compiled English-like computer programming language designed for business use in banking,
296 insurance, and enterprise",
297 },
298 {
299     name: "Objective-C",
300     category: "Apple Legacy",
301     extension: ".m",
302     description: "General-purpose, object-oriented programming language that adds Smalltalk-style messaging to the
303 C programming language",
304 },
305 {
306     name: "F#",
307     category: "Functional / .NET",
308     extension: ".fs",
309     description: "Cross-platform, open-source, functional-first programming language for .NET.",
310     starterCode: `let square x = x * x\nprintfn "Square of 5 is %d" (square 5)\`,
311 },
312 {
313     name: "OCaml",
314     category: "Functional",
315     extension: ".ml",
316     description: "Industrial strength programming language supporting functional, imperative and object-oriented
317 styles with powerful type inference",
318 },
319 {
320     name: "Move",
321     category: "Blockchain",
322     extension: ".move",
323     description: "Safe and secure programming language for Web3 smart contracts originally developed for Aptos and
324 Sui blockchains",
325 },
326 {
327     name: "HCL / Terraform",
328     category: "Infrastructure",
329     extension: ".tf",
330     description: "HashiCorp Configuration Language used to describe infrastructure as code across clouds.",
331     starterCode: `resource "google_compute_instance" "default" {\n  name = "coding-super-hub-vm"\n  machine_type = "e2-medium"\n  zone = "us-east1-a"\n}\`,
332 },
333 {
334     name: "Protocol Buffers",
335     category: "Data / RPC",
336     extension: ".proto",
337     description: "Language-neutral, platform-neutral, extensible mechanism for serializing structured data used in
338 gRPC",
339 },
340 ];
341

```

FILE [19/25]

src/data/toolsMetadata.ts

1411 lines • 22.1 KB • Registry & metadata of all 150 dev tools

```
1 export const ALL_TOOLS_METADATA = [  
2   {  
3     "id": 2,  
4     "title": "Text Case Converter",  
5     "category": "Text",  
6     "keywords": [  
7       "uppercase",  
8       "lowercase",  
9       "titlecase",  
10      "camelcase",  
11      "slug"  
12    ]  
13  },  
14  {  
15    "id": 3,  
16    "title": "Word & Character Counter",  
17    "category": "Text",  
18    "keywords": [  
19      "words",  
20      "chars",  
21      "length",  
22      "reading time"  
23    ]  
24  },  
25  {  
26    "id": 4,  
27    "title": "JSON Formatter",  
28    "category": "Formatting",  
29    "keywords": [  
30      "pretty",  
31      "minify",  
32      "json",  
33      "beautify"  
34    ]  
35  },  
36  {  
37    "id": 5,  
38    "title": "Base64 Encoder / Decoder",  
39    "category": "Converters",  
40    "keywords": [  
41      "btoa",  
42      "atob",  
43      "encode",  
44      "decode"  
45    ]  
46  },  
47  {  
48    "id": 6,  
49    "title": "URL Encoder / Decoder",  
50    "category": "Converters",  
51    "keywords": [  
52      "uri",  
53      "encodeURIComponent"  
54    ]  
55  },  
56  {  
57    "id": 7,  
58    "title": "Password Generator",  
59    "category": "Security",  
60    "keywords": [  
61      "password",  
62      "random",  
63      "generator"  
64    ]  
65  },  
66  {
```

src/data/toolsMetadata.ts (continued)

```
67     "id": 8,
68     "title": "Color Converter",
69     "category": "Color",
70     "keywords": [
71       "hex",
72       "rgb",
73       "hsl"
74     ]
75   },
76   {
77     "id": 9,
78     "title": "Timestamp Converter",
79     "category": "Time",
80     "keywords": [
81       "unix",
82       "epoch",
83       "date"
84     ]
85   },
86   {
87     "id": 10,
88     "title": "HTML Entity Encoder / Decoder",
89     "category": "HTML",
90     "keywords": [
91       "entities",
92       "escape",
93       "unescape"
94     ]
95   },
96   {
97     "id": 11,
98     "title": "UUID Generator",
99     "category": "Generators",
100    "keywords": [
101      "uuid",
102      "guid",
103      "v4"
104    ]
105  },
106  {
107    "id": 12,
108    "title": "Text to Slug Converter",
109    "category": "Text",
110    "keywords": [
111      "slug",
112      "url",
113      "seo"
114    ]
115  },
116  {
117    "id": 13,
118    "title": "Regex Tester",
119    "category": "Text",
120    "keywords": [
121      "regular expression",
122      "match"
123    ]
124  },
125  {
126    "id": 14,
127    "title": "HTML Previewer",
128    "category": "HTML",
129    "keywords": [
130      "sandbox",
131      "preview",
132      "iframe"
133    ]
134  },
135  {
```

src/data/toolsMetadata.ts (continued)

```
136     "id": 15,
137     "title": "CSS Gradient Generator",
138     "category": "CSS",
139     "keywords": [
140       "gradient",
141       "linear-gradient"
142     ]
143   },
144   {
145     "id": 16,
146     "title": "Markdown Previewer",
147     "category": "Formatting",
148     "keywords": [
149       "markdown",
150       "md",
151       "preview"
152     ]
153   },
154   {
155     "id": 17,
156     "title": "Number Base Converter",
157     "category": "Math",
158     "keywords": [
159       "binary",
160       "decimal",
161       "hexadecimal",
162       "octal"
163     ]
164   },
165   {
166     "id": 18,
167     "title": "JWT Decoder",
168     "category": "Security",
169     "keywords": [
170       "jwt",
171       "token",
172       "payload",
173       "json web token"
174     ]
175   },
176   {
177     "id": 19,
178     "title": "HTML/CSS/JS Minifier",
179     "category": "Formatting",
180     "keywords": [
181       "minify",
182       "compress",
183       "strip"
184     ]
185   },
186   {
187     "id": 20,
188     "title": "Lorem Ipsum Generator",
189     "category": "Generators",
190     "keywords": [
191       "lorem",
192       "ipsum",
193       "placeholder text"
194     ]
195   },
196   {
197     "id": 21,
198     "title": "Color Palette Generator",
199     "category": "Color",
200     "keywords": [
201       "palette",
202       "tints",
203       "shades"
204     ]
205   }
```

src/data/toolsMetadata.ts (continued)

```
205     },
206     {
207       "id": 22,
208       "title": "QR Code Generator",
209       "category": "Generators",
210       "keywords": [
211         "qrcode",
212         "barcode"
213       ]
214     },
215     {
216       "id": 23,
217       "title": "IP Address Info",
218       "category": "Utilities",
219       "keywords": [
220         "ip",
221         "geo",
222         "network",
223         "client"
224       ]
225     },
226     {
227       "id": 24,
228       "title": "Age Calculator",
229       "category": "Math",
230       "keywords": [
231         "age",
232         "birthday",
233         "years"
234       ]
235     },
236     {
237       "id": 25,
238       "title": "Random Number Generator",
239       "category": "Generators",
240       "keywords": [
241         "random",
242         "dice",
243         "number"
244       ]
245     },
246     {
247       "id": 26,
248       "title": "Temperature Converter",
249       "category": "Converters",
250       "keywords": [
251         "celsius",
252         "fahrenheit",
253         "kelvin"
254       ]
255     },
256     {
257       "id": 27,
258       "title": "BMI Calculator",
259       "category": "Math",
260       "keywords": [
261         "bmi",
262         "body mass",
263         "health"
264       ]
265     },
266     {
267       "id": 28,
268       "title": "Percentage Calculator",
269       "category": "Math",
270       "keywords": [
271         "percent",
272         "ratio",
273         "fraction"
```

src/data/toolsMetadata.ts (continued)

```
274     ]
275   },
276   {
277     "id": 29,
278     "title": "Tip Calculator",
279     "category": "Math",
280     "keywords": [
281       "tip",
282       "bill",
283       "restaurant"
284     ]
285   },
286   {
287     "id": 30,
288     "title": "Discount Calculator",
289     "category": "Math",
290     "keywords": [
291       "discount",
292       "sale",
293       "price"
294     ]
295   },
296   {
297     "id": 31,
298     "title": "EMI Calculator",
299     "category": "Math",
300     "keywords": [
301       "loan",
302       "emi",
303       "mortgage"
304     ]
305   },
306   {
307     "id": 32,
308     "title": "Compound Interest Calculator",
309     "category": "Math",
310     "keywords": [
311       "interest",
312       "finance"
313     ]
314   },
315   {
316     "id": 33,
317     "title": "Simple Interest Calculator",
318     "category": "Math",
319     "keywords": [
320       "simple interest",
321       "principal"
322     ]
323   },
324   {
325     "id": 34,
326     "title": "Average Calculator",
327     "category": "Math",
328     "keywords": [
329       "mean",
330       "average"
331     ]
332   },
333   {
334     "id": 35,
335     "title": "Sum Calculator",
336     "category": "Math",
337     "keywords": [
338       "addition",
339       "total"
340     ]
341   },
342   {
```

src/data/toolsMetadata.ts (continued)

```
343     "id": 36,
344     "title": "Unit Converter",
345     "category": "Converters",
346     "keywords": [
347       "km",
348       "miles",
349       "kg",
350       "pounds",
351       "meters",
352       "feet"
353   ],
354 },
355 {
356     "id": 37,
357     "title": "Time Converter",
358     "category": "Time",
359     "keywords": [
360       "minutes",
361       "hours"
362   ],
363 },
364 {
365     "id": 38,
366     "title": "Seconds Converter",
367     "category": "Time",
368     "keywords": [
369       "seconds",
370       "duration"
371   ],
372 },
373 {
374     "id": 39,
375     "title": "Days Calculator",
376     "category": "Time",
377     "keywords": [
378       "days difference",
379       "dates"
380   ],
381 },
382 {
383     "id": 40,
384     "title": "Leap Year Checker",
385     "category": "Math",
386     "keywords": [
387       "leap",
388       "calendar",
389       "february"
390   ],
391 },
392 {
393     "id": 41,
394     "title": "Prime Number Checker",
395     "category": "Math",
396     "keywords": [
397       "prime",
398       "divisible"
399   ],
400 },
401 {
402     "id": 42,
403     "title": "Even / Odd Checker",
404     "category": "Math",
405     "keywords": [
406       "even",
407       "odd",
408       "parity"
409   ],
410 },
411 {
```


src/data/toolsMetadata.ts (continued)

```
412     "id": 43,
413     "title": "Factorial Calculator",
414     "category": "Math",
415     "keywords": [
416       "factorial",
417       "n!"
418     ]
419   },
420   {
421     "id": 44,
422     "title": "Fibonacci Generator",
423     "category": "Math",
424     "keywords": [
425       "fibonacci",
426       "sequence"
427     ]
428   },
429   {
430     "id": 45,
431     "title": "GCD Calculator",
432     "category": "Math",
433     "keywords": [
434       "gcd",
435       "greatest common divisor"
436     ]
437   },
438   {
439     "id": 46,
440     "title": "LCM Calculator",
441     "category": "Math",
442     "keywords": [
443       "lcm",
444       "least common multiple"
445     ]
446   },
447   {
448     "id": 47,
449     "title": "Binary Calculator",
450     "category": "Math",
451     "keywords": [
452       "binary to decimal"
453     ]
454   },
455   {
456     "id": 48,
457     "title": "Hexadecimal Calculator",
458     "category": "Math",
459     "keywords": [
460       "hex to decimal"
461     ]
462   },
463   {
464     "id": 49,
465     "title": "Random Password Generator",
466     "category": "Security",
467     "keywords": [
468       "password",
469       "strong"
470     ]
471   },
472   {
473     "id": 50,
474     "title": "Strong Password Checker",
475     "category": "Security",
476     "keywords": [
477       "password strength",
478       "entropy"
479     ]
480   },
```

src/data/toolsMetadata.ts (continued)

```
481   {
482     "id": 51,
483     "title": "JSON Validator",
484     "category": "Data",
485     "keywords": [
486       "json",
487       "validate",
488       "syntax"
489     ]
490   },
491   {
492     "id": 52,
493     "title": "JSON to CSV",
494     "category": "Converters",
495     "keywords": [
496       "csv",
497       "table",
498       "export"
499     ]
500   },
501   {
502     "id": 53,
503     "title": "CSV to JSON",
504     "category": "Converters",
505     "keywords": [
506       "csv to json",
507       "table"
508     ]
509   },
510   {
511     "id": 54,
512     "title": "JSON Sorter",
513     "category": "Formatting",
514     "keywords": [
515       "sort keys",
516       "alphabetical"
517     ]
518   },
519   {
520     "id": 55,
521     "title": "XML Formatter",
522     "category": "Formatting",
523     "keywords": [
524       "xml",
525       "pretty",
526       "indent"
527     ]
528   },
529   {
530     "id": 56,
531     "title": "Text Sorter",
532     "category": "Text",
533     "keywords": [
534       "sort lines",
535       "alphabetical"
536     ]
537   },
538   {
539     "id": 57,
540     "title": "Duplicate Line Remover",
541     "category": "Text",
542     "keywords": [
543       "dedupe",
544       "unique"
545     ]
546   },
547   {
548     "id": 58,
549     "title": "Line Counter",
```

src/data/toolsMetadata.ts (continued)

```
550     "category": "Text",
551     "keywords": [
552       "lines",
553       "empty lines"
554     ]
555   },
556   {
557     "id": 59,
558     "title": "Whitespace Cleaner",
559     "category": "Text",
560     "keywords": [
561       "whitespace",
562       "trim",
563       "collapse"
564     ]
565   },
566   {
567     "id": 60,
568     "title": "Reverse Text",
569     "category": "Text",
570     "keywords": [
571       "reverse",
572       "backwards"
573     ]
574   },
575   {
576     "id": 61,
577     "title": "Palindrome Checker",
578     "category": "Text",
579     "keywords": [
580       "palindrome"
581     ]
582   },
583   {
584     "id": 62,
585     "title": "Anagram Checker",
586     "category": "Text",
587     "keywords": [
588       "anagram"
589     ]
590   },
591   {
592     "id": 63,
593     "title": "Word Frequency Counter",
594     "category": "Text",
595     "keywords": [
596       "frequency",
597       "occurrence"
598     ]
599   },
600   {
601     "id": 64,
602     "title": "Text Deduplicator",
603     "category": "Text",
604     "keywords": [
605       "dedup words"
606     ]
607   },
608   {
609     "id": 65,
610     "title": "Character Frequency",
611     "category": "Text",
612     "keywords": [
613       "letters count"
614     ]
615   },
616   {
617     "id": 66,
618     "title": "Morse Code Converter",
```

src/data/toolsMetadata.ts (continued)

```
619     "category": "Converters",
620     "keywords": [
621       "morse",
622       "telegraph"
623     ]
624   },
625   {
626     "id": 67,
627     "title": "Binary Text Converter",
628     "category": "Converters",
629     "keywords": [
630       "binary text",
631       "ascii binary"
632     ]
633   },
634   {
635     "id": 68,
636     "title": "SHA-256 Hash Generator",
637     "category": "Hashes",
638     "keywords": [
639       "sha256",
640       "checksum"
641     ]
642   },
643   {
644     "id": 69,
645     "title": "URL Parser",
646     "category": "Web",
647     "keywords": [
648       "url breakdown",
649       "protocol",
650       "pathname"
651     ]
652   },
653   {
654     "id": 70,
655     "title": "Query String Parser",
656     "category": "Web",
657     "keywords": [
658       "querystring",
659       "params"
660     ]
661   },
662   {
663     "id": 71,
664     "title": "Browser Information",
665     "category": "Utilities",
666     "keywords": [
667       "browser",
668       "user-agent",
669       "platform"
670     ]
671   },
672   {
673     "id": 72,
674     "title": "Screen & Viewport Info",
675     "category": "Utilities",
676     "keywords": [
677       "resolution",
678       "viewport"
679     ]
680   },
681   {
682     "id": 73,
683     "title": "Meta Tag Generator",
684     "category": "Web",
685     "keywords": [
686       "seo",
687       "html meta",
```

src/data/toolsMetadata.ts (continued)

```
688     "opengraph"
689   ]
690 },
691 {
692   "id": 74,
693   "title": "HTML Boilerplate Generator",
694   "category": "HTML",
695   "keywords": [
696     "template",
697     "html5"
698   ]
699 },
700 {
701   "id": 75,
702   "title": "CSS Reset Generator",
703   "category": "CSS",
704   "keywords": [
705     "reset",
706     "normalize",
707     "box-sizing"
708   ]
709 },
710 {
711   "id": 76,
712   "title": "CSS Flexbox Generator",
713   "category": "CSS",
714   "keywords": [
715     "flex",
716     "justify-content",
717     "align-items"
718   ]
719 },
720 {
721   "id": 77,
722   "title": "CSS Grid Generator",
723   "category": "CSS",
724   "keywords": [
725     "grid",
726     "columns"
727   ]
728 },
729 {
730   "id": 78,
731   "title": "Box Shadow Generator",
732   "category": "CSS",
733   "keywords": [
734     "shadow",
735     "box-shadow"
736   ]
737 },
738 {
739   "id": 79,
740   "title": "Border Radius Generator",
741   "category": "CSS",
742   "keywords": [
743     "rounded",
744     "border-radius"
745   ]
746 },
747 {
748   "id": 80,
749   "title": "CSS Clamp Generator",
750   "category": "CSS",
751   "keywords": [
752     "clamp",
753     "fluid typography"
754   ]
755 },
756 {
```

src/data/toolsMetadata.ts (continued)

```
757     "id": 81,
758     "title": "PX <-> REM Converter",
759     "category": "Converters",
760     "keywords": [
761       "px to rem",
762       "rem to px"
763     ]
764   },
765   {
766     "id": 82,
767     "title": "Aspect Ratio Calculator",
768     "category": "Math",
769     "keywords": [
770       "aspect ratio",
771       "16:9",
772       "4:3"
773     ]
774   },
775   {
776     "id": 83,
777     "title": "Random Color Generator",
778     "category": "Color",
779     "keywords": [
780       "random color",
781       "hex"
782     ]
783   },
784   {
785     "id": 84,
786     "title": "Color Contrast Checker",
787     "category": "Color",
788     "keywords": [
789       "contrast ratio",
790       "wcag"
791     ]
792   },
793   {
794     "id": 85,
795     "title": "Data URI Generator",
796     "category": "Converters",
797     "keywords": [
798       "data uri",
799       "base64 text"
800     ]
801   },
802   {
803     "id": 86,
804     "title": "Favicon HTML Generator",
805     "category": "HTML",
806     "keywords": [
807       "favicon",
808       "icon"
809     ]
810   },
811   {
812     "id": 87,
813     "title": "Robots.txt Generator",
814     "category": "Web",
815     "keywords": [
816       "robots",
817       "crawler"
818     ]
819   },
820   {
821     "id": 88,
822     "title": "Sitemap XML Generator",
823     "category": "Web",
824     "keywords": [
825       "sitemap",
```

src/data/toolsMetadata.ts (continued)

```
826         "xml"
827     ]
828 },
829 {
830     "id": 89,
831     "title": "Gitignore Generator",
832     "category": "DevOps",
833     "keywords": [
834         "gitignore",
835         "git"
836     ]
837 },
838 {
839     "id": 90,
840     "title": "README Generator",
841     "category": "DevOps",
842     "keywords": [
843         "readme",
844         "markdown"
845     ]
846 },
847 {
848     "id": 91,
849     "title": "JS Function Generator",
850     "category": "JavaScript",
851     "keywords": [
852         "function template",
853         "snippet"
854     ]
855 },
856 {
857     "id": 92,
858     "title": "Fetch API Snippet",
859     "category": "JavaScript",
860     "keywords": [
861         "fetch",
862         "ajax",
863         "api call"
864     ]
865 },
866 {
867     "id": 93,
868     "title": "SQL Formatter",
869     "category": "Formatting",
870     "keywords": [
871         "sql",
872         "query"
873     ]
874 },
875 {
876     "id": 94,
877     "title": "SQL String Escaper",
878     "category": "Security",
879     "keywords": [
880         "sql injection",
881         "escape single quote"
882     ]
883 },
884 {
885     "id": 95,
886     "title": "Date Formatter",
887     "category": "Time",
888     "keywords": [
889         "date format",
890         "iso",
891         "utc"
892     ]
893 },
894 {
```

src/data/toolsMetadata.ts (continued)

```
895     "id": 96,
896     "title": "Unix Timestamp Now",
897     "category": "Time",
898     "keywords": [
899       "current epoch",
900       "now"
901   ],
902 },
903 {
904     "id": 97,
905     "title": "HMAC SHA-256 Generator",
906     "category": "Hashes",
907     "keywords": [
908       "hmac",
909       "secret signature"
910   ],
911 },
912 {
913     "id": 98,
914     "title": "UUID Bulk Generator",
915     "category": "Generators",
916     "keywords": [
917       "multiple uuids",
918       "bulk guid"
919   ],
920 },
921 {
922     "id": 99,
923     "title": "HTTP Status Code Lookup",
924     "category": "API",
925     "keywords": [
926       "status code",
927       "200",
928       "404",
929       "500"
930   ],
931 },
932 {
933     "id": 100,
934     "title": "Developer Cheat Sheet",
935     "category": "Utilities",
936     "keywords": [
937       "cheat sheet",
938       "git",
939       "docker",
940       "linux"
941   ],
942 },
943 {
944     "id": 101,
945     "title": "Text Diff Checker",
946     "category": "Text",
947     "keywords": [
948       "diff",
949       "compare",
950       "changes"
951   ],
952 },
953 {
954     "id": 102,
955     "title": "Text Joiner",
956     "category": "Text",
957     "keywords": [
958       "join",
959       "delimiter",
960       "combine"
961   ],
962 },
963 {
```


src/data/toolsMetadata.ts (continued)

```
964     "id": 103,
965     "title": "Line Break Converter",
966     "category": "Formatting",
967     "keywords": [
968       "crlf",
969       "lf",
970       "windows",
971       "unix"
972   ]
973 },
974 {
975     "id": 104,
976     "title": "Tabs <-> Spaces Converter",
977     "category": "Formatting",
978     "keywords": [
979       "tabs",
980       "spaces",
981       "indentation"
982   ]
983 },
984 {
985     "id": 105,
986     "title": "Indent Formatter",
987     "category": "Formatting",
988     "keywords": [
989       "indent",
990       "add spaces"
991   ]
992 },
993 {
994     "id": 106,
995     "title": "Text Extractor",
996     "category": "Text",
997     "keywords": [
998       "emails",
999       "links",
1000       "urls"
1001   ]
1002 },
1003 {
1004     "id": 107,
1005     "title": "Text Trimmer",
1006     "category": "Text",
1007     "keywords": [
1008       "trim",
1009       "strip spaces"
1010   ]
1011 },
1012 {
1013     "id": 108,
1014     "title": "Number List Generator",
1015     "category": "Generators",
1016     "keywords": [
1017       "sequence",
1018       "range",
1019       "numbers"
1020   ]
1021 },
1022 {
1023     "id": 109,
1024     "title": "Text Padding Generator",
1025     "category": "Text",
1026     "keywords": [
1027       "padstart",
1028       "padend",
1029       "zero-fill"
1030   ]
1031 },
1032 {
```

src/data/toolsMetadata.ts (continued)

```
1033     "id": 110,
1034     "title": "Text Wrap Converter",
1035     "category": "Text",
1036     "keywords": [
1037       "word wrap",
1038       "column width"
1039   ],
1040 },
1041 {
1042     "id": 111,
1043     "title": "MD5 Hash Generator",
1044     "category": "Hashes",
1045     "keywords": [
1046       "md5",
1047       "checksum"
1048   ],
1049 },
1050 {
1051     "id": 112,
1052     "title": "SHA-1 Hash Generator",
1053     "category": "Hashes",
1054     "keywords": [
1055       "sha1",
1056       "git hash"
1057   ],
1058 },
1059 {
1060     "id": 113,
1061     "title": "SHA-384 Hash Generator",
1062     "category": "Hashes",
1063     "keywords": [
1064       "sha384",
1065       "sri"
1066   ],
1067 },
1068 {
1069     "id": 114,
1070     "title": "SHA-512 Hash Generator",
1071     "category": "Hashes",
1072     "keywords": [
1073       "sha512"
1074   ],
1075 },
1076 {
1077     "id": 115,
1078     "title": "HMAC SHA-512 Generator",
1079     "category": "Hashes",
1080     "keywords": [
1081       "hmac sha512"
1082   ],
1083 },
1084 {
1085     "id": 116,
1086     "title": "Hex Encoder / Decoder",
1087     "category": "Converters",
1088     "keywords": [
1089       "hex to text",
1090       "text to hex"
1091   ],
1092 },
1093 {
1094     "id": 117,
1095     "title": "ROT13 Encoder / Decoder",
1096     "category": "Security",
1097     "keywords": [
1098       "rot13",
1099       "caesar"
1100   ],
1101 },
```

src/data/toolsMetadata.ts (continued)

```
1102 {
1103   "id": 118,
1104   "title": "ROT47 Encoder / Decoder",
1105   "category": "Security",
1106   "keywords": [
1107     "rot47",
1108     "ascii rotate"
1109   ]
1110 },
1111 {
1112   "id": 119,
1113   "title": "Caesar Cipher",
1114   "category": "Security",
1115   "keywords": [
1116     "caesar",
1117     "shift"
1118   ]
1119 },
1120 {
1121   "id": 120,
1122   "title": "AES-GCM Key Generator",
1123   "category": "Security",
1124   "keywords": [
1125     "aes",
1126     "key",
1127     "web crypto"
1128   ]
1129 },
1130 {
1131   "id": 121,
1132   "title": "HTTP Header Generator",
1133   "category": "Security",
1134   "keywords": [
1135     "cors",
1136     "csp",
1137     "security headers"
1138   ]
1139 },
1140 {
1141   "id": 122,
1142   "title": "cURL Request Builder",
1143   "category": "API",
1144   "keywords": [
1145     "curl",
1146     "bash http"
1147   ]
1148 },
1149 {
1150   "id": 123,
1151   "title": "HTTP Response Header Viewer",
1152   "category": "API",
1153   "keywords": [
1154     "headers",
1155     "inspect"
1156   ]
1157 },
1158 {
1159   "id": 124,
1160   "title": "MIME Type Lookup",
1161   "category": "Web",
1162   "keywords": [
1163     "mime",
1164     "content-type",
1165     "extension"
1166   ]
1167 },
1168 {
1169   "id": 125,
1170   "title": "User-Agent Parser",
```

src/data/toolsMetadata.ts (continued)

```
1171     "category": "Utilities",
1172     "keywords": [
1173       "user-agent",
1174       "browser engine"
1175     ]
1176   },
1177   {
1178     "id": 126,
1179     "title": "URL Shortener Generator",
1180     "category": "Web",
1181     "keywords": [
1182       "shortener",
1183       "hashid",
1184       "slug"
1185     ]
1186   },
1187   {
1188     "id": 127,
1189     "title": "Open Graph Meta Generator",
1190     "category": "Web",
1191     "keywords": [
1192       "og:image",
1193       "facebook",
1194       "linkedin"
1195     ]
1196   },
1197   {
1198     "id": 128,
1199     "title": "Twitter Card Generator",
1200     "category": "Web",
1201     "keywords": [
1202       "twitter:card",
1203       "social cards"
1204     ]
1205   },
1206   {
1207     "id": 129,
1208     "title": "Canonical URL Generator",
1209     "category": "Web",
1210     "keywords": [
1211       "canonical",
1212       "duplicate content"
1213     ]
1214   },
1215   {
1216     "id": 130,
1217     "title": "Web Manifest Generator",
1218     "category": "Web",
1219     "keywords": [
1220       "manifest.json",
1221       "pwa"
1222     ]
1223   },
1224   {
1225     "id": 131,
1226     "title": "CSS Button Generator",
1227     "category": "CSS",
1228     "keywords": [
1229       "button styles",
1230       "custom button"
1231     ]
1232   },
1233   {
1234     "id": 132,
1235     "title": "CSS Text Shadow Generator",
1236     "category": "CSS",
1237     "keywords": [
1238       "text-shadow",
1239       "typography glow"
```

src/data/toolsMetadata.ts (continued)

```
1240     ]
1241   },
1242   {
1243     "id": 133,
1244     "title": "CSS Animation Generator",
1245     "category": "CSS",
1246     "keywords": [
1247       "@keyframes",
1248       "animation"
1249     ]
1250   },
1251   {
1252     "id": 134,
1253     "title": "CSS Transform Generator",
1254     "category": "CSS",
1255     "keywords": [
1256       "rotate",
1257       "scale",
1258       "transform"
1259     ]
1260   },
1261   {
1262     "id": 135,
1263     "title": "CSS Transition Generator",
1264     "category": "CSS",
1265     "keywords": [
1266       "transition",
1267       "timing-function"
1268     ]
1269   },
1270   {
1271     "id": 136,
1272     "title": "CSS Filter Generator",
1273     "category": "CSS",
1274     "keywords": [
1275       "filter",
1276       "blur",
1277       "brightness"
1278     ]
1279   },
1280   {
1281     "id": 137,
1282     "title": "CSS Glassmorphism Generator",
1283     "category": "CSS",
1284     "keywords": [
1285       "glassmorphism",
1286       "backdrop-filter"
1287     ]
1288   },
1289   {
1290     "id": 138,
1291     "title": "CSS Neumorphism Generator",
1292     "category": "CSS",
1293     "keywords": [
1294       "neumorphism",
1295       "soft ui"
1296     ]
1297   },
1298   {
1299     "id": 139,
1300     "title": "CSS Triangle Generator",
1301     "category": "CSS",
1302     "keywords": [
1303       "triangle",
1304       "border hack"
1305     ]
1306   },
1307   {
1308     "id": 140,
```

src/data/toolsMetadata.ts (continued)

```
1309     "title": "CSS Spinner Generator",
1310     "category": "CSS",
1311     "keywords": [
1312       "spinner",
1313       "loading"
1314   ],
1315 },
1316 {
1317     "id": 141,
1318     "title": "JS Deep Clone Helper",
1319     "category": "JavaScript",
1320     "keywords": [
1321       "clone",
1322       "structuredclone",
1323       "deep copy"
1324   ],
1325 },
1326 {
1327     "id": 142,
1328     "title": "JS Array Generator",
1329     "category": "JavaScript",
1330     "keywords": [
1331       "array.from",
1332       "fill"
1333   ],
1334 },
1335 {
1336     "id": 143,
1337     "title": "Array Methods Cheat Sheet",
1338     "category": "JavaScript",
1339     "keywords": [
1340       "map",
1341       "filter",
1342       "reduce",
1343       "find"
1344   ],
1345 },
1346 {
1347     "id": 144,
1348     "title": "Date Helper Snippets",
1349     "category": "JavaScript",
1350     "keywords": [
1351       "date difference",
1352       "add days"
1353   ],
1354 },
1355 {
1356     "id": 145,
1357     "title": "Debounce Function Snippet",
1358     "category": "JavaScript",
1359     "keywords": [
1360       "debounce",
1361       "rate limit",
1362       "search input"
1363   ],
1364 },
1365 {
1366     "id": 146,
1367     "title": "Throttle Function Snippet",
1368     "category": "JavaScript",
1369     "keywords": [
1370       "throttle",
1371       "scroll performance"
1372   ],
1373 },
1374 {
1375     "id": 147,
1376     "title": "Event Listener Snippet",
1377     "category": "JavaScript",
```

src/data/toolsMetadata.ts (continued)

```
1378     "keywords": [  
1379         "addeventlistener",  
1380         "dom event"  
1381     ]  
1382 },  
1383 {  
1384     "id": 148,  
1385     "title": "DOM Selector Generator",  
1386     "category": "JavaScript",  
1387     "keywords": [  
1388         "queryselector",  
1389         "queryselectorall"  
1390     ]  
1391 },  
1392 {  
1393     "id": 149,  
1394     "title": "LocalStorage Code Helper",  
1395     "category": "JavaScript",  
1396     "keywords": [  
1397         "localstorage",  
1398         "json storage"  
1399     ]  
1400 },  
1401 {  
1402     "id": 150,  
1403     "title": "SessionStorage Code Helper",  
1404     "category": "JavaScript",  
1405     "keywords": [  
1406         "sessionstorage",  
1407         "tab session"  
1408     ]  
1409 }  
1410 ] as const;  
1411
```

FILE [20/25]

src/utils/helpers.ts

260 lines • 7.3 KB • Conversion, formatting, math & string utilities

```
1 // General Utilities for 150 Coding Super Hub Tools
2
3 export function safeBase64Encode(str: string): string {
4     return btoa(unescape(encodeURIComponent(str)));
5 }
6
7 export function safeBase64Decode(str: string): string {
8     return decodeURIComponent(escape(atob(str)));
9 }
10
11 export function generateUUIDv4(): string {
12     if (typeof crypto !== "undefined" && crypto.randomUUID) {
13         return crypto.randomUUID();
14     }
15     return "xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx".replace(/[xy]/g, (c) => {
16         const r = (Math.random() * 16) | 0;
17         const v = c === "x" ? r : (r & 0x3) | 0x8;
18         return v.toString(16);
19     });
20 }
21
22 export function textToSlug(text: string): string {
23     return text
24         .toLowerCase()
25         .normalize("NFD")
26         .replace(/[\u0300-\u036f]/g, "")
27         .replace(/[\^a-z0-9\s-]/g, "")
28         .trim()
29         .replace(/\s+/g, "-")
30         .replace(/-+/g, "-")
31         .replace(/^\^-|$/g, "");
32 }
33
34 export function decodeJWTToken(token: string): { header: any; payload: any; isExpired?: boolean } {
35     const parts = token.trim().split(".");
36     if (parts.length !== 3) {
37         throw new Error("Invalid JWT token format (expected 3 parts separated by dots)");
38     }
39
40     const decodePart = (part: string) => {
41         const base64 = part.replace(/-/g, "+").replace(/_/g, "/");
42         const padded = base64 + "=".repeat((4 - (base64.length % 4)) % 4);
43         return JSON.parse(atob(padded));
44     };
45
46     const header = decodePart(parts[0]);
47     const payload = decodePart(parts[1]);
48
49     let isExpired: boolean | undefined = undefined;
50     if (payload && typeof payload.exp === "number") {
51         isExpired = Date.now() >= payload.exp * 1000;
52     }
53
54     return { header, payload, isExpired };
55 }
56
57 export function hexToRgb(hex: string): { r: number; g: number; b: number } | null {
58     const cleanHex = hex.replace("#", "").trim();
59     if (!/^[0-9A-Fa-f]{6}$/i.test(cleanHex)) return null;
60     const n = parseInt(cleanHex, 16);
61     return {
62         r: (n >> 16) & 255,
63         g: (n >> 8) & 255,
64         b: n & 255,
65     };
66 }
```


src/utils/helpers.ts (continued)

```

67
68 export function getLuminance(r: number, g: number, b: number): number {
69   const a = [r, g, b].map((v) => {
70     const val = v / 255;
71     return val <= 0.03928 ? val / 12.92 : Math.pow((val + 0.055) / 1.055, 2.4);
72   });
73   return 0.2126 * a[0] + 0.7152 * a[1] + 0.0722 * a[2];
74 }
75
76 export function getContrastRatio(hex1: string, hex2: string): number {
77   const rgb1 = hexToRgb(hex1) || { r: 255, g: 255, b: 255 };
78   const rgb2 = hexToRgb(hex2) || { r: 0, g: 0, b: 0 };
79   const l1 = getLuminance(rgb1.r, rgb1.g, rgb1.b);
80   const l2 = getLuminance(rgb2.r, rgb2.g, rgb2.b);
81   const lighter = Math.max(l1, l2);
82   const darker = Math.min(l1, l2);
83   return (lighter + 0.05) / (darker + 0.05);
84 }
85
86 export function sortJsonKeys(obj: any): any {
87   if (Array.isArray(obj)) return obj.map(sortJsonKeys);
88   if (obj !== null && typeof obj === "object") {
89     return Object.keys(obj)
90       .sort()
91       .reduce((res: Record<string, any>, key: string) => {
92         res[key] = sortJsonKeys(obj[key]);
93         return res;
94       }, {});
95   }
96   return obj;
97 }
98
99 export function jsonToCsv(jsonInput: any[] | string): string {
100   let jsonArray: any[];
101   if (typeof jsonInput === "string") {
102     try {
103       jsonArray = JSON.parse(jsonInput);
104     } catch {
105       throw new Error("Invalid JSON string. Please provide a valid JSON array.");
106     }
107   } else {
108     jsonArray = jsonInput;
109   }
110   if (!Array.isArray(jsonArray) || jsonArray.length === 0) {
111     throw new Error("Input must be a non-empty array of objects");
112   }
113   const headers = Array.from(new Set(jsonArray.flatMap((item) => (typeof item === "object" && item !== null ?
114     Object.keys(item) : []))));
115   const escapeCell = (val: any) => `${String(val ?? "")}.replace(/"/g, '""')}`;
116   const headerRow = headers.map(escapeCell).join(",");
117   const rows = jsonArray.map((row) =>
118     headers.map((h) => escapeCell(row ? row[h] : "")).join(",")
119   );
120   return [headerRow, ...rows].join("\n");
121 }
122
123 export function csvToJson(csvText: string): any[] {
124   const lines = csvText.trim().split(/\r?\n/).filter(Boolean);
125   if (lines.length < 2) {
126     throw new Error("CSV must include a header and at least one data row");
127   }
128
129   const parseLine = (line: string) => {
130     const result: string[] = [];
131     let current = "";
132     let inQuotes = false;
133     for (let i = 0; i < line.length; i++) {
134       const ch = line[i];
135       if (ch === '"') {

```

src/utls/helpers.ts (continued)

```
136         if (inQuotes && line[i + 1] === '') {
137             current += '';
138             i++;
139         } else {
140             inQuotes = !inQuotes;
141         }
142     } else if (ch === "," && !inQuotes) {
143         result.push(current);
144         current = "";
145     } else {
146         current += ch;
147     }
148 }
149 result.push(current);
150 return result;
151 };
152
153 const headers = parseLine(lines[0]);
154 return lines.slice(1).map((line) => {
155     const values = parseLine(line);
156     const obj: Record<string, string> = {};
157     headers.forEach((h, i) => {
158         obj[h] = values[i] ?? "";
159     });
160     return obj;
161 });
162 }
163
164 export const HTTP_STATUS_CODES: Record<number, string> = {
165     100: "Continue",
166     101: "Switching Protocols",
167     200: "OK",
168     201: "Created",
169     202: "Accepted",
170     204: "No Content",
171     301: "Moved Permanently",
172     302: "Found",
173     304: "Not Modified",
174     307: "Temporary Redirect",
175     308: "Permanent Redirect",
176     400: "Bad Request",
177     401: "Unauthorized",
178     403: "Forbidden",
179     404: "Not Found",
180     405: "Method Not Allowed",
181     408: "Request Timeout",
182     409: "Conflict",
183     410: "Gone",
184     413: "Payload Too Large",
185     415: "Unsupported Media Type",
186     422: "Unprocessable Entity",
187     429: "Too Many Requests",
188     500: "Internal Server Error",
189     501: "Not Implemented",
190     502: "Bad Gateway",
191     503: "Service Unavailable",
192     504: "Gateway Timeout",
193 };
194
195 export {
196     rot13,
197     rot47,
198     caesarCipher,
199     textToMorse as morseEncode,
200     morseToText as morseDecode,
201 } from "./crypto";
202
203 export function textToBinaryString(text: string): string {
204     return Array.from(new TextEncoder().encode(text))
```

src/utils/helpers.ts (continued)

```
205     .map((b) => b.toString(2).padStart(8, "0"))
206     .join(" ");
207 }
208
209 export function binaryStringToText(binaryStr: string): string {
210     const clean = binaryStr.trim().split(/\s+/);
211     const bytes = new Uint8Array(clean.map((bin) => parseInt(bin, 2)));
212     return new TextDecoder().decode(bytes);
213 }
214
215 export function stringToHex(text: string): string {
216     return Array.from(new TextEncoder().encode(text))
217         .map((b) => b.toString(16).padStart(2, "0"))
218         .join(" ");
219 }
220
221 export function hexToString(hexStr: string): string {
222     const clean = hexStr.replace(/^[^0-9A-Fa-f]/g, "");
223     const bytes = new Uint8Array(clean.length / 2);
224     for (let i = 0; i < clean.length; i += 2) {
225         bytes[i / 2] = parseInt(clean.substring(i, i + 2), 16);
226     }
227     return new TextDecoder().decode(bytes);
228 }
229
230 export function safeStorageGet(key: string): string | null {
231     try {
232         if (typeof window !== "undefined" && window.localStorage) {
233             return window.localStorage.getItem(key);
234         }
235     } catch {
236         // Gracefully handle iframe or restricted cookie exceptions
237     }
238     return null;
239 }
240
241 export function safeStorageSet(key: string, value: string): void {
242     try {
243         if (typeof window !== "undefined" && window.localStorage) {
244             window.localStorage.setItem(key, value);
245         }
246     } catch {
247         // Gracefully handle iframe or restricted cookie exceptions
248     }
249 }
250
251 export function safeStorageRemove(key: string): void {
252     try {
253         if (typeof window !== "undefined" && window.localStorage) {
254             window.localStorage.removeItem(key);
255         }
256     } catch {
257         // Gracefully handle iframe or restricted cookie exceptions
258     }
259 }
260
```

FILE [21/25]

src/utls/crypto.ts

282 lines • 10.0 KB • Hash, cipher & encoding cryptographic helpers

```
1 // Standalone MD5 Implementation
2 export function md5(str: string): string {
3     function safeAdd(x: number, y: number): number {
4         const l = (x & 0xffff) + (y & 0xffff);
5         return (((x >> 16) + (y >> 16) + (l >> 16)) << 16) | (l & 0xffff);
6     }
7
8     function rol(num: number, cnt: number): number {
9         return (num << cnt) | (num >> (32 - cnt));
10    }
11
12    function cmn(q: number, a: number, b: number, x: number, s: number, t: number): number {
13        return safeAdd(rol(safeAdd(safeAdd(a, q), safeAdd(x, t)), s), b);
14    }
15
16    function ff(a: number, b: number, c: number, d: number, x: number, s: number, t: number): number {
17        return cmn((b & c) | (~b & d), a, b, x, s, t);
18    }
19
20    function gg(a: number, b: number, c: number, d: number, x: number, s: number, t: number): number {
21        return cmn((b & d) | (c & ~d), a, b, x, s, t);
22    }
23
24    function hh(a: number, b: number, c: number, d: number, x: number, s: number, t: number): number {
25        return cmn(b ^ c ^ d, a, b, x, s, t);
26    }
27
28    function ii(a: number, b: number, c: number, d: number, x: number, s: number, t: number): number {
29        return cmn(c ^ (b | ~d), a, b, x, s, t);
30    }
31
32    const bytes = new TextEncoder().encode(str);
33    const len = bytes.length;
34    const bitLen = len * 8;
35    const total = (((len + 8) >> 6) + 1) * 16;
36    const x = new Uint32Array(total);
37
38    for (let i = 0; i < len; i++) {
39        x[i >> 2] |= bytes[i] << ((i % 4) * 8);
40    }
41    x[len >> 2] |= 0x80 << ((len % 4) * 8);
42    x[total - 2] = bitLen >>> 0;
43    x[total - 1] = Math.floor(bitLen / 4294967296);
44
45    let a = 0x67452301;
46    let b = 0xefcdab89;
47    let c = 0x98badcfe;
48    let d = 0x10325476;
49
50    for (let i = 0; i < total; i += 16) {
51        const oa = a;
52        const ob = b;
53        const oc = c;
54        const od = d;
55
56        a = ff(a, b, c, d, x[i + 0], 7, 0xd76aa478);
57        d = ff(d, a, b, c, x[i + 1], 12, 0xe8c7b756);
58        c = ff(c, d, a, b, x[i + 2], 17, 0x242070db);
59        b = ff(b, c, d, a, x[i + 3], 22, 0xc1bdcee);
60
61        a = ff(a, b, c, d, x[i + 4], 7, 0xf57c0faf);
62        d = ff(d, a, b, c, x[i + 5], 12, 0x4787c62a);
63        c = ff(c, d, a, b, x[i + 6], 17, 0xa8304613);
64        b = ff(b, c, d, a, x[i + 7], 22, 0xfd469501);
65
66        a = ff(a, b, c, d, x[i + 8], 7, 0x698098d8);
```

src/utls/crypto.ts (continued)

```
67     d = ff(d, a, b, c, x[i + 9], 12, 0x8b44f7af);
68     c = ff(c, d, a, b, x[i + 10], 17, 0xfffff5bb1);
69     b = ff(b, c, d, a, x[i + 11], 22, 0x895cd7be);
70
71     a = ff(a, b, c, d, x[i + 12], 7, 0x6b901122);
72     d = ff(d, a, b, c, x[i + 13], 12, 0xfd987193);
73     c = ff(c, d, a, b, x[i + 14], 17, 0xa679438e);
74     b = ff(b, c, d, a, x[i + 15], 22, 0x49b40821);
75
76     a = gg(a, b, c, d, x[i + 1], 5, 0xf61e2562);
77     d = gg(d, a, b, c, x[i + 6], 9, 0xc040b340);
78     c = gg(c, d, a, b, x[i + 11], 14, 0x265e5a51);
79     b = gg(b, c, d, a, x[i + 0], 20, 0xe9b6c7aa);
80
81     a = gg(a, b, c, d, x[i + 5], 5, 0xd62f105d);
82     d = gg(d, a, b, c, x[i + 10], 9, 0x02441453);
83     c = gg(c, d, a, b, x[i + 15], 14, 0xd8a1e681);
84     b = gg(b, c, d, a, x[i + 4], 20, 0xe7d3fbc8);
85
86     a = gg(a, b, c, d, x[i + 9], 5, 0x21e1cde6);
87     d = gg(d, a, b, c, x[i + 14], 9, 0xc33707d6);
88     c = gg(c, d, a, b, x[i + 3], 14, 0xf4d50d87);
89     b = gg(b, c, d, a, x[i + 8], 20, 0x455a14ed);
90
91     a = gg(a, b, c, d, x[i + 13], 5, 0xa9e3e905);
92     d = gg(d, a, b, c, x[i + 2], 9, 0xfcefa3f8);
93     c = gg(c, d, a, b, x[i + 7], 14, 0x676f02d9);
94     b = gg(b, c, d, a, x[i + 12], 20, 0x8d2a4c8a);
95
96     a = hh(a, b, c, d, x[i + 5], 4, 0xfffa3942);
97     d = hh(d, a, b, c, x[i + 8], 11, 0x8771f681);
98     c = hh(c, d, a, b, x[i + 11], 16, 0x6d9d6122);
99     b = hh(b, c, d, a, x[i + 14], 23, 0xfde5380c);
100
101     a = hh(a, b, c, d, x[i + 1], 4, 0xa4beea44);
102     d = hh(d, a, b, c, x[i + 4], 11, 0x4bdecfa9);
103     c = hh(c, d, a, b, x[i + 7], 16, 0xf6bb4b60);
104     b = hh(b, c, d, a, x[i + 10], 23, 0xbebfb70);
105
106     a = hh(a, b, c, d, x[i + 13], 4, 0x289b7ec6);
107     d = hh(d, a, b, c, x[i + 0], 11, 0xeaa127fa);
108     c = hh(c, d, a, b, x[i + 3], 16, 0xd4ef3085);
109     b = hh(b, c, d, a, x[i + 6], 23, 0x04881d05);
110
111     a = hh(a, b, c, d, x[i + 9], 4, 0xd9d4d039);
112     d = hh(d, a, b, c, x[i + 12], 11, 0xe6db99e5);
113     c = hh(c, d, a, b, x[i + 15], 16, 0x1fa27cf8);
114     b = hh(b, c, d, a, x[i + 2], 23, 0xc4ac5665);
115
116     a = ii(a, b, c, d, x[i + 0], 6, 0xf4292244);
117     d = ii(d, a, b, c, x[i + 7], 10, 0x432aff97);
118     c = ii(c, d, a, b, x[i + 14], 15, 0xab9423a7);
119     b = ii(b, c, d, a, x[i + 5], 21, 0xfc93a039);
120
121     a = ii(a, b, c, d, x[i + 12], 6, 0x655b59c3);
122     d = ii(d, a, b, c, x[i + 3], 10, 0x8f0ccc92);
123     c = ii(c, d, a, b, x[i + 10], 15, 0xffefff47d);
124     b = ii(b, c, d, a, x[i + 1], 21, 0x85845dd1);
125
126     a = ii(a, b, c, d, x[i + 8], 6, 0x6fa87e4f);
127     d = ii(d, a, b, c, x[i + 15], 10, 0xfe2ce6e0);
128     c = ii(c, d, a, b, x[i + 6], 15, 0xa3014314);
129     b = ii(b, c, d, a, x[i + 13], 21, 0x4e0811a1);
130
131     a = ii(a, b, c, d, x[i + 4], 6, 0xf7537e82);
132     d = ii(d, a, b, c, x[i + 11], 10, 0xbd3af235);
133     c = ii(c, d, a, b, x[i + 2], 15, 0x2ad7d2bb);
134     b = ii(b, c, d, a, x[i + 9], 21, 0xeb86d391);
135
```

src/utils/crypto.ts (continued)

```

136     a = safeAdd(a, oa);
137     b = safeAdd(b, ob);
138     c = safeAdd(c, oc);
139     d = safeAdd(d, od);
140 }
141
142 const words = [a, b, c, d];
143 let out = "";
144 for (const word of words) {
145     for (let i = 0; i < 4; i++) {
146         out += ((word >>> (i * 8)) & 255).toString(16).padStart(2, "0");
147     }
148 }
149 return out;
150 }
151
152 export const simpleMd5 = md5;
153
154 export async function subtleHash(text: string, algorithm: "SHA-1" | "SHA-256" | "SHA-384" | "SHA-512"):
155 Promise<string> = {
156     const buffer = await crypto.subtle.digest(algorithm, new TextEncoder().encode(text));
157     return Array.from(new Uint8Array(buffer))
158         .map((b) => b.toString(16).padStart(2, "0"))
159         .join("");
160 }
161
162 export const sha1Hex = (text: string) => subtleHash(text, "SHA-1");
163 export const sha256Hex = (text: string) => subtleHash(text, "SHA-256");
164 export const sha384Hex = (text: string) => subtleHash(text, "SHA-384");
165 export const sha512Hex = (text: string) => subtleHash(text, "SHA-512");
166
167 export async function subtleHmac(keyStr: string, message: string, hashAlgo: "SHA-256" | "SHA-512"): Promise<string>
168 {
169     const key = await crypto.subtle.importKey(
170         "raw",
171         new TextEncoder().encode(keyStr),
172         { name: "HMAC", hash: hashAlgo },
173         false,
174         ["sign"]
175     );
176     const signature = await crypto.subtle.sign("HMAC", key, new TextEncoder().encode(message));
177     return Array.from(new Uint8Array(signature))
178         .map((b) => b.toString(16).padStart(2, "0"))
179         .join("");
180 }
181
182 export const hmacSha256Hex = (key: string, msg: string) => subtleHmac(key, msg, "SHA-256");
183 export const hmacSha512Hex = (key: string, msg: string) => subtleHmac(key, msg, "SHA-512");
184
185 export function bytesToBase64(bytes: Uint8Array): string {
186     let binary = "";
187     for (let i = 0; i < bytes.byteLength; i++) {
188         binary += String.fromCharCode(bytes[i]);
189     }
190     return btoa(binary);
191 }
192
193 export function base64ToBytes(str: string): Uint8Array {
194     const binary = atob(str);
195     const bytes = new Uint8Array(binary.length);
196     for (let i = 0; i < binary.length; i++) {
197         bytes[i] = binary.charCodeAt(i);
198     }
199     return bytes;
200 }
201
202 export async function aesEncrypt(password: string, plaintext: string): Promise<string> {
203     const keyHash = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(password));
204     const key = await crypto.subtle.importKey("raw", keyHash, { name: "AES-GCM" }, false, ["encrypt"]);
205     const iv = crypto.getRandomValues(new Uint8Array(12));
206     const encrypted = await crypto.subtle.encrypt(

```

src/utils/crypto.ts (continued)

```

205     { name: "AES-GCM", iv },
206     key,
207     new TextEncoder().encode(plaintext)
208   );
209
210   return `${bytesToBase64(iv)}.${bytesToBase64(new Uint8Array(encrypted))}`;
211 }
212
213 export async function aesDecrypt(password: string, packageText: string): Promise<string> {
214   const parts = packageText.trim().split(".");
215   if (parts.length !== 2) throw new Error("Invalid encrypted package. Expected IV.CIPHERTEXT");
216   const iv = base64ToBytes(parts[0]);
217   const cipher = base64ToBytes(parts[1]);
218   if (iv.length !== 12) throw new Error("Invalid IV length. Expected 12 bytes.");
219
220   const keyHash = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(password));
221   const key = await crypto.subtle.importKey("raw", keyHash, { name: "AES-GCM" }, false, ["decrypt"]);
222   const decrypted = await crypto.subtle.decrypt({ name: "AES-GCM", iv }, key, cipher);
223   return new TextDecoder().decode(decrypted);
224 }
225
226 export function rot13(text: string): string {
227   return text.replace(/[A-Za-z]/g, (c) => {
228     const base = c <= "Z" ? 65 : 97;
229     return String.fromCharCode(((c.charCodeAt(0) - base + 13) % 26) + base);
230   });
231 }
232
233 export function rot47(text: string): string {
234   return Array.from(text)
235     .map((c) => {
236       const code = c.charCodeAt(0);
237       return code >= 33 && code <= 126 ? String.fromCharCode(33 + ((code - 33 + 47) % 94)) : c;
238     })
239     .join("");
240 }
241
242 export function caesarCipher(text: string, shift: number, decrypt = false): string {
243   const s = ((shift % 26) + 26) % 26 * (decrypt ? -1 : 1);
244   return Array.from(text)
245     .map((c) => {
246       if (/[A-Z]/.test(c)) {
247         return String.fromCharCode((((c.charCodeAt(0) - 65 + s) % 26) + 26) % 26 + 65);
248       }
249       if (/[a-z]/.test(c)) {
250         return String.fromCharCode((((c.charCodeAt(0) - 97 + s) % 26) + 26) % 26 + 97);
251       }
252       return c;
253     })
254     .join("");
255 }
256
257 export const MORSE_CODE_MAP: Record<string, string> = {
258   A: ".-.", B: "-...", C: "-.-.", D: "-..", E: ".", F: "...-", G: "--.", H: "....",
259   I: "..", J: ".---", K: "-.-", L: ".-..", M: "--", N: "-.", O: "---", P: "--..",
260   Q: "--.-", R: ".-.", S: "...", T: "-", U: "..-", V: "...-", W: "--.", X: "-.-.",
261   Y: "-.-.", Z: "...", "0": "-----", "1": ".----", "2": "--...", "3": "...--",
262   "4": "....-", "5": ".....", "6": "-....", "7": "--...", "8": "---..", "9": "----.",
263   " ": "/", ".": ".-.-.-", ",": "-.-.-.-", "?": ".-.-.-.", "!": "-.-.-.-", "@": "-.-.-.-"
264 };
265
266 export function textToMorse(text: string): string {
267   return text
268     .toUpperCase()
269     .split("")
270     .map((c) => MORSE_CODE_MAP[c] || c)
271     .join(" ");
272 }
273

```

src/utils/crypto.ts (continued)

```
274 export function morseToText(morse: string): string {
275   const reverseMap = Object.fromEntries(Object.entries(MORSE_CODE_MAP).map(([k, v]) => [v, k]));
276   return morse
277     .trim()
278     .split(/\s+/)
279     .map((token) => (token === "/" ? " " : reverseMap[token] || "?"))
280     .join("");
281 }
282
```


FILE [22/25]

src/components/tools/ToolsTextFormat.jsx • Tools #2 - #20 (Text, Formatting, Conversion, Hashes)

```
1 import React, { useState } from "react";
2 import { ToolCard } from "../ToolCard";
3 import { ToolCategory } from "../../types";
4 import {
5   safeBase64Encode,
6   safeBase64Decode,
7   generateUUIDv4,
8   textToSlug,
9   decodeJWTToken,
10 } from "../../utils/helpers";
11
12 interface ToolsProps {
13   searchQuery: string;
14   selectedCategory: ToolCategory;
15 }
16
17 export const ToolsTextFormat: React.FC<ToolsProps> = ({
18   searchQuery,
19   selectedCategory,
20 }) => {
21   // Helper to check filter visibility
22   const isVisible = (id: number, title: string, category: ToolCategory, keywords: string[] = []) => {
23     const matchesCategory = selectedCategory === "All" || selectedCategory === category;
24     if (!matchesCategory) return false;
25     if (!searchQuery.trim()) return true;
26     const q = searchQuery.toLowerCase().trim();
27     return (
28       title.toLowerCase().includes(q) ||
29       category.toLowerCase().includes(q) ||
30       `#${id}`.includes(q) ||
31       keywords.some((k) => k.toLowerCase().includes(q))
32     );
33   };
34
35   // Tool 2 - Text Case Converter
36   const [t2Input, setT2Input] = useState("Hello World! Welcome to Coding Super Hub.");
37   const [t2Output, setT2Output] = useState("");
38
39   // Tool 3 - Word & Character Counter
40   const [t3Input, setT3Input] = useState("Coding Super Hub provides over 150 developer tools in one place.");
41
42   // Tool 4 - JSON Formatter
43   const [t4Input, setT4Input] = useState('{"name": "Coding Super Hub", "tools": 150, "fast": true}');
44   const [t4Output, setT4Output] = useState("");
45   const [t4Error, setT4Error] = useState(false);
46
47   // Tool 5 - Base64
48   const [t5Input, setT5Input] = useState("Developer Superpowers 0=b€");
49   const [t5Output, setT5Output] = useState("");
50   const [t5Error, setT5Error] = useState(false);
51
52   // Tool 6 - URL Encoder / Decoder
53   const [t6Input, setT6Input] = useState("https://example.com/search?q=developer tools & code=100%");
54   const [t6Output, setT6Output] = useState("");
55
56   // Tool 7 - Password Generator
57   const [t7Length, setT7Length] = useState(16);
58   const [t7Output, setT7Output] = useState("");
59
60   // Tool 8 - Color Converter
61   const [t8Hex, setT8Hex] = useState("#2563eb");
62   const [t8Output, setT8Output] = useState("");
63
64   // Tool 9 - Timestamp Converter
65   const [t9Ts, setT9Ts] = useState(String(Math.floor(Date.now() / 1000)));
66   const [t9Date, setT9Date] = useState("");
```

src/components/tools/ToolsTextFormat.tsx (continued)

```
const [t9Output, setT9Output] = useState("");

// Tool 10 - HTML Entity Encoder / Decoder
const [t10Input, setT10Input] = useState('<div class="badge">Coding & Design</div>');
const [t10Output, setT10Output] = useState("");

// Tool 11 - UUID Generator
const [t11Output, setT11Output] = useState("");

// Tool 12 - Text to Slug
const [t12Input, setT12Input] = useState("10 Super Fast Ways to Code in 2026!");
const [t12Output, setT12Output] = useState("");

// Tool 13 - Regex Tester
const [t13Pattern, setT13Pattern] = useState("[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$");
const [t13Flags, setT13Flags] = useState("i");
const [t13Text, setT13Text] = useState("developer@example.com");
const [t13Output, setT13Output] = useState("");

// Tool 14 - HTML Previewer
const [t14Input, setT14Input] = useState(
`<div style="padding:16px;background:#e0f2fe;color:#0369a1;border-radius:8px;font-family:sans-serif;"><h3>HTML
Preview Active</h3><p>Ins
`,
);

// Tool 15 - CSS Gradient Generator
const [t15Color1, setT15Color1] = useState("#2563eb");
const [t15Color2, setT15Color2] = useState("#9333ea");
const [t15Angle, setT15Angle] = useState(90);

// Tool 16 - Markdown Previewer
const [t16Input, setT16Input] = useState(
`# Coding Super Hub\n\n**Feature List:**\n- 150 Tools\n- *Markdown parser*\n- `Code snippets`\n\n>
Craftsmanship in every pixel`
);

// Tool 17 - Number Base Converter
const [t17Input, setT17Input] = useState("255");
const [t17From, setT17From] = useState(10);
const [t17To, setT17To] = useState(16);
const [t17Output, setT17Output] = useState("");

// Tool 18 - JWT Decoder
const [t18Input, setT18Input] = useState(
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkJkMTUxNjZlOTAyMn0.Sf
TKxwRJSMekKF2QT4fwpmEJf3
");
const [t18Output, setT18Output] = useState("");
const [t18Error, setT18Error] = useState(false);

// Tool 19 - Code Minifier
const [t19Input, setT19Input] = useState("body {\n margin: 0;\n padding: 0;\n}");
const [t19Type, setT19Type] = useState("<html" | "css" | "js">("css");
const [t19Output, setT19Output] = useState("");

// Tool 20 - Lorem Ipsum Generator
const [t20Count, setT20Count] = useState(3);
const [t20Output, setT20Output] = useState("");

return (
<div>
{ /* TOOL 2 - Text Case Converter */ }
{ isVisible(2, "Text Case Converter", "Text", ["uppercase", "lowercase", "titlecase", "camelcase", "slug"]) &&
    <ToolCard
      id={2}
      title="Text Case Converter"
      category="Text"
      description="Convert text into UPPERCASE, lowercase, Title Case, camelCase, or snake_case"
      output={t20Output}
    >
      <textarea
        value={t2Input}

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

136         onChange={(e) => setT2Input(e.target.value)}
137         rows={3}
138         placeholder="Enter text here..."
139         className="w-full text-xs p-2.5 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
140         dark:bg-slate-900 text-slate
141       <div className="flex flex-wrap gap-1.5">
142         <button
143           type="button"
144           onClick={() => setT2Output(t2Input.toUpperCase())}
145           className="px-2.5 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
146         >
147           UPPERCASE
148         </button>
149         <button
150           type="button"
151           onClick={() => setT2Output(t2Input.toLowerCase())}
152           className="px-2.5 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
153         >
154           lowercase
155         </button>
156         <button
157           type="button"
158           onClick={() =>
159             setT2Output(
160               t2Input
161                 .toLowerCase()
162                 .split(" ")
163                 .map((w) => (w ? w.charAt(0).toUpperCase() + w.slice(1) : ""))
164                 .join(" ")
165             )
166           }
167           className="px-2.5 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
168         >
169           Title Case
170         </button>
171         <button
172           type="button"
173           onClick={() =>
174             setT2Output(
175               t2Input
176                 .toLowerCase()
177                 .replace(/^[^a-zA-Z0-9]+(.)/g, (_, chr) => chr.toUpperCase())
178             )
179           }
180           className="px-2.5 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
181         >
182           camelCase
183         </button>
184         <button
185           type="button"
186           onClick={() =>
187             setT2Output(
188               t2Input
189                 .trim()
190                 .toLowerCase()
191                 .replace(/\s+/g, "_")
192             )
193           }
194           className="px-2.5 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
195         >
196           snake_case
197         </button>
198       </div>
199     </ToolCard>
200   )}
201
202   {/* TOOL 3 - Word & Character Counter */}
203   {isVisible(3, "Word & Character Counter", "Text", ["words", "chars", "length", "reading time"]) && (
204     <ToolCard

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

205         id={3}
206         title="Word & Character Counter"
207         category="Text"
208         description="Real-time statistics for words, characters, whitespace, and reading estimate"
209     >
210         <textarea
211             value={t3Input}
212             onChange={(e) => setT3Input(e.target.value)}
213             rows={3}
214             placeholder="Type or paste your text..."
215             className="w-full text-xs p-2.5 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
216 dark:bg-slate-900 text-slate-800">
217             <div className="grid grid-cols-2 sm:grid-cols-4 gap-2 pt-1 text-center text-xs">
218                 <div className="p-2 bg-slate-100 dark:bg-slate-900 rounded-lg border border-slate-200 dark:border-
219 slate-800">
220                     <span className="text-[10px] text-slate-500 block">Words</span>
221                     <strong className="text-base text-blue-600 dark:text-blue-400">
222                         {t3Input.trim() ? t3Input.trim().split(/\s+/).length : 0}
223                     </strong>
224                 </div>
225                 <div className="p-2 bg-slate-100 dark:bg-slate-900 rounded-lg border border-slate-200 dark:border-
226 slate-800">
227                     <span className="text-[10px] text-slate-500 block">Characters</span>
228                     <strong className="text-base text-slate-900 dark:text-slate-100">
229                         {t3Input.length}
230                     </strong>
231                 </div>
232                 <div className="p-2 bg-slate-100 dark:bg-slate-900 rounded-lg border border-slate-200 dark:border-
233 slate-800">
234                     <span className="text-[10px] text-slate-500 block">No Spaces</span>
235                     <strong className="text-base text-slate-900 dark:text-slate-100">
236                         {t3Input.replace(/\s/g, "").length}
237                     </strong>
238                 </div>
239                 <div className="p-2 bg-slate-100 dark:bg-slate-900 rounded-lg border border-slate-200 dark:border-
240 slate-800">
241                     <span className="text-[10px] text-slate-500 block">Read Time</span>
242                     <strong className="text-base text-slate-900 dark:text-slate-100">
243                         ~{Math.max(1, Math.ceil((t3Input.trim().split(/\s+/).length || 0) / 200))}m
244                     </strong>
245                 </div>
246             </div>
247         </ToolCard>
248     )}
249
250     {/* TOOL 4 - JSON Formatter */}
251     {isVisible(4, "JSON Formatter", "Formatting", ["pretty", "minify", "json", "beautify"])} && (
252         <ToolCard
253             id={4}
254             title="JSON Formatter & Minifier"
255             category="Formatting"
256             description="Format with 2-space indentation or minify JSON payloads"
257             output={t4Output}
258             isError={t4Error}
259         >
260             <textarea
261                 value={t4Input}
262                 onChange={(e) => setT4Input(e.target.value)}
263                 rows={3}
264                 placeholder='{"key": "value"}'
265                 className="w-full text-xs p-2.5 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
266 dark:bg-slate-900 text-slate-800">
267             <div className="flex gap-1.5">
268                 <button
269                     type="button"
270                     onClick={() => {
271                         try {
272                             const parsed = JSON.parse(t4Input);
273                             setT4Output(JSON.stringify(parsed, null, 2));
274                             setT4Error(false);
275                         } catch (e: any) {
276                             setT4Output("'L Invalid JSON: " + e.message);
277                             setT4Error(true);

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

    }
  }}
  className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
>
  Format (Pretty)
</button>
<button
  type="button"
  onClick={() => {
    try {
      const parsed = JSON.parse(t4Input);
      setT4Output(JSON.stringify(parsed));
      setT4Error(false);
    } catch (e: any) {
      setT4Output("'L Invalid JSON: " + e.message);
      setT4Error(true);
    }
  }}
  className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs"
>
  Minify
</button>
</div>
</ToolCard>
)}

{/* T00L 5 - Base64 Encoder / Decoder */}
{isVisible(5, "Base64 Encoder / Decoder", "Converters", ["btoa", "atob", "encode", "decode"]) && (
  <ToolCard
    id={5}
    title="Base64 Encoder / Decoder"
    category="Converters"
    description="UTF-8 safe Base64 encoding and decoding"
    output={t5Output}
    isError={t5Error}
  >
    <textarea
      value={t5Input}
      onChange={(e) => setT5Input(e.target.value)}
      rows={3}
      placeholder="Enter plain text or base64..."
      className="w-full text-xs p-2.5 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
dark:bg-slate-900 text-slat
    >
    <div className="flex gap-1.5">
      <button
        type="button"
        onClick={() => {
          try {
            setT5Output(safeBase64Encode(t5Input));
            setT5Error(false);
          } catch {
            setT5Output("'L Encoding error");
            setT5Error(true);
          }
        }}
        className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
      >
        Encode
      </button>
      <button
        type="button"
        onClick={() => {
          try {
            setT5Output(safeBase64Decode(t5Input));
            setT5Error(false);
          } catch {
            setT5Output("'L Invalid Base64 string");
            setT5Error(true);
          }
        }}
      >
        Decode
      </button>
    </div>
  </ToolCard>
)
}

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

343         }}
344         className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs"
345     >
346         Decode
347     </button>
348 </div>
349 </ToolCard>
350 }}
351
352 {/* TOOL 6 - URL Encoder / Decoder */}
353 {isVisible(6, "URL Encoder / Decoder", "Converters", ["uri", "encodeuricomponent"]) && (
354     <ToolCard
355         id={6}
356         title="URL Encoder / Decoder"
357         category="Converters"
358         description="Encode query parameters or decode URL-encoded components"
359         output={t6Output}
360     >
361         <textarea
362             value={t6Input}
363             onChange={(e) => setT6Input(e.target.value)}
364             rows={3}
365             placeholder="Enter URL or text..."
366             className="w-full text-xs p-2.5 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
367 dark:bg-slate-900 text-slat
368         <div className="flex gap-1.5">
369             <button
370                 type="button"
371                 onClick={() => setT6Output(encodeURIComponent(t6Input))}
372                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
373             >
374                 Encode URL
375             </button>
376             <button
377                 type="button"
378                 onClick={() => {
379                     try {
380                         setT6Output(decodeURIComponent(t6Input));
381                     } catch {
382                         setT6Output("'L Invalid encoded URL string");
383                     }
384                 }}
385                 className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs"
386             >
387                 Decode URL
388             </button>
389         </div>
390     </ToolCard>
391 }}
392
393 {/* TOOL 7 - Password Generator */}
394 {isVisible(7, "Password Generator", "Security", ["password", "random", "generator"]) && (
395     <ToolCard
396         id={7}
397         title="Password Generator"
398         category="Security"
399         description="Generate cryptographically secure randomized passwords"
400         output={t7Output}
401     >
402         <div className="flex items-center gap-2">
403             <label className="text-xs text-slate-600 dark:text-slate-400">Length:</label>
404             <input
405                 type="number"
406                 min={6}
407                 max={128}
408                 value={t7Length}
409                 onChange={(e) => setT7Length(Number(e.target.value))}
410                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
411 dark:bg-slate-900 text-sla

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

412         <button
413             type="button"
414             onClick={() => {
415                 const chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$$%^&*()_+~[]{}|";
416                 const array = new Uint32Array(Math.max(6, Math.min(128, t7Length)));
417                 crypto.getRandomValues(array);
418                 const pwd = Array.from(array, (x) => chars[x % chars.length]).join("");
419                 setT7Output(pwd);
420             }}
421             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-medium"
422         >
423             Generate Password
424         </button>
425     </div>
426 </ToolCard>
427 }}
428
429 {/* TOOL 8 - Color Converter */}
430 {isVisible(8, "Color Converter", "Color", ["hex", "rgb", "hsl"]) && (
431     <ToolCard
432         id={8}
433         title="Color Converter (HEX to RGB & HSL)"
434         category="Color"
435         description="Convert hex colors to RGB and HSL with interactive color picker"
436         output={t8Output}
437     >
438         <div className="flex items-center gap-2">
439             <input
440                 type="color"
441                 value={t8Hex}
442                 onChange={(e) => setT8Hex(e.target.value)}
443                 className="w-10 h-8 rounded border border-slate-300 cursor-pointer"
444             />
445             <input
446                 type="text"
447                 value={t8Hex}
448                 onChange={(e) => setT8Hex(e.target.value)}
449                 placeholder="#2563eb"
450                 className="flex-1 text-xs px-2.5 py-1.5 rounded border border-slate-300 dark:border-slate-700 bg-
451 slate-50 dark:bg-slate-900 te
452             <button
453                 type="button"
454                 onClick={() => {
455                     const hex = t8Hex.trim();
456                     if (!/^#[0-9A-Fa-f]{6}$/.test(hex)) {
457                         setT8Output("'L Enter a valid 6-digit hex like #2563eb");
458                         return;
459                     }
460                     const r = parseInt(hex.substring(1, 3), 16);
461                     const g = parseInt(hex.substring(3, 5), 16);
462                     const b = parseInt(hex.substring(5, 7), 16);
463                     setT8Output(`HEX: ${hex}\nRGB: rgb(${r}, ${g}, ${b})\nCSS: rgba(${r}, ${g}, ${b}, 1)`);
464                 }}
465                 className="px-3 py-1.5 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-medium"
466             >
467                 Convert
468             </button>
469         </div>
470     </ToolCard>
471 }}
472
473 {/* TOOL 9 - Timestamp Converter */}
474 {isVisible(9, "Timestamp Converter", "Time", ["unix", "epoch", "date"]) && (
475     <ToolCard
476         id={9}
477         title="Unix Timestamp Converter"
478         category="Time"
479         description="Convert between Unix epoch timestamps and human-readable dates"
480         output={t9Output}

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

481         >
482         <div className="space-y-2">
483             <div className="flex gap-2">
484                 <input
485                     type="text"
486                     value={t9Ts}
487                     onChange={(e) => setT9Ts(e.target.value)}
488                     placeholder="Unix timestamp e.g. 1750000000"
489                     className="flex-1 text-xs px-2.5 py-1 rounded border border-slate-300 dark:border-slate-700 bg-
490 slate-50 dark:bg-slate-900 te
491                 <button
492                     type="button"
493                     onClick={() => {
494                         const val = Number(t9Ts.trim());
495                         if (!Number.isFinite(val)) {
496                             setT9Output("'L Enter valid numeric timestamp");
497                             return;
498                         }
499                         const ms = t9Ts.length <= 10 ? val * 1000 : val;
500                         const d = new Date(ms);
501                         setT9Output(`UTC: ${d.toUTCString()}\nLocal: ${d.toLocaleString()}\nISO: ${d.toISOString()}`);
502                     }}
503                     className="px-2.5 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
504                 >
505                     Timestamp !' Date
506                 </button>
507             </div>
508             <div className="flex gap-2">
509                 <input
510                     type="datetime-local"
511                     value={t9Date}
512                     onChange={(e) => setT9Date(e.target.value)}
513                     className="flex-1 text-xs px-2.5 py-1 rounded border border-slate-300 dark:border-slate-700 bg-
514 slate-50 dark:bg-slate-900 te
515                 <button
516                     type="button"
517                     onClick={() => {
518                         if (!t9Date) {
519                             setT9Output("'L Select a date");
520                             return;
521                         }
522                         const d = new Date(t9Date);
523                         setT9Output(`Seconds: ${Math.floor(d.getTime() / 1000)}\nMilliseconds: ${d.getTime()}`);
524                     }}
525                     className="px-2.5 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs"
526                 >
527                     Date !' Timestamp
528                 </button>
529             </div>
530         </div>
531     </ToolCard>
532 ))
533
534 {/* TOOL 10 - HTML Entity Encoder / Decoder */}
535 {isVisible(10, "HTML Entity Encoder / Decoder", "HTML", ["entities", "escape", "unescape"]) && (
536     <ToolCard
537         id={10}
538         title="HTML Entity Encoder / Decoder"
539         category="HTML"
540         description="Convert reserved HTML characters to HTML entities (&, <, >, &quot;)"
541         output={t10Output}
542     >
543         <textarea
544             value={t10Input}
545             onChange={(e) => setT10Input(e.target.value)}
546             rows={2}
547             className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
548 dark:bg-slate-900 text-slate-
549         <div className="flex gap-1.5">

```


src/components/tools/ToolsTextFormat.tsx (continued)

```

550         <button
551             type="button"
552             onClick={() => {
553                 const encoded = t10Input
554                     .replace(/&/g, "&amp;")
555                     .replace(/</g, "&lt;")
556                     .replace(/>/g, "&gt;")
557                     .replace(/"/g, "&quot;")
558                     .replace(/'/g, "&#39;");
559                 setT10Output(encoded);
560             }}
561             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs"
562         >
563             Encode
564         </button>
565         <button
566             type="button"
567             onClick={() => {
568                 const doc = new DOMParser().parseFromString(t10Input, "text/html");
569                 setT10Output(doc.documentElement.textContent || "");
570             }}
571             className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs"
572         >
573             Decode
574         </button>
575     </div>
576 </ToolCard>
577 )}
578
579 /* TOOL 11 - UUID Generator */
580 {isVisible(11, "UUID Generator", "Generators", ["uuid", "guid", "v4"]) && (
581     <ToolCard
582         id={11}
583         title="UUID v4 Generator"
584         category="Generators"
585         description="Generate RFC 4122 compliant version 4 UUIDs"
586         output={t11Output}
587     >
588         <button
589             type="button"
590             onClick={() => setT11Output(generateUUIDv4())}
591             className="px-4 py-1.5 bg-blue-600 hover:bg-blue-500 text-white rounded-lg text-xs font-semibold"
592         >
593             Generate New UUID
594         </button>
595     </ToolCard>
596 )}
597
598 /* TOOL 12 - Text to Slug */
599 {isVisible(12, "Text to Slug Converter", "Text", ["slug", "url", "seo"]) && (
600     <ToolCard
601         id={12}
602         title="Text to Slug Converter"
603         category="Text"
604         description="Convert post titles into SEO-friendly URL slugs"
605         output={t12Output}
606     >
607         <div className="flex gap-2">
608             <input
609                 type="text"
610                 value={t12Input}
611                 onChange={(e) => setT12Input(e.target.value)}
612                 placeholder="Enter title..."
613                 className="flex-1 text-xs px-2.5 py-1.5 rounded border border-slate-300 dark:border-slate-700 bg-
614 slate-50 dark:bg-slate-900 te
615             </input>
616             <button
617                 type="button"
618                 onClick={() => setT12Output(textToSlug(t12Input))}
619                 className="px-3 py-1.5 bg-blue-600 hover:bg-blue-500 text-white rounded-lg text-xs font-semibold"

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

619         >
620         Convert
621     </button>
622 </div>
623 </ToolCard>
624 }}
625
626 {/* TOOL 13 - Regex Tester */}
627 {isVisible(13, "Regex Tester", "Text", ["regular expression", "match"]) && (
628     <ToolCard
629         id={13}
630         title="Regex Tester"
631         category="Text"
632         description="Test JavaScript regular expressions against input strings"
633         output={t13Output}
634     >
635         <div className="space-y-2">
636             <div className="flex gap-2">
637                 <input
638                     type="text"
639                     value={t13Pattern}
640                     onChange={(e) => setT13Pattern(e.target.value)}
641                     placeholder="Regex pattern..."
642                     className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-
643 slate-50 dark:bg-slate-900 text
644                 <input
645                     type="text"
646                     value={t13Flags}
647                     onChange={(e) => setT13Flags(e.target.value)}
648                     placeholder="Flags (gi)"
649                     className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
650 dark:bg-slate-900 text-s
651                 </div>
652                 <input
653                     type="text"
654                     value={t13Text}
655                     onChange={(e) => setT13Text(e.target.value)}
656                     placeholder="Test string..."
657                     className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
658 dark:bg-slate-900 text-s
659                 <button
660                     type="button"
661                     onClick={() => {
662                         try {
663                             const reg = new RegExp(t13Pattern, t13Flags);
664                             const matches = t13Text.match(reg);
665                             if (matches) {
666                                 setT13Output(` Match found!\nMatches: ${matches.join(", ")}`);
667                             } else {
668                                 setT13Output("'L No match found.");
669                             }
670                         } catch (e: any) {
671                             setT13Output("'L Invalid regex: " + e.message);
672                         }
673                     }}
674                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
675                 >
676                     Test Regex
677                 </button>
678             </div>
679         </ToolCard>
680     )}
681
682 {/* TOOL 14 - HTML Previewer */}
683 {isVisible(14, "HTML Previewer", "HTML", ["sandbox", "preview", "iframe"]) && (
684     <ToolCard
685         id={14}
686         title="HTML Previewer"
687         category="HTML"

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

688         description="Live sandbox preview of custom HTML snippets"
689     >
690         <textarea
691             value={t14Input}
692             onChange={(e) => setT14Input(e.target.value)}
693             rows={3}
694             className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
695             dark:bg-slate-900 text-slate-
696         <div className="mt-2 border border-slate-300 dark:border-slate-700 rounded-lg overflow-hidden bg-white">
697             <iframe
698                 srcDoc={t14Input}
699                 sandbox="allow-scripts"
700                 title="HTML Sandbox Preview"
701                 className="w-full h-32 border-none"
702             />
703         </div>
704     </ToolCard>
705 })
706
707 {/* TOOL 15 - CSS Gradient Generator */}
708 {isVisible(15, "CSS Gradient Generator", "CSS", ["gradient", "linear-gradient"]) && (
709     <ToolCard
710         id={15}
711         title="CSS Gradient Generator"
712         category="CSS"
713         description="Interactive two-color linear gradient builder"
714         output={`background: linear-gradient(${t15Angle}deg, ${t15Color1}, ${t15Color2});`}
715     >
716         <div className="flex flex-wrap items-center gap-3">
717             <div className="flex items-center gap-1.5">
718                 <label className="text-xs text-slate-600 dark:text-slate-400">Color 1:</label>
719                 <input
720                     type="color"
721                     value={t15Color1}
722                     onChange={(e) => setT15Color1(e.target.value)}
723                     className="w-8 h-7 rounded border cursor-pointer"
724                 />
725             </div>
726             <div className="flex items-center gap-1.5">
727                 <label className="text-xs text-slate-600 dark:text-slate-400">Color 2:</label>
728                 <input
729                     type="color"
730                     value={t15Color2}
731                     onChange={(e) => setT15Color2(e.target.value)}
732                     className="w-8 h-7 rounded border cursor-pointer"
733                 />
734             </div>
735             <div className="flex items-center gap-1.5">
736                 <label className="text-xs text-slate-600 dark:text-slate-400">Angle:</label>
737                 <input
738                     type="number"
739                     min={0}
740                     max={360}
741                     value={t15Angle}
742                     onChange={(e) => setT15Angle(Number(e.target.value))}
743                     className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
744                     dark:bg-slate-900 font-m
745                 <span className="text-xs text-slate-500">deg</span>
746             </div>
747         </div>
748         <div
749             className="w-full h-14 rounded-xl border border-slate-300 dark:border-slate-700 shadow-inner mt-2"
750             style={{
751                 background: `linear-gradient(${t15Angle}deg, ${t15Color1}, ${t15Color2})`,
752             }}
753         />
754     </ToolCard>
755 })
756

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

757     {/* TOOL 16 - Markdown Previewer */}
758     {isVisible(16, "Markdown Previewer", "Formatting", ["markdown", "md", "preview"]) && (
759         <ToolCard
760             id={16}
761             title="Markdown Previewer"
762             category="Formatting"
763             description="Live formatting of headings, lists, bold text, and code"
764         >
765             <textarea
766                 value={t16Input}
767                 onChange={(e) => setT16Input(e.target.value)}
768                 rows={3}
769                 className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
770 dark:bg-slate-900 text-slate-
771 <div className="p-3 bg-slate-50 dark:bg-slate-900 border border-slate-200 dark:border-slate-800 rounded-
772 xl text-xs space-y-1.5 text
773 {t16Input.split("\n").map((line, idx) => {
774     if (line.startsWith("# ")) {
775         return <h3 key={idx} className="text-sm font-bold text-blue-600 dark:text-blue-400">{line.slice(2)}
776     }
777     if (line.startsWith("## ")) {
778         return <h4 key={idx} className="text-xs font-bold text-slate-800 dark:text-
779 slate-200">{line.slice(3)}</h4>;
780     if (line.startsWith("- ")) {
781         return <li key={idx} className="ml-4 list-disc">{line.slice(2)}</li>;
782     }
783     if (line.startsWith("> ")) {
784         return <blockquote key={idx} className="pl-3 border-l-2 border-blue-500 italic text-
785 slate-500">{line.slice(2)}</blockquote>;
786     return <p key={idx}>{line}</p>;
787     }}}
788 </div>
789 </ToolCard>
790 ))
791
792     {/* TOOL 17 - Number Base Converter */}
793     {isVisible(17, "Number Base Converter", "Math", ["binary", "decimal", "hexadecimal", "octal"]) && (
794         <ToolCard
795             id={17}
796             title="Number Base Converter"
797             category="Math"
798             description="Convert numbers between Decimal, Binary, Hex, and Octal bases"
799             output={t17Output}
800         >
801             <div className="flex flex-wrap items-center gap-2">
802                 <input
803                     type="text"
804                     value={t17Input}
805                     onChange={(e) => setT17Input(e.target.value)}
806                     placeholder="Value"
807                     className="w-28 text-xs px-2.5 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
808 dark:bg-slate-900 font-m
809                 <span className="text-xs text-slate-500">From:</span>
810                 <select
811                     value={t17From}
812                     onChange={(e) => setT17From(Number(e.target.value))}
813                     className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
814 dark:bg-slate-900"
815                 >
816                     <option value={10}>Decimal (10)</option>
817                     <option value={2}>Binary (2)</option>
818                     <option value={16}>Hexadecimal (16)</option>
819                     <option value={8}>Octal (8)</option>
820                 </select>
821                 <span className="text-xs text-slate-500">To:</span>
822                 <select
823                     value={t17To}
824                     onChange={(e) => setT17To(Number(e.target.value))}
825                     className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
826 dark:bg-slate-900"
827                 >
828                     <option value={16}>Hexadecimal (16)</option>

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

826         <option value={10}>Decimal (10)</option>
827         <option value={2}>Binary (2)</option>
828         <option value={8}>Octal (8)</option>
829     </select>
830     <button
831         type="button"
832         onClick={() => {
833             const num = parseInt(t17Input.trim(), t17From);
834             if (isNaN(num)) {
835                 setT17Output("'L Invalid input for selected source base");
836             } else {
837                 setT17Output(num.toString(t17To).toUpperCase());
838             }
839         }}
840         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
841     >
842         Convert
843     </button>
844 </div>
845 </ToolCard>
846 }}
847
848 {/* TOOL 18 - JWT Decoder */}
849 {isVisible(18, "JWT Decoder", "Security", ["jwt", "token", "payload", "json web token"]) && (
850     <ToolCard
851         id={18}
852         title="JWT Decoder"
853         category="Security"
854         description="Inspect header, claims payload, and expiration timestamp of JSON Web Tokens"
855         output={t18Output}
856         isError={t18Error}
857     >
858         <textarea
859             value={t18Input}
860             onChange={(e) => setT18Input(e.target.value)}
861             rows={3}
862             placeholder="Paste JWT here..."
863             className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
864             dark:bg-slate-900 text-slate-
865         >
866         <button
867             type="button"
868             onClick={() => {
869                 try {
870                     const res = decodeJWTToken(t18Input);
871                     const expInfo =
872                         res.isExpired !== undefined
873                         ? res.isExpired
874                           ? "\n& p Token is EXPIRED"
875                           : "\n' Token is currently ACTIVE"
876                         : "";
877                     setT18Output(
878                         `HEADER:\n${JSON.stringify(res.header, null, 2)}\n\nPAYLOAD:\n${JSON.stringify(
879                             res.payload,
880                             null,
881                             2
882                         )}\n\n(Note: Decoded locally; signature verification requires private key)`
883                     );
884                     setT18Error(false);
885                 } catch (e: any) {
886                     setT18Output("'L " + e.message);
887                     setT18Error(true);
888                 }
889             }}
890             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
891         >
892             Decode JWT
893         </button>
894     </ToolCard>
895 }}

```

src/components/tools/ToolsTextFormat.tsx (continued)

```

895
896     { /* TOOL 19 - Code Minifier */ }
897     { isVisible(19, "HTML/CSS/JS Minifier", "Formatting", ["minify", "compress", "strip"]) && (
898         <ToolCard
899             id={19}
900             title="HTML / CSS / JS Minifier"
901             category="Formatting"
902             description="Strip comments and redundant whitespace from code"
903             output={t19Output}
904         >
905             <textarea
906                 value={t19Input}
907                 onChange={(e) => setT19Input(e.target.value)}
908                 rows={3}
909                 className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
910                 dark:bg-slate-900 text-slate-
911             <div className="flex gap-2">
912                 <select
913                     value={t19Type}
914                     onChange={(e) => setT19Type(e.target.value as any)}
915                     className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
916                     dark:bg-slate-900"
917                     <option value="css">CSS</option>
918                     <option value="html">HTML</option>
919                     <option value="js">JavaScript</option>
920                 </select>
921                 <button
922                     type="button"
923                     onClick={() => {
924                         let out = t19Input;
925                         if (t19Type === "html") {
926                             out = out.replace(/<!--[\s\S]*?-->/g, "").replace(/>\s+</g, "><").replace(/\\s{2,}/g, " ").trim();
927                         } else if (t19Type === "css") {
928                             out = out.replace(/\\/\\*[\s\S]*?\\*\\/ /g, "").replace(/\\s+ /g, " ").replace(/\\s*([{}:;])\\s*/g,
929                             "$1").trim();
930                         } else {
931                             out = out.replace(/\\/\\*[\s\S]*?\\*\\/ /g, "").replace(/\\s*\\*\\/\\*.$/gm, "").replace(/\\s+ /g, "
932                             ").trim();
933                         }
934                         setT19Output(out);
935                     }}
936                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
937                 >
938                     Minify
939                 </button>
940             </div>
941         </ToolCard>
942     )}
943
944     { /* TOOL 20 - Lorem Ipsum Generator */ }
945     { isVisible(20, "Lorem Ipsum Generator", "Generators", ["lorem", "ipsum", "placeholder text"]) && (
946         <ToolCard
947             id={20}
948             title="Lorem Ipsum Generator"
949             category="Generators"
950             description="Generate placeholder filler paragraphs for layouts and mockups"
951             output={t20Output}
952         >
953             <div className="flex items-center gap-2">
954                 <label className="text-xs text-slate-600 dark:text-slate-400">Paragraphs:</label>
955                 <input
956                     type="number"
957                     min={1}
958                     max={15}
959                     value={t20Count}
960                     onChange={(e) => setT20Count(Number(e.target.value))}
961                     className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
962                     dark:bg-slate-900 font-mon
963                 >
964                 <button
965                     type="button"
966                     onClick={() => {

```

src/components/tools/ToolsTextFormat.tsx (continued)

```
964         const sample = [  
965             "Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut  
966             labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco  
967             pariat. Excepteur sint occididunt pariatur. Excepteur sint occididunt pariatur. Excepteur  
968             pharetra, est eros bibendum dapibus, neque id cursus faucibus, tortor neque egestas augue, eu vulputate magna eros  
969             eu erat. Aliquam erat vol  
970         ],  
971         const res = Array.from(  
972             { length: Math.max(1, Math.min(15, t20Count)) },  
973             (_, i) => sample[i % sample.length]  
974         ).join("\n\n");  
975         setT20Output(res);  
976     }}  
977     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"  
978 >  
979     Generate Lorem  
980 </button>  
981 </div>  
982 </ToolCard>  
983 )}  
984 </>  
985 );  
986 };
```

FILE [23/25]

src/components/tools/ToolsMathCalculators.tsx Tools #21 - #50 (Math, Calculations, Financial, Conversion)

```
1 import React, { useState } from "react";
2 import { ToolCard } from "../ToolCard";
3 import { ToolCategory } from "../../types";
4
5 interface ToolsProps {
6   searchQuery: string;
7   selectedCategory: ToolCategory;
8 }
9
10 export const ToolsMathCalculators: React.FC<ToolsProps> = ({
11   searchQuery,
12   selectedCategory,
13 }) => {
14   const isVisible = (id: number, title: string, category: ToolCategory, keywords: string[] = []) => {
15     const matchesCategory = selectedCategory === "All" || selectedCategory === category;
16     if (!matchesCategory) return false;
17     if (!searchQuery.trim()) return true;
18     const q = searchQuery.toLowerCase().trim();
19     return (
20       title.toLowerCase().includes(q) ||
21       category.toLowerCase().includes(q) ||
22       `#${id}`.includes(q) ||
23       keywords.some((k) => k.toLowerCase().includes(q))
24     );
25   };
26
27   // Tool 21 - Color Palette
28   const [t21Base, setT21Base] = useState("#2563eb");
29   const [t21Output, setT21Output] = useState<string[]>([]);
30
31   // Tool 22 - QR Code
32   const [t22Input, setT22Input] = useState("https://ai.studio");
33
34   // Tool 23 - IP Address Info
35   const [t23Output, setT23Output] = useState("");
36   const [t23Loading, setT23Loading] = useState(false);
37
38   // Tool 24 - Age Calculator
39   const [t24Birth, setT24Birth] = useState("2000-01-01");
40   const [t24Output, setT24Output] = useState("");
41
42   // Tool 25 - Random Number
43   const [t25Min, setT25Min] = useState(1);
44   const [t25Max, setT25Max] = useState(100);
45   const [t25Output, setT25Output] = useState("");
46
47   // Tool 26 - Temperature Converter
48   const [t26Val, setT26Val] = useState("100");
49   const [t26From, setT26From] = useState("c");
50   const [t26To, setT26To] = useState("f");
51   const [t26Output, setT26Output] = useState("");
52
53   // Tool 27 - BMI Calculator
54   const [t27Weight, setT27Weight] = useState("70");
55   const [t27Height, setT27Height] = useState("175");
56   const [t27Output, setT27Output] = useState("");
57
58   // Tool 28 - Percentage Calculator
59   const [t28Val, setT28Val] = useState("25");
60   const [t28Total, setT28Total] = useState("200");
61   const [t28Output, setT28Output] = useState("");
62
63   // Tool 29 - Tip Calculator
64   const [t29Bill, setT29Bill] = useState("120");
65   const [t29Tip, setT29Tip] = useState("15");
66   const [t29Output, setT29Output] = useState("");
```


src/components/tools/ToolsMathCalculators.tsx (continued)

```
67
68 // Tool 30 - Discount Calculator
69 const [t30Price, setT30Price] = useState("80");
70 const [t30Disc, setT30Disc] = useState("20");
71 const [t30Output, setT30Output] = useState("");
72
73 // Tool 31 - EMI Calculator
74 const [t31P, setT31P] = useState("50000");
75 const [t31R, setT31R] = useState("8.5");
76 const [t31M, setT31M] = useState("24");
77 const [t31Output, setT31Output] = useState("");
78
79 // Tool 32 - Compound Interest
80 const [t32P, setT32P] = useState("10000");
81 const [t32R, setT32R] = useState("7");
82 const [t32Y, setT32Y] = useState("5");
83 const [t32Output, setT32Output] = useState("");
84
85 // Tool 33 - Simple Interest
86 const [t33P, setT33P] = useState("10000");
87 const [t33R, setT33R] = useState("5");
88 const [t33T, setT33T] = useState("3");
89 const [t33Output, setT33Output] = useState("");
90
91 // Tool 34 - Average Calculator
92 const [t34Input, setT34Input] = useState("12, 24, 36, 48, 60");
93 const [t34Output, setT34Output] = useState("");
94
95 // Tool 35 - Sum Calculator
96 const [t35Input, setT35Input] = useState("10, 25, 45, 120");
97 const [t35Output, setT35Output] = useState("");
98
99 // Tool 36 - Unit Converter
100 const [t36Val, setT36Val] = useState("10");
101 const [t36Type, setT36Type] = useState("km-mi");
102 const [t36Output, setT36Output] = useState("");
103
104 // Tool 37 - Time Converter (minutes to hours)
105 const [t37Min, setT37Min] = useState("135");
106 const [t37Output, setT37Output] = useState("");
107
108 // Tool 38 - Seconds Converter
109 const [t38Sec, setT38Sec] = useState("7325");
110 const [t38Output, setT38Output] = useState("");
111
112 // Tool 39 - Days Calculator
113 const [t39Start, setT39Start] = useState("2026-01-01");
114 const [t39End, setT39End] = useState("2026-12-31");
115 const [t39Output, setT39Output] = useState("");
116
117 // Tool 40 - Leap Year Checker
118 const [t40Year, setT40Year] = useState("2028");
119 const [t40Output, setT40Output] = useState("");
120
121 // Tool 41 - Prime Number Checker
122 const [t41Num, setT41Num] = useState("97");
123 const [t41Output, setT41Output] = useState("");
124
125 // Tool 42 - Even / Odd Checker
126 const [t42Num, setT42Num] = useState("42");
127 const [t42Output, setT42Output] = useState("");
128
129 // Tool 43 - Factorial Calculator
130 const [t43Num, setT43Num] = useState("7");
131 const [t43Output, setT43Output] = useState("");
132
133 // Tool 44 - Fibonacci Generator
134 const [t44Count, setT44Count] = useState("10");
135 const [t44Output, setT44Output] = useState("");
```

src/components/tools/ToolsMathCalculators.tsx (continued)

```
// Tool 45 - GCD Calculator
const [t45A, setT45A] = useState("48");
const [t45B, setT45B] = useState("18");
const [t45Output, setT45Output] = useState("");

// Tool 46 - LCM Calculator
const [t46A, setT46A] = useState("12");
const [t46B, setT46B] = useState("15");
const [t46Output, setT46Output] = useState("");

// Tool 47 - Binary Calculator
const [t47Bin, setT47Bin] = useState("110101");
const [t47Output, setT47Output] = useState("");

// Tool 48 - Hexadecimal Calculator
const [t48Hex, setT48Hex] = useState("FF");
const [t48Output, setT48Output] = useState("");

// Tool 49 - Random Password Generator
const [t49Len, setT49Len] = useState(16);
const [t49Output, setT49Output] = useState("");

// Tool 50 - Strong Password Checker
const [t50Pass, setT50Pass] = useState("Coding#2026_Secure!");
const [t50Output, setT50Output] = useState("");

return (
  <>
    {/* TOOL 21 - Color Palette */}
    {isVisible(21, "Color Palette Generator", "Color", ["palette", "tints", "shades"]) && (
      <ToolCard
        id={21}
        title="Color Palette Generator"
        category="Color"
        description="Generate a harmonious 5-tier tonal color palette"
        output={t21Output.length ? t21Output.join(", ") : undefined}
      >
        <div className="flex items-center gap-2">
          <input
            type="color"
            value={t21Base}
            onChange={(e) => setT21Base(e.target.value)}
            className="w-10 h-8 rounded border cursor-pointer"
          />
          <input
            type="text"
            value={t21Base}
            onChange={(e) => setT21Base(e.target.value)}
            className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-slate-900 font-mon"
          />
          <button
            type="button"
            onClick={() => {
              const hex = t21Base.replace("#", "");
              const r = parseInt(hex.substring(0, 2), 16) || 0;
              const g = parseInt(hex.substring(2, 4), 16) || 0;
              const b = parseInt(hex.substring(4, 6), 16) || 0;
              const colors = [];
              for (let i = 0; i < 5; i++) {
                const factor = 0.5 + i * 0.25;
                const nr = Math.min(255, Math.round(r * factor));
                const ng = Math.min(255, Math.round(g * factor));
                const nb = Math.min(255, Math.round(b * factor));
                colors.push(
                  "#" +
                    nr.toString(16).padStart(2, "0") +
                    ng.toString(16).padStart(2, "0") +
                    nb.toString(16).padStart(2, "0")
                );
              }
              setT21Output(colors);
            }}
          />
        </div>
      </>
    )}
  </>
)
```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

205         );
206     }
207     setT21Output(colors);
208   }}
209   className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
210 >
211   Generate Palette
212 </button>
213 </div>
214 {t21Output.length > 0 && (
215   <div className="flex h-10 rounded-xl overflow-hidden mt-2 border border-slate-300 dark:border-slate-700
shadow-sm">
216     {t21Output.map((c, i) => (
217       <div
218         key={i}
219         style={{ background: c }}
220         className="flex-1 flex items-center justify-center text-[10px] font-mono text-white font-bold
drop-shadow-sm" >
221         {c}
222       </div>
223     ))}
224   </div>
225 )}
226 </ToolCard>
227 )}
228
229
230 {/* TOOL 22 - QR Code Generator */}
231 {isVisible(22, "QR Code Generator", "Generators", ["qrcode", "barcode"]) && (
232   <ToolCard
233     id={22}
234     title="QR Code Generator"
235     category="Generators"
236     description="Create high-resolution QR codes for URLs and text"
237   >
238     <div className="flex gap-2">
239       <input
240         type="text"
241         value={t22Input}
242         onChange={(e) => setT22Input(e.target.value)}
243         placeholder="URL or text..."
244         className="flex-1 text-xs px-2.5 py-1 rounded border border-slate-300 dark:border-slate-700 bg-
245 slate-50 dark:bg-slate-900 font
246       </div>
247       {t22Input.trim() && (
248         <div className="mt-3 flex flex-col items-center p-3 bg-white dark:bg-slate-900 border border-slate-200
dark:border-slate-800 rou
249         <img
250           src={`https://api.qrserver.com/v1/create-qr-code/?size=160x160&data=${encodeURIComponent(
251             t22Input
252           )}`}
253           alt="QR Code"
254           className="w-36 h-36 border border-slate-200 dark:border-slate-800 rounded-lg p-1 bg-white shadow-
xs"
255         </div>
256         <span className="text-[11px] text-slate-500 mt-2 font-mono break-all text-center">
257           {t22Input}
258         </span>
259       </div>
260     )}
261   </ToolCard>
262 )}
263
264 {/* TOOL 23 - IP & Client Info */}
265 {isVisible(23, "IP Address Info", "Utilities", ["ip", "geo", "network", "client"]) && (
266   <ToolCard
267     id={23}
268     title="IP & Client Network Information"
269     category="Utilities"
270     description="Fetch public IP details and client environment metrics"
271     output={t23Output}
272   >
273     <button

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

274         type="button"
275         disabled={t23Loading}
276         onClick={async () => {
277             setT23Loading(true);
278             setT23Output("Fetching IP and network telemetry...");
279             try {
280                 const res = await fetch("https://ipapi.co/json/");
281                 if (!res.ok) throw new Error("Network request failed");
282                 const data = await res.json();
283                 setT23Output(
284                     `IP: ${data.ip || "N/A"}\nCity: ${data.city || "N/A"}\nRegion: ${
285                         data.region || "N/A"
286                     }\nCountry: ${data.country_name || "N/A"}\nISP: ${
287                         data.org || "N/A"
288                     }\nTimezone: ${data.timezone || "N/A"}`
289                 );
290             } catch {
291                 setT23Output(
292                     `Client Info (Offline Mode):\nUser-Agent: ${navigator.userAgent}\nLanguage:
293 ${navigator.language}\nOnLine: ${navigator.onLine ? "Yes" : "No"}
294 \nScreen: ${screen.width}x${screen.height}`
295                 );
296             } finally {
297                 setT23Loading(false);
298             }
299         }}
300         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
301     >
302         {t23Loading ? "Loading..." : "Get Info"}
303     </button>
304 </ToolCard>
305 ))
306
307 {/* TOOL 24 - Age Calculator */}
308 {isVisible(24, "Age Calculator", "Math", ["age", "birthday", "years"]) && (
309     <ToolCard
310         id={24}
311         title="Age Calculator"
312         category="Math"
313         description="Calculate exact chronological age in years and months"
314         output={t24Output}
315     >
316         <div className="flex gap-2">
317             <input
318                 type="date"
319                 value={t24Birth}
320                 onChange={(e) => setT24Birth(e.target.value)}
321                 className="text-xs px-2.5 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
322                 dark:bg-slate-900"
323             />
324             <button
325                 type="button"
326                 onClick={() => {
327                     if (!t24Birth) {
328                         setT24Output("'L Select a birth date");
329                         return;
330                     }
331                     const b = new Date(t24Birth);
332                     const now = new Date();
333                     let age = now.getFullYear() - b.getFullYear();
334                     const m = now.getMonth() - b.getMonth();
335                     if (m < 0 || (m === 0 && now.getDate() < b.getDate())) {
336                         age--;
337                     }
338                     const diffDays = Math.floor((now.getTime() - b.getTime()) / (1000 * 60 * 60 * 24));
339                     setT24Output(`Age: ${age} years\nTotal Days lived: ~${diffDays} days`);
340                 }}
341             >
342                 Calculate Age

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

343         </button>
344     </div>
345 </ToolCard>
346 }}
347
348 {/* TOOL 25 - Random Number Generator */}
349 {isVisible(25, "Random Number Generator", "Generators", ["random", "dice", "number"]) && (
350     <ToolCard
351         id={25}
352         title="Random Number Generator"
353         category="Generators"
354         description="Generate random integer within custom minimum and maximum bounds"
355         output={t25Output}
356     >
357         <div className="flex items-center gap-2">
358             <input
359                 type="number"
360                 value={t25Min}
361                 onChange={(e) => setT25Min(Number(e.target.value))}
362                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
363                 dark:bg-slate-900 font-mon"
364             />
365             <span className="text-xs text-slate-500">to</span>
366             <input
367                 type="number"
368                 value={t25Max}
369                 onChange={(e) => setT25Max(Number(e.target.value))}
370                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
371                 dark:bg-slate-900 font-mon"
372             />
373             <button
374                 type="button"
375                 onClick={() => {
376                     if (t25Min > t25Max) {
377                         setT25Output("L Minimum cannot exceed Maximum");
378                         return;
379                     }
380                     const rand = Math.floor(Math.random() * (t25Max - t25Min + 1)) + t25Min;
381                     setT25Output(`Random Number: ${rand}`);
382                 }}
383                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
384             >
385                 Generate
386             </button>
387         </div>
388     </ToolCard>
389 }}
390
391 {/* TOOL 26 - Temperature Converter */}
392 {isVisible(26, "Temperature Converter", "Converters", ["celsius", "fahrenheit", "kelvin"]) && (
393     <ToolCard
394         id={26}
395         title="Temperature Converter"
396         category="Converters"
397         description="Convert between Celsius, Fahrenheit, and Kelvin scales"
398         output={t26Output}
399     >
400         <div className="flex flex-wrap items-center gap-2">
401             <input
402                 type="number"
403                 value={t26Val}
404                 onChange={(e) => setT26Val(e.target.value)}
405                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
406                 dark:bg-slate-900 font-mon"
407             />
408             <select
409                 value={t26From}
410                 onChange={(e) => setT26From(e.target.value)}
411                 className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
412                 dark:bg-slate-900"

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

412         <option value="c">°C</option>
413         <option value="f">°F</option>
414         <option value="k">K</option>
415     </select>
416     <span className="text-xs text-slate-500">!</span>
417     <select
418         value={t26To}
419         onChange={(e) => setT26To(e.target.value)}
420         className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
421         dark:bg-slate-900"
422         <option value="f">°F</option>
423         <option value="c">°C</option>
424         <option value="k">K</option>
425     </select>
426     <button
427         type="button"
428         onClick={() => {
429             const v = Number(t26Val);
430             if (!Number.isFinite(v)) return;
431             let c = v;
432             if (t26From === "f") c = ((v - 32) * 5) / 9;
433             if (t26From === "k") c = v - 273.15;
434             let res = c;
435             if (t26To === "f") res = (c * 9) / 5 + 32;
436             if (t26To === "k") res = c + 273.15;
437             setT26Output(`Result: ${res.toFixed(2)} °${t26To.toUpperCase()}`);
438         }}
439         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
440     >
441         Convert
442     </button>
443 </div>
444 </ToolCard>
445 )}
446
447 {/* TOOL 27 - BMI Calculator */}
448 {isVisible(27, "BMI Calculator", "Math", ["bmi", "body mass", "health"]) && (
449     <ToolCard
450         id={27}
451         title="BMI Calculator"
452         category="Math"
453         description="Calculate Body Mass Index and healthy weight categories"
454         output={t27Output}
455     >
456         <div className="flex flex-wrap items-center gap-2">
457             <input
458                 type="number"
459                 value={t27Weight}
460                 onChange={(e) => setT27Weight(e.target.value)}
461                 placeholder="Weight (kg)"
462                 className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
463                 dark:bg-slate-900 font-mon"
464             />
465             <input
466                 type="number"
467                 value={t27Height}
468                 onChange={(e) => setT27Height(e.target.value)}
469                 placeholder="Height (cm)"
470                 className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
471                 dark:bg-slate-900 font-mon"
472             />
473             <button
474                 type="button"
475                 onClick={() => {
476                     const w = Number(t27Weight);
477                     const h = Number(t27Height) / 100;
478                     if (w <= 0 || h <= 0) {
479                         setT27Output("L Enter positive values for weight and height");
480                     }
481                     const bmi = w / (h * h);

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

481         const cat =
482             bmi < 18.5
483                 ? "Underweight"
484                 : bmi < 25
485                 ? "Normal weight"
486                 : bmi < 30
487                 ? "Overweight"
488                 : "Obese";
489         setT27Output(`BMI: ${bmi.toFixed(2)} (${cat})`);
490     }}
491     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
492 >
493     Calculate BMI
494 </button>
495 </div>
496 </ToolCard>
497 )}
498
499 /* TOOL 28 - Percentage Calculator */
500 {isVisible(28, "Percentage Calculator", "Math", ["percent", "ratio", "fraction"]) && (
501     <ToolCard
502         id={28}
503         title="Percentage Calculator"
504         category="Math"
505         description="Find what percentage one number is of another"
506         output={t28Output}
507     >
508         <div className="flex items-center gap-2">
509             <input
510                 type="number"
511                 value={t28Val}
512                 onChange={(e) => setT28Val(e.target.value)}
513                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
514                 dark:bg-slate-900 font-mon
515                 <span className="text-xs text-slate-500">out of</span>
516             <input
517                 type="number"
518                 value={t28Total}
519                 onChange={(e) => setT28Total(e.target.value)}
520                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
521                 dark:bg-slate-900 font-mon
522             <button
523                 type="button"
524                 onClick={() => {
525                     const v = Number(t28Val);
526                     const t = Number(t28Total);
527                     if (t === 0) {
528                         setT28Output("'L Total cannot be 0");
529                         return;
530                     }
531                     setT28Output(`Result: ${((v / t) * 100).toFixed(2)}%`);
532                 }}
533                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
534             >
535                 Calculate
536             </button>
537         </div>
538     </ToolCard>
539 )}
540
541 /* TOOL 29 - Tip Calculator */
542 {isVisible(29, "Tip Calculator", "Math", ["tip", "bill", "restaurant"]) && (
543     <ToolCard
544         id={29}
545         title="Tip Calculator"
546         category="Math"
547         description="Calculate gratuity amount and bill grand total"
548         output={t29Output}
549     >

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

550         <div className="flex items-center gap-2">
551             <input
552                 type="number"
553                 value={t29Bill}
554                 onChange={(e) => setT29Bill(e.target.value)}
555                 placeholder="Bill ($)"
556                 className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
557 dark:bg-slate-900 font-mon
558             <input
559                 type="number"
560                 value={t29Tip}
561                 onChange={(e) => setT29Tip(e.target.value)}
562                 placeholder="Tip %"
563                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
564 dark:bg-slate-900 font-mon
565             <span className="text-xs text-slate-500">%/</span>
566             <button
567                 type="button"
568                 onClick={() => {
569                     const b = Number(t29Bill);
570                     const p = Number(t29Tip);
571                     const tip = (b * p) / 100;
572                     setT29Output(`Tip: ${tip.toFixed(2)}\nTotal: ${((b + tip).toFixed(2))}`);
573                 }}
574                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
575             >
576                 Calculate Tip
577             </button>
578         </div>
579     </ToolCard>
580 ))
581
582 {/* TOOL 30 - Discount Calculator */}
583 {isVisible(30, "Discount Calculator", "Math", ["discount", "sale", "price"])} && (
584     <ToolCard
585         id={30}
586         title="Discount Calculator"
587         category="Math"
588         description="Calculate final discounted price and savings"
589         output={t30Output}
590     >
591         <div className="flex items-center gap-2">
592             <input
593                 type="number"
594                 value={t30Price}
595                 onChange={(e) => setT30Price(e.target.value)}
596                 placeholder="Original Price"
597                 className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
598 dark:bg-slate-900 font-mon
599             <input
600                 type="number"
601                 value={t30Disc}
602                 onChange={(e) => setT30Disc(e.target.value)}
603                 placeholder="Discount %"
604                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
605 dark:bg-slate-900 font-mon
606             <span className="text-xs text-slate-500">%/</span>
607             <button
608                 type="button"
609                 onClick={() => {
610                     const p = Number(t30Price);
611                     const d = Number(t30Disc);
612                     const savings = (p * d) / 100;
613                     setT30Output(`You Save: ${savings.toFixed(2)}\nFinal Price: ${((p - savings).toFixed(2))}`);
614                 }}
615                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
616             >
617                 Calculate
618             </button>

```


src/components/tools/ToolsMathCalculators.tsx (continued)

```

619         </div>
620     </ToolCard>
621 })
622
623 {/* TOOL 31 - EMI Calculator */}
624 {isVisible(31, "EMI Calculator", "Math", ["loan", "emi", "mortgage"]) && (
625     <ToolCard
626         id={31}
627         title="EMI Loan Calculator"
628         category="Math"
629         description="Calculate monthly installment and total payable loan interest"
630         output={t31Output}
631     >
632         <div className="flex flex-wrap items-center gap-2">
633             <input
634                 type="number"
635                 value={t31P}
636                 onChange={(e) => setT31P(e.target.value)}
637                 placeholder="Loan amount"
638                 className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
639                 dark:bg-slate-900 font-mon
640             />
641             <input
642                 type="number"
643                 value={t31R}
644                 onChange={(e) => setT31R(e.target.value)}
645                 placeholder="Annual Rate %"
646                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
647                 dark:bg-slate-900 font-mon
648             />
649             <input
650                 type="number"
651                 value={t31M}
652                 onChange={(e) => setT31M(e.target.value)}
653                 placeholder="Months"
654                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
655                 dark:bg-slate-900 font-mon
656             />
657             <button
658                 type="button"
659                 onClick={() => {
660                     const p = Number(t31P);
661                     const r = Number(t31R) / 12 / 100;
662                     const m = Number(t31M);
663                     if (p <= 0 || m <= 0) return;
664                     const emi = (p * r * Math.pow(1 + r, m)) / (Math.pow(1 + r, m) - 1);
665                     const total = emi * m;
666                     setT31Output(`Monthly EMI: ${emi.toFixed(2)}\nTotal Interest: ${((total - p).toFixed(2))}\nTotal
667                     Amount: ${total.toFixed(2)}`)
668                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
669                 }
670             />
671             Calculate EMI
672         </button>
673     </div>
674 </ToolCard>
675 })
676
677 {/* TOOL 32 - Compound Interest */}
678 {isVisible(32, "Compound Interest Calculator", "Math", ["interest", "finance"]) && (
679     <ToolCard
680         id={32}
681         title="Compound Interest Calculator"
682         category="Math"
683         description="Calculate future value with compounding interest"
684         output={t32Output}
685     >
686         <div className="flex flex-wrap items-center gap-2">
687             <input
688                 type="number"
689                 value={t32P}
690                 onChange={(e) => setT32P(e.target.value)}
691                 placeholder="Principal"

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

688         className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
689 dark:bg-slate-900 font-mon
690         <input
691           type="number"
692           value={t32R}
693           onChange={(e) => setT32R(e.target.value)}
694           placeholder="Annual %"
695         className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
696 dark:bg-slate-900 font-mon
697         <input
698           type="number"
699           value={t32Y}
700           onChange={(e) => setT32Y(e.target.value)}
701           placeholder="Years"
702         className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
703 dark:bg-slate-900 font-mon
704         <button
705           type="button"
706           onClick={() => {
707             const p = Number(t32P);
708             const r = Number(t32R) / 100;
709             const y = Number(t32Y);
710             const amount = p * Math.pow(1 + r, y);
711             setT320Output(`Total Amount: $${amount.toFixed(2)}\nInterest Earned: $${(amount - p).toFixed(2)}`);
712           }}
713         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
714       >
715         Calculate
716     </button>
717 </div>
718 </ToolCard>
719 }}
720
721 /* TOOL 33 - Simple Interest */
722 {isVisible(33, "Simple Interest Calculator", "Math", ["simple interest", "principal"]) && (
723   <ToolCard
724     id={33}
725     title="Simple Interest Calculator"
726     category="Math"
727     description="Compute simple interest (P * R * T / 100)"
728     output={t330Output}
729   >
730     <div className="flex flex-wrap items-center gap-2">
731       <input
732         type="number"
733         value={t33P}
734         onChange={(e) => setT33P(e.target.value)}
735         placeholder="Principal"
736       className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
737 dark:bg-slate-900 font-mon
738       <input
739         type="number"
740         value={t33R}
741         onChange={(e) => setT33R(e.target.value)}
742         placeholder="Rate %"
743       className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
744 dark:bg-slate-900 font-mon
745       <input
746         type="number"
747         value={t33T}
748         onChange={(e) => setT33T(e.target.value)}
749         placeholder="Time"
750       className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
751 dark:bg-slate-900 font-mon
752       <button
753         type="button"
754         onClick={() => {
755           const p = Number(t33P);
756           const r = Number(t33R);

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

757         const t = Number(t33T);
758         const interest = (p * r * t) / 100;
759         setT33Output(`Interest: ${interest.toFixed(2)}\nTotal: ${ (p + interest).toFixed(2)}`);
760     }}
761     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
762     >
763         Calculate
764     </button>
765 </div>
766 </ToolCard>
767 }}
768
769 {/* TOOL 34 - Average Calculator */}
770 {isVisible(34, "Average Calculator", "Math", ["mean", "average"]) && (
771     <ToolCard
772         id={34}
773         title="Average (Mean) Calculator"
774         category="Math"
775         description="Compute arithmetic mean for comma-separated numbers"
776         output={t34Output}
777     >
778         <div className="flex gap-2">
779             <input
780                 type="text"
781                 value={t34Input}
782                 onChange={(e) => setT34Input(e.target.value)}
783                 placeholder="e.g. 10, 20, 35, 50"
784                 className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
785                 dark:bg-slate-900 font-m
786             >
787             <button
788                 type="button"
789                 onClick={() => {
790                     const nums = t34Input.split(",").map(Number).filter(Number.isFinite);
791                     if (!nums.length) return setT34Output("'L Enter valid numbers");
792                     const avg = nums.reduce((a, b) => a + b, 0) / nums.length;
793                     setT34Output(`Average: ${avg.toFixed(2)} (Count: ${nums.length})`);
794                 }}
795                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
796             >
797                 Calculate
798             </button>
799         </div>
800     </ToolCard>
801 }}
802
803 {/* TOOL 35 - Sum Calculator */}
804 {isVisible(35, "Sum Calculator", "Math", ["addition", "total"]) && (
805     <ToolCard
806         id={35}
807         title="Sum Calculator"
808         category="Math"
809         description="Add up comma-separated numbers"
810         output={t35Output}
811     >
812         <div className="flex gap-2">
813             <input
814                 type="text"
815                 value={t35Input}
816                 onChange={(e) => setT35Input(e.target.value)}
817                 placeholder="e.g. 10, 20, 30"
818                 className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
819                 dark:bg-slate-900 font-m
820             >
821             <button
822                 type="button"
823                 onClick={() => {
824                     const nums = t35Input.split(",").map(Number).filter(Number.isFinite);
825                     const sum = nums.reduce((a, b) => a + b, 0);
826                     setT35Output(`Sum: ${sum}`);
827                 }}

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```
826         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
827     >
828         Sum
829     </button>
830 </div>
831 </ToolCard>
832 }}
833
834 {/* TOOL 36 - Unit Converter */}
835 {isVisible(36, "Unit Converter", "Converters", ["km", "miles", "kg", "pounds", "meters", "feet"]) && (
836     <ToolCard
837         id={36}
838         title="Unit Converter"
839         category="Converters"
840         description="Convert distances, lengths, and weights"
841         output={t36Output}
842     >
843         <div className="flex flex-wrap items-center gap-2">
844             <input
845                 type="number"
846                 value={t36Val}
847                 onChange={(e) => setT36Val(e.target.value)}
848                 className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
849                 dark:bg-slate-900 font-mon"
850             />
851             <select
852                 value={t36Type}
853                 onChange={(e) => setT36Type(e.target.value)}
854                 className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
855                 dark:bg-slate-900"
856             >
857                 <option value="km-mi">Kilometers !' Miles</option>
858                 <option value="mi-km">Miles !' Kilometers</option>
859                 <option value="m-ft">Meters !' Feet</option>
860                 <option value="ft-m">Feet !' Meters</option>
861                 <option value="kg-lb">Kg !' Pounds</option>
862                 <option value="lb-kg">Pounds !' Kg</option>
863             </select>
864             <button
865                 type="button"
866                 onClick={() => {
867                     const v = Number(t36Val);
868                     let res = 0;
869                     let u = "";
870                     if (t36Type === "km-mi") { res = v * 0.621371; u = "miles"; }
871                     if (t36Type === "mi-km") { res = v * 1.60934; u = "km"; }
872                     if (t36Type === "m-ft") { res = v * 3.28084; u = "feet"; }
873                     if (t36Type === "ft-m") { res = v * 0.3048; u = "meters"; }
874                     if (t36Type === "kg-lb") { res = v * 2.20462; u = "pounds"; }
875                     if (t36Type === "lb-kg") { res = v * 0.453592; u = "kg"; }
876                     setT36Output(`Result: ${res.toFixed(4)} ${u}`);
877                 }}
878                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
879             >
880                 Convert
881             </button>
882         </div>
883     </ToolCard>
884 }}
885
886 {/* TOOL 37 - Time Converter */}
887 {isVisible(37, "Time Converter", "Time", ["minutes", "hours"]) && (
888     <ToolCard
889         id={37}
890         title="Minutes to Hours Converter"
891         category="Time"
892         description="Convert minutes into hours and remaining minutes"
893         output={t37Output}
894     >
895         <div className="flex gap-2">
896             <input
```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

895         type="number"
896         value={t37Min}
897         onChange={(e) => setT37Min(e.target.value)}
898         placeholder="Minutes"
899         className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
900 dark:bg-slate-900 font-mon
901       <button
902         type="button"
903         onClick={() => {
904           const m = Number(t37Min);
905           setT370Output(`${Math.floor(m / 60)} hours ${m % 60} minutes`);
906         }}
907         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
908       >
909         Convert
910       </button>
911     </div>
912   </ToolCard>
913 })
914
915 {/* TOOL 38 - Seconds Converter */}
916 {isVisible(38, "Seconds Converter", "Time", ["seconds", "duration"]) && (
917   <ToolCard
918     id={38}
919     title="Seconds to Hours, Min & Sec"
920     category="Time"
921     description="Convert raw seconds to formatted duration"
922     output={t380Output}
923   >
924     <div className="flex gap-2">
925       <input
926         type="number"
927         value={t38Sec}
928         onChange={(e) => setT38Sec(e.target.value)}
929         placeholder="Seconds"
930         className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
931 dark:bg-slate-900 font-mon
932       <button
933         type="button"
934         onClick={() => {
935           const s = Number(t38Sec);
936           const h = Math.floor(s / 3600);
937           const m = Math.floor((s % 3600) / 60);
938           const sec = s % 60;
939           setT380Output(`${h}h ${m}m ${sec}s`);
940         }}
941         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
942       >
943         Convert
944       </button>
945     </div>
946   </ToolCard>
947 })
948
949 {/* TOOL 39 - Days Calculator */}
950 {isVisible(39, "Days Calculator", "Time", ["days difference", "dates"]) && (
951   <ToolCard
952     id={39}
953     title="Days Between Dates Calculator"
954     category="Time"
955     description="Find total calendar days between two dates"
956     output={t390Output}
957   >
958     <div className="flex flex-wrap items-center gap-2">
959       <input
960         type="date"
961         value={t39Start}
962         onChange={(e) => setT39Start(e.target.value)}
963         className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
dark:bg-slate-900 font-mono"

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

964         />
965         <span className="text-xs text-slate-500">to</span>
966         <input
967             type="date"
968             value={t39End}
969             onChange={(e) => setT39End(e.target.value)}
970             className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
971             dark:bg-slate-900 font-mono"
972         <button
973             type="button"
974             onClick={() => {
975                 const s = new Date(t39Start);
976                 const e = new Date(t39End);
977                 const diff = Math.round(Math.abs(e.getTime() - s.getTime()) / (1000 * 60 * 60 * 24));
978                 setT39Output(`Difference: ${diff} days`);
979             }}
980             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
981         >
982             Calculate
983         </button>
984     </div>
985 </ToolCard>
986 }}
987
988 /* TOOL 40 - Leap Year Checker */
989 {isVisible(40, "Leap Year Checker", "Math", ["leap", "calendar", "february"]) && (
990     <ToolCard
991         id={40}
992         title="Leap Year Checker"
993         category="Math"
994         description="Check if a year has 366 days"
995         output={t40Output}
996     >
997         <div className="flex gap-2">
998             <input
999                 type="number"
1000                 value={t40Year}
1001                 onChange={(e) => setT40Year(e.target.value)}
1002                 placeholder="Year"
1003                 className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1004                 dark:bg-slate-900 font-mon
1005             <button
1006                 type="button"
1007                 onClick={() => {
1008                     const y = Number(t40Year);
1009                     const isLeap = (y % 4 === 0 && y % 100 !== 0) || y % 400 === 0;
1010                     setT40Output(isLeap ? ` ${y} is a Leap Year` : `L ${y} is NOT a Leap Year`);
1011                 }}
1012                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1013             >
1014                 Check
1015             </button>
1016         </div>
1017     </ToolCard>
1018 }}
1019
1020 /* TOOL 41 - Prime Number Checker */
1021 {isVisible(41, "Prime Number Checker", "Math", ["prime", "divisible"]) && (
1022     <ToolCard
1023         id={41}
1024         title="Prime Number Checker"
1025         category="Math"
1026         description="Test if an integer is prime"
1027         output={t41Output}
1028     >
1029         <div className="flex gap-2">
1030             <input
1031                 type="number"
1032                 value={t41Num}

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

1033         onChange={(e) => setT41Num(e.target.value)}
1034         placeholder="Number"
1035         className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1036         dark:bg-slate-900 font-mon
1037         <button
1038             type="button"
1039             onClick={() => {
1040                 const n = Number(t41Num);
1041                 if (n < 2) return setT41Output("'L Must be greater than 1");
1042                 let isP = true;
1043                 for (let i = 2; i <= Math.sqrt(n); i++) {
1044                     if (n % i === 0) { isP = false; break; }
1045                 }
1046                 setT41Output(isP ? ` ${n} is Prime` : `L ${n} is Composite (Not Prime)`);
1047             }}
1048             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1049         >
1050             Check Prime
1051         </button>
1052     </div>
1053 </ToolCard>
1054 }}
1055
1056 {/* TOOL 42 - Even / Odd Checker */}
1057 {isVisible(42, "Even / Odd Checker", "Math", ["even", "odd", "parity"]) && (
1058     <ToolCard
1059         id={42}
1060         title="Even / Odd Checker"
1061         category="Math"
1062         description="Check integer parity"
1063         output={t42Output}
1064     >
1065         <div className="flex gap-2">
1066             <input
1067                 type="number"
1068                 value={t42Num}
1069                 onChange={(e) => setT42Num(e.target.value)}
1070                 className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1071                 dark:bg-slate-900 font-mon
1072             <button
1073                 type="button"
1074                 onClick={() => {
1075                     const n = Number(t42Num);
1076                     setT42Output(n % 2 === 0 ? ` ${n} is Even` : ` ${n} is Odd`);
1077                 }}
1078                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1079             >
1080                 Check
1081             </button>
1082         </div>
1083     </ToolCard>
1084 }}
1085
1086 {/* TOOL 43 - Factorial Calculator */}
1087 {isVisible(43, "Factorial Calculator", "Math", ["factorial", "n!"]) && (
1088     <ToolCard
1089         id={43}
1090         title="Factorial Calculator (n!)"
1091         category="Math"
1092         description="Calculate n! for numbers 0 to 170"
1093         output={t43Output}
1094     >
1095         <div className="flex gap-2">
1096             <input
1097                 type="number"
1098                 min={0}
1099                 max={170}
1100                 value={t43Num}
1101                 onChange={(e) => setT43Num(e.target.value)}

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

1102         className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1103 dark:bg-slate-900 font-mon
1104         <button
1105           type="button"
1106           onClick={() => {
1107             const n = Number(t43Num);
1108             if (n < 0 || n > 170) return setT43Output("'L Between 0 and 170");
1109             let res = 1;
1110             for (let i = 2; i <= n; i++) res *= i;
1111             setT43Output(`${n}! = ${res}`);
1112           }}
1113           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1114         >
1115           Calculate
1116         </button>
1117       </div>
1118     </ToolCard>
1119   )}
1120
1121   {/* TOOL 44 - Fibonacci Generator */}
1122   {isVisible(44, "Fibonacci Generator", "Math", ["fibonacci", "sequence"]) && (
1123     <ToolCard
1124       id={44}
1125       title="Fibonacci Sequence Generator"
1126       category="Math"
1127       description="Generate first N numbers of the Fibonacci sequence"
1128       output={t44Output}
1129     >
1130       <div className="flex gap-2">
1131         <input
1132           type="number"
1133           min={1}
1134           max={50}
1135           value={t44Count}
1136           onChange={(e) => setT44Count(e.target.value)}
1137         />
1138         dark:bg-slate-900 font-mon
1139         <button
1140           type="button"
1141           onClick={() => {
1142             const count = Math.min(50, Number(t44Count));
1143             const res = [];
1144             let a = 0, b = 1;
1145             for (let i = 0; i < count; i++) {
1146               res.push(a);
1147               const next = a + b;
1148               a = b;
1149               b = next;
1150             }
1151             setT44Output(res.join(", "));
1152           }}
1153           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1154         >
1155           Generate
1156         </button>
1157       </div>
1158     </ToolCard>
1159   )}
1160
1161   {/* TOOL 45 - GCD Calculator */}
1162   {isVisible(45, "GCD Calculator", "Math", ["gcd", "greatest common divisor"]) && (
1163     <ToolCard
1164       id={45}
1165       title="GCD Calculator"
1166       category="Math"
1167       description="Find greatest common divisor of two integers"
1168       output={t45Output}
1169     >
1170       <div className="flex gap-2">

```


src/components/tools/ToolsMathCalculators.tsx (continued)

```

1171         <input
1172             type="number"
1173             value={t45A}
1174             onChange={(e) => setT45A(e.target.value)}
1175             className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1176             dark:bg-slate-900 font-mon">
1177         <input
1178             type="number"
1179             value={t45B}
1180             onChange={(e) => setT45B(e.target.value)}
1181             className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1182             dark:bg-slate-900 font-mon">
1183         <button
1184             type="button"
1185             onClick={() => {
1186                 let a = Math.abs(Number(t45A));
1187                 let b = Math.abs(Number(t45B));
1188                 while (b !== 0) {
1189                     const temp = b;
1190                     b = a % b;
1191                     a = temp;
1192                 }
1193                 setT45Output(`GCD: ${a}`);
1194             }}
1195             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1196         >
1197             GCD
1198         </button>
1199     </div>
1200 </ToolCard>
1201 }}
1202
1203 /* TOOL 46 - LCM Calculator */
1204 {isVisible(46, "LCM Calculator", "Math", ["lcm", "least common multiple"]) && (
1205     <ToolCard
1206         id={46}
1207         title="LCM Calculator"
1208         category="Math"
1209         description="Find least common multiple of two integers"
1210         output={t46Output}
1211     >
1212         <div className="flex gap-2">
1213             <input
1214                 type="number"
1215                 value={t46A}
1216                 onChange={(e) => setT46A(e.target.value)}
1217                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1218                 dark:bg-slate-900 font-mon">
1219             <input
1220                 type="number"
1221                 value={t46B}
1222                 onChange={(e) => setT46B(e.target.value)}
1223                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1224                 dark:bg-slate-900 font-mon">
1225             <button
1226                 type="button"
1227                 onClick={() => {
1228                     const a = Math.abs(Number(t46A));
1229                     const b = Math.abs(Number(t46B));
1230                     const gcd = (x: number, y: number): number => (y === 0 ? x : gcd(y, x % y));
1231                     const lcm = (a * b) / gcd(a, b);
1232                     setT46Output(`LCM: ${lcm}`);
1233                 }}
1234                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1235             >
1236                 LCM
1237             </button>
1238         </div>
1239     </ToolCard>

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```
1240     })
1241
1242     {/* TOOL 47 - Binary Calculator */}
1243     {isVisible(47, "Binary Calculator", "Math", ["binary to decimal"]) && (
1244       <ToolCard
1245         id={47}
1246         title="Binary to Decimal Calculator"
1247         category="Math"
1248         description="Convert binary sequence to base 10 integer"
1249         output={t47Output}
1250       >
1251         <div className="flex gap-2">
1252           <input
1253             type="text"
1254             value={t47Bin}
1255             onChange={(e) => setT47Bin(e.target.value)}
1256             placeholder="e.g. 101010"
1257             className="w-36 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1258             dark:bg-slate-900 font-mon">
1259           <button
1260             type="button"
1261             onClick={() => {
1262               if (!/^[01]+$/.test(t47Bin.trim())) return setT47Output("L Enter valid binary (0 and 1 only)");
1263               setT47Output(`Decimal: ${parseInt(t47Bin.trim(), 2)}`);
1264             }}
1265             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1266           >
1267             Convert
1268           </button>
1269         </div>
1270       </ToolCard>
1271     )}
1272
1273     {/* TOOL 48 - Hexadecimal Calculator */}
1274     {isVisible(48, "Hexadecimal Calculator", "Math", ["hex to decimal"]) && (
1275       <ToolCard
1276         id={48}
1277         title="Hexadecimal to Decimal Calculator"
1278         category="Math"
1279         description="Convert hex number to base 10 integer"
1280         output={t48Output}
1281       >
1282         <div className="flex gap-2">
1283           <input
1284             type="text"
1285             value={t48Hex}
1286             onChange={(e) => setT48Hex(e.target.value)}
1287             placeholder="e.g. 1A3F"
1288             className="w-36 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1289             dark:bg-slate-900 font-mon">
1290           <button
1291             type="button"
1292             onClick={() => {
1293               if (!/^[0-9a-fA-F]+$/.test(t48Hex.trim())) return setT48Output("L Enter valid hex string");
1294               setT48Output(`Decimal: ${parseInt(t48Hex.trim(), 16)}`);
1295             }}
1296             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1297           >
1298             Convert
1299           </button>
1300         </div>
1301       </ToolCard>
1302     )}
1303
1304     {/* TOOL 49 - Random Password Generator */}
1305     {isVisible(49, "Random Password Generator", "Security", ["password", "strong"]) && (
1306       <ToolCard
1307         id={49}
1308         title="Advanced Password Generator"
```

src/components/tools/ToolsMathCalculators.tsx (continued)

```

1309         category="Security"
1310         description="Generate complex passwords with uppercase, lowercase, numbers, and symbols"
1311         output={t49Output}
1312     >
1313         <div className="flex items-center gap-2">
1314             <input
1315                 type="number"
1316                 min={6}
1317                 max={64}
1318                 value={t49Len}
1319                 onChange={(e) => setT49Len(Number(e.target.value))}
1320                 className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1321                 dark:bg-slate-900 font-mon
1322             <button
1323                 type="button"
1324                 onClick={() => {
1325                     const chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789!@#$$%^&*()_+!=";
1326                     const arr = new Uint8Array(t49Len);
1327                     crypto.getRandomValues(arr);
1328                     setT49Output(Array.from(arr, (x) => chars[x % chars.length]).join(""));
1329                 }}
1330                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1331             >
1332                 Generate
1333             </button>
1334         </div>
1335     </ToolCard>
1336 }}
1337
1338 /* TOOL 50 - Strong Password Checker */
1339 {isVisible(50, "Strong Password Checker", "Security", ["password strength", "entropy"]) && (
1340     <ToolCard
1341         id={50}
1342         title="Strong Password Checker"
1343         category="Security"
1344         description="Evaluate password entropy against length and diversity criteria"
1345         output={t50Output}
1346     >
1347         <div className="flex gap-2">
1348             <input
1349                 type="text"
1350                 value={t50Pass}
1351                 onChange={(e) => setT50Pass(e.target.value)}
1352                 placeholder="Enter password..."
1353                 className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1354                 dark:bg-slate-900 font-m
1355             <button
1356                 type="button"
1357                 onClick={() => {
1358                     let score = 0;
1359                     if (t50Pass.length >= 8) score++;
1360                     if (t50Pass.length >= 12) score++;
1361                     if (/[A-Z]/.test(t50Pass)) score++;
1362                     if (/[a-z]/.test(t50Pass)) score++;
1363                     if (/[0-9]/.test(t50Pass)) score++;
1364                     if (/^[A-Za-z0-9]/.test(t50Pass)) score++;
1365                     const rating =
1366                         score <= 2
1367                             ? "ðŸŒˆ Weak Password (Score: " + score + "/6)"
1368                             : score <= 4
1369                             ? "ðŸŒˆ Medium Password (Score: " + score + "/6)"
1370                             : "ðŸŒˆ Strong Password (Score: " + score + "/6)";
1371                     setT50Output(rating);
1372                 }}
1373                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1374             >
1375                 Check Strength
1376             </button>
1377         </div>

```

src/components/tools/ToolsMathCalculators.tsx (continued)

```
1378         </ToolCard>
1379     )}
1380 </>
1381 );
1382 };
1383
```

FILE [24/25]

src/components/tools/ToolsDataWeb.tsx 74.3 KB • Tools #51 - #100 (Data, Web, CSS, JSON, SEO, Dev Cheat Sheets)

```
1 import React, { useState } from "react";
2 import { ToolCard } from "../ToolCard";
3 import { ToolCategory } from "../../types";
4 import { sha256Hex, hmacSha256Hex } from "../../utils/crypto";
5 import {
6   jsonToCsv,
7   csvToJson,
8   morseEncode,
9   morseDecode,
10  textToBinaryString,
11  binaryStringToText,
12  generateUUIDv4,
13 } from "../../utils/helpers";
14
15 interface ToolsProps {
16   searchQuery: string;
17   selectedCategory: ToolCategory;
18 }
19
20 export const ToolsDataWeb: React.FC<ToolsProps> = ({
21   searchQuery,
22   selectedCategory,
23 }) => {
24   const isVisible = (id: number, title: string, category: ToolCategory, keywords: string[] = []) => {
25     const matchesCategory = selectedCategory === "All" || selectedCategory === category;
26     if (!matchesCategory) return false;
27     if (!searchQuery.trim()) return true;
28     const q = searchQuery.toLowerCase().trim();
29     return (
30       title.toLowerCase().includes(q) ||
31       category.toLowerCase().includes(q) ||
32       `#${id}`.includes(q) ||
33       keywords.some((k) => k.toLowerCase().includes(q))
34     );
35   };
36
37   // Tool 51 - JSON Validator
38   const [t51In, setT51In] = useState('{ "status": "ok", "count": 10 }');
39   const [t51Out, setT51Out] = useState("");
40   const [t51Err, setT51Err] = useState(false);
41
42   // Tool 52 - JSON to CSV
43   const [t52In, setT52In] = useState('{"name": "Alice", "age": 28}, {"name": "Bob", "age": 34}');
44   const [t52Out, setT52Out] = useState("");
45
46   // Tool 53 - CSV to JSON
47   const [t53In, setT53In] = useState("name,age\nAlice,28\nBob,34");
48   const [t53Out, setT53Out] = useState("");
49
50   // Tool 54 - JSON Sorter
51   const [t54In, setT54In] = useState('{ "zebra": 1, "apple": 2, "mango": 3 }');
52   const [t54Out, setT54Out] = useState("");
53
54   // Tool 55 - XML Formatter
55   const [t55In, setT55In] = useState("<root><user><name>Dev</name><role>Admin</role></user></root>");
56   const [t55Out, setT55Out] = useState("");
57
58   // Tool 56 - Text Sorter
59   const [t56In, setT56In] = useState("Banana\nApple\nOrange\nMango");
60   const [t56Out, setT56Out] = useState("");
61
62   // Tool 57 - Duplicate Line Remover
63   const [t57In, setT57In] = useState("Line A\nLine B\nLine A\nLine C\nLine B");
64   const [t57Out, setT57Out] = useState("");
65
66   // Tool 58 - Line Counter
```

src/components/tools/ToolsDataWeb.tsx (continued)

```
67   const [t58In, setT58In] = useState("One\nTwo\nThree\nFour\nFive");
68
69   // Tool 59 - Whitespace Cleaner
70   const [t59In, setT59In] = useState(" Clean excessive spacing and tabs. ");
71   const [t59Out, setT59Out] = useState("");
72
73   // Tool 60 - Reverse Text
74   const [t60In, setT60In] = useState("Coding Super Hub");
75   const [t60Out, setT60Out] = useState("");
76
77   // Tool 61 - Palindrome Checker
78   const [t61In, setT61In] = useState("A man a plan a canal Panama");
79   const [t61Out, setT61Out] = useState("");
80
81   // Tool 62 - Anagram Checker
82   const [t62A, setT62A] = useState("listen");
83   const [t62B, setT62B] = useState("silent");
84   const [t62Out, setT62Out] = useState("");
85
86   // Tool 63 - Word Frequency
87   const [t63In, setT63In] = useState("code build test code deploy code");
88   const [t63Out, setT63Out] = useState("");
89
90   // Tool 64 - Text Deduplicator
91   const [t64In, setT64In] = useState("apple banana apple orange banana");
92   const [t64Out, setT64Out] = useState("");
93
94   // Tool 65 - Character Frequency
95   const [t65In, setT65In] = useState("developer");
96   const [t65Out, setT65Out] = useState("");
97
98   // Tool 66 - Morse Code
99   const [t66In, setT66In] = useState("SOS HELLO");
100  const [t66Out, setT66Out] = useState("");
101
102  // Tool 67 - Binary Text
103  const [t67In, setT67In] = useState("AI");
104  const [t67Out, setT67Out] = useState("");
105
106  // Tool 68 - SHA-256
107  const [t68In, setT68In] = useState("secure_password_123");
108  const [t68Out, setT68Out] = useState("");
109
110  // Tool 69 - URL Parser
111  const [t69In, setT69In] = useState("https://api.github.com:443/repos/octocat/Hello-World?
112  filter=active#section=1");
113  const [t69Out, setT69Out] = useState("");
114
115  // Tool 70 - Query String Parser
116  const [t70In, setT70In] = useState("category=tech&page=2&sort=desc&tag=react&tag=node");
117  const [t70Out, setT70Out] = useState("");
118
119  // Tool 71 - Browser Information
120  const [t71Out, setT71Out] = useState("");
121
122  // Tool 72 - Screen Info
123  const [t72Out, setT72Out] = useState("");
124
125  // Tool 73 - Meta Tag Generator
126  const [t73Title, setT73Title] = useState("Coding Super Hub");
127  const [t73Desc, setT73Desc] = useState("150+ developer tools & AI coding assistant in one app");
128  const [t73Out, setT73Out] = useState("");
129
130  // Tool 74 - HTML Boilerplate Generator
131  const [t74Out, setT74Out] = useState("");
132
133  // Tool 75 - CSS Reset Generator
134  const [t75Out, setT75Out] = useState("");
135
136  // Tool 76 - CSS Flexbox Generator
```

src/components/tools/ToolsDataWeb.tsx (continued)

```
136 const [t76Justify, setT76Justify] = useState("center");
137 const [t76Align, setT76Align] = useState("center");
138 const [t76Dir, setT76Dir] = useState("row");
139
140 // Tool 77 - CSS Grid Generator
141 const [t77Cols, setT77Cols] = useState(3);
142 const [t77Gap, setT77Gap] = useState(16);
143
144 // Tool 78 - Box Shadow Generator
145 const [t78X, setT78X] = useState(0);
146 const [t78Y, setT78Y] = useState(10);
147 const [t78Blur, setT78Blur] = useState(15);
148 const [t78Spread, setT78Spread] = useState(-3);
149
150 // Tool 79 - Border Radius Generator
151 const [t79Radius, setT79Radius] = useState(16);
152
153 // Tool 80 - CSS Clamp Generator
154 const [t80Min, setT80Min] = useState("1rem");
155 const [t80Val, setT80Val] = useState("2.5vw");
156 const [t80Max, setT80Max] = useState("2rem");
157
158 // Tool 81 - PX to REM
159 const [t81Px, setT81Px] = useState("16");
160 const [t81Rem, setT81Rem] = useState("1");
161 const [t81Out, setT81Out] = useState("");
162
163 // Tool 82 - Aspect Ratio Calculator
164 const [t82W, setT82W] = useState("1920");
165 const [t82H, setT82H] = useState("1080");
166 const [t82Out, setT82Out] = useState("");
167
168 // Tool 83 - Random Color Generator
169 const [t83Out, setT83Out] = useState("#2563eb");
170
171 // Tool 84 - Color Contrast
172 const [t84Fg, setT84Fg] = useState("#ffffff");
173 const [t84Bg, setT84Bg] = useState("#2563eb");
174 const [t84Out, setT84Out] = useState("");
175
176 // Tool 85 - Data URI Generator
177 const [t85In, setT85In] = useState("console.log('Hello World!');");
178 const [t85Out, setT85Out] = useState("");
179
180 // Tool 86 - Favicon Generator
181 const [t86Href, setT86Href] = useState("/favicon.ico");
182 const [t86Out, setT86Out] = useState("");
183
184 // Tool 87 - Robots.txt Generator
185 const [t87Site, setT87Site] = useState("https://example.com");
186 const [t87Out, setT87Out] = useState("");
187
188 // Tool 88 - Sitemap XML Generator
189 const [t88Site, setT88Site] = useState("https://example.com");
190 const [t88Out, setT88Out] = useState("");
191
192 // Tool 89 - Gitignore Generator
193 const [t89Type, setT89Type] = useState("node");
194 const [t89Out, setT89Out] = useState("");
195
196 // Tool 90 - README Generator
197 const [t90Name, setT90Name] = useState("My Project");
198 const [t90Desc, setT90Desc] = useState("A powerful project built with TypeScript.");
199 const [t90Out, setT90Out] = useState("");
200
201 // Tool 91 - JS Function Generator
202 const [t91Name, setT91Name] = useState("fetchUserData");
203 const [t91Args, setT91Args] = useState("userId, token");
204 const [t91Out, setT91Out] = useState("");
```

src/components/tools/ToolsDataWeb.tsx (continued)

```
205
206 // Tool 92 - Fetch API Snippet
207 const [t92Url, setT92Url] = useState("https://api.example.com/data");
208 const [t92Method, setT92Method] = useState("GET");
209 const [t92Out, setT92Out] = useState("");
210
211 // Tool 93 - SQL Formatter
212 const [t93In, setT93In] = useState("SELECT id, name, email FROM users WHERE status = 'active' ORDER BY created_at
DESC");
213 const [t93Out, setT93Out] = useState("");
214
215 // Tool 94 - SQL String Escaper
216 const [t94In, setT94In] = useState("O'Reilly; DROP TABLE users;");
217 const [t94Out, setT94Out] = useState("");
218
219 // Tool 95 - Date Formatter
220 const [t95Date, setT95Date] = useState("2026-06-15T12:00:00");
221 const [t95Out, setT95Out] = useState("");
222
223 // Tool 96 - Unix Timestamp Now
224 const [t96Out, setT96Out] = useState("");
225
226 // Tool 97 - HMAC SHA-256
227 const [t97Key, setT97Key] = useState("secret_key_42");
228 const [t97Msg, setT97Msg] = useState("payload_message");
229 const [t97Out, setT97Out] = useState("");
230
231 // Tool 98 - UUID Bulk Generator
232 const [t98Count, setT98Count] = useState(5);
233 const [t98Out, setT98Out] = useState("");
234
235 // Tool 99 - HTTP Status Lookup
236 const [t99Code, setT99Code] = useState("200");
237 const [t99Out, setT99Out] = useState("");
238
239 // Tool 100 - Dev Cheat Sheet
240 const [t100Topic, setT100Topic] = useState("git");
241 const [t100Out, setT100Out] = useState("");
242
243 return (
244   <>
245     {/* TOOL 51 - JSON Validator */}
246     {isVisible(51, "JSON Validator", "Data", ["json", "validate", "syntax"]) && (
247       <ToolCard
248         id={51}
249         title="JSON Validator"
250         category="Data"
251         description="Validate JSON string and show pinpoint syntax error positions"
252         output={t51Out}
253         isError={t51Err}
254       >
255         <textarea
256           value={t51In}
257           onChange={(e) => setT51In(e.target.value)}
258           rows={3}
259           className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
dark:bg-slate-900 font-mono"
260         />
261         <button
262           type="button"
263           onClick={() => {
264             try {
265               JSON.parse(t51In);
266               setT51Out("' Valid JSON syntax!");
267               setT51Err(false);
268             } catch (e: any) {
269               setT51Out("'L " + e.message);
270               setT51Err(true);
271             }
272           }}
273         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
```


src/components/tools/ToolsDataWeb.tsx (continued)

```
274         >
275         Validate JSON
276       </button>
277     </ToolCard>
278   )}
279
280   {/* TOOL 52 - JSON to CSV */}
281   {isVisible(52, "JSON to CSV", "Converters", ["csv", "table", "export"]) && (
282     <ToolCard
283       id={52}
284       title="JSON to CSV Converter"
285       category="Converters"
286       description="Convert an array of JSON objects to standard CSV format"
287       output={t52Out}
288     >
289       <textarea
290         value={t52In}
291         onChange={(e) => setT52In(e.target.value)}
292         rows={3}
293         className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
294         dark:bg-slate-900 font-mono"
295       >
296         <button
297           type="button"
298           onClick={() => {
299             try {
300               setT52Out(jsonToCsv(t52In));
301             } catch (e: any) {
302               setT52Out("'L " + e.message);
303             }
304           }}
305           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
306         >
307           Convert to CSV
308         </button>
309       </ToolCard>
310     )}
311
312   {/* TOOL 53 - CSV to JSON */}
313   {isVisible(53, "CSV to JSON", "Converters", ["csv to json", "table"]) && (
314     <ToolCard
315       id={53}
316       title="CSV to JSON Converter"
317       category="Converters"
318       description="Parse comma-separated values into JSON records"
319       output={t53Out}
320     >
321       <textarea
322         value={t53In}
323         onChange={(e) => setT53In(e.target.value)}
324         rows={3}
325         className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
326         dark:bg-slate-900 font-mono"
327       >
328         <button
329           type="button"
330           onClick={() => {
331             try {
332               setT53Out(JSON.stringify(csvToJson(t53In), null, 2));
333             } catch (e: any) {
334               setT53Out("'L " + e.message);
335             }
336           }}
337           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
338         >
339           Convert to JSON
340         </button>
341       </ToolCard>
342     )}
343
344   {/* TOOL 54 - JSON Sorter */}
```

src/components/tools/ToolsDataWeb.tsx (continued)

```

343     {isVisible(54, "JSON Sorter", "Formatting", ["sort keys", "alphabetical"]) && (
344       <ToolCard
345         id={54}
346         title="JSON Key Sorter"
347         category="Formatting"
348         description="Sort all keys in a JSON object alphabetically"
349         output={t54Out}
350       >
351         <textarea
352           value={t54In}
353           onChange={(e) => setT54In(e.target.value)}
354           rows={3}
355           className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
356 dark:bg-slate-900 font-mono"
357         <button
358           type="button"
359           onClick={() => {
360             try {
361               const parsed = JSON.parse(t54In);
362               const sortKeys = (obj: any): any => {
363                 if (typeof obj !== "object" || obj === null) return obj;
364                 if (Array.isArray(obj)) return obj.map(sortKeys);
365                 return Object.keys(obj)
366                   .sort()
367                   .reduce((acc: any, key: string) => {
368                     acc[key] = sortKeys(obj[key]);
369                     return acc;
370                   }, {});
371             };
372             setT54Out(JSON.stringify(sortKeys(parsed), null, 2));
373           } catch (e: any) {
374             setT54Out("'L " + e.message);
375           }
376         }}
377         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
378       >
379         Sort Keys
380     </button>
381   </ToolCard>
382 )}
383
384   {/* TOOL 55 - XML Formatter */}
385   {isVisible(55, "XML Formatter", "Formatting", ["xml", "pretty", "indent"]) && (
386     <ToolCard
387       id={55}
388       title="XML Formatter"
389       category="Formatting"
390       description="Beautify XML structures with indentation"
391       output={t55Out}
392     >
393       <textarea
394         value={t55In}
395         onChange={(e) => setT55In(e.target.value)}
396         rows={3}
397         className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
398 dark:bg-slate-900 font-mono"
399       <button
400         type="button"
401         onClick={() => {
402           try {
403             const reg = /(\/>)(<)(\/\*\/)/g;
404             let formatted = t55In.replace(reg, "$1\r\n$2$3");
405             let pad = 0;
406             const lines = formatted.split("\r\n").map((node) => {
407               let indent = 0;
408               if (node.match(/\.+<\/\w[^\>]*>$/)) {
409                 indent = 0;
410               } else if (node.match(/^<\/\w\/)) {
411                 if (pad !== 0) pad -= 1;

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

412         } else if (node.match(/^<\w[^>]*[^\>]>.*$/)) {
413             indent = 1;
414         } else {
415             indent = 0;
416         }
417         const padding = " ".repeat(pad);
418         pad += indent;
419         return padding + node;
420     });
421     setT550Out(lines.join("\n"));
422 } catch (e: any) {
423     setT550Out("'L " + e.message);
424 }
425 }}
426 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
427 >
428     Format XML
429 </button>
430 </ToolCard>
431 }}
432
433 {/* T00L 56 - Text Sorter */}
434 {isVisible(56, "Text Sorter", "Text", ["sort lines", "alphabetical"]) && (
435     <ToolCard
436         id={56}
437         title="Text Line Sorter"
438         category="Text"
439         description="Sort lines alphabetically A-Z or Z-A"
440         output={t560Out}
441     >
442         <textarea
443             value={t56In}
444             onChange={(e) => setT56In(e.target.value)}
445             rows={3}
446             className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
447             dark:bg-slate-900 font-mono"
448         >
449             <div className="flex gap-2">
450                 <button
451                     type="button"
452                     onClick={() => {
453                         const lines = t56In.split("\n").sort((a, b) => a.localeCompare(b));
454                         setT560Out(lines.join("\n"));
455                     }}
456                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
457                 >
458                     Sort A-Z
459                 </button>
460                 <button
461                     type="button"
462                     onClick={() => {
463                         const lines = t56In.split("\n").sort((a, b) => b.localeCompare(a));
464                         setT560Out(lines.join("\n"));
465                     }}
466                     className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs font-semibold"
467                 >
468                     Sort Z-A
469                 </button>
470             </div>
471         </ToolCard>
472     )}
473
474 {/* T00L 57 - Duplicate Line Remover */}
475 {isVisible(57, "Duplicate Line Remover", "Text", ["dedupe", "unique"]) && (
476     <ToolCard
477         id={57}
478         title="Duplicate Line Remover"
479         category="Text"
480         description="Filter out repeated lines to keep unique entries"
481         output={t570Out}

```

src/components/tools/ToolsDataWeb.tsx (continued)

```
481         >
482         <textarea
483             value={t57In}
484             onChange={(e) => setT57In(e.target.value)}
485             rows={3}
486             className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
487 dark:bg-slate-900 font-mono"
488         <button
489             type="button"
490             onClick={() => {
491                 const lines = Array.from(new Set(t57In.split("\n")));
492                 setT57Out(lines.join("\n"));
493             }}
494             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
495         >
496         Remove Duplicates
497     </button>
498 </ToolCard>
499 }}
500
501 {/* TOOL 58 - Line Counter */}
502 {isVisible(58, "Line Counter", "Text", ["lines", "empty lines"]) && (
503     <ToolCard
504         id={58}
505         title="Line & Non-Empty Line Counter"
506         category="Text"
507         description="Count total lines and non-blank lines"
508     >
509         <textarea
510             value={t58In}
511             onChange={(e) => setT58In(e.target.value)}
512             rows={3}
513             className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
514 dark:bg-slate-900 font-mono"
515         <div className="flex gap-4 pt-1 text-xs">
516             <span className="text-slate-600 dark:text-slate-400">
517                 Total Lines: <strong>{t58In ? t58In.split("\n").length : 0}</strong>
518             </span>
519             <span className="text-slate-600 dark:text-slate-400">
520                 Non-Empty Lines:{" "}
521                 <strong>{t58In.split("\n").filter((l) => l.trim() !== "").length}</strong>
522             </span>
523         </div>
524     </ToolCard>
525 )}
526
527 {/* TOOL 59 - Whitespace Cleaner */}
528 {isVisible(59, "Whitespace Cleaner", "Text", ["whitespace", "trim", "collapse"]) && (
529     <ToolCard
530         id={59}
531         title="Whitespace Cleaner"
532         category="Text"
533         description="Collapse consecutive spaces and trim edges"
534         output={t59Out}
535     >
536         <textarea
537             value={t59In}
538             onChange={(e) => setT59In(e.target.value)}
539             rows={2}
540             className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
541 dark:bg-slate-900 font-mono"
542         <button
543             type="button"
544             onClick={() => setT59Out(t59In.replace(/\s+/g, " ").trim())}
545             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
546         >
547         Clean Whitespace
548     </button>
549 </ToolCard>
```

src/components/tools/ToolsDataWeb.tsx (continued)

```

550     })
551
552     {/ * TOOL 60 - Reverse Text */}
553     {isVisible(60, "Reverse Text", "Text", ["reverse", "backwards"]) && (
554         <ToolCard
555             id={60}
556             title="Reverse Text"
557             category="Text"
558             description="Invert text character by character"
559             output={t60Out}
560         >
561             <input
562                 type="text"
563                 value={t60In}
564                 onChange={(e) => setT60In(e.target.value)}
565                 className="w-full text-xs px-2.5 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
566                 dark:bg-slate-900 font-m
567             <button
568                 type="button"
569                 onClick={() => setT60Out(t60In.split("").reverse().join(""))}
570                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
571             >
572                 Reverse
573             </button>
574         </ToolCard>
575     )}
576
577     {/ * TOOL 61 - Palindrome Checker */}
578     {isVisible(61, "Palindrome Checker", "Text", ["palindrome"]) && (
579         <ToolCard
580             id={61}
581             title="Palindrome Checker"
582             category="Text"
583             description="Check if text reads the same backwards"
584             output={t61Out}
585         >
586             <div className="flex gap-2">
587                 <input
588                     type="text"
589                     value={t61In}
590                     onChange={(e) => setT61In(e.target.value)}
591                     className="flex-1 text-xs px-2.5 py-1 rounded border border-slate-300 dark:border-slate-700 bg-
592                     slate-50 dark:bg-slate-900 font
593                 <button
594                     type="button"
595                     onClick={() => {
596                         const clean = t61In.toLowerCase().replace(/^[a-z0-9]/g, "");
597                         const isPal = clean === clean.split("").reverse().join("");
598                         setT61Out(isPal ? ` "${t61In}" is a Palindrome!` : `L "${t61In}" is NOT a Palindrome.`);
599                     }}
600                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
601                 >
602                     Check
603                 </button>
604             </div>
605         </ToolCard>
606     )}
607
608     {/ * TOOL 62 - Anagram Checker */}
609     {isVisible(62, "Anagram Checker", "Text", ["anagram"]) && (
610         <ToolCard
611             id={62}
612             title="Anagram Checker"
613             category="Text"
614             description="Determine if two words contain identical letters"
615             output={t62Out}
616         >
617             <div className="flex gap-2">
618                 <input

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

619         type="text"
620         value={t62A}
621         onChange={(e) => setT62A(e.target.value)}
622         className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
623         dark:bg-slate-900 placeholder="word 1"
624     />
625     <input
626         type="text"
627         value={t62B}
628         onChange={(e) => setT62B(e.target.value)}
629         className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
630         dark:bg-slate-900 placeholder="word 2"
631     />
632     <button
633         type="button"
634         onClick={() => {
635             const norm = (s: string) => s.toLowerCase().replace(/^[a-z0-9]/g, "").split("").sort().join("");
636             const isAna = norm(t62A) === norm(t62B);
637             setT62Out(isAna ? ` "${t62A}" and "${t62B}" are Anagrams!` : `!L Not Anagrams.`);
638         }}
639         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
640     >
641         Check
642     </button>
643 </div>
644 </ToolCard>
645 }}
646
647 /* TOOL 63 - Word Frequency Counter */
648 {isVisible(63, "Word Frequency Counter", "Text", ["frequency", "occurrence"]) && (
649     <ToolCard
650         id={63}
651         title="Word Frequency Counter"
652         category="Text"
653         description="Compute count of each word in text"
654         output={t63Out}
655     >
656         <textarea
657             value={t63In}
658             onChange={(e) => setT63In(e.target.value)}
659             rows={2}
660             className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
661             dark:bg-slate-900 font-mono"
662         >
663             <button
664                 type="button"
665                 onClick={() => {
666                     const words = t63In.toLowerCase().match(/\b\w+\b/g) || [];
667                     const map: Record<string, number> = {};
668                     words.forEach((w) => (map[w] = (map[w] || 0) + 1));
669                     const res = Object.entries(map)
670                         .sort((a, b) => b[1] - a[1])
671                         .map(([w, c]) => `${w}: ${c}`)
672                         .join(", ");
673                     setT63Out(res || "No words detected");
674                 }}
675                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
676             >
677                 Count Frequency
678             </button>
679         </ToolCard>
680     )}
681
682 /* TOOL 64 - Text Deduplicator */
683 {isVisible(64, "Text Deduplicator", "Text", ["dedup words"]) && (
684     <ToolCard
685         id={64}
686         title="Word Deduplicator"
687         category="Text"
688         description="Remove duplicate words while preserving order"

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

688         output={t64Out}
689       >
690       <input
691         type="text"
692         value={t64In}
693         onChange={(e) => setT64In(e.target.value)}
694         className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
695         dark:bg-slate-900 font-mon
696       <button
697         type="button"
698         onClick={() => {
699           const words = t64In.trim().split(/\s+/);
700           setT64Out(Array.from(new Set(words)).join(" "));
701         }}
702         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
703       >
704         Deduplicate Words
705     </button>
706 </ToolCard>
707 }}
708
709 {/* TOOL 65 - Character Frequency */}
710 {isVisible(65, "Character Frequency", "Text", ["letters count"])} && (
711   <ToolCard
712     id={65}
713     title="Character Frequency"
714     category="Text"
715     description="Count occurrences of each letter"
716     output={t65Out}
717   >
718     <input
719       type="text"
720       value={t65In}
721       onChange={(e) => setT65In(e.target.value)}
722       className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
723       dark:bg-slate-900 font-mon
724     <button
725       type="button"
726       onClick={() => {
727         const map: Record<string, number> = {};
728         for (const ch of t65In) {
729           if (ch.trim()) map[ch] = (map[ch] || 0) + 1;
730         }
731         const res = Object.entries(map)
732           .sort((a, b) => b[1] - a[1])
733           .map([c, n]) => `'{c}': ${n}`)
734           .join(", ");
735         setT65Out(res || "No characters");
736       }}
737       className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
738     >
739       Count Letters
740     </button>
741   </ToolCard>
742 )}
743
744 {/* TOOL 66 - Morse Code */}
745 {isVisible(66, "Morse Code Converter", "Converters", ["morse", "telegraph"])} && (
746   <ToolCard
747     id={66}
748     title="Morse Code Converter"
749     category="Converters"
750     description="Translate text to Morse code and decode Morse to text"
751     output={t66Out}
752   >
753     <input
754       type="text"
755       value={t66In}
756       onChange={(e) => setT66In(e.target.value)}

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

757         className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
758 dark:bg-slate-900 font-mon
759         <div className="flex gap-2">
760             <button
761                 type="button"
762                 onClick={() => setT66Out(morseEncode(t66In))}
763                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
764             >
765                 Encode to Morse
766             </button>
767             <button
768                 type="button"
769                 onClick={() => setT66Out(morseDecode(t66In))}
770                 className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs font-semibold"
771             >
772                 Decode Morse
773             </button>
774         </div>
775     </ToolCard>
776 }}
777
778 {/* TOOL 67 - Binary Text Converter */}
779 {isVisible(67, "Binary Text Converter", "Converters", ["binary text", "ascii binary"]) && (
780     <ToolCard
781         id={67}
782         title="Binary Text Converter"
783         category="Converters"
784         description="Convert text to 8-bit binary and decode back"
785         output={t67Out}
786     >
787         <input
788             type="text"
789             value={t67In}
790             onChange={(e) => setT67In(e.target.value)}
791             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
792 dark:bg-slate-900 font-mon
793         <div className="flex gap-2">
794             <button
795                 type="button"
796                 onClick={() => setT67Out(textToBinaryString(t67In))}
797                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
798             >
799                 Text !' Binary
800             </button>
801             <button
802                 type="button"
803                 onClick={() => setT67Out(binaryStringToText(t67In))}
804                 className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs font-semibold"
805             >
806                 Binary !' Text
807             </button>
808         </div>
809     </ToolCard>
810 }}
811
812 {/* TOOL 68 - SHA-256 Hash */}
813 {isVisible(68, "SHA-256 Hash Generator", "Hashes", ["sha256", "checksum"]) && (
814     <ToolCard
815         id={68}
816         title="SHA-256 Hash Generator"
817         category="Hashes"
818         description="Generate standard 256-bit cryptographic digest"
819         output={t68Out}
820     >
821         <input
822             type="text"
823             value={t68In}
824             onChange={(e) => setT68In(e.target.value)}
825             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
dark:bg-slate-900 font-mon

```


src/components/tools/ToolsDataWeb.tsx (continued)

```

826         />
827         <button
828             type="button"
829             onClick={async () => setT68Out(await sha256Hex(t68In))}
830             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
831         >
832             Compute SHA-256
833         </button>
834     </ToolCard>
835 }}
836
837 {/* TOOL 69 - URL Parser */}
838 {isVisible(69, "URL Parser", "Web", ["url breakdown", "protocol", "pathname"]) && (
839     <ToolCard
840         id={69}
841         title="URL Component Parser"
842         category="Web"
843         description="Deconstruct URL into protocol, host, port, path, query, hash"
844         output={t69Out}
845     >
846         <input
847             type="text"
848             value={t69In}
849             onChange={(e) => setT69In(e.target.value)}
850             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
851             dark:bg-slate-900 font-mon
852         >
853             <button
854                 type="button"
855                 onClick={() => {
856                     try {
857                         const u = new URL(t69In);
858                         setT69Out(
859                             `Protocol: ${u.protocol}\nHost: ${u.host}\nHostname: ${u.hostname}\nPort: ${u.port ||
860                             "default"}\nPathname: ${u.pathname}\n
861                             } catch (e: any) {
862                                 setT69Out("'L " + e.message);
863                             }
864                         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
865                     >
866                         Parse URL
867                     </button>
868                 </ToolCard>
869             )}
870
871 {/* TOOL 70 - Query String Parser */}
872 {isVisible(70, "Query String Parser", "Web", ["querystring", "params"]) && (
873     <ToolCard
874         id={70}
875         title="URL Query String Parser"
876         category="Web"
877         description="Parse URL query search parameters into JSON"
878         output={t70Out}
879     >
880         <input
881             type="text"
882             value={t70In}
883             onChange={(e) => setT70In(e.target.value)}
884             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
885             dark:bg-slate-900 font-mon
886         >
887             <button
888                 type="button"
889                 onClick={() => {
890                     const sp = new URLSearchParams(t70In);
891                     const obj: Record<string, any> = {};
892                     sp.forEach((val, key) => {
893                         if (obj[key]) {
894                             if (Array.isArray(obj[key])) obj[key].push(val);

```

src/components/tools/ToolsDataWeb.tsx (continued)

```
895         } else {
896             obj[key] = val;
897         }
898     });
899     setT70Out(JSON.stringify(obj, null, 2));
900 }
901 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
902 >
903     Parse Query Params
904 </button>
905 </ToolCard>
906 }}
907
908 {/* TOOL 71 - Browser Information */}
909 {isVisible(71, "Browser Information", "Utilities", ["browser", "user-agent", "platform"]) && (
910     <ToolCard
911         id={71}
912         title="Browser & Platform Information"
913         category="Utilities"
914         description="Inspect active client browser properties"
915         output={t71Out}
916     >
917         <button
918             type="button"
919             onClick={() => {
920                 setT71Out(
921                     `User-Agent: ${navigator.userAgent}\nLanguage: ${navigator.language}\nOnline: ${
922                         navigator.onLine ? "Yes" : "No"
923                     }\nCookies Enabled: ${navigator.cookieEnabled ? "Yes" : "No"}\nPlatform: ${navigator.platform}`
924                 );
925             }}
926             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
927         >
928             Get Browser Info
929         </button>
930     </ToolCard>
931 )}
932
933 {/* TOOL 72 - Screen & Viewport Info */}
934 {isVisible(72, "Screen & Viewport Info", "Utilities", ["resolution", "viewport"]) && (
935     <ToolCard
936         id={72}
937         title="Screen & Viewport Resolution"
938         category="Utilities"
939         description="Inspect monitor resolution and window viewport size"
940         output={t72Out}
941     >
942         <button
943             type="button"
944             onClick={() => {
945                 setT72Out(
946                     `Screen Resolution: ${window.screen.width} x ${window.screen.height}\nAvailable Screen: ${
947                         window.screen.availWidth
948                     } x ${window.screen.availHeight}\nWindow Inner Size: ${window.innerWidth} x ${
949                         window.innerHeight
950                     }\nDevice Pixel Ratio: ${window.devicePixelRatio}`
951                 );
952             }}
953             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
954         >
955             Inspect Viewport
956         </button>
957     </ToolCard>
958 )}
959
960 {/* TOOL 73 - Meta Tag Generator */}
961 {isVisible(73, "Meta Tag Generator", "Web", ["seo", "html meta", "opengraph"]) && (
962     <ToolCard
963         id={73}
```

src/components/tools/ToolsDataWeb.tsx (continued)

```

964         title="HTML Meta Tag Generator"
965         category="Web"
966         description="Generate responsive SEO and OpenGraph meta elements"
967         output={t73Out}
968     >
969     <div className="space-y-2">
970         <input
971             type="text"
972             value={t73Title}
973             onChange={(e) => setT73Title(e.target.value)}
974             placeholder="Page Title"
975             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
976             dark:bg-slate-900 font-m
977         <input
978             type="text"
979             value={t73Desc}
980             onChange={(e) => setT73Desc(e.target.value)}
981             placeholder="Meta Description"
982             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
983             dark:bg-slate-900 font-m
984         <button
985             type="button"
986             onClick={() => {
987                 setT73Out(
988                     `<title>${t73Title}</title>\n<meta name="description" content="${t73Desc}">\n<meta
989                     name="viewport" content="width=device-w
990                 `)
991             }}
992             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
993         >
994             Generate Tags
995         </button>
996     </div>
997 </ToolCard>
998 }}
999
1000 {/* TOOL 74 - HTML Boilerplate Generator */}
1001 {isVisible(74, "HTML Boilerplate Generator", "HTML", ["template", "html5"])} && (
1002     <ToolCard
1003         id={74}
1004         title="HTML5 Boilerplate Template"
1005         category="HTML"
1006         description="Starter HTML5 document structure with viewport and UTF-8 charset"
1007         output={t74Out}
1008     >
1009         <button
1010             type="button"
1011             onClick={() => {
1012                 setT74Out(
1013                     `<!DOCTYPE html>\n<html lang="en">\n<head>\n  <meta charset="UTF-8">\n  <meta name="viewport"
1014                     content="width=device-width, i
1015                 `)
1016             }}
1017             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1018         >
1019             Generate HTML5 Boilerplate
1020         </button>
1021     </ToolCard>
1022 }}
1023
1024 {/* TOOL 75 - CSS Reset Generator */}
1025 {isVisible(75, "CSS Reset Generator", "CSS", ["reset", "normalize", "box-sizing"])} && (
1026     <ToolCard
1027         id={75}
1028         title="Modern CSS Reset Snippet"
1029         category="CSS"
1030         description="Production modern CSS reset including border-box and margin zero"
1031         output={t75Out}
1032     >
1033         <button
1034             type="button"

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

1033         onClick={() => {
1034             setT75Out(
1035                 `*::before, *::after {\n box-sizing: border-box;\n margin: 0;\n padding: 0;\n}\n\nhtml, body
1036 {\n height: 100%;\n lin
1037 }}
1038 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1039 >
1040     Generate CSS Reset
1041 </button>
1042 </ToolCard>
1043 })}
1044
1045 {/* TOOL 76 - CSS Flexbox Generator */}
1046 {isVisible(76, "CSS Flexbox Generator", "CSS", ["flex", "justify-content", "align-items"]) && (
1047     <ToolCard
1048         id={76}
1049         title="CSS Flexbox Generator"
1050         category="CSS"
1051         description="Visual helper to generate flex layout rules"
1052         output={`display: flex;\nflex-direction: ${t76Dir};\njustify-content: ${t76Justify};\nalign-items:
1053 ${t76Align};`}
1054         <div className="flex flex-wrap gap-2 text-xs">
1055             <select
1056                 value={t76Dir}
1057                 onChange={(e) => setT76Dir(e.target.value)}
1058                 className="px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1059 slate-900"
1060             >
1061                 <option value="row">row</option>
1062                 <option value="column">column</option>
1063                 <option value="row-reverse">row-reverse</option>
1064             </select>
1065             <select
1066                 value={t76Justify}
1067                 onChange={(e) => setT76Justify(e.target.value)}
1068                 className="px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1069 slate-900"
1070             >
1071                 <option value="flex-start">justify: flex-start</option>
1072                 <option value="center">justify: center</option>
1073                 <option value="space-between">justify: space-between</option>
1074                 <option value="space-around">justify: space-around</option>
1075                 <option value="flex-end">justify: flex-end</option>
1076             </select>
1077             <select
1078                 value={t76Align}
1079                 onChange={(e) => setT76Align(e.target.value)}
1080                 className="px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1081 slate-900"
1082             >
1083                 <option value="stretch">align: stretch</option>
1084                 <option value="center">align: center</option>
1085                 <option value="flex-start">align: flex-start</option>
1086                 <option value="flex-end">align: flex-end</option>
1087             </select>
1088         </div>
1089     </ToolCard>
1090 })}
1091
1092 {/* TOOL 77 - CSS Grid Generator */}
1093 {isVisible(77, "CSS Grid Generator", "CSS", ["grid", "columns"]) && (
1094     <ToolCard
1095         id={77}
1096         title="CSS Grid Generator"
1097         category="CSS"
1098         description="Generate CSS Grid template columns and gap rules"
1099         output={`display: grid;\ngrid-template-columns: repeat(${t77Cols}, 1fr);\ngap: ${t77Gap}px;`}
1100         >
1101         <div className="flex gap-2 text-xs">
1102             <div className="flex items-center gap-1">
1103                 <span>Columns:</span>
1104                 <input

```

src/components/tools/ToolsDataWeb.tsx (continued)

```
1102         type="number"
1103         min={1}
1104         max={12}
1105         value={t77Cols}
1106         onChange={(e) => setT77Cols(Number(e.target.value))}
1107         className="w-14 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1108 slate-900 font-mono"
1109     </div>
1110     <div className="flex items-center gap-1">
1111         <span>Gap (px):</span>
1112         <input
1113             type="number"
1114             min={0}
1115             max={64}
1116             value={t77Gap}
1117             onChange={(e) => setT77Gap(Number(e.target.value))}
1118             className="w-14 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1119 slate-900 font-mono"
1120         </div>
1121     </div>
1122 </ToolCard>
1123 }}
1124
1125 {/* TOOL 78 - Box Shadow Generator */}
1126 {isVisible(78, "Box Shadow Generator", "CSS", ["shadow", "box-shadow"]) && (
1127     <ToolCard
1128         id={78}
1129         title="CSS Box Shadow Generator"
1130         category="CSS"
1131         description="Tune offset, blur, and spread with live CSS output"
1132         output={`box-shadow: ${t78X}px ${t78Y}px ${t78Blur}px ${t78Spread}px rgba(0, 0, 0, 0.1);`}
1133     >
1134         <div className="grid grid-cols-2 sm:grid-cols-4 gap-2 text-xs">
1135             <div>
1136                 <span>X ({t78X}px)</span>
1137                 <input
1138                     type="range"
1139                     min={-30}
1140                     max={30}
1141                     value={t78X}
1142                     onChange={(e) => setT78X(Number(e.target.value))}
1143                     className="w-full"
1144                 />
1145             </div>
1146             <div>
1147                 <span>Y ({t78Y}px)</span>
1148                 <input
1149                     type="range"
1150                     min={-30}
1151                     max={30}
1152                     value={t78Y}
1153                     onChange={(e) => setT78Y(Number(e.target.value))}
1154                     className="w-full"
1155                 />
1156             </div>
1157             <div>
1158                 <span>Blur ({t78Blur}px)</span>
1159                 <input
1160                     type="range"
1161                     min={0}
1162                     max={50}
1163                     value={t78Blur}
1164                     onChange={(e) => setT78Blur(Number(e.target.value))}
1165                     className="w-full"
1166                 />
1167             </div>
1168             <div>
1169                 <span>Spread ({t78Spread}px)</span>
1170                 <input
```

src/components/tools/ToolsDataWeb.tsx (continued)

```

1171         type="range"
1172         min={-20}
1173         max={20}
1174         value={t78Spread}
1175         onChange={(e) => setT78Spread(Number(e.target.value))}
1176         className="w-full"
1177       />
1178     </div>
1179   </div>
1180   <div
1181     className="w-24 h-12 bg-white dark:bg-slate-700 rounded-lg mx-auto mt-2 border border-slate-200
1182     dark:border-slate-900"
1183     style={{
1184       boxShadow: `${t78X}px ${t78Y}px ${t78Blur}px ${t78Spread}px rgba(0,0,0,0.15)` ,
1185     }}
1186   />
1187 </ToolCard>
1188 ))
1189
1190 /* TOOL 79 - Border Radius Generator */
1191 {isVisible(79, "Border Radius Generator", "CSS", ["rounded", "border-radius"]) && (
1192   <ToolCard
1193     id={79}
1194     title="CSS Border Radius Generator"
1195     category="CSS"
1196     description="Adjust rounded corners smoothly"
1197     output={`border-radius: ${t79Radius}px;`}
1198   >
1199     <div className="flex items-center gap-3">
1200       <input
1201         type="range"
1202         min={0}
1203         max={64}
1204         value={t79Radius}
1205         onChange={(e) => setT79Radius(Number(e.target.value))}
1206         className="flex-1"
1207       />
1208       <span className="text-xs font-mono">{t79Radius}px</span>
1209     </div>
1210   </ToolCard>
1211 ))
1212
1213 /* TOOL 80 - CSS Clamp Generator */
1214 {isVisible(80, "CSS Clamp Generator", "CSS", ["clamp", "fluid typography"]) && (
1215   <ToolCard
1216     id={80}
1217     title="CSS clamp() Fluid Value Generator"
1218     category="CSS"
1219     description="Generate responsive fluid sizing using CSS clamp()"
1220     output={`font-size: clamp(${t80Min}, ${t80Val}, ${t80Max});`}
1221   >
1222     <div className="flex gap-2 text-xs">
1223       <input
1224         type="text"
1225         value={t80Min}
1226         onChange={(e) => setT80Min(e.target.value)}
1227         placeholder="Min e.g. 1rem"
1228         className="w-24 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1229         slate-900 font-mono"
1230       />
1231       <input
1232         type="text"
1233         value={t80Val}
1234         onChange={(e) => setT80Val(e.target.value)}
1235         placeholder="Val e.g. 2.5vw"
1236         className="w-24 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1237         slate-900 font-mono"
1238       />
1239       <input
1240         type="text"
1241         value={t80Max}
1242         onChange={(e) => setT80Max(e.target.value)}

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

1240         placeholder="Max e.g. 2rem"
1241         className="w-24 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1242 slate-900 font-mono"
1243     </div>
1244     </ToolCard>
1245 }}
1246
1247 {/* TOOL 81 - PX <-> REM Converter */}
1248 {isVisible(81, "PX <-> REM Converter", "Converters", ["px to rem", "rem to px"]) && (
1249     <ToolCard
1250         id={81}
1251         title="PX to REM Converter"
1252         category="Converters"
1253         description="Convert pixel units to root em (rem) with 16px baseline"
1254         output={t81Out}
1255     >
1256         <div className="flex gap-2">
1257             <input
1258                 type="number"
1259                 value={t81Px}
1260                 onChange={(e) => setT81Px(e.target.value)}
1261                 placeholder="Pixels (px)"
1262                 className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1263 dark:bg-slate-900 font-mon
1264             <button
1265                 type="button"
1266                 onClick={() => {
1267                     const px = Number(t81Px);
1268                     setT81Out(`${px}px = ${((px / 16).toFixed(3))}rem`);
1269                 }}
1270                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1271             >
1272                 PX !' REM
1273             </button>
1274             <input
1275                 type="number"
1276                 value={t81Rem}
1277                 onChange={(e) => setT81Rem(e.target.value)}
1278                 placeholder="REM"
1279                 className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1280 dark:bg-slate-900 font-mon
1281             <button
1282                 type="button"
1283                 onClick={() => {
1284                     const rem = Number(t81Rem);
1285                     setT81Out(`${rem}rem = ${rem * 16}px`);
1286                 }}
1287                 className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs font-semibold"
1288             >
1289                 REM !' PX
1290             </button>
1291         </div>
1292     </ToolCard>
1293 }}
1294
1295 {/* TOOL 82 - Aspect Ratio Calculator */}
1296 {isVisible(82, "Aspect Ratio Calculator", "Math", ["aspect ratio", "16:9", "4:3"]) && (
1297     <ToolCard
1298         id={82}
1299         title="Aspect Ratio Calculator"
1300         category="Math"
1301         description="Compute proportional ratio (e.g. 16:9) and greatest common divisor"
1302         output={t82Out}
1303     >
1304         <div className="flex items-center gap-2">
1305             <input
1306                 type="number"
1307                 value={t82W}
1308                 onChange={(e) => setT82W(e.target.value)}

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

1309         placeholder="Width"
1310         className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1311 dark:bg-slate-900 font-mon
1312         <span>:</span>
1313         <input
1314             type="number"
1315             value={t82H}
1316             onChange={(e) => setT82H(e.target.value)}
1317             placeholder="Height"
1318             className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1319 dark:bg-slate-900 font-mon
1320         <button
1321             type="button"
1322             onClick={() => {
1323                 const w = Number(t82W);
1324                 const h = Number(t82H);
1325                 const gcd = (a: number, b: number): number => (b === 0 ? a : gcd(b, a % b));
1326                 const divisor = gcd(w, h);
1327                 setT82Out(`Ratio: ${w / divisor}:${h / divisor} (${(w / h).toFixed(2)})`);
1328             }}
1329             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1330         >
1331             Calculate Ratio
1332         </button>
1333     </div>
1334 </ToolCard>
1335 }}
1336
1337 {/* TOOL 83 - Random Color Generator */}
1338 {isVisible(83, "Random Color Generator", "Color", ["random color", "hex"]) && (
1339     <ToolCard
1340         id={83}
1341         title="Random Color Generator"
1342         category="Color"
1343         description="Generate random HEX color codes"
1344         output={t830Out}
1345     >
1346         <div className="flex items-center gap-3">
1347             <button
1348                 type="button"
1349                 onClick={() => {
1350                     const hex = "#" + Math.floor(Math.random() * 16777215).toString(16).padStart(6, "0");
1351                     setT830Out(hex);
1352                 }}
1353                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1354             >
1355                 Pick Random Color
1356             </button>
1357             <div
1358                 className="w-10 h-6 rounded border border-slate-300 shadow-xs"
1359                 style={{ background: t830Out }}
1360             />
1361         </div>
1362     </ToolCard>
1363 })
1364
1365 {/* TOOL 84 - Color Contrast Checker */}
1366 {isVisible(84, "Color Contrast Checker", "Color", ["contrast ratio", "wcag"]) && (
1367     <ToolCard
1368         id={84}
1369         title="WCAG Color Contrast Checker"
1370         category="Color"
1371         description="Check contrast ratio between foreground and background colors"
1372         output={t840Out}
1373     >
1374         <div className="flex items-center gap-2">
1375             <span className="text-xs">Text:</span>
1376             <input
1377                 type="color"

```


src/components/tools/ToolsDataWeb.tsx (continued)

```

1378         value={t84Fg}
1379         onChange={(e) => setT84Fg(e.target.value)}
1380         className="w-8 h-6 rounded border cursor-pointer"
1381       />
1382       <span className="text-xs">BG:</span>
1383       <input
1384         type="color"
1385         value={t84Bg}
1386         onChange={(e) => setT84Bg(e.target.value)}
1387         className="w-8 h-6 rounded border cursor-pointer"
1388       />
1389       <button
1390         type="button"
1391         onClick={() => {
1392           const getL = (hex: string) => {
1393             const rgb = [
1394               parseInt(hex.slice(1, 3), 16) / 255,
1395               parseInt(hex.slice(3, 5), 16) / 255,
1396               parseInt(hex.slice(5, 7), 16) / 255,
1397             ].map((v) => (v <= 0.03928 ? v / 12.92 : Math.pow((v + 0.055) / 1.055, 2.4)));
1398             return 0.2126 * rgb[0] + 0.7152 * rgb[1] + 0.0722 * rgb[2];
1399           };
1400           const l1 = getL(t84Fg);
1401           const l2 = getL(t84Bg);
1402           const ratio = (Math.max(l1, l2) + 0.05) / (Math.min(l1, l2) + 0.05);
1403           const passAA = ratio >= 4.5 ? "' AA Normal Pass" : "'L AA Normal Fail";
1404           setT84Out(`Ratio: ${ratio.toFixed(2)}:1\n${passAA}`);
1405         }}
1406         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1407       >
1408         Check Ratio
1409       </button>
1410     </div>
1411   </ToolCard>
1412 })
1413
1414   {/* TOOL 85 - Data URI Generator */}
1415   {isVisible(85, "Data URI Generator", "Converters", ["data uri", "base64 text"]) && (
1416     <ToolCard
1417       id={85}
1418       title="Data URI Generator"
1419       category="Converters"
1420       description="Convert text into standard data:text/plain;base64 URIs"
1421       output={t85Out}
1422     >
1423       <input
1424         type="text"
1425         value={t85In}
1426         onChange={(e) => setT85In(e.target.value)}
1427         className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1428         dark:bg-slate-900 font-mon"
1429       >
1430       <button
1431         type="button"
1432         onClick={() => {
1433           const b64 = btoa(unescape(encodeURIComponent(t85In)));
1434           setT85Out(`data:text/plain;charset=utf-8;base64,${b64}`);
1435         }}
1436         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1437       >
1438         Generate Data URI
1439       </button>
1440     </ToolCard>
1441   )}
1442
1443   {/* TOOL 86 - Favicon HTML Generator */}
1444   {isVisible(86, "Favicon HTML Generator", "HTML", ["favicon", "icon"]) && (
1445     <ToolCard
1446       id={86}
1447       title="Favicon HTML Tag Generator"

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

1447         category="HTML"
1448         description="Generate link rel tags for standard and apple touch favicons"
1449         output={t86Out}
1450     >
1451         <input
1452             type="text"
1453             value={t86Href}
1454             onChange={(e) => setT86Href(e.target.value)}
1455             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1456             dark:bg-slate-900 font-mon
1457         <button
1458             type="button"
1459             onClick={() => {
1460                 setT86Out(
1461                     `<link rel="icon" type="image/x-icon" href="${t86Href}">\n<link rel="apple-touch-icon" href="/apple-
1462                     touch-icon.png`);
1463                 }}
1464             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1465         >
1466             Generate Favicon Links
1467         </button>
1468     </ToolCard>
1469 }}
1470
1471 {/* TOOL 87 - Robots.txt Generator */}
1472 {isVisible(87, "Robots.txt Generator", "Web", ["robots", "crawler"])} && (
1473     <ToolCard
1474         id={87}
1475         title="Robots.txt Generator"
1476         category="Web"
1477         description="Generate crawler directive file with sitemap pointer"
1478         output={t87Out}
1479     >
1480         <input
1481             type="text"
1482             value={t87Site}
1483             onChange={(e) => setT87Site(e.target.value)}
1484             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1485             dark:bg-slate-900 font-mon
1486         <button
1487             type="button"
1488             onClick={() => {
1489                 setT87Out(
1490                     `User-agent: *\nAllow: /\nDisallow: /admin/\nDisallow: /private/\n\nSitemap: ${t87Site}/sitemap.xml`
1491                 );
1492             }}
1493             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1494         >
1495             Generate Robots.txt
1496         </button>
1497     </ToolCard>
1498 }}
1499
1500 {/* TOOL 88 - Sitemap XML Generator */}
1501 {isVisible(88, "Sitemap XML Generator", "Web", ["sitemap", "xml"])} && (
1502     <ToolCard
1503         id={88}
1504         title="Sitemap.xml Generator"
1505         category="Web"
1506         description="Create indexable XML sitemap template"
1507         output={t88Out}
1508     >
1509         <input
1510             type="text"
1511             value={t88Site}
1512             onChange={(e) => setT88Site(e.target.value)}
1513             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1514             dark:bg-slate-900 font-mon
1515         <button

```

src/components/tools/ToolsDataWeb.tsx (continued)

```

1516         type="button"
1517         onClick={() => {
1518             const now = new Date().toISOString().split("T")[0];
1519             setT880Out(
1520                 `<?xml version="1.0" encoding="UTF-8"?>\n<urlset xmlns="http://www.sitemaps.org/schemas/
1521 sitemap/0.9">\n; <url>\n      <loc>${t
1522             }}
1523             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1524         >
1525             Generate Sitemap.xml
1526         </button>
1527     </ToolCard>
1528 }}
1529
1530 {/* TOOL 89 - Gitignore Generator */}
1531 {isVisible(89, "Gitignore Generator", "DevOps", ["gitignore", "git"]) && (
1532     <ToolCard
1533         id={89}
1534         title=".gitignore Generator"
1535         category="DevOps"
1536         description="Generate standard .gitignore for Node.js, Python, or React"
1537         output={t890Out}
1538     >
1539         <div className="flex gap-2">
1540             <select
1541                 value={t89Type}
1542                 onChange={(e) => setT89Type(e.target.value)}
1543                 className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1544                 dark:bg-slate-900"
1545                 <option value="node">Node.js / React</option>
1546                 <option value="python">Python</option>
1547                 <option value="general">General macOS/Windows</option>
1548             </select>
1549             <button
1550                 type="button"
1551                 onClick={() => {
1552                     if (t89Type === "node") {
1553                         setT890Out("node_modules/\ndist/\nbuild/\n.env\n.env.local\n*.log\n.DS_Store");
1554                     } else if (t89Type === "python") {
1555                         setT890Out("__pycache__/\n*.py[cod]\n*$py.class\n.venv/\nenv/\n*.log\n.env");
1556                     } else {
1557                         setT890Out(".DS_Store\nThumbs.db\n*.tmp\n.vscode/\n.idea/");
1558                     }
1559                 }}
1560                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1561             >
1562                 Generate .gitignore
1563             </button>
1564         </div>
1565     </ToolCard>
1566 }}
1567
1568 {/* TOOL 90 - README Generator */}
1569 {isVisible(90, "README Generator", "DevOps", ["readme", "markdown"]) && (
1570     <ToolCard
1571         id={90}
1572         title="README.md Starter Generator"
1573         category="DevOps"
1574         description="Generate well-formatted project README template"
1575         output={t900Out}
1576     >
1577         <div className="space-y-2">
1578             <input
1579                 type="text"
1580                 value={t90Name}
1581                 onChange={(e) => setT90Name(e.target.value)}
1582                 placeholder="Project Name"
1583                 className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1584                 dark:bg-slate-900 font-m

```

src/components/tools/ToolsDataWeb.tsx (continued)

```
1585         <input
1586           type="text"
1587           value={t90Desc}
1588           onChange={(e) => setT90Desc(e.target.value)}
1589           placeholder="Short Description"
1590           className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1591 dark:bg-slate-900 font-m
1592         <button
1593           type="button"
1594           onClick={() => {
1595             setT90Out(
1596               `# ${t90Name}\n\n${t90Desc}\n\n## Installation\n\n\`\`\`bash\nnpm install\n\`\`\`\n\n##
1597 Usage\n\n\`\`\`bash\nnpm run dev\n
1598             `)
1599           }}
1600           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1601         >
1602           Generate README
1603         </button>
1604       </div>
1605     </ToolCard>
1606   )}
1607
1608   {/* TOOL 91 - JS Function Generator */}
1609   {isVisible(91, "JS Function Generator", "JavaScript", ["function template", "snippet"])} && (
1610     <ToolCard
1611       id={91}
1612       title="JavaScript Function Snippet Generator"
1613       category="JavaScript"
1614       description="Generate modern async / arrow function declarations"
1615       output={t91Out}
1616     >
1617       <div className="flex gap-2">
1618         <input
1619           type="text"
1620           value={t91Name}
1621           onChange={(e) => setT91Name(e.target.value)}
1622           placeholder="Function Name"
1623           className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1624 dark:bg-slate-900 font-m
1625         <input
1626           type="text"
1627           value={t91Args}
1628           onChange={(e) => setT91Args(e.target.value)}
1629           placeholder="Arguments"
1630           className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1631 dark:bg-slate-900 font-m
1632         <button
1633           type="button"
1634           onClick={() => {
1635             setT91Out(
1636               `export const ${t91Name} = async (${t91Args}) => {\n  try {\n    // Implementation logic
1637             here\n    return true;\n  } catch
1638             `)
1639           }}
1640           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1641         >
1642           Generate
1643         </button>
1644       </div>
1645     </ToolCard>
1646   )}
1647
1648   {/* TOOL 92 - Fetch API Snippet */}
1649   {isVisible(92, "Fetch API Snippet", "JavaScript", ["fetch", "ajax", "api call"])} && (
1650     <ToolCard
1651       id={92}
1652       title="Fetch API Snippet Generator"
1653       category="JavaScript"
1654       description="Generate fetch() calls with async/await and response checks"
1655       output={t92Out}
```

src/components/tools/ToolsDataWeb.tsx (continued)

```

1654         >
1655         <div className="flex gap-2">
1656             <select
1657                 value={t92Method}
1658                 onChange={(e) => setT92Method(e.target.value)}
1659                 className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1660 dark:bg-slate-900"
1661                 <option value="GET">GET</option>
1662                 <option value="POST">POST</option>
1663                 <option value="PUT">PUT</option>
1664                 <option value="DELETE">DELETE</option>
1665             </select>
1666             <input
1667                 type="text"
1668                 value={t92Url}
1669                 onChange={(e) => setT92Url(e.target.value)}
1670                 className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1671 dark:bg-slate-900 font-m
1672             <button
1673                 type="button"
1674                 onClick={() => {
1675                     setT92Out(
1676                         `const res = await fetch("${t92Url}", {\n  method: "${t92Method}",\n  headers: { "Content-Type":
1677 "application/json"},\n})
1678                     }
1679                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1680                 >
1681                     Generate Fetch
1682             </button>
1683         </div>
1684     </ToolCard>
1685 })
1686
1687     /* TOOL 93 - SQL Formatter */
1688     {isVisible(93, "SQL Formatter", "Formatting", ["sql", "query"]) && (
1689         <ToolCard
1690             id={93}
1691             title="SQL Query Formatter"
1692             category="Formatting"
1693             description="Capitalize and indent standard SQL keywords"
1694             output={t93Out}
1695         >
1696             <textarea
1697                 value={t93In}
1698                 onChange={(e) => setT93In(e.target.value)}
1699                 rows={2}
1700                 className="w-full text-xs p-2 rounded-lg border border-slate-300 dark:border-slate-700 bg-slate-50
1701 dark:bg-slate-900 font-mono"
1702             <button
1703                 type="button"
1704                 onClick={() => {
1705                     const keywords = [
1706                         "SELECT", "FROM", "WHERE", "ORDER BY", "GROUP BY", "HAVING", "LIMIT",
1707                         "JOIN", "LEFT JOIN", "RIGHT JOIN", "INNER JOIN", "INSERT INTO", "VALUES",
1708                         "UPDATE", "SET", "DELETE FROM"
1709                     ];
1710                     let res = t93In;
1711                     keywords.forEach((k) => {
1712                         const regex = new RegExp(`\\b${k}\\b`, "gi");
1713                         res = res.replace(regex, `\\n${k}`);
1714                     });
1715                     setT93Out(res.trim());
1716                 }
1717                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1718             >
1719                 Format SQL
1720             </button>
1721         </ToolCard>
1722     )}

```

src/components/tools/ToolsDataWeb.tsx (continued)

```
1723
1724     {/* TOOL 94 - SQL String Escaper */}
1725     {isVisible(94, "SQL String Escaper", "Security", ["sql injection", "escape single quote"]) && (
1726         <ToolCard
1727             id={94}
1728             title="SQL String Escaper"
1729             category="Security"
1730             description="Escape single quotes and special characters for SQL literals"
1731             output={t94Out}
1732         >
1733             <input
1734                 type="text"
1735                 value={t94In}
1736                 onChange={(e) => setT94In(e.target.value)}
1737                 className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1738                 dark:bg-slate-900 font-mon
1739             />
1740             <button
1741                 type="button"
1742                 onClick={() => setT94Out(t94In.replace(/'/g, "'").replace(/\\/g, "\\"))}
1743                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1744             >
1745                 Escape String
1746             </button>
1747         </ToolCard>
1748     )}
1749
1750     {/* TOOL 95 - Date Formatter */}
1751     {isVisible(95, "Date Formatter", "Time", ["date format", "iso", "utc"]) && (
1752         <ToolCard
1753             id={95}
1754             title="Date Formatter"
1755             category="Time"
1756             description="Display date in ISO, UTC, and locale variations"
1757             output={t95Out}
1758         >
1759             <div className="flex gap-2">
1760                 <input
1761                     type="datetime-local"
1762                     value={t95Date}
1763                     onChange={(e) => setT95Date(e.target.value)}
1764                     className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1765                     dark:bg-slate-900 font-m
1766                 />
1767                 <button
1768                     type="button"
1769                     onClick={() => {
1770                         const d = new Date(t95Date);
1771                         setT95Out(
1772                             `ISO: ${d.toISOString()}\nUTC: ${d.toUTCString()}\nDate: ${d.toString()}\nTime:
1773                             ${d.toTimeString()}\nLocale: ${d.toLocale
1774                             }`
1775                         )
1776                     }}
1777                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1778                 >
1779                     Format Date
1780                 </button>
1781             </div>
1782         </ToolCard>
1783     )}
1784
1785     {/* TOOL 96 - Unix Timestamp Now */}
1786     {isVisible(96, "Unix Timestamp Now", "Time", ["current epoch", "now"]) && (
1787         <ToolCard
1788             id={96}
1789             title="Current Unix Timestamp (Epoch)"
1790             category="Time"
1791             description="Get accurate current Unix timestamp in seconds and milliseconds"
1792             output={t96Out}
1793         >
1794             <button
1795                 type="button"
```

src/components/tools/ToolsDataWeb.tsx (continued)

```
1792         onClick={() => {
1793             const now = Date.now();
1794             setT96Out(`Seconds: ${Math.floor(now / 1000)}\nMilliseconds: ${now}`);
1795         }}
1796         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1797     >
1798         Get Current Epoch
1799     </button>
1800 </ToolCard>
1801 }}
1802
1803 {/* TOOL 97 - HMAC SHA-256 */}
1804 {isVisible(97, "HMAC SHA-256 Generator", "Hashes", ["hmac", "secret signature"]) && (
1805     <ToolCard
1806         id={97}
1807         title="HMAC SHA-256 Generator"
1808         category="Hashes"
1809         description="Create cryptographic HMAC signature using key and payload"
1810         output={t97Out}
1811     >
1812         <div className="space-y-2">
1813             <input
1814                 type="text"
1815                 value={t97Key}
1816                 onChange={(e) => setT97Key(e.target.value)}
1817                 placeholder="Secret Key"
1818                 className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1819 dark:bg-slate-900 font-m
1820             <input
1821                 type="text"
1822                 value={t97Msg}
1823                 onChange={(e) => setT97Msg(e.target.value)}
1824                 placeholder="Message / Payload"
1825                 className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1826 dark:bg-slate-900 font-m
1827             <button
1828                 type="button"
1829                 onClick={async () => setT97Out(await hmacSha256Hex(t97Key, t97Msg))}
1830                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1831             >
1832                 Generate HMAC
1833             </button>
1834         </div>
1835     </ToolCard>
1836 }}
1837
1838 {/* TOOL 98 - UUID Bulk Generator */}
1839 {isVisible(98, "UUID Bulk Generator", "Generators", ["multiple uuids", "bulk guid"]) && (
1840     <ToolCard
1841         id={98}
1842         title="Bulk UUID Generator"
1843         category="Generators"
1844         description="Generate multiple RFC 4122 v4 UUIDs at once"
1845         output={t98Out}
1846     >
1847         <div className="flex items-center gap-2">
1848             <label className="text-xs">Count:</label>
1849             <input
1850                 type="number"
1851                 min={1}
1852                 max={50}
1853                 value={t98Count}
1854                 onChange={(e) => setT98Count(Number(e.target.value))}
1855                 className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1856 dark:bg-slate-900 font-mon
1857             <button
1858                 type="button"
1859                 onClick={() => {
1860                     const list = Array.from(
```

src/components/tools/ToolsDataWeb.tsx (continued)

```

1861         { length: Math.min(50, Math.max(1, t98Count)) },
1862         () => generateUUIDv4()
1863     );
1864     setT98Out(list.join("\n"));
1865   }}
1866   className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1867 >
1868   Generate
1869 </button>
1870 </div>
1871 </ToolCard>
1872 })
1873
1874 { /* TOOL 99 - HTTP Status Code Lookup */}
1875 {isVisible(99, "HTTP Status Code Lookup", "API", ["status code", "200", "404", "500"]) && (
1876   <ToolCard
1877     id={99}
1878     title="HTTP Status Code Lookup"
1879     category="API"
1880     description="Lookup standard RFC HTTP status codes and explanations"
1881     output={t99Out}
1882   >
1883     <div className="flex gap-2">
1884       <input
1885         type="text"
1886         value={t99Code}
1887         onChange={(e) => setT99Code(e.target.value)}
1888         placeholder="e.g. 200, 404, 500"
1889         className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1890         dark:bg-slate-900 font-mon
1891       >
1892       <button
1893         type="button"
1894         onClick={() => {
1895           const map: Record<string, string> = {
1896             "200": "200 OK: Standard successful HTTP request.",
1897             "201": "201 Created: New resource successfully created.",
1898             "204": "204 No Content: Request succeeded with no content returned.",
1899             "301": "301 Moved Permanently: Resource URL permanently changed.",
1900             "302": "302 Found: Temporary redirection to new URI.",
1901             "304": "304 Not Modified: Cached response is still valid.",
1902             "400": "400 Bad Request: Invalid syntax or missing required parameters.",
1903             "401": "401 Unauthorized: Authentication credentials required or invalid.",
1904             "403": "403 Forbidden: Server understood request but refuses authorization.",
1905             "404": "404 Not Found: Requested resource could not be located.",
1906             "409": "409 Conflict: Request conflicts with current state of resource.",
1907             "429": "429 Too Many Requests: Rate limiting triggered.",
1908             "500": "500 Internal Server Error: Unexpected server failure.",
1909             "502": "502 Bad Gateway: Upstream gateway received invalid response.",
1910             "503": "503 Service Unavailable: Server currently overloaded or in maintenance.",
1911           };
1912           setT99Out(map[t99Code.trim()] || `HTTP ${t99Code}: Consult RFC 9110 specification.`);
1913         }}
1914       >
1915       Lookup
1916     </button>
1917   </div>
1918 </ToolCard>
1919 })
1920
1921 { /* TOOL 100 - Developer Cheat Sheet */}
1922 {isVisible(100, "Developer Cheat Sheet", "Utilities", ["cheat sheet", "git", "docker", "linux"]) && (
1923   <ToolCard
1924     id={100}
1925     title="Developer Cheat Sheet Reference"
1926     category="Utilities"
1927     description="Instant command references for Git, Docker, and Linux terminal"
1928     output={t100Out}
1929   >

```


src/components/tools/ToolsDataWeb.tsx (continued)

```
1930         <div className="flex gap-2">
1931             <select
1932                 value={t100Topic}
1933                 onChange={(e) => setT100Topic(e.target.value)}
1934                 className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1935                 dark:bg-slate-900">
1936                 <option value="git">Git Commands</option>
1937                 <option value="docker">Docker Commands</option>
1938                 <option value="linux">Linux Bash Commands</option>
1939             </select>
1940             <button
1941                 type="button"
1942                 onClick={() => {
1943                     if (t100Topic === "git") {
1944                         setT100Out(
1945                             "git status\ngit add .\ngit commit -m 'feat: message'\ngit pull --rebase\ngit push origin
1946                             main\ngit branch -q\ngit check
1947                         )
1948                     } else if (t100Topic === "docker") {
1949                         setT100Out(
1950                             "docker build -t app:latest .\ndocker run -p 3000:3000 app:latest\ndocker ps\ndocker stop
1951                             <container_id>\ndocker compose
1952                         )
1953                     } else {
1954                         setT100Out(
1955                             "ls -la\ncd /path/to/dir\nmkdir -p new_folder\nrm -rf target_folder\nps aux | grep node\nkill
1956                             -9 <PID>\nchmod +x\script.
1957                         )
1958                     }
1959                 }}
1960             </button>
1961         </div>
1962     </ToolCard>
1963   )}
1964 </>
1965 );
1966 };
1967
```

FILE [25/25]

src/components/tools/ToolsAdvancedDev.jsx • Tools #101 - #150 (Diff, Ciphers, Crypto, CSS & JS Snippets)

```
1 import React, { useState } from "react";
2 import { ToolCard } from "../ToolCard";
3 import { ToolCategory } from "../../types";
4 import {
5   sha1Hex,
6   sha384Hex,
7   sha512Hex,
8   hmacSha512Hex,
9   simpleMd5,
10 } from "../../utils/crypto";
11 import {
12   rot13,
13   rot47,
14   caesarCipher,
15   stringToHex,
16   hexToString,
17 } from "../../utils/helpers";
18
19 interface ToolsProps {
20   searchQuery: string;
21   selectedCategory: ToolCategory;
22 }
23
24 export const ToolsAdvancedDev: React.FC<ToolsProps> = ({
25   searchQuery,
26   selectedCategory,
27 }) => {
28   const isVisible = (id: number, title: string, category: ToolCategory, keywords: string[] = []) => {
29     const matchesCategory = selectedCategory === "All" || selectedCategory === category;
30     if (!matchesCategory) return false;
31     if (!searchQuery.trim()) return true;
32     const q = searchQuery.toLowerCase().trim();
33     return (
34       title.toLowerCase().includes(q) ||
35       category.toLowerCase().includes(q) ||
36       `#${id}`.includes(q) ||
37       keywords.some((k) => k.toLowerCase().includes(q))
38     );
39   };
40
41   // 101 - Text Diff
42   const [t101A, setT101A] = useState("Hello World\nCoding is great");
43   const [t101B, setT101B] = useState("Hello World!\nCoding is super great");
44   const [t101Out, setT101Out] = useState("");
45
46   // 102 - Text Joiner
47   const [t102In, setT102In] = useState("apple\nbanana\norange");
48   const [t102Delim, setT102Delim] = useState(", ");
49   const [t102Out, setT102Out] = useState("");
50
51   // 103 - Line Break Converter
52   const [t103In, setT103In] = useState("First Line\r\nSecond Line\r\nThird Line");
53   const [t103Out, setT103Out] = useState("");
54
55   // 104 - Tabs <-> Spaces
56   const [t104In, setT104In] = useState("    const x = 10;\n    const y = 20;");
57   const [t104Out, setT104Out] = useState("");
58
59   // 105 - Indent Formatter
60   const [t105In, setT105In] = useState("function test() {\nreturn true;\n}");
61   const [t105Out, setT105Out] = useState("");
62
63   // 106 - Text Extractor
64   const [t106In, setT106In] = useState("Contact us at support@example.com or admin@hub.dev for assistance.");
65   const [t106Out, setT106Out] = useState("");
66 }
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
67 // 107 - Text Trimmer
68 const [t107In, setT107In] = useState("    Leading and trailing whitespace    ");
69 const [t107Out, setT107Out] = useState("");
70
71 // 108 - Number List Generator
72 const [t108Start, setT108Start] = useState(1);
73 const [t108End, setT108End] = useState(10);
74 const [t108Step, setT108Step] = useState(1);
75 const [t108Out, setT108Out] = useState("");
76
77 // 109 - Text Padding Generator
78 const [t109In, setT109In] = useState("42");
79 const [t109Len, setT109Len] = useState(6);
80 const [t109Char, setT109Char] = useState("0");
81 const [t109Out, setT109Out] = useState("");
82
83 // 110 - Text Wrap Converter
84 const [t110In, setT110In] = useState("Coding Super Hub provides developers with over 150 real-time tools for
85 daily workflows.");
86 const [t110Width, setT110Width] = useState(30);
87 const [t110Out, setT110Out] = useState("");
88
89 // 111 - MD5
90 const [t111In, setT111In] = useState("secret_message");
91 const [t111Out, setT111Out] = useState("");
92
93 // 112 - SHA-1
94 const [t112In, setT112In] = useState("git commit message");
95 const [t112Out, setT112Out] = useState("");
96
97 // 113 - SHA-384
98 const [t113In, setT113In] = useState("subresource_integrity_check");
99 const [t113Out, setT113Out] = useState("");
100
101 // 114 - SHA-512
102 const [t114In, setT114In] = useState("high_security_hash");
103 const [t114Out, setT114Out] = useState("");
104
105 // 115 - HMAC SHA-512
106 const [t115Key, setT115Key] = useState("secret_key_512");
107 const [t115Msg, setT115Msg] = useState("payload_data");
108 const [t115Out, setT115Out] = useState("");
109
110 // 116 - Hex Encoder / Decoder
111 const [t116In, setT116In] = useState("Super Hub");
112 const [t116Out, setT116Out] = useState("");
113
114 // 117 - ROT13
115 const [t117In, setT117In] = useState("Hello World");
116 const [t117Out, setT117Out] = useState("");
117
118 // 118 - ROT47
119 const [t118In, setT118In] = useState("Hello World 123!");
120 const [t118Out, setT118Out] = useState("");
121
122 // 119 - Caesar Cipher
123 const [t119In, setT119In] = useState("ATTACK AT DAWN");
124 const [t119Shift, setT119Shift] = useState(3);
125 const [t119Out, setT119Out] = useState("");
126
127 // 120 - AES-GCM Key Generator
128 const [t120Out, setT120Out] = useState("");
129
130 // 121 - HTTP Header Generator
131 const [t121Type, setT121Type] = useState("cors");
132 const [t121Out, setT121Out] = useState("");
133
134 // 122 - cURL Request Builder
135 const [t122Url, setT122Url] = useState("https://api.example.com/items");
136 const [t122Method, setT122Method] = useState("POST");
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
136 const [t122Data, setT122Data] = useState('{ "title": "Tool"}');
137 const [t122Out, setT122Out] = useState("");
138
139 // 123 - HTTP Response Header Viewer
140 const [t123Out, setT123Out] = useState("");
141
142 // 124 - MIME Type Lookup
143 const [t124Ext, setT124Ext] = useState("json");
144 const [t124Out, setT124Out] = useState("");
145
146 // 125 - User-Agent Parser
147 const [t125Ua, setT125Ua] = useState(typeof navigator !== "undefined" ? navigator.userAgent : "");
148 const [t125Out, setT125Out] = useState("");
149
150 // 126 - URL Shortener Slug Generator
151 const [t126Url, setT126Url] = useState("https://ai.studio/build/app");
152 const [t126Out, setT126Out] = useState("");
153
154 // 127 - Open Graph Meta Generator
155 const [t127Title, setT127Title] = useState("Coding Super Hub");
156 const [t127Img, setT127Img] = useState("https://example.com/og.png");
157 const [t127Out, setT127Out] = useState("");
158
159 // 128 - Twitter Card Generator
160 const [t128Title, setT128Title] = useState("Coding Super Hub");
161 const [t128Handle, setT128Handle] = useState("@developer");
162 const [t128Out, setT128Out] = useState("");
163
164 // 129 - Canonical URL Generator
165 const [t129Url, setT129Url] = useState("https://example.com/page");
166 const [t129Out, setT129Out] = useState("");
167
168 // 130 - Web App Manifest Generator
169 const [t130Name, setT130Name] = useState("Coding Super Hub");
170 const [t130Color, setT130Color] = useState("#2563eb");
171 const [t130Out, setT130Out] = useState("");
172
173 // 131 - CSS Button Generator
174 const [t131Bg, setT131Bg] = useState("#2563eb");
175 const [t131Color, setT131Color] = useState("ffffff");
176 const [t131Radius, setT131Radius] = useState(8);
177
178 // 132 - CSS Text Shadow Generator
179 const [t132X, setT132X] = useState(2);
180 const [t132Y, setT132Y] = useState(2);
181 const [t132Blur, setT132Blur] = useState(4);
182
183 // 133 - CSS Animation Keyframe Generator
184 const [t133Name, setT133Name] = useState("fadeInUp");
185 const [t133Out, setT133Out] = useState("");
186
187 // 134 - CSS Transform Generator
188 const [t134Rot, setT134Rot] = useState(15);
189 const [t134Scale, setT134Scale] = useState(1.1);
190
191 // 135 - CSS Transition Generator
192 const [t135Prop, setT135Prop] = useState("all");
193 const [t135Duration, setT135Duration] = useState("0.3s");
194 const [t135Ease, setT135Ease] = useState("ease-in-out");
195
196 // 136 - CSS Filter Generator
197 const [t136Blur, setT136Blur] = useState(0);
198 const [t136Brightness, setT136Brightness] = useState(100);
199 const [t136Contrast, setT136Contrast] = useState(100);
200
201 // 137 - CSS Glassmorphism Generator
202 const [t137Blur, setT137Blur] = useState(12);
203 const [t137Opacity, setT137Opacity] = useState(0.2);
204
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
205 // 138 - CSS Neumorphism Generator
206 const [t138Dist, setT138Dist] = useState(8);
207 const [t138Blur, setT138Blur] = useState(16);
208
209 // 139 - CSS Triangle Generator
210 const [t139Dir, setT139Dir] = useState("up");
211 const [t139Size, setT139Size] = useState(16);
212 const [t139Color, setT139Color] = useState("#2563eb");
213
214 // 140 - CSS Spinner Generator
215 const [t140Size, setT140Size] = useState(32);
216 const [t140Color, setT140Color] = useState("#2563eb");
217
218 // 141 - JS Deep Clone Helper
219 const [t141In, setT141In] = useState('{"user": {"name": "Dev", "roles": ["admin"]}}');
220 const [t141Out, setT141Out] = useState("");
221
222 // 142 - JS Array Generator
223 const [t142Len, setT142Len] = useState(10);
224 const [t142Out, setT142Out] = useState("");
225
226 // 143 - Array Methods Cheat Sheet
227 const [t143Method, setT143Method] = useState("map");
228 const [t143Out, setT143Out] = useState("");
229
230 // 144 - Date Helper Snippets
231 const [t144Type, setT144Type] = useState("diff");
232 const [t144Out, setT144Out] = useState("");
233
234 // 145 - Debounce Function Snippet
235 const [t145Out, setT145Out] = useState("");
236
237 // 146 - Throttle Function Snippet
238 const [t146Out, setT146Out] = useState("");
239
240 // 147 - Event Listener Snippet
241 const [t147Event, setT147Event] = useState("click");
242 const [t147Target, setT147Target] = useState("myButton");
243 const [t147Out, setT147Out] = useState("");
244
245 // 148 - DOM Selector Generator
246 const [t148Selector, setT148Selector] = useState(".nav-item");
247 const [t148Out, setT148Out] = useState("");
248
249 // 149 - LocalStorage Code Helper
250 const [t149Key, setT149Key] = useState("app_settings");
251 const [t149Out, setT149Out] = useState("");
252
253 // 150 - SessionStorage Code Helper
254 const [t150Key, setT150Key] = useState("auth_session");
255 const [t150Out, setT150Out] = useState("");
256
257 return (
258   <>
259     {/* TOOL 101 - Text Diff Checker */}
260     {isVisible(101, "Text Diff Checker", "Text", ["diff", "compare", "changes"]) && (
261       <ToolCard
262         id={101}
263         title="Text Diff Checker"
264         category="Text"
265         description="Compare two text blocks side-by-side to highlight additions and deletions"
266         output={t101Out}
267       >
268         <div className="grid grid-cols-1 sm:grid-cols-2 gap-2">
269           <textarea
270             value={t101A}
271             onChange={(e) => setT101A(e.target.value)}
272             rows={3}
273             placeholder="Original text..."
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

274         className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
275 dark:bg-slate-900 font-mono"
276         <textarea
277           value={t101B}
278           onChange={(e) => setT101B(e.target.value)}
279           rows={3}
280           placeholder="Modified text..."
281           className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
282 dark:bg-slate-900 font-mono"
283         </div>
284         <button
285           type="button"
286           onClick={() => {
287             const linesA = t101A.split("\n");
288             const linesB = t101B.split("\n");
289             const max = Math.max(linesA.length, linesB.length);
290             const diff: string[] = [];
291             for (let i = 0; i < max; i++) {
292               const a = linesA[i];
293               const b = linesB[i];
294               if (a === b) {
295                 diff.push(` [Line ${i + 1}] ${a} || ""`);
296               } else {
297                 if (a !== undefined) diff.push(`- [Line ${i + 1}] ${a}`);
298                 if (b !== undefined) diff.push(`+ [Line ${i + 1}] ${b}`);
299               }
300             }
301             setT101Out(diff.join("\n"));
302           }}
303           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
304         >
305           Compare Text
306         </button>
307       </ToolCard>
308     )}
309
310     {/* TOOL 102 - Text Joiner */}
311     {isVisible(102, "Text Joiner", "Text", ["join", "delimiter", "combine"]) && (
312       <ToolCard
313         id={102}
314         title="Text Line Joiner"
315         category="Text"
316         description="Merge multiple lines using custom delimiter (e.g. comma, pipe, semicolon)"
317         output={t102Out}
318       >
319         <textarea
320           value={t102In}
321           onChange={(e) => setT102In(e.target.value)}
322           rows={2}
323           className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
324 slate-900 font-mono"
325         <div className="flex items-center gap-2">
326           <input
327             type="text"
328             value={t102Delim}
329             onChange={(e) => setT102Delim(e.target.value)}
330             placeholder="Delimiter"
331             className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
332 dark:bg-slate-900 font-mon
333           <button
334             type="button"
335             onClick={() => {
336               setT102Out(t102In.split("\n").join(t102Delim));
337             }}
338             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
339           >
340             Join Lines
341           </button>
342         </div>

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

343         </ToolCard>
344     )}
345
346     {/* TOOL 103 - Line Break Converter */}
347     {isVisible(103, "Line Break Converter", "Formatting", ["crlf", "lf", "windows", "unix"]) && (
348         <ToolCard
349             id={103}
350             title="Line Break Converter (CRLF <-> LF)"
351             category="Formatting"
352             description="Convert between Windows (CRLF) and Unix / Linux (LF) line terminators"
353             output={t103Out}
354         >
355             <textarea
356                 value={t103In}
357                 onChange={(e) => setT103In(e.target.value)}
358                 rows={2}
359                 className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
360 slate-900 font-mono"
361             >
362                 <div className="flex gap-2">
363                     <button
364                         type="button"
365                         onClick={() => {
366                             const lf = t103In.replace(/\r\n/g, "\n");
367                             setT103Out(`Normalized to Unix (LF) - Length: ${lf.length} chars`);
368                         }}
369                         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
370                     >
371                         Convert to LF (Unix)
372                     </button>
373                     <button
374                         type="button"
375                         onClick={() => {
376                             const crlf = t103In.replace(/\r?\n/g, "\r\n");
377                             setT103Out(`Normalized to Windows (CRLF) - Length: ${crlf.length} chars`);
378                         }}
379                         className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs font-semibold"
380                     >
381                         Convert to CRLF (Windows)
382                     </button>
383                 </div>
384             </ToolCard>
385         )}
386
387     {/* TOOL 104 - Tabs <-> Spaces */}
388     {isVisible(104, "Tabs <-> Spaces Converter", "Formatting", ["tabs", "spaces", "indentation"]) && (
389         <ToolCard
390             id={104}
391             title="Tabs to Spaces Converter"
392             category="Formatting"
393             description="Convert tab indents to 2/4 spaces or vice-versa"
394             output={t104Out}
395         >
396             <textarea
397                 value={t104In}
398                 onChange={(e) => setT104In(e.target.value)}
399                 rows={2}
400                 className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
401 slate-900 font-mono"
402             >
403                 <div className="flex gap-2">
404                     <button
405                         type="button"
406                         onClick={() => setT104Out(t104In.replace(/\t/g, " "))}
407                         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
408                     >
409                         Tabs !' 2 Spaces
410                     </button>
411                     <button
412                         type="button"
413                         onClick={() => setT104Out(t104In.replace(/\t/g, "    "))}

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
412         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
413       >
414         Tabs !' 4 Spaces
415       </button>
416       <button
417         type="button"
418         onClick={() => setT104Out(t104In.replace(/ {4}/g, "\t"))}
419         className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs font-semibold"
420       >
421         4 Spaces !' Tabs
422       </button>
423     </div>
424   </ToolCard>
425 )}
426
427 /* TOOL 105 - Indent Formatter */
428 {isVisible(105, "Indent Formatter", "Formatting", ["indent", "add spaces"]) && (
429   <ToolCard
430     id={105}
431     title="Text Indent Formatter"
432     category="Formatting"
433     description="Prefix all lines with uniform indentation"
434     output={t105Out}
435   >
436     <textarea
437       value={t105In}
438       onChange={(e) => setT105In(e.target.value)}
439       rows={2}
440       className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
441 slate-900 font-mono"
442     <button
443       type="button"
444       onClick={() => {
445         setT105Out(t105In.split("\n").map((l) => "  " + l).join("\n"));
446       }}
447       className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
448     >
449       Indent +2 Spaces
450     </button>
451   </ToolCard>
452 )}
453
454 /* TOOL 106 - Text Extractor */
455 {isVisible(106, "Text Extractor", "Text", ["emails", "links", "urls"]) && (
456   <ToolCard
457     id={106}
458     title="Email & URL Extractor"
459     category="Text"
460     description="Extract all email addresses and web URLs from unstructured text"
461     output={t106Out}
462   >
463     <textarea
464       value={t106In}
465       onChange={(e) => setT106In(e.target.value)}
466       rows={2}
467       className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
468 slate-900 font-mono"
469     <button
470       type="button"
471       onClick={() => {
472         const emails = t106In.match(/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g) || [];
473         const urls = t106In.match(/https?:\/\/[^\s]+/g) || [];
474         setT106Out(`Emails Found:\n${emails.join("\n")} || "None"`\n\nURLs Found:\n${urls.join("\n")} || "None"
475 `);
476       }}
477       className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
478     >
479       Extract Links & Emails
480     </button>
481   </ToolCard>
```


src/components/tools/ToolsAdvancedDev.tsx (continued)

```

481     })
482
483     {/* TOOL 107 - Text Trimmer */}
484     {isVisible(107, "Text Trimmer", "Text", ["trim", "strip spaces"]) && (
485       <ToolCard
486         id={107}
487         title="Text Line Trimmer"
488         category="Text"
489         description="Trim leading and trailing spaces from every individual line"
490         output={t107Out}
491       >
492         <textarea
493           value={t107In}
494           onChange={(e) => setT107In(e.target.value)}
495           rows={2}
496           className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
497             slate-900 font-mono"
498         >
499           <button
500             type="button"
501             onClick={() => {
502               setT107Out(t107In.split("\n").map((l) => l.trim()).join("\n"));
503             }}
504             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
505           >
506             Trim Each Line
507           </button>
508         </ToolCard>
509       )}
510
511     {/* TOOL 108 - Number List Generator */}
512     {isVisible(108, "Number List Generator", "Generators", ["sequence", "range", "numbers"]) && (
513       <ToolCard
514         id={108}
515         title="Sequential Number List Generator"
516         category="Generators"
517         description="Generate list of numbers from start to end with step size"
518         output={t108Out}
519       >
520         <div className="flex flex-wrap items-center gap-2">
521           <input
522             type="number"
523             value={t108Start}
524             onChange={(e) => setT108Start(Number(e.target.value))}
525             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
526               dark:bg-slate-900 font-mono"
527             placeholder="Start"
528           />
529           <span className="text-xs">to</span>
530           <input
531             type="number"
532             value={t108End}
533             onChange={(e) => setT108End(Number(e.target.value))}
534             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
535               dark:bg-slate-900 font-mono"
536             placeholder="End"
537           />
538           <span className="text-xs">step:</span>
539           <input
540             type="number"
541             min={1}
542             value={t108Step}
543             onChange={(e) => setT108Step(Number(e.target.value))}
544             className="w-14 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
545               dark:bg-slate-900 font-mon
546           >
547           <button
548             type="button"
549             onClick={() => {
550               const step = Math.max(1, t108Step);
551               const nums: number[] = [];
552               for (let i = t108Start; i <= t108End; i += step) nums.push(i);
553               setT108Out(nums.join(", "));
554             }}
555           >
556             Generate
557           </button>
558         </div>
559       </ToolCard>
560     )}
561
562     {/* TOOL 109 - Random String Generator */}
563     {isVisible(109, "Random String Generator", "Generators", ["sequence", "range", "numbers"]) && (
564       <ToolCard
565         id={109}
566         title="Random String Generator"
567         category="Generators"
568         description="Generate random strings of specified length and character set"
569         output={t109Out}
570       >
571         <div className="flex flex-wrap items-center gap-2">
572           <input
573             type="text"
574             value={t109Length}
575             onChange={(e) => setT109Length(e.target.value)}
576             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
577               dark:bg-slate-900 font-mono"
578             placeholder="Length"
579           />
580           <input
581             type="text"
582             value={t109Chars}
583             onChange={(e) => setT109Chars(e.target.value)}
584             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
585               dark:bg-slate-900 font-mono"
586             placeholder="Characters"
587           />
588           <span className="text-xs">Generate</span>
589           <button
590             type="button"
591             onClick={() => {
592               const length = parseInt(t109Length, 10);
593               const chars = t109Chars;
594               if (length < 1 || chars.length < 1) return;
595               let result = "";
596               for (let i = 0; i < length; i++) {
597                 const randomIndex = Math.floor(Math.random() * chars.length);
598                 result += chars[randomIndex];
599               }
600               setT109Out(result);
601             }}
602           >
603             Generate
604           </button>
605         </div>
606       </ToolCard>
607     )}
608
609     {/* TOOL 110 - Password Generator */}
610     {isVisible(110, "Password Generator", "Generators", ["sequence", "range", "numbers"]) && (
611       <ToolCard
612         id={110}
613         title="Password Generator"
614         category="Generators"
615         description="Generate secure passwords with customizable options"
616         output={t110Out}
617       >
618         <div className="flex flex-wrap items-center gap-2">
619           <input
620             type="text"
621             value={t110Length}
622             onChange={(e) => setT110Length(e.target.value)}
623             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
624               dark:bg-slate-900 font-mono"
625             placeholder="Length"
626           />
627           <input
628             type="text"
629             value={t110Chars}
630             onChange={(e) => setT110Chars(e.target.value)}
631             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
632               dark:bg-slate-900 font-mono"
633             placeholder="Characters"
634           />
635           <input
636             type="checkbox"
637             checked={t110IncludeSpecial}
638             onChange={(e) => setT110IncludeSpecial(e.target.checked)}
639             className="text-slate-500 dark:text-slate-400"
640             placeholder="Include Special"
641           />
642           <span className="text-xs">Generate</span>
643           <button
644             type="button"
645             onClick={() => {
646               const length = parseInt(t110Length, 10);
647               const chars = t110Chars;
648               const includeSpecial = t110IncludeSpecial;
649               if (length < 1 || chars.length < 1) return;
650               let result = "";
651               for (let i = 0; i < length; i++) {
652                 let randomIndex;
653                 if (includeSpecial) {
654                   randomIndex = Math.floor(Math.random() * (chars.length + 3));
655                 } else {
656                   randomIndex = Math.floor(Math.random() * chars.length);
657                 }
658                 result += chars[randomIndex];
659               }
660               setT110Out(result);
661             }}
662           >
663             Generate
664           </button>
665         </div>
666       </ToolCard>
667     )}
668
669     {/* TOOL 111 - QR Code Generator */}
670     {isVisible(111, "QR Code Generator", "Generators", ["sequence", "range", "numbers"]) && (
671       <ToolCard
672         id={111}
673         title="QR Code Generator"
674         category="Generators"
675         description="Generate QR codes from text or URLs"
676         output={t111Out}
677       >
678         <div className="flex flex-wrap items-center gap-2">
679           <input
680             type="text"
681             value={t111Text}
682             onChange={(e) => setT111Text(e.target.value)}
683             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
684               dark:bg-slate-900 font-mono"
685             placeholder="Text/URL"
686           />
687           <span className="text-xs">Generate</span>
688           <button
689             type="button"
690             onClick={() => {
691               const text = t111Text;
692               if (text.length < 1) return;
693               const qr = new QRCode.QRCode(1, text);
694               setT111Out(qr.toDataURL());
695             }}
696           >
697             Generate
698           </button>
699         </div>
700       </ToolCard>
701     )}
702
703     {/* TOOL 112 - Base64 Encoder/Decoder */}
704     {isVisible(112, "Base64 Encoder/Decoder", "Generators", ["sequence", "range", "numbers"]) && (
705       <ToolCard
706         id={112}
707         title="Base64 Encoder/Decoder"
708         category="Generators"
709         description="Encode or decode text to/from Base64"
710         output={t112Out}
711       >
712         <div className="flex flex-wrap items-center gap-2">
713           <input
714             type="text"
715             value={t112Text}
716             onChange={(e) => setT112Text(e.target.value)}
717             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
718               dark:bg-slate-900 font-mono"
719             placeholder="Text"
720           />
721           <span className="text-xs">Encode</span>
722           <input
723             type="checkbox"
724             checked={t112Decode}
725             onChange={(e) => setT112Decode(e.target.checked)}
726             className="text-slate-500 dark:text-slate-400"
727             placeholder="Decode"
728           />
729           <span className="text-xs">Generate</span>
730           <button
731             type="button"
732             onClick={() => {
733               const text = t112Text;
734               if (text.length < 1) return;
735               const encoder = new TextEncoder();
736               const decoder = new TextDecoder();
737               if (t112Decode) {
738                 const decoded = decoder.decode(atob(text));
739                 setT112Out(decoded);
740               } else {
741                 const encoded = btoa(encoder.encode(text));
742                 setT112Out(encoded);
743               }
744             }}
745           >
746             Generate
747           </button>
748         </div>
749       </ToolCard>
750     )}
751
752     {/* TOOL 113 - JSON Validator */}
753     {isVisible(113, "JSON Validator", "Generators", ["sequence", "range", "numbers"]) && (
754       <ToolCard
755         id={113}
756         title="JSON Validator"
757         category="Generators"
758         description="Validate JSON syntax and structure"
759         output={t113Out}
760       >
761         <div className="flex flex-wrap items-center gap-2">
762           <input
763             type="text"
764             value={t113Text}
765             onChange={(e) => setT113Text(e.target.value)}
766             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
767               dark:bg-slate-900 font-mono"
768             placeholder="JSON Text"
769           />
770           <span className="text-xs">Validate</span>
771           <button
772             type="button"
773             onClick={() => {
774               const text = t113Text;
775               if (text.length < 1) return;
776               try {
777                 const json = JSON.parse(text);
778                 setT113Out(JSON.stringify(json, null, 2));
779               } catch (error) {
780                 setT113Out("Invalid JSON");
781               }
782             }}
783           >
784             Validate
785           </button>
786         </div>
787       </ToolCard>
788     )}
789
790     {/* TOOL 114 - CSV Generator */}
791     {isVisible(114, "CSV Generator", "Generators", ["sequence", "range", "numbers"]) && (
792       <ToolCard
793         id={114}
794         title="CSV Generator"
795         category="Generators"
796         description="Generate CSV data from a template"
797         output={t114Out}
798       >
799         <div className="flex flex-wrap items-center gap-2">
800           <input
801             type="text"
802             value={t114Template}
803             onChange={(e) => setT114Template(e.target.value)}
804             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
805               dark:bg-slate-900 font-mono"
806             placeholder="Template"
807           />
808           <span className="text-xs">Generate</span>
809           <button
810             type="button"
811             onClick={() => {
812               const template = t114Template;
813               if (template.length < 1) return;
814               const rows = template.split("\n");
815               const csv = rows.map((row) => row.replace(/"/g, '""')).join("\n");
816               setT114Out(csv);
817             }}
818           >
819             Generate
820           </button>
821         </div>
822       </ToolCard>
823     )}
824
825     {/* TOOL 115 - Markdown Generator */}
826     {isVisible(115, "Markdown Generator", "Generators", ["sequence", "range", "numbers"]) && (
827       <ToolCard
828         id={115}
829         title="Markdown Generator"
830         category="Generators"
831         description="Generate Markdown content from a template"
832         output={t115Out}
833       >
834         <div className="flex flex-wrap items-center gap-2">
835           <input
836             type="text"
837             value={t115Template}
838             onChange={(e) => setT115Template(e.target.value)}
839             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
840               dark:bg-slate-900 font-mono"
841             placeholder="Template"
842           />
843           <span className="text-xs">Generate</span>
844           <button
845             type="button"
846             onClick={() => {
847               const template = t115Template;
848               if (template.length < 1) return;
849               const rows = template.split("\n");
850               const md = rows.map((row) => row.replace(/"/g, '""')).join("\n");
851               setT115Out(md);
852             }}
853           >
854             Generate
855           </button>
856         </div>
857       </ToolCard>
858     )}
859
860     {/* TOOL 116 - HTML Generator */}
861     {isVisible(116, "HTML Generator", "Generators", ["sequence", "range", "numbers"]) && (
862       <ToolCard
863         id={116}
864         title="HTML Generator"
865         category="Generators"
866         description="Generate HTML content from a template"
867         output={t116Out}
868       >
869         <div className="flex flex-wrap items-center gap-2">
870           <input
871             type="text"
872             value={t116Template}
873             onChange={(e) => setT116Template(e.target.value)}
874             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
875               dark:bg-slate-900 font-mono"
876             placeholder="Template"
877           />
878           <span className="text-xs">Generate</span>
879           <button
880             type="button"
881             onClick={() => {
882               const template = t116Template;
883               if (template.length < 1) return;
884               const rows = template.split("\n");
885               const html = rows.map((row) => row.replace(/"/g, '""')).join("\n");
886               setT116Out(html);
887             }}
888           >
889             Generate
890           </button>
891         </div>
892       </ToolCard>
893     )}
894
895     {/* TOOL 117 - XML Generator */}
896     {isVisible(117, "XML Generator", "Generators", ["sequence", "range", "numbers"]) && (
897       <ToolCard
898         id={117}
899         title="XML Generator"
900         category="Generators"
901         description="Generate XML content from a template"
902         output={t117Out}
903       >
904         <div className="flex flex-wrap items-center gap-2">
905           <input
906             type="text"
907             value={t117Template}
908             onChange={(e) => setT117Template(e.target.value)}
909             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
910               dark:bg-slate-900 font-mono"
911             placeholder="Template"
912           />
913           <span className="text-xs">Generate</span>
914           <button
915             type="button"
916             onClick={() => {
917               const template = t117Template;
918               if (template.length < 1) return;
919               const rows = template.split("\n");
920               const xml = rows.map((row) => row.replace(/"/g, '""')).join("\n");
921               setT117Out(xml);
922             }}
923           >
924             Generate
925           </button>
926         </div>
927       </ToolCard>
928     )}
929
930     {/* TOOL 118 - YAML Generator */}
931     {isVisible(118, "YAML Generator", "Generators", ["sequence", "range", "numbers"]) && (
932       <ToolCard
933         id={118}
934         title="YAML Generator"
935         category="Generators"
936         description="Generate YAML content from a template"
937         output={t118Out}
938       >
939         <div className="flex flex-wrap items-center gap-2">
940           <input
941             type="text"
942             value={t118Template}
943             onChange={(e) => setT118Template(e.target.value)}
944             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
945               dark:bg-slate-900 font-mono"
946             placeholder="Template"
947           />
948           <span className="text-xs">Generate</span>
949           <button
950             type="button"
951             onClick={() => {
952               const template = t118Template;
953               if (template.length < 1) return;
954               const rows = template.split("\n");
955               const yaml = rows.map((row) => row.replace(/"/g, '""')).join("\n");
956               setT118Out(yaml);
957             }}
958           >
959             Generate
960           </button>
961         </div>
962       </ToolCard>
963     )}
964
965     {/* TOOL 119 - SQL Generator */}
966     {isVisible(119, "SQL Generator", "Generators", ["sequence", "range", "numbers"]) && (
967       <ToolCard
968         id={119}
969         title="SQL Generator"
970         category="Generators"
971         description="Generate SQL queries from a template"
972         output={t119Out}
973       >
974         <div className="flex flex-wrap items-center gap-2">
975           <input
976             type="text"
977             value={t119Template}
978             onChange={(e) => setT119Template(e.target.value)}
979             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
980               dark:bg-slate-900 font-mono"
981             placeholder="Template"
982           />
983           <span className="text-xs">Generate</span>
984           <button
985             type="button"
986             onClick={() => {
987               const template = t119Template;
988               if (template.length < 1) return;
989               const rows = template.split("\n");
990               const sql = rows.map((row) => row.replace(/"/g, '""')).join("\n");
991               setT119Out(sql);
992             }}
993           >
994             Generate
995           </button>
996         </div>
997       </ToolCard>
998     )}
999
1000     {/* TOOL 120 - Bash Script Generator */}
1001     {isVisible(120, "Bash Script Generator", "Generators", ["sequence", "range", "numbers"]) && (
1002       <ToolCard
1003         id={120}
1004         title="Bash Script Generator"
1005         category="Generators"
1006         description="Generate Bash scripts from a template"
1007         output={t120Out}
1008       >
1009         <div className="flex flex-wrap items-center gap-2">
1010           <input
1011             type="text"
1012             value={t120Template}
1013             onChange={(e) => setT120Template(e.target.value)}
1014             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1015               dark:bg-slate-900 font-mono"
1016             placeholder="Template"
1017           />
1018           <span className="text-xs">Generate</span>
1019           <button
1020             type="button"
1021             onClick={() => {
1022               const template = t120Template;
1023               if (template.length < 1) return;
1024               const rows = template.split("\n");
1025               const bash = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1026               setT120Out(bash);
1027             }}
1028           >
1029             Generate
1030           </button>
1031         </div>
1032       </ToolCard>
1033     )}
1034
1035     {/* TOOL 121 - Python Script Generator */}
1036     {isVisible(121, "Python Script Generator", "Generators", ["sequence", "range", "numbers"]) && (
1037       <ToolCard
1038         id={121}
1039         title="Python Script Generator"
1040         category="Generators"
1041         description="Generate Python scripts from a template"
1042         output={t121Out}
1043       >
1044         <div className="flex flex-wrap items-center gap-2">
1045           <input
1046             type="text"
1047             value={t121Template}
1048             onChange={(e) => setT121Template(e.target.value)}
1049             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1050               dark:bg-slate-900 font-mono"
1051             placeholder="Template"
1052           />
1053           <span className="text-xs">Generate</span>
1054           <button
1055             type="button"
1056             onClick={() => {
1057               const template = t121Template;
1058               if (template.length < 1) return;
1059               const rows = template.split("\n");
1060               const python = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1061               setT121Out(python);
1062             }}
1063           >
1064             Generate
1065           </button>
1066         </div>
1067       </ToolCard>
1068     )}
1069
1070     {/* TOOL 122 - JavaScript Script Generator */}
1071     {isVisible(122, "JavaScript Script Generator", "Generators", ["sequence", "range", "numbers"]) && (
1072       <ToolCard
1073         id={122}
1074         title="JavaScript Script Generator"
1075         category="Generators"
1076         description="Generate JavaScript scripts from a template"
1077         output={t122Out}
1078       >
1079         <div className="flex flex-wrap items-center gap-2">
1080           <input
1081             type="text"
1082             value={t122Template}
1083             onChange={(e) => setT122Template(e.target.value)}
1084             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1085               dark:bg-slate-900 font-mono"
1086             placeholder="Template"
1087           />
1088           <span className="text-xs">Generate</span>
1089           <button
1090             type="button"
1091             onClick={() => {
1092               const template = t122Template;
1093               if (template.length < 1) return;
1094               const rows = template.split("\n");
1095               const js = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1096               setT122Out(js);
1097             }}
1098           >
1099             Generate
1100           </button>
1101         </div>
1102       </ToolCard>
1103     )}
1104
1105     {/* TOOL 123 - CSS Generator */}
1106     {isVisible(123, "CSS Generator", "Generators", ["sequence", "range", "numbers"]) && (
1107       <ToolCard
1108         id={123}
1109         title="CSS Generator"
1110         category="Generators"
1111         description="Generate CSS styles from a template"
1112         output={t123Out}
1113       >
1114         <div className="flex flex-wrap items-center gap-2">
1115           <input
1116             type="text"
1117             value={t123Template}
1118             onChange={(e) => setT123Template(e.target.value)}
1119             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1120               dark:bg-slate-900 font-mono"
1121             placeholder="Template"
1122           />
1123           <span className="text-xs">Generate</span>
1124           <button
1125             type="button"
1126             onClick={() => {
1127               const template = t123Template;
1128               if (template.length < 1) return;
1129               const rows = template.split("\n");
1130               const css = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1131               setT123Out(css);
1132             }}
1133           >
1134             Generate
1135           </button>
1136         </div>
1137       </ToolCard>
1138     )}
1139
1140     {/* TOOL 124 - JSON Schema Generator */}
1141     {isVisible(124, "JSON Schema Generator", "Generators", ["sequence", "range", "numbers"]) && (
1142       <ToolCard
1143         id={124}
1144         title="JSON Schema Generator"
1145         category="Generators"
1146         description="Generate JSON schemas from a template"
1147         output={t124Out}
1148       >
1149         <div className="flex flex-wrap items-center gap-2">
1150           <input
1151             type="text"
1152             value={t124Template}
1153             onChange={(e) => setT124Template(e.target.value)}
1154             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1155               dark:bg-slate-900 font-mono"
1156             placeholder="Template"
1157           />
1158           <span className="text-xs">Generate</span>
1159           <button
1160             type="button"
1161             onClick={() => {
1162               const template = t124Template;
1163               if (template.length < 1) return;
1164               const rows = template.split("\n");
1165               const schema = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1166               setT124Out(schema);
1167             }}
1168           >
1169             Generate
1170           </button>
1171         </div>
1172       </ToolCard>
1173     )}
1174
1175     {/* TOOL 125 - XML Schema Generator */}
1176     {isVisible(125, "XML Schema Generator", "Generators", ["sequence", "range", "numbers"]) && (
1177       <ToolCard
1178         id={125}
1179         title="XML Schema Generator"
1180         category="Generators"
1181         description="Generate XML schemas from a template"
1182         output={t125Out}
1183       >
1184         <div className="flex flex-wrap items-center gap-2">
1185           <input
1186             type="text"
1187             value={t125Template}
1188             onChange={(e) => setT125Template(e.target.value)}
1189             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1190               dark:bg-slate-900 font-mono"
1191             placeholder="Template"
1192           />
1193           <span className="text-xs">Generate</span>
1194           <button
1195             type="button"
1196             onClick={() => {
1197               const template = t125Template;
1198               if (template.length < 1) return;
1199               const rows = template.split("\n");
1200               const schema = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1201               setT125Out(schema);
1202             }}
1203           >
1204             Generate
1205           </button>
1206         </div>
1207       </ToolCard>
1208     )}
1209
1210     {/* TOOL 126 - YAML Schema Generator */}
1211     {isVisible(126, "YAML Schema Generator", "Generators", ["sequence", "range", "numbers"]) && (
1212       <ToolCard
1213         id={126}
1214         title="YAML Schema Generator"
1215         category="Generators"
1216         description="Generate YAML schemas from a template"
1217         output={t126Out}
1218       >
1219         <div className="flex flex-wrap items-center gap-2">
1220           <input
1221             type="text"
1222             value={t126Template}
1223             onChange={(e) => setT126Template(e.target.value)}
1224             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1225               dark:bg-slate-900 font-mono"
1226             placeholder="Template"
1227           />
1228           <span className="text-xs">Generate</span>
1229           <button
1230             type="button"
1231             onClick={() => {
1232               const template = t126Template;
1233               if (template.length < 1) return;
1234               const rows = template.split("\n");
1235               const schema = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1236               setT126Out(schema);
1237             }}
1238           >
1239             Generate
1240           </button>
1241         </div>
1242       </ToolCard>
1243     )}
1244
1245     {/* TOOL 127 - SQL Schema Generator */}
1246     {isVisible(127, "SQL Schema Generator", "Generators", ["sequence", "range", "numbers"]) && (
1247       <ToolCard
1248         id={127}
1249         title="SQL Schema Generator"
1250         category="Generators"
1251         description="Generate SQL schemas from a template"
1252         output={t127Out}
1253       >
1254         <div className="flex flex-wrap items-center gap-2">
1255           <input
1256             type="text"
1257             value={t127Template}
1258             onChange={(e) => setT127Template(e.target.value)}
1259             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1260               dark:bg-slate-900 font-mono"
1261             placeholder="Template"
1262           />
1263           <span className="text-xs">Generate</span>
1264           <button
1265             type="button"
1266             onClick={() => {
1267               const template = t127Template;
1268               if (template.length < 1) return;
1269               const rows = template.split("\n");
1270               const schema = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1271               setT127Out(schema);
1272             }}
1273           >
1274             Generate
1275           </button>
1276         </div>
1277       </ToolCard>
1278     )}
1279
1280     {/* TOOL 128 - Bash Script Generator (continued) */}
1281     {isVisible(128, "Bash Script Generator", "Generators", ["sequence", "range", "numbers"]) && (
1282       <ToolCard
1283         id={128}
1284         title="Bash Script Generator"
1285         category="Generators"
1286         description="Generate Bash scripts from a template"
1287         output={t128Out}
1288       >
1289         <div className="flex flex-wrap items-center gap-2">
1290           <input
1291             type="text"
1292             value={t128Template}
1293             onChange={(e) => setT128Template(e.target.value)}
1294             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1295               dark:bg-slate-900 font-mono"
1296             placeholder="Template"
1297           />
1298           <span className="text-xs">Generate</span>
1299           <button
1300             type="button"
1301             onClick={() => {
1302               const template = t128Template;
1303               if (template.length < 1) return;
1304               const rows = template.split("\n");
1305               const bash = rows.map((row) => row.replace(/"/g, '""')).join("\n");
1306               setT128Out(bash);
1307             }}
1308           >
1309             Generate
1310           </button>
1311         </div>
1312       </ToolCard>
1313     )}
1314
1315     {/* TOOL 129 - Python Script Generator (continued) */}
1316     {isVisible(129, "Python Script Generator", "Generators", ["sequence", "range", "numbers"]) && (
1317       <ToolCard
1318         id={129}
1319         title="Python Script Generator"
1320         category="Generators"
1321         description="Generate Python scripts from a template"
1322         output={t129Out}
1323       >
1324         <div className="flex flex-wrap items-center gap-2">
1325           <input
1326             type="text"
1327             value={t129Template}
1328             onChange={(e) => setT129Template(e.target.value)}
1329             className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1330               dark:bg-slate-900 font-mono"
1331             placeholder="Template"
1332           />
1333           <span className="text-xs">Generate</span>
1334           <button
1335             type="button"
1336             onClick={() => {
1337               const template
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

550         }}
551         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
552     >
553         Generate
554     </button>
555 </div>
556 </ToolCard>
557 }}
558
559 {/* TOOL 109 - Text Padding Generator */}
560 {isVisible(109, "Text Padding Generator", "Text", ["padstart", "padend", "zero-fill"]) && (
561     <ToolCard
562         id={109}
563         title="Text Padding (PadStart / PadEnd)"
564         category="Text"
565         description="Pad strings with characters up to target length"
566         output={t109Out}
567     >
568         <div className="flex flex-wrap items-center gap-2">
569             <input
570                 type="text"
571                 value={t109In}
572                 onChange={(e) => setT109In(e.target.value)}
573                 className="w-24 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
574                 dark:bg-slate-900 font-mon
575             <input
576                 type="number"
577                 min={1}
578                 max={30}
579                 value={t109Len}
580                 onChange={(e) => setT109Len(Number(e.target.value))}
581                 className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
582                 dark:bg-slate-900 font-mon
583             <input
584                 type="text"
585                 maxLength={1}
586                 value={t109Char}
587                 onChange={(e) => setT109Char(e.target.value)}
588                 className="w-10 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
589                 dark:bg-slate-900 font-mon
590             <button
591                 type="button"
592                 onClick={() => setT109Out(t109In.padStart(t109Len, t109Char))}
593                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
594             >
595                 Pad Start
596             </button>
597             <button
598                 type="button"
599                 onClick={() => setT109Out(t109In.padEnd(t109Len, t109Char))}
600                 className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs font-semibold"
601             >
602                 Pad End
603             </button>
604         </div>
605     </ToolCard>
606 }}
607
608 {/* TOOL 110 - Text Wrap Converter */}
609 {isVisible(110, "Text Wrap Converter", "Text", ["word wrap", "column width"]) && (
610     <ToolCard
611         id={110}
612         title="Text Word-Wrap at N Columns"
613         category="Text"
614         description="Break long paragraphs into lines of fixed character width"
615         output={t110Out}
616     >
617         <textarea
618             value={t110In}

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

619         onChange={(e) => setT110In(e.target.value)}
620         rows={2}
621         className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
622 slate-900 font-mono"
623         <div className="flex items-center gap-2">
624             <span className="text-xs">Max width:</span>
625             <input
626                 type="number"
627                 min={10}
628                 max={120}
629                 value={t110Width}
630                 onChange={(e) => setT110Width(Number(e.target.value))}
631                 className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
632 dark:bg-slate-900 font-mon
633             <button
634                 type="button"
635                 onClick={() => {
636                     const words = t110In.split(" ");
637                     const lines: string[] = [];
638                     let cur = "";
639                     words.forEach((w) => {
640                         if ((cur + " " + w).trim().length <= t110Width) {
641                             cur = (cur + " " + w).trim();
642                         } else {
643                             if (cur) lines.push(cur);
644                             cur = w;
645                         }
646                     });
647                     if (cur) lines.push(cur);
648                     setT110Out(lines.join("\n"));
649                 }}
650                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
651             >
652                 Wrap Text
653             </button>
654         </div>
655     </ToolCard>
656 }}
657
658     {/* TOOL 111 - MD5 Hash */}
659     {isVisible(111, "MD5 Hash Generator", "Hashes", ["md5", "checksum"]) && (
660         <ToolCard
661             id={111}
662             title="MD5 Hash Generator"
663             category="Hashes"
664             description="Calculate 128-bit MD5 checksum for input text"
665             output={t111Out}
666         >
667             <input
668                 type="text"
669                 value={t111In}
670                 onChange={(e) => setT111In(e.target.value)}
671                 className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
672 dark:bg-slate-900 font-mon
673             <button
674                 type="button"
675                 onClick={() => setT111Out(simpleMd5(t111In))}
676                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
677             >
678                 Compute MD5
679             </button>
680         </ToolCard>
681     )}
682
683     {/* TOOL 112 - SHA-1 Hash */}
684     {isVisible(112, "SHA-1 Hash Generator", "Hashes", ["sha1", "git hash"]) && (
685         <ToolCard
686             id={112}
687             title="SHA-1 Hash Generator"

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
688         category="Hashes"
689         description="Generate standard 160-bit cryptographic hash"
690         output={t112Out}
691       >
692         <input
693           type="text"
694           value={t112In}
695           onChange={(e) => setT112In(e.target.value)}
696           className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
697 dark:bg-slate-900 font-mon
698         <button
699           type="button"
700           onClick={async () => setT112Out(await sha1Hex(t112In))}
701           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
702         >
703           Compute SHA-1
704         </button>
705       </ToolCard>
706     )}
707
708     {/* TOOL 113 - SHA-384 Hash */}
709     {isVisible(113, "SHA-384 Hash Generator", "Hashes", ["sha384", "sri"]) && (
710       <ToolCard
711         id={113}
712         title="SHA-384 Hash Generator"
713         category="Hashes"
714         description="Generate 384-bit cryptographic hash digest"
715         output={t113Out}
716       >
717         <input
718           type="text"
719           value={t113In}
720           onChange={(e) => setT113In(e.target.value)}
721           className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
722 dark:bg-slate-900 font-mon
723         <button
724           type="button"
725           onClick={async () => setT113Out(await sha384Hex(t113In))}
726           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
727         >
728           Compute SHA-384
729         </button>
730       </ToolCard>
731     )}
732
733     {/* TOOL 114 - SHA-512 Hash */}
734     {isVisible(114, "SHA-512 Hash Generator", "Hashes", ["sha512"]) && (
735       <ToolCard
736         id={114}
737         title="SHA-512 Hash Generator"
738         category="Hashes"
739         description="Generate 512-bit maximum strength cryptographic hash"
740         output={t114Out}
741       >
742         <input
743           type="text"
744           value={t114In}
745           onChange={(e) => setT114In(e.target.value)}
746           className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
747 dark:bg-slate-900 font-mon
748         <button
749           type="button"
750           onClick={async () => setT114Out(await sha512Hex(t114In))}
751           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
752         >
753           Compute SHA-512
754         </button>
755       </ToolCard>
756     )}
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

757
758     { /* TOOL 115 - HMAC SHA-512 */ }
759     { isVisible(115, "HMAC SHA-512 Generator", "Hashes", ["hmac sha512"]) && (
760         <ToolCard
761             id={115}
762             title="HMAC SHA-512 Generator"
763             category="Hashes"
764             description="Create high-security HMAC authentication tag"
765             output={t115Out}
766         >
767             <div className="space-y-2">
768                 <input
769                     type="text"
770                     value={t115Key}
771                     onChange={(e) => setT115Key(e.target.value)}
772                     placeholder="Key"
773                     className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
774 dark:bg-slate-900 font-m
775                 <input
776                     type="text"
777                     value={t115Msg}
778                     onChange={(e) => setT115Msg(e.target.value)}
779                     placeholder="Message"
780                     className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
781 dark:bg-slate-900 font-m
782                 <button
783                     type="button"
784                     onClick={async () => setT115Out(await hmacSha512Hex(t115Key, t115Msg))}
785                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
786                 >
787                     Generate HMAC SHA-512
788                 </button>
789             </div>
790         </ToolCard>
791     )}
792
793     { /* TOOL 116 - Hex Encoder / Decoder */ }
794     { isVisible(116, "Hex Encoder / Decoder", "Converters", ["hex to text", "text to hex"]) && (
795         <ToolCard
796             id={116}
797             title="Hexadecimal Text Encoder / Decoder"
798             category="Converters"
799             description="Convert strings into hexadecimal byte pairs and decode back"
800             output={t116Out}
801         >
802             <input
803                 type="text"
804                 value={t116In}
805                 onChange={(e) => setT116In(e.target.value)}
806                 className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
807 dark:bg-slate-900 font-mon
808             <div className="flex gap-2">
809                 <button
810                     type="button"
811                     onClick={() => setT116Out(stringToHex(t116In))}
812                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
813                 >
814                     Encode to Hex
815                 </button>
816                 <button
817                     type="button"
818                     onClick={() => setT116Out(hexToString(t116In))}
819                     className="px-3 py-1 bg-slate-700 hover:bg-slate-600 text-white rounded-md text-xs font-semibold"
820                 >
821                     Decode from Hex
822                 </button>
823             </div>
824         </ToolCard>
825     )}

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

826
827 { /* TOOL 117 - ROT13 */}
828 {isVisible(117, "ROT13 Encoder / Decoder", "Security", ["rot13", "caesar"])} && (
829   <ToolCard
830     id={117}
831     title="ROT13 Cipher"
832     category="Security"
833     description="Rotate letters by 13 positions (symmetric cipher)"
834     output={t117Out}
835   >
836     <input
837       type="text"
838       value={t117In}
839       onChange={(e) => setT117In(e.target.value)}
840       className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
841       dark:bg-slate-900 font-mon
842     <button
843       type="button"
844       onClick={() => setT117Out(rot13(t117In))}
845       className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
846     >
847       Apply ROT13
848     </button>
849   </ToolCard>
850 )}
851
852 { /* TOOL 118 - ROT47 */}
853 {isVisible(118, "ROT47 Encoder / Decoder", "Security", ["rot47", "ascii rotate"])} && (
854   <ToolCard
855     id={118}
856     title="ROT47 Cipher"
857     category="Security"
858     description="Rotate ASCII printable characters between 33 and 126 by 47"
859     output={t118Out}
860   >
861     <input
862       type="text"
863       value={t118In}
864       onChange={(e) => setT118In(e.target.value)}
865       className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
866       dark:bg-slate-900 font-mon
867     <button
868       type="button"
869       onClick={() => setT118Out(rot47(t118In))}
870       className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
871     >
872       Apply ROT47
873     </button>
874   </ToolCard>
875 )}
876
877 { /* TOOL 119 - Caesar Cipher */}
878 {isVisible(119, "Caesar Cipher", "Security", ["caesar", "shift"])} && (
879   <ToolCard
880     id={119}
881     title="Caesar Cipher with Custom Shift"
882     category="Security"
883     description="Shift alphabetic letters forward or backward by custom integer"
884     output={t119Out}
885   >
886     <div className="flex items-center gap-2">
887       <input
888         type="text"
889         value={t119In}
890         onChange={(e) => setT119In(e.target.value)}
891         className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
892         dark:bg-slate-900 font-m
893       <input
894         type="number"

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

895         min={-25}
896         max={25}
897         value={t119Shift}
898         onChange={(e) => setT119Shift(Number(e.target.value))}
899         className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
900 dark:bg-slate-900 font-mon
901         <button
902           type="button"
903           onClick={() => setT119Out(caesarCipher(t119In, t119Shift))}
904           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
905         >
906           Encrypt / Shift
907         </button>
908       </div>
909     </ToolCard>
910   )}
911
912   /* TOOL 120 - AES-GCM Key Generator */
913   {isVisible(120, "AES-GCM Key Generator", "Security", ["aes", "key", "web crypto"]) && (
914     <ToolCard
915       id={120}
916       title="AES-GCM 256-Bit Cryptographic Key Generator"
917       category="Security"
918       description="Generate cryptographically random 256-bit AES symmetric key in Hex"
919       output={t120Out}
920     >
921       <button
922         type="button"
923         onClick={() => {
924           const arr = new Uint8Array(32);
925           crypto.getRandomValues(arr);
926           const hex = Array.from(arr, (b) => b.toString(16).padStart(2, "0")).join("");
927           setT120Out(`AES-256 Key (Hex):\n${hex}\n\nIV / Nonce (12-byte):\n${Array.from(arr.slice(0, 12), (b)
928 => b.toString(16).padStart
929           className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
930         >
931           Generate Key & IV
932         </button>
933       </ToolCard>
934     )}
935
936   /* TOOL 121 - HTTP Header Generator */
937   {isVisible(121, "HTTP Header Generator", "Security", ["cors", "csp", "security headers"]) && (
938     <ToolCard
939       id={121}
940       title="Security HTTP Headers Generator"
941       category="Security"
942       description="Generate CORS, CSP, and HSTS headers for production servers"
943       output={t121Out}
944     >
945       <div className="flex gap-2">
946         <select
947           value={t121Type}
948           onChange={(e) => setT121Type(e.target.value)}
949           className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
950 dark:bg-slate-900"
951           <option value="cors">CORS Headers</option>
952           <option value="csp">Content-Security-Policy</option>
953           <option value="hsts">Strict-Transport-Security (HSTS)</option>
954         </select>
955         <button
956           type="button"
957           onClick={() => {
958             if (t121Type === "cors") {
959               setT121Out(
960                 "Access-Control-Allow-Origin: *\nAccess-Control-Allow-Methods: GET, POST, PUT, DELETE,
961                 OPTIONS\nAccess-Control-Allow-Hea
962             } else if (t121Type === "csp") {
963               setT121Out(

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

964         "Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src
965         'self' 'unsafe-inline'; img-s
966         } else {
967             setT121Out(
968                 "Strict-Transport-Security: max-age=31536000; includeSubDomains; preload\nX-Content-Type-
969 Options: nosniff\nX-Frame-Optio
970             )
971         }}
972         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
973     >
974         Generate Headers
975     </button>
976 </div>
977 </ToolCard>
978 )}
979
980 {/* TOOL 122 - cURL Request Builder */}
981 {isVisible(122, "cURL Request Builder", "API", ["curl", "bash http"]) && (
982     <ToolCard
983         id={122}
984         title="cURL Command Generator"
985         category="API"
986         description="Generate terminal cURL commands for API calls"
987         output={t122Out}
988     >
989         <div className="space-y-2">
990             <div className="flex gap-2">
991                 <select
992                     value={t122Method}
993                     onChange={(e) => setT122Method(e.target.value)}
994                     className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
995 dark:bg-slate-900"
996                     <option value="GET">GET</option>
997                     <option value="POST">POST</option>
998                     <option value="PUT">PUT</option>
999                     <option value="DELETE">DELETE</option>
1000                 </select>
1001                 <input
1002                     type="text"
1003                     value={t122Url}
1004                     onChange={(e) => setT122Url(e.target.value)}
1005                     className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-
1006 slate-50 dark:bg-slate-900 font
1007                 </div>
1008                 <button
1009                     type="button"
1010                     onClick={() => {
1011                         const body = t122Method !== "GET" ? ` -H "Content-Type: application/json" -d '${t122Data}'` : "";
1012                         setT122Out(`curl -X ${t122Method} "${t122Url}"${body}`);
1013                     }}
1014                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1015                 >
1016                     Generate cURL
1017                 </button>
1018             </div>
1019         </ToolCard>
1020     )}
1021
1022 {/* TOOL 123 - HTTP Response Header Viewer */}
1023 {isVisible(123, "HTTP Response Header Viewer", "API", ["headers", "inspect"]) && (
1024     <ToolCard
1025         id={123}
1026         title="HTTP Response Header Inspector"
1027         category="API"
1028         description="Display standard response header schema and cache instructions"
1029         output={t123Out}
1030     >
1031         <button
1032             type="button"

```


src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1033         onClick={() => {
1034             setT123Out(
1035                 "HTTP/1.1 200 OK\nDate: " +
1036                 new Date().toUTCString() +
1037                 "\nContent-Type: application/json; charset=utf-8\nCache-Control: public, max-age=3600\nETag: W/
1038                 \"33a-1cd9aa\"",
1039             ),
1040             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1041         }
1042         >
1043         Show Example Headers
1044     </button>
1045 </ToolCard>
1046 }}
1047
1048 {/* TOOL 124 - MIME Type Lookup */}
1049 {isVisible(124, "MIME Type Lookup", "Web", ["mime", "content-type", "extension"]) && (
1050     <ToolCard
1051         id={124}
1052         title="MIME Type Lookup"
1053         category="Web"
1054         description="Find Content-Type MIME strings for common file extensions"
1055         output={t124Out}
1056     >
1057         <div className="flex gap-2">
1058             <input
1059                 type="text"
1060                 value={t124Ext}
1061                 onChange={(e) => setT124Ext(e.target.value)}
1062                 placeholder="e.g. json, png, pdf, mp4"
1063                 className="w-32 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1064                 dark:bg-slate-900 font-mon"
1065             />
1066             <button
1067                 type="button"
1068                 onClick={() => {
1069                     const map: Record<string, string> = {
1070                         json: "application/json",
1071                         html: "text/html",
1072                         css: "text/css",
1073                         js: "application/javascript",
1074                         ts: "application/typescript",
1075                         png: "image/png",
1076                         jpg: "image/jpeg",
1077                         jpeg: "image/jpeg",
1078                         svg: "image/svg+xml",
1079                         pdf: "application/pdf",
1080                         zip: "application/zip",
1081                         mp4: "video/mp4",
1082                         mp3: "audio/mpeg",
1083                         woff2: "font/woff2",
1084                     };
1085                     const clean = t124Ext.trim().toLowerCase().replace(/^\./, "");
1086                     setT124Out(map[clean] || `Unknown MIME type for ${clean}`);
1087                 }}
1088                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1089             >
1090             Lookup MIME
1091         </button>
1092     </div>
1093 </ToolCard>
1094 }}
1095
1096 {/* TOOL 125 - User-Agent Parser */}
1097 {isVisible(125, "User-Agent Parser", "Utilities", ["user-agent", "browser engine"]) && (
1098     <ToolCard
1099         id={125}
1100         title="User-Agent Parser"
1101         category="Utilities"
1102         description="Inspect browser engine and OS version tokens from UA string"
1103         output={t125Out}

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1102         >
1103         <textarea
1104             value={t125Ua}
1105             onChange={(e) => setT125Ua(e.target.value)}
1106             rows={2}
1107             className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1108 slate-900 font-mono"
1109         <button
1110             type="button"
1111             onClick={() => {
1112                 const isChrome = /Chrome/.test(t125Ua) && !/Edg/.test(t125Ua);
1113                 const isFirefox = /Firefox/.test(t125Ua);
1114                 const isSafari = /Safari/.test(t125Ua) && !/Chrome/.test(t125Ua);
1115                 const isEdge = /Edg/.test(t125Ua);
1116                 const isMobile = /Mobile|Android|iPhone/.test(t125Ua);
1117                 setT125Out(
1118                     `Browser: ${
1119                         isChrome
1120                             ? "Google Chrome"
1121                             : isFirefox
1122                             ? "Mozilla Firefox"
1123                             : isSafari
1124                             ? "Apple Safari"
1125                             : isEdge
1126                             ? "Microsoft Edge"
1127                             : "Other / Unknown"
1128                     }\nDevice Category: ${isMobile ? "Mobile / Tablet" : "Desktop"}`
1129                 );
1130             }}
1131             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1132         >
1133             Parse User-Agent
1134         </button>
1135     </ToolCard>
1136 }}
1137
1138     {/* TOOL 126 - URL Shortener Slug Generator */}
1139     {isVisible(126, "URL Shortener Generator", "Web", ["shortener", "hashid", "slug"]) && (
1140         <ToolCard
1141             id={126}
1142             title="URL Shortener Hash Slug Generator"
1143             category="Web"
1144             description="Generate base62 short slug from destination URL"
1145             output={t126Out}
1146         >
1147             <input
1148                 type="text"
1149                 value={t126Url}
1150                 onChange={(e) => setT126Url(e.target.value)}
1151                 className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1152 dark:bg-slate-900 font-mon
1153             >
1154             <button
1155                 type="button"
1156                 onClick={() => {
1157                     let hash = 0;
1158                     for (let i = 0; i < t126Url.length; i++) {
1159                         hash = (hash << 5) - hash + t126Url.charCodeAt(i);
1160                         hash |= 0;
1161                     }
1162                     const slug = Math.abs(hash).toString(36).slice(0, 6);
1163                     setT126Out(`Short Slug: ${slug}\nExample URL: https://hub.to/${slug}`);
1164                 }}
1165                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1166             >
1167                 Generate Slug
1168             </button>
1169         </ToolCard>
1170     )}
1171

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1171     { /* TOOL 127 - Open Graph Meta Generator */ }
1172     { isVisible(127, "Open Graph Meta Generator", "Web", ["og:image", "facebook", "linkedin"]) && (
1173         <ToolCard
1174             id={127}
1175             title="Open Graph Social Preview Meta Tags"
1176             category="Web"
1177             description="Generate standard og:title, og:image, and og:type tags"
1178             output={t127Out}
1179         >
1180             <div className="space-y-2">
1181                 <input
1182                     type="text"
1183                     value={t127Title}
1184                     onChange={(e) => setT127Title(e.target.value)}
1185                     placeholder="Title"
1186                     className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1187 dark:bg-slate-900 font-m
1188                 <input
1189                     type="text"
1190                     value={t127Img}
1191                     onChange={(e) => setT127Img(e.target.value)}
1192                     placeholder="Image URL"
1193                     className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1194 dark:bg-slate-900 font-m
1195                 <button
1196                     type="button"
1197                     onClick={() => {
1198                         setT127Out(
1199                             `<meta property="og:type" content="website">\n<meta property="og:title"
1200 content="\${t127Title}>\n<meta property="og:image"
1201                             `)
1202                         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1203                     }
1204                 >
1205                     Generate OG Meta
1206                 </button>
1207             </div>
1208         </ToolCard>
1209     )}
1210     { /* TOOL 128 - Twitter Card Generator */ }
1211     { isVisible(128, "Twitter Card Generator", "Web", ["twitter:card", "social cards"]) && (
1212         <ToolCard
1213             id={128}
1214             title="Twitter / X Card Meta Tags"
1215             category="Web"
1216             description="Generate summary_large_image card tags"
1217             output={t128Out}
1218         >
1219             <div className="flex gap-2">
1220                 <input
1221                     type="text"
1222                     value={t128Title}
1223                     onChange={(e) => setT128Title(e.target.value)}
1224                     placeholder="Title"
1225                     className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1226 dark:bg-slate-900 font-m
1227                 <input
1228                     type="text"
1229                     value={t128Handle}
1230                     onChange={(e) => setT128Handle(e.target.value)}
1231                     placeholder="@handle"
1232                     className="w-28 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1233 dark:bg-slate-900 font-mon
1234                 <button
1235                     type="button"
1236                     onClick={() => {
1237                         setT128Out(
1238                             `<meta name="twitter:card" content="summary_large_image">\n<meta name="twitter:site"
1239 content="\${t128Handle}>\n<meta name=

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1240         }}
1241         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1242     >
1243         Generate
1244     </button>
1245 </div>
1246 </ToolCard>
1247 )}
1248
1249 {/* TOOL 129 - Canonical URL Generator */}
1250 {isVisible(129, "Canonical URL Generator", "Web", ["canonical", "duplicate content"]) && (
1251     <ToolCard
1252         id={129}
1253         title="Canonical Link Element Generator"
1254         category="Web"
1255         description="Generate link rel=canonical tags to avoid search engine duplicate indexing"
1256         output={t129Out}
1257     >
1258         <input
1259             type="text"
1260             value={t129Url}
1261             onChange={(e) => setT129Url(e.target.value)}
1262             className="w-full text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1263             dark:bg-slate-900 font-mon
1264         >
1265             <button
1266                 type="button"
1267                 onClick={() => setT129Out(`<link rel="canonical" href="${t129Url}">`)}
1268                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1269             >
1270                 Generate Canonical Link
1271             </button>
1272         </ToolCard>
1273     )}
1274
1275 {/* TOOL 130 - Web Manifest Generator */}
1276 {isVisible(130, "Web Manifest Generator", "Web", ["manifest.json", "pwa"]) && (
1277     <ToolCard
1278         id={130}
1279         title="Web App Manifest.json Generator"
1280         category="Web"
1281         description="Create standard PWA manifest definition"
1282         output={t130Out}
1283     >
1284         <div className="flex items-center gap-2">
1285             <input
1286                 type="text"
1287                 value={t130Name}
1288                 onChange={(e) => setT130Name(e.target.value)}
1289                 className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1290                 dark:bg-slate-900 font-m
1291             >
1292             <input
1293                 type="color"
1294                 value={t130Color}
1295                 onChange={(e) => setT130Color(e.target.value)}
1296                 className="w-8 h-7 rounded border cursor-pointer"
1297             />
1298             <button
1299                 type="button"
1300                 onClick={() => {
1301                     setT130Out(
1302                         JSON.stringify(
1303                             {
1304                                 name: t130Name,
1305                                 short_name: t130Name.slice(0, 12),
1306                                 start_url: "/",
1307                                 display: "standalone",
1308                                 background_color: "#ffffff",
1309                                 theme_color: t130Color,
1310                             },

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
1309         null,
1310         2
1311     )
1312     );
1313     }}
1314     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1315     >
1316     Generate Manifest
1317     </button>
1318     </div>
1319     </ToolCard>
1320     )}
1321
1322     {/* TOOL 131 - CSS Button Generator */}
1323     {isVisible(131, "CSS Button Generator", "CSS", ["button styles", "custom button"]) && (
1324     <ToolCard
1325     id={131}
1326     title="CSS Button Style Generator"
1327     category="CSS"
1328     description="Generate button CSS with live preview"
1329     output={`background-color: ${t131Bg};\ncolor: ${t131Color};\nborder-radius: ${t131Radius}px;\npadding:
1330     8px 16px;\nborder: none;\nc
1331     <div className="flex flex-wrap items-center gap-2">
1332     <input
1333     type="color"
1334     value={t131Bg}
1335     onChange={(e) => setT131Bg(e.target.value)}
1336     className="w-8 h-7 rounded border cursor-pointer"
1337     />
1338     <input
1339     type="color"
1340     value={t131Color}
1341     onChange={(e) => setT131Color(e.target.value)}
1342     className="w-8 h-7 rounded border cursor-pointer"
1343     />
1344     <input
1345     type="range"
1346     min={0}
1347     max={24}
1348     value={t131Radius}
1349     onChange={(e) => setT131Radius(Number(e.target.value))}
1350     className="w-24"
1351     />
1352     <button
1353     type="button"
1354     style={{
1355     backgroundColor: t131Bg,
1356     color: t131Color,
1357     borderRadius: `${t131Radius}px`,
1358     }}
1359     className="px-4 py-1.5 text-xs font-semibold shadow-sm ml-auto"
1360     >
1361     Sample Button
1362     </button>
1363     </div>
1364     </ToolCard>
1365     )}
1366
1367     {/* TOOL 132 - CSS Text Shadow Generator */}
1368     {isVisible(132, "CSS Text Shadow Generator", "CSS", ["text-shadow", "typography glow"]) && (
1369     <ToolCard
1370     id={132}
1371     title="CSS Text Shadow Generator"
1372     category="CSS"
1373     description="Configure text-shadow with offsets and blur radius"
1374     output={`text-shadow: ${t132X}px ${t132Y}px ${t132Blur}px rgba(0, 0, 0, 0.5);`}
1375     >
1376     <div className="flex items-center gap-2">
1377     <input
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
1378         type="number"
1379         value={t132X}
1380         onChange={(e) => setT132X(Number(e.target.value))}
1381         className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1382 dark:bg-slate-900 font-mon
1383       <input
1384         type="number"
1385         value={t132Y}
1386         onChange={(e) => setT132Y(Number(e.target.value))}
1387         className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1388 dark:bg-slate-900 font-mon
1389       <input
1390         type="number"
1391         value={t132Blur}
1392         onChange={(e) => setT132Blur(Number(e.target.value))}
1393         className="w-16 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1394 dark:bg-slate-900 font-mon
1395     </div>
1396   </ToolCard>
1397 )}
1398
1399   {/* TOOL 133 - CSS Animation Keyframe Generator */}
1400   {isVisible(133, "CSS Animation Generator", "CSS", ["@keyframes", "animation"])} && (
1401     <ToolCard
1402       id={133}
1403       title="CSS @keyframes Animation Generator"
1404       category="CSS"
1405       description="Create smooth CSS animation keyframes"
1406       output={t133Out}
1407     >
1408       <div className="flex gap-2">
1409         <input
1410           type="text"
1411           value={t133Name}
1412           onChange={(e) => setT133Name(e.target.value)}
1413           className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1414 dark:bg-slate-900 font-m
1415         <button
1416           type="button"
1417           onClick={() => {
1418             setT133Out(
1419               {\n      opacity: 1;\n      @keyframes ${t133Name} {\n        0% {\n          opacity: 0;\n          transform: translateY(20px);\n        }\n        100%
1420               }\n      opacity: 1;\n      }\n
1421             )}
1422             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1423           >
1424             Generate Keyframes
1425           </button>
1426         </div>
1427       </ToolCard>
1428     )}
1429
1430   {/* TOOL 134 - CSS Transform Generator */}
1431   {isVisible(134, "CSS Transform Generator", "CSS", ["rotate", "scale", "transform"])} && (
1432     <ToolCard
1433       id={134}
1434       title="CSS Transform Generator (Rotate & Scale)"
1435       category="CSS"
1436       description="Configure 2D transform matrix rules"
1437       output={`transform: rotate(${t134Rot}deg) scale(${t134Scale});`}
1438     >
1439       <div className="flex items-center gap-3 text-xs">
1440         <span>Rot:</span>
1441         <input
1442           type="range"
1443           min={-180}
1444           max={180}
1445           value={t134Rot}
1446           onChange={(e) => setT134Rot(Number(e.target.value))}
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
1447         className="w-28"
1448       />
1449     <span>Scale:</span>
1450     <input
1451       type="range"
1452       min={0.5}
1453       max={2}
1454       step={0.1}
1455       value={t134Scale}
1456       onChange={(e) => setT134Scale(Number(e.target.value))}
1457       className="w-28"
1458     />
1459   </div>
1460 </ToolCard>
1461 }}
1462
1463 {/* TOOL 135 - CSS Transition Generator */}
1464 {isVisible(135, "CSS Transition Generator", "CSS", ["transition", "timing-function"]) && (
1465   <ToolCard
1466     id={135}
1467     title="CSS Transition Generator"
1468     category="CSS"
1469     description="Generate smooth transition property definitions"
1470     output={`transition: ${t135Prop} ${t135Duration} ${t135Ease};`}
1471   >
1472     <div className="flex gap-2 text-xs">
1473       <input
1474         type="text"
1475         value={t135Prop}
1476         onChange={(e) => setT135Prop(e.target.value)}
1477         className="w-20 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1478 slate-900 font-mono"
1479       />
1480       <input
1481         type="text"
1482         value={t135Duration}
1483         onChange={(e) => setT135Duration(e.target.value)}
1484         className="w-16 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1485 slate-900 font-mono"
1486       />
1487       <select
1488         value={t135Ease}
1489         onChange={(e) => setT135Ease(e.target.value)}
1490         className="px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1491 slate-900"
1492       >
1493         <option value="ease">ease</option>
1494         <option value="ease-in">ease-in</option>
1495         <option value="ease-out">ease-out</option>
1496         <option value="ease-in-out">ease-in-out</option>
1497         <option value="linear">linear</option>
1498       </select>
1499     </div>
1500   </ToolCard>
1501 )}
1502
1503 {/* TOOL 136 - CSS Filter Generator */}
1504 {isVisible(136, "CSS Filter Generator", "CSS", ["filter", "blur", "brightness"]) && (
1505   <ToolCard
1506     id={136}
1507     title="CSS Filter Generator"
1508     category="CSS"
1509     description="Tune blur, brightness, and contrast filters"
1510     output={`filter: blur(${t136Blur}px) brightness(${t136Brightness}%) contrast(${t136Contrast}%)`}
1511   >
1512     <div className="grid grid-cols-3 gap-2 text-xs">
1513       <div>
1514         <span>Blur ({t136Blur}px)</span>
1515         <input
1516           type="range"
1517           min={0}
1518           max={20}
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
1516         value={t136Blur}
1517         onChange={(e) => setT136Blur(Number(e.target.value))}
1518         className="w-full"
1519       />
1520     </div>
1521     <div>
1522       <span>Bright ({t136Brightness}%)</span>
1523       <input
1524         type="range"
1525         min={50}
1526         max={150}
1527         value={t136Brightness}
1528         onChange={(e) => setT136Brightness(Number(e.target.value))}
1529         className="w-full"
1530       />
1531     </div>
1532     <div>
1533       <span>Contrast ({t136Contrast}%)</span>
1534       <input
1535         type="range"
1536         min={50}
1537         max={150}
1538         value={t136Contrast}
1539         onChange={(e) => setT136Contrast(Number(e.target.value))}
1540         className="w-full"
1541       />
1542     </div>
1543   </div>
1544 </ToolCard>
1545 }}
1546
1547 /* TOOL 137 - CSS Glassmorphism Generator */
1548 {isVisible(137, "CSS Glassmorphism Generator", "CSS", ["glassmorphism", "backdrop-filter"]) && (
1549   <ToolCard
1550     id={137}
1551     title="CSS Glassmorphism Generator"
1552     category="CSS"
1553     description="Generate frosted glass card styles with backdrop blur"
1554     output={`background: rgba(255, 255, 255, ${t137Opacity});\nbackdrop-filter: blur(${t137Blur}px);\n-webkit-
1555     backdrop-filter: blur(${
1556       <div className="flex items-center gap-3 text-xs">
1557         <span>Blur:</span>
1558         <input
1559           type="range"
1560           min={1}
1561           max={30}
1562           value={t137Blur}
1563           onChange={(e) => setT137Blur(Number(e.target.value))}
1564           className="w-24"
1565         />
1566         <span>Opacity:</span>
1567         <input
1568           type="range"
1569           min={0.05}
1570           max={0.8}
1571           step={0.05}
1572           value={t137Opacity}
1573           onChange={(e) => setT137Opacity(Number(e.target.value))}
1574           className="w-24"
1575         />
1576       </div>
1577     </ToolCard>
1578   )}
1579
1580 /* TOOL 138 - CSS Neumorphism Generator */
1581 {isVisible(138, "CSS Neumorphism Generator", "CSS", ["neumorphism", "soft ui"]) && (
1582   <ToolCard
1583     id={138}
1584     title="CSS Neumorphism Soft UI Generator"
```


src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1585         category="CSS"
1586         description="Create dual-shadow soft extruded surface styles"
1587         output={`background: #e0e5ec;\nbox-shadow: ${t138Dist}px ${t138Dist}px ${t138Blur}px #a3b1c6, -${t138Dist}
1588 px -${t138Dist}px ${t138Blur}px #a3b1c6, -${t138Dist}
1589         <div className="flex items-center gap-3 text-xs">
1590             <span>Distance:</span>
1591             <input
1592                 type="range"
1593                 min={2}
1594                 max={20}
1595                 value={t138Dist}
1596                 onChange={(e) => setT138Dist(Number(e.target.value))}
1597                 className="w-24"
1598             />
1599             <span>Blur:</span>
1600             <input
1601                 type="range"
1602                 min={4}
1603                 max={30}
1604                 value={t138Blur}
1605                 onChange={(e) => setT138Blur(Number(e.target.value))}
1606                 className="w-24"
1607             />
1608         </div>
1609     </ToolCard>
1610 }}
1611
1612     {/* TOOL 139 - CSS Triangle Generator */}
1613     {isVisible(139, "CSS Triangle Generator", "CSS", ["triangle", "border hack"]) && (
1614         <ToolCard
1615             id={139}
1616             title="CSS Pure Border Triangle Generator"
1617             category="CSS"
1618             description="Generate clean CSS border arrows without images"
1619             output={`width: 0;\nheight: 0;\nborder-left: ${t139Size}px solid ${t139Dir === "right" ? t139Color :
1620 "transparent"};\nborder-right
1621         <div className="flex items-center gap-2 text-xs">
1622             <select
1623                 value={t139Dir}
1624                 onChange={(e) => setT139Dir(e.target.value)}
1625                 className="px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1626 slate-900"
1627             >
1628                 <option value="up">Up</option>
1629                 <option value="down">Down</option>
1630                 <option value="left">Left</option>
1631                 <option value="right">Right</option>
1632             </select>
1633             <input
1634                 type="number"
1635                 min={4}
1636                 max={64}
1637                 value={t139Size}
1638                 onChange={(e) => setT139Size(Number(e.target.value))}
1639                 className="w-16 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1640 slate-900 font-mono"
1641             <input
1642                 type="color"
1643                 value={t139Color}
1644                 onChange={(e) => setT139Color(e.target.value)}
1645                 className="w-8 h-7 rounded border cursor-pointer"
1646             />
1647         </div>
1648     </ToolCard>
1649 }}
1650
1651     {/* TOOL 140 - CSS Spinner Generator */}
1652     {isVisible(140, "CSS Spinner Generator", "CSS", ["spinner", "loading"]) && (
1653         <ToolCard
1654             id={140}

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1654         title="CSS Circular Loading Spinner"
1655         category="CSS"
1656         description="Generate lightweight animated loading spinner"
1657         output={`.spinner {\n  width: ${t140Size}px;\n  height: ${t140Size}px;\n  border: 4px solid rgba(0, 0, 0,
1658 0.1);\n  border-top-color
1659         <div className="flex items-center gap-2 text-xs">
1660             <input
1661                 type="number"
1662                 min={16}
1663                 max={80}
1664                 value={t140Size}
1665                 onChange={(e) => setT140Size(Number(e.target.value))}
1666                 className="w-16 px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1667 slate-900 font-mono"
1668             <input
1669                 type="color"
1670                 value={t140Color}
1671                 onChange={(e) => setT140Color(e.target.value)}
1672                 className="w-8 h-7 rounded border cursor-pointer"
1673             />
1674             <div
1675                 className="rounded-full animate-spin border-4 border-slate-200 ml-auto"
1676                 style={{
1677                     width: `${t140Size}px`,
1678                     height: `${t140Size}px`,
1679                     borderTopColor: t140Color,
1680                 }}
1681             />
1682         </div>
1683     </ToolCard>
1684 })
1685
1686     /* TOOL 141 - JS Deep Clone Helper */
1687     {isVisible(141, "JS Deep Clone Helper", "JavaScript", ["clone", "structuredclone", "deep copy"]) && (
1688         <ToolCard
1689             id={141}
1690             title="JavaScript Deep Clone & Formatter"
1691             category="JavaScript"
1692             description="Deep clone complex nested data structures with structuredClone"
1693             output={t141Out}
1694         >
1695             <textarea
1696                 value={t141In}
1697                 onChange={(e) => setT141In(e.target.value)}
1698                 rows={2}
1699                 className="w-full text-xs p-2 rounded border border-slate-300 dark:border-slate-700 bg-slate-50 dark:bg-
1700 slate-900 font-mono"
1701             <button
1702                 type="button"
1703                 onClick={() => {
1704                     try {
1705                         const parsed = JSON.parse(t141In);
1706                         const cloned = structuredClone(parsed);
1707                         setT141Out(`// Cloned via structuredClone:\n${JSON.stringify(cloned, null, 2)}`);
1708                     } catch (e: any) {
1709                         setT141Out(`'L ' + e.message);
1710                     }
1711                 }}
1712                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1713             >
1714                 Deep Clone
1715             </button>
1716         </ToolCard>
1717     )}
1718
1719     /* TOOL 142 - JS Array Generator */
1720     {isVisible(142, "JS Array Generator", "JavaScript", ["array.from", "fill"]) && (
1721         <ToolCard
1722             id={142}

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1723         title="JavaScript Array.from Generator"
1724         category="JavaScript"
1725         description="Create initialized arrays of length N"
1726         output={t142Out}
1727     >
1728     <div className="flex items-center gap-2">
1729         <input
1730             type="number"
1731             min={1}
1732             max={100}
1733             value={t142Len}
1734             onChange={(e) => setT142Len(Number(e.target.value))}
1735             className="w-20 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1736             dark:bg-slate-900 font-mon
1737         <button
1738             type="button"
1739             onClick={() => {
1740                 const arr = Array.from({ length: t142Len }, (_, i) => i + 1);
1741                 setT142Out(JSON.stringify(arr));
1742             }}
1743             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1744         >
1745             Generate Array
1746         </button>
1747     </div>
1748 </ToolCard>
1749 }}
1750
1751 {/* TOOL 143 - Array Methods Cheat Sheet */}
1752 {isVisible(143, "Array Methods Cheat Sheet", "JavaScript", ["map", "filter", "reduce", "find"]) && (
1753     <ToolCard
1754         id={143}
1755         title="JS Array Methods Quick Reference"
1756         category="JavaScript"
1757         description="Essential code patterns for array transformations"
1758         output={t143Out}
1759     >
1760     <div className="flex gap-2">
1761         <select
1762             value={t143Method}
1763             onChange={(e) => setT143Method(e.target.value)}
1764             className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1765             dark:bg-slate-900"
1766             <option value="map">.map()</option>
1767             <option value="filter">.filter()</option>
1768             <option value="reduce">.reduce()</option>
1769             <option value="find">.find()</option>
1770         </select>
1771         <button
1772             type="button"
1773             onClick={() => {
1774                 if (t143Method === "map") {
1775                     setT143Out("const doubled = numbers.map(n => n * 2);");
1776                 } else if (t143Method === "filter") {
1777                     setT143Out("const evens = numbers.filter(n => n % 2 === 0);");
1778                 } else if (t143Method === "reduce") {
1779                     setT143Out("const sum = numbers.reduce((acc, curr) => acc + curr, 0);");
1780                 } else {
1781                     setT143Out("const user = users.find(u => u.id === targetId);");
1782                 }
1783             }}
1784             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1785         >
1786             Show Example
1787         </button>
1788     </div>
1789 </ToolCard>
1790 }}
1791

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
1792     { /* TOOL 144 - Date Helper Snippets */ }
1793     { isVisible(144, "Date Helper Snippets", "JavaScript", ["date difference", "add days"]) && (
1794         <ToolCard
1795             id={144}
1796             title="JavaScript Date Calculation Snippets"
1797             category="JavaScript"
1798             description="Copyable snippets for adding days and calculating duration differences"
1799             output={t144Out}
1800         >
1801             <div className="flex gap-2">
1802                 <select
1803                     value={t144Type}
1804                     onChange={(e) => setT144Type(e.target.value)}
1805                     className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1806                     dark:bg-slate-900">
1807                     <option value="diff">Days Difference</option>
1808                     <option value="add">Add N Days</option>
1809                 </select>
1810                 <button
1811                     type="button"
1812                     onClick={() => {
1813                         if (t144Type === "diff") {
1814                             setT144Out("const diffDays = Math.ceil(Math.abs(d2 - d1) / (1000 * 60 * 60 * 24));");
1815                         } else {
1816                             setT144Out("const future = new Date();\nfuture.setDate(future.getDate() + 7);");
1817                         }
1818                     }}
1819                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1820                 >
1821                     Generate Snippet
1822                 </button>
1823             </div>
1824         </ToolCard>
1825     )}
1826
1827     { /* TOOL 145 - Debounce Function Snippet */ }
1828     { isVisible(145, "Debounce Function Snippet", "JavaScript", ["debounce", "rate limit", "search input"]) && (
1829         <ToolCard
1830             id={145}
1831             title="JavaScript Debounce Function Snippet"
1832             category="JavaScript"
1833             description="Delay function execution until typing or actions pause"
1834             output={t145Out}
1835         >
1836             <button
1837                 type="button"
1838                 onClick={() => {
1839                     setT145Out(
1840                         `function debounce(fn, delay = 300) {\n  let timer;\n  return function (...args) {\n
1841                         clearTimeout(timer);\n    timer = se
1842                     }
1843                     className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1844                 >
1845                     Generate Debounce
1846                 </button>
1847             </ToolCard>
1848     )}
1849
1850     { /* TOOL 146 - Throttle Function Snippet */ }
1851     { isVisible(146, "Throttle Function Snippet", "JavaScript", ["throttle", "scroll performance"]) && (
1852         <ToolCard
1853             id={146}
1854             title="JavaScript Throttle Function Snippet"
1855             category="JavaScript"
1856             description="Enforce maximum execution frequency for scroll and resize events"
1857             output={t146Out}
1858         >
1859             <button
1860                 type="button"
```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1861         onClick={() => {
1862             setT146Out(
1863                 `function throttle(fn, limit = 200) {\n  let waiting = false;\n  return function (...args) {\n
1864 if (!waiting) {\n      fn.
1865         }\n    }\n  }\n`
1866             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1867         >
1868             Generate Throttle
1869         </button>
1870     </ToolCard>
1871 })
1872
1873 /* TOOL 147 - Event Listener Snippet */
1874 {isVisible(147, "Event Listener Snippet", "JavaScript", ["addeventlistener", "dom event"])} && (
1875     <ToolCard
1876         id={147}
1877         title="DOM Event Listener Generator"
1878         category="JavaScript"
1879         description="Generate event listeners with cleanup patterns"
1880         output={t147Out}
1881     >
1882         <div className="flex gap-2">
1883             <input
1884                 type="text"
1885                 value={t147Target}
1886                 onChange={(e) => setT147Target(e.target.value)}
1887                 placeholder="Element variable"
1888                 className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1889 dark:bg-slate-900 font-m
1890             <select
1891                 value={t147Event}
1892                 onChange={(e) => setT147Event(e.target.value)}
1893                 className="text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1894 dark:bg-slate-900"
1895                 <option value="click">click</option>
1896                 <option value="submit">submit</option>
1897                 <option value="keydown">keydown</option>
1898                 <option value="change">change</option>
1899             </select>
1900             <button
1901                 type="button"
1902                 onClick={() => {
1903                     setT147Out(
1904                         `const handler = (event) => {\n  console.log("${t147Event} triggered", event);\n};
1905 \n${t147Target}.addEventListener("${t147
1906                 }\n`
1907             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1908         >
1909             Generate
1910         </button>
1911     </div>
1912 </ToolCard>
1913 })
1914
1915 /* TOOL 148 - DOM Selector Generator */
1916 {isVisible(148, "DOM Selector Generator", "JavaScript", ["querySelector", "querySelectorall"])} && (
1917     <ToolCard
1918         id={148}
1919         title="DOM Query Selector Generator"
1920         category="JavaScript"
1921         description="Generate type-safe DOM querying code"
1922         output={t148Out}
1923     >
1924         <div className="flex gap-2">
1925             <input
1926                 type="text"
1927                 value={t148Selector}
1928                 onChange={(e) => setT148Selector(e.target.value)}
1929                 className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
dark:bg-slate-900 font-m

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```

1930         />
1931         <button
1932             type="button"
1933             onClick={() => {
1934                 setT148Out(
1935                     `// Single element:\nconst element = document.querySelector("${t148Selector}");\n\n// Multiple
1936 elements as Array;\nconst i
1937                 }}
1938                 className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1939             >
1940                 Generate
1941             </button>
1942         </div>
1943     </ToolCard>
1944 }}
1945
1946     {/* TOOL 149 - LocalStorage Code Helper */}
1947     {isVisible(149, "LocalStorage Code Helper", "JavaScript", ["localStorage", "json storage"]) && (
1948         <ToolCard
1949             id={149}
1950             title="LocalStorage Type-Safe Storage Helper"
1951             category="JavaScript"
1952             description="Read and write JSON objects to browser localStorage"
1953             output={t149Out}
1954         >
1955             <div className="flex gap-2">
1956                 <input
1957                     type="text"
1958                     value={t149Key}
1959                     onChange={(e) => setT149Key(e.target.value)}
1960                     className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1961 dark:bg-slate-900 font-m
1962                 <button
1963                     type="button"
1964                     onClick={() => {
1965                         setT149Out(
1966                             `// Save item:\nlocalStorage.setItem("${t149Key}", JSON.stringify(data));\n\n// Retrieve item:
1967 \nconst raw = localStorage.g
1968                             }}
1969                             className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
1970                         >
1971                             Generate Code
1972                         </button>
1973                     </div>
1974                 </ToolCard>
1975             )}
1976
1977     {/* TOOL 150 - SessionStorage Code Helper */}
1978     {isVisible(150, "SessionStorage Code Helper", "JavaScript", ["sessionstorage", "tab session"]) && (
1979         <ToolCard
1980             id={150}
1981             title="SessionStorage Code Helper"
1982             category="JavaScript"
1983             description="Read and write session-scoped tab storage"
1984             output={t150Out}
1985         >
1986             <div className="flex gap-2">
1987                 <input
1988                     type="text"
1989                     value={t150Key}
1990                     onChange={(e) => setT150Key(e.target.value)}
1991                     className="flex-1 text-xs px-2 py-1 rounded border border-slate-300 dark:border-slate-700 bg-slate-50
1992 dark:bg-slate-900 font-m
1993                 <button
1994                     type="button"
1995                     onClick={() => {
1996                         setT150Out(
1997                             `// Save session:\nsessionStorage.setItem("${t150Key}", JSON.stringify(sessionData));\n\n// Clear
1998 on logout:\nsessionStora

```

src/components/tools/ToolsAdvancedDev.tsx (continued)

```
1999         }}
2000         className="px-3 py-1 bg-blue-600 hover:bg-blue-500 text-white rounded-md text-xs font-semibold"
2001       >
2002         Generate Code
2003       </button>
2004     </div>
2005   </ToolCard>
2006   )}
2007 </>
2008 );
2009 };
2010
```


